# AX foundation and Trend 2025

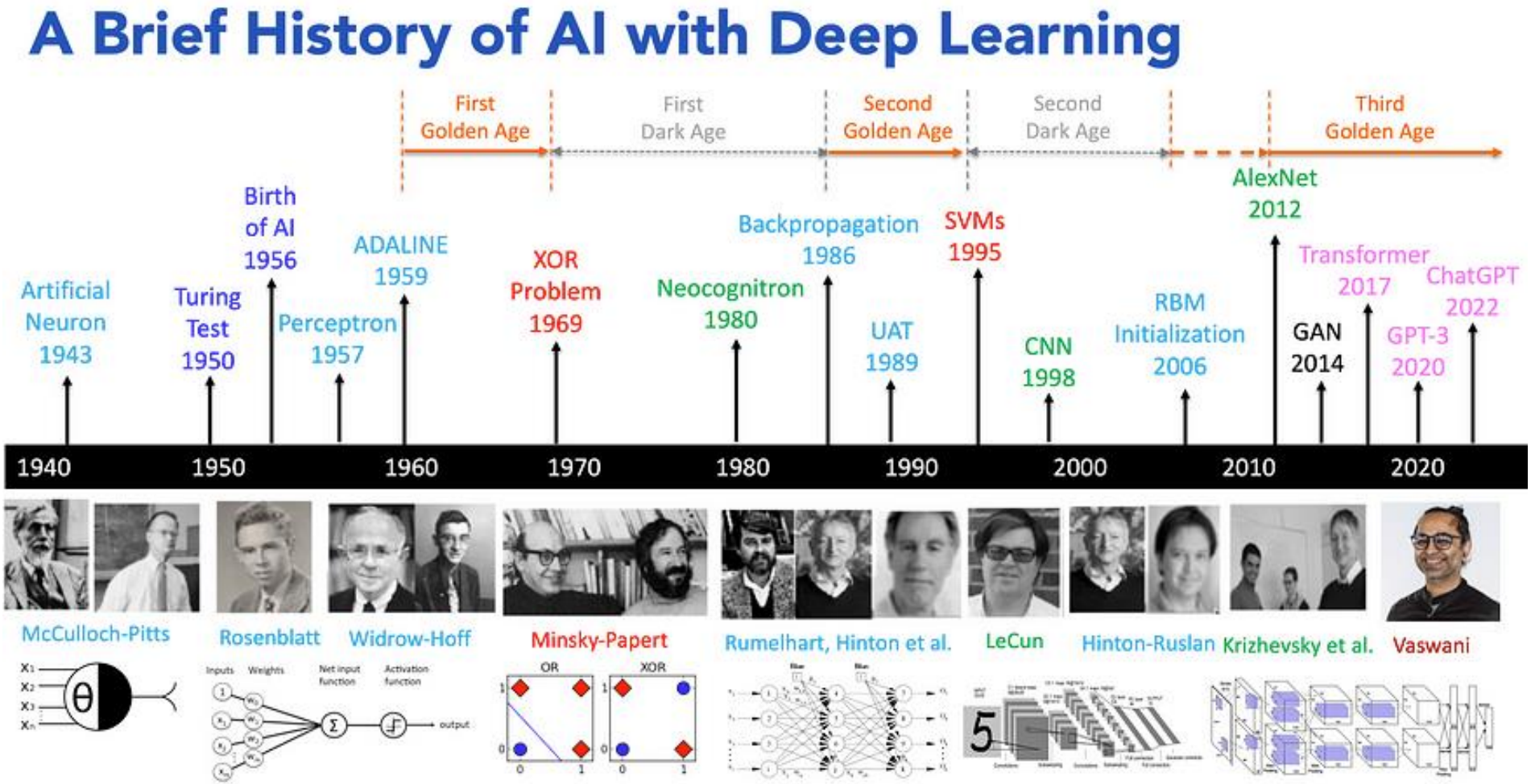
Kang Taewook
laputa99999@gmail.com
https://daddynkidsmakers.blogspot.com

2025/7

# AI 서비스란 무엇인가?

- AI vs 일반 소프트웨어 구조 비교
- AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

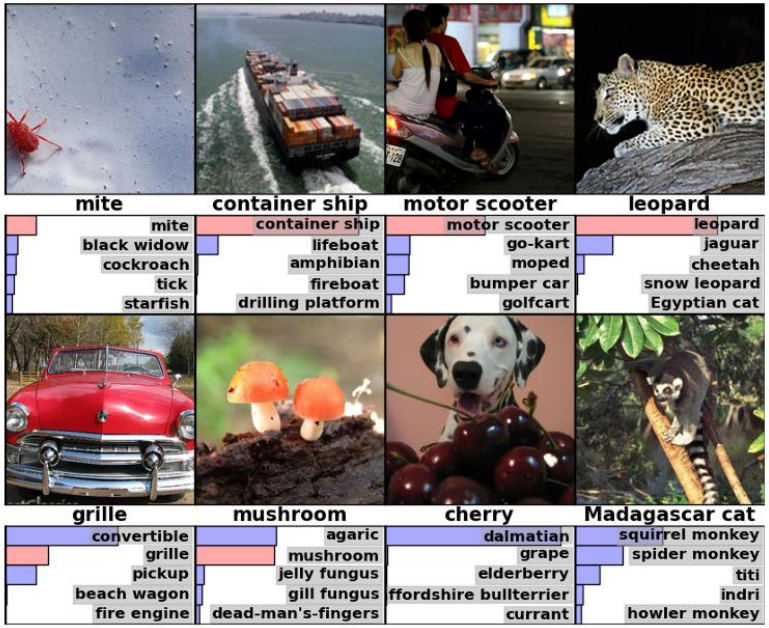
# AI 서비스 AI vs 일반 소프트웨어 구조




A Brief History of AI with Deep Learning | by LM Po | Medium

# AI 서비스 AI vs 일반 소프트웨어 구조

Convolutional neural networks running on GPUs (2012) Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton, Advances in Neural Information Processing Systems 2012




Deep CNN, ReLU, Dropout

| Model | Top-1 (val) | Top-5 (val) | Top-5 (test) |
|---|---|---|---|
| *SIFT + FVs [7]* | — | — | *26.2%* |
| 1 CNN | 40.7% | 18.2% | — |
| 5 CNNs | 38.1% | 16.4% | **16.4%** |
| 1 CNN* | 39.0% | 16.6% | — |
| 7 CNNs* | 36.7% | 15.4% | **15.3%** |

ImageNet Classification with Deep Convolutional Neural Networks

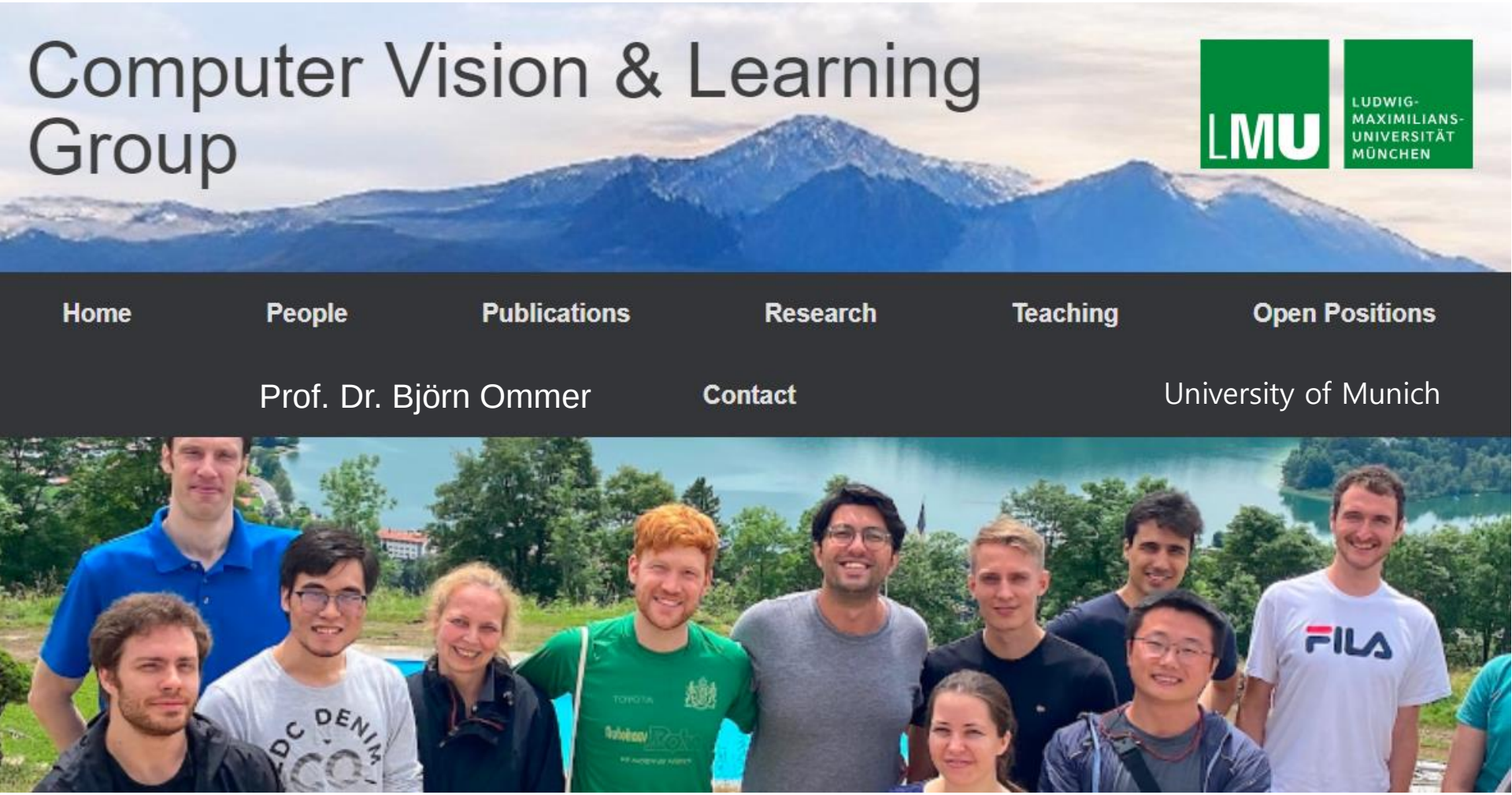
Google images

2013. Google acquired DNN

Facebook AI Research

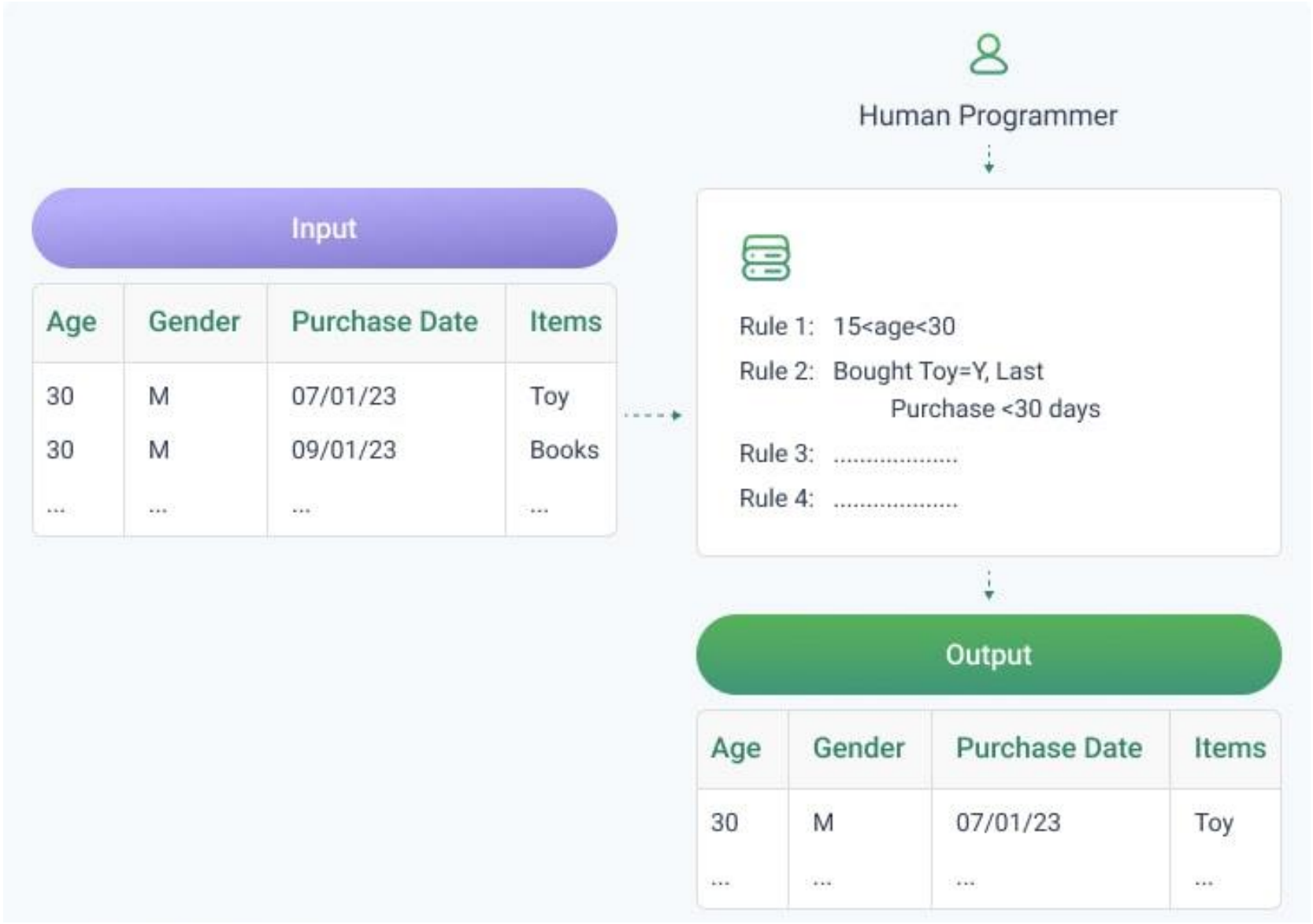
Yann LeCun.
Chief AI Scientist for Facebook AI Research (FAIR)

# AI 서비스 AI vs 일반 소프트웨어 구조



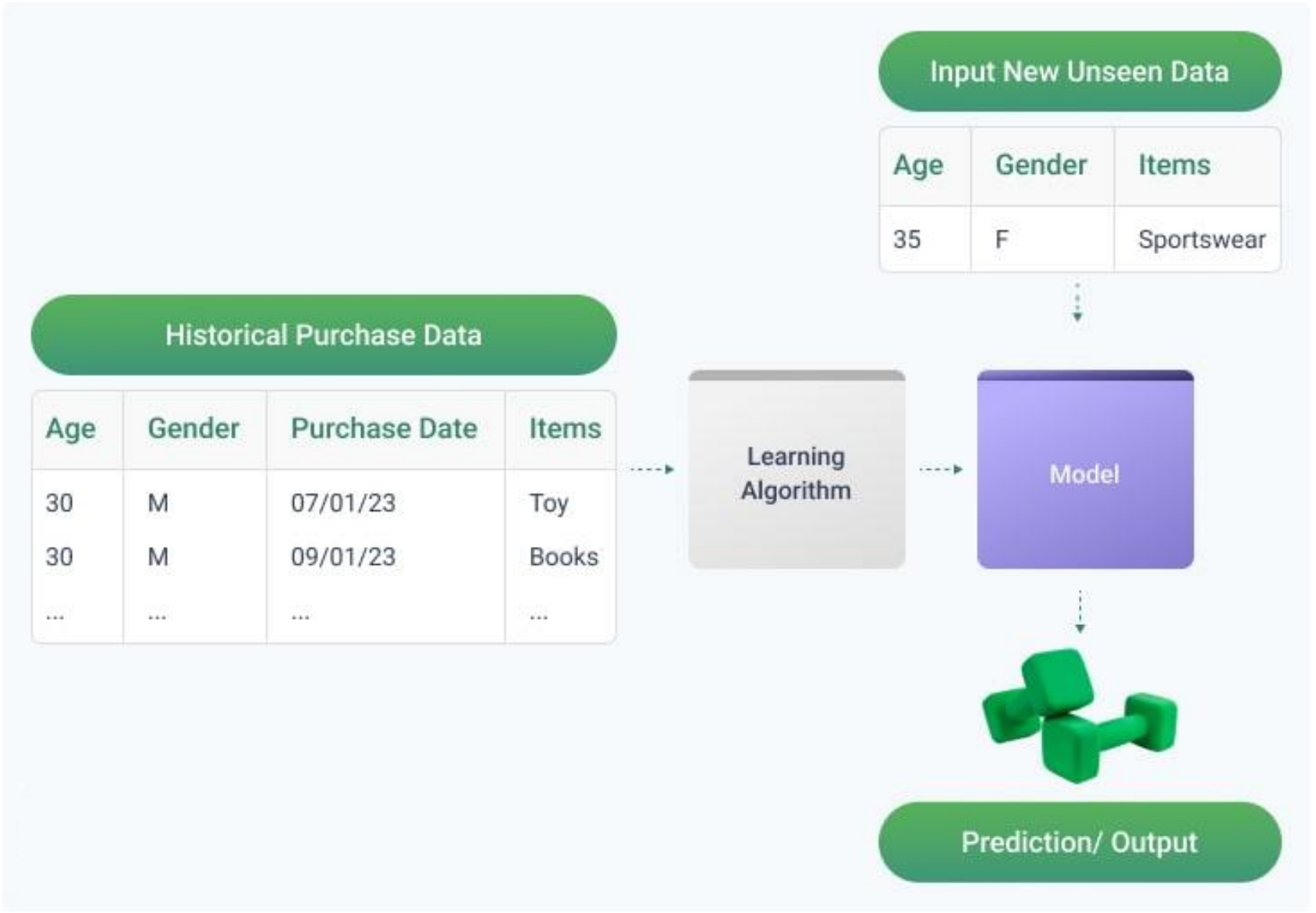

Stable Diffusion Developer [Computer Vision & Learning Group (ommer-lab.com)]

# AI 서비스 AI vs 일반 소프트웨어 구조




Artificial intelligence vs Traditional Software | Belitsoft

# AI 서비스 AI vs 일반 소프트웨어 구조




Artificial intelligence vs Traditional Software | Belitsoft

# AI 서비스 AI vs 일반 소프트웨어 구조

## Traditional Software

```
def is_even(number):
    return number % 2 == 0

# Test
numbers = [1, 2, 3, 4]
for num in numbers:
    result = "Even" if is_even(num) else "Odd"
    print(f"{num} is {result}")
```

## AI Software

```
from sklearn.linear_model import LogisticRegression
import numpy as np

# Prepare training data
X = np.array([[x] for x in range(1, 101)])  # Numbers 1 to 100
y = np.array([0 if x % 2 == 0 else 1 for x in range(1, 101)])  # 0: Even, 1: Odd

# Train model
model = LogisticRegression()
model.fit(X, y)

# Test
test_numbers = [1, 2, 3, 4]
for num in test_numbers:
    prediction = model.predict([[num]])[0]
    result = "Even" if prediction == 0 else "Odd"
    print(f"{num} is {result}")
```

AI Software Development Vs Traditional Software Development : PART1 | by Dr. Sathiskumar Jothi | Medium

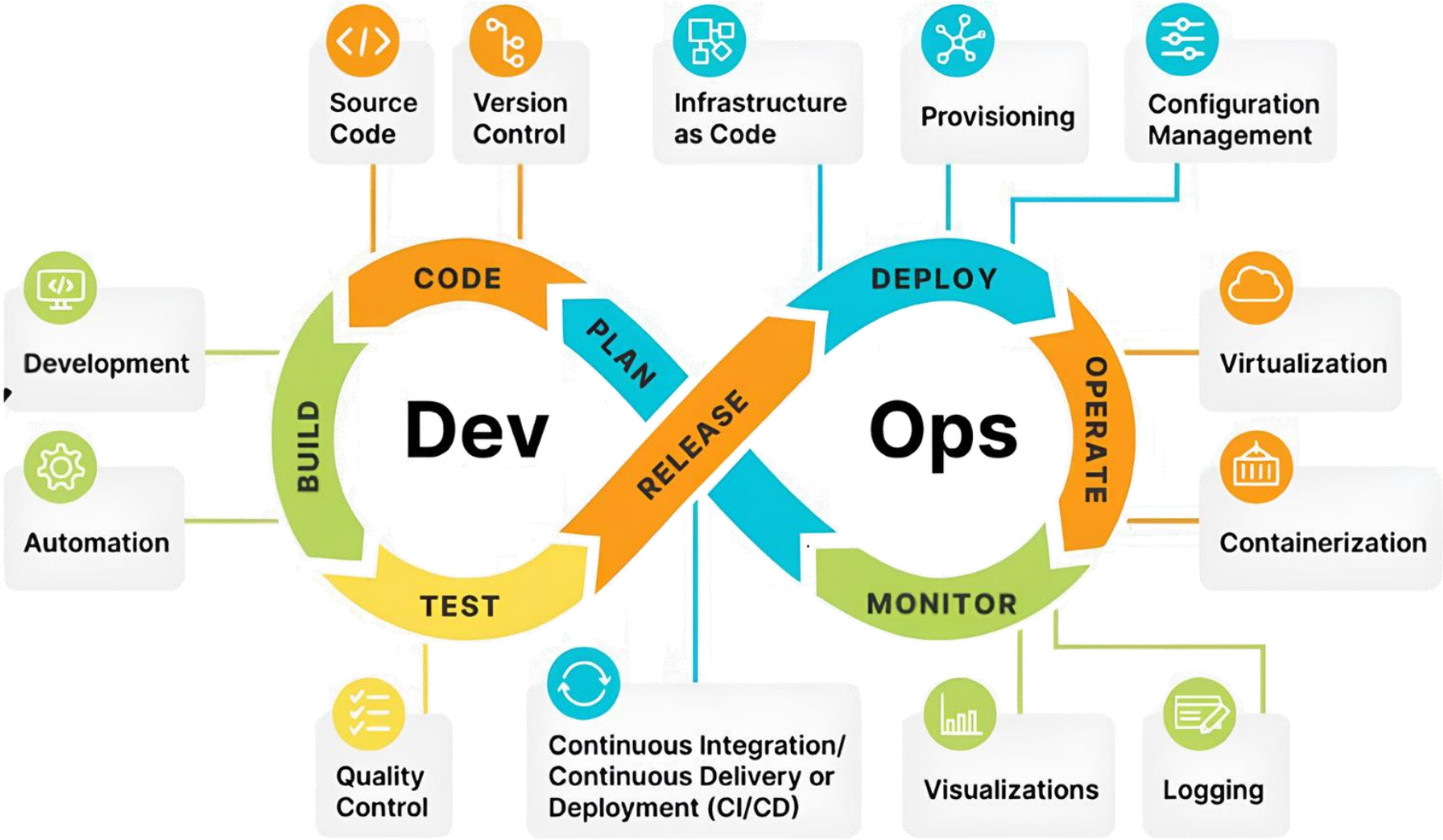
# AI 서비스 AI vs 일반 소프트웨어 구조

| 구분 | 일반 소프트웨어 (전통적 방식) | AI 서비스 |
|---|---|---|
| 핵심 로직 | 개발자가 직접 코딩한 명시적 규칙과 알고리즘 | 데이터로부터 학습된 패턴 (모델) |
| 동작 방식 | **결정론적 (Deterministic)**<br>동일한 입력에 대해 항상 동일한 결과를 출력 | **확률적 (Probabilistic) & 적응적 (Adaptive)**<br>학습 데이터와 모델 상태에 따라 결과가 달라질 수 있으며, 새로운 데이터로 지속적 성능 개선 가능 |
| 개발 과정 | 요구사항 분석 → 설계 → 코딩(구현) → 테스트 | 데이터 수집/가공 → 모델 학습 및 평가 → 서빙 → 모니터링 및 재학습 |
| 데이터 역할 | 시스템이 처리해야 할 입력값 | 모델을 학습시키고 검증하는 핵심 재료 |
| 유지보수 | 버그 수정, 기능 추가 등 코드 업데이트 | 성능 저하(Drift) 모니터링, 모델 재학습, 데이터 편향(Bias) 관리 |
| 장점 | - 예측 가능성 및 신뢰성이 높음<br>- 로직이 명시적이므로 디버깅 및 검증이 용이<br>- 요구사항이 명확하면 개발 기간과 비용 예측 용이 | - 복잡하고 확률적 비정형적인 문제 해결<br>- 데이터에 스스로 적응/학습/성능 향상<br>- 인간 직관이 필요한 패턴 인식. 개인화 강점 |
| 단점 | - 예상치 못한 데이터나 변화에 대응 난해<br>- 규칙을 정의할 수 없는 문제(예. 이미지 인식)는 해결이 불가능<br>- 로직이 복잡해질수록 개발 유지보수가 기하급수적으로 어려워짐 | - '블랙박스' 문제: 왜 그런 결정을 내렸는지 설명하기 어려움<br>- 데이터 의존성: 대량의 고품질 데이터가 필수적이며, 데이터 편향에 취약<br>- 학습 및 운영에 높은 컴퓨팅 비용과 전문 인력이 필요<br>- 환각현상. 100% 정확성을 보장할 수 없음 |

# AI 서비스

AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

## DevOps



# AI 서비스

AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

## DevOps

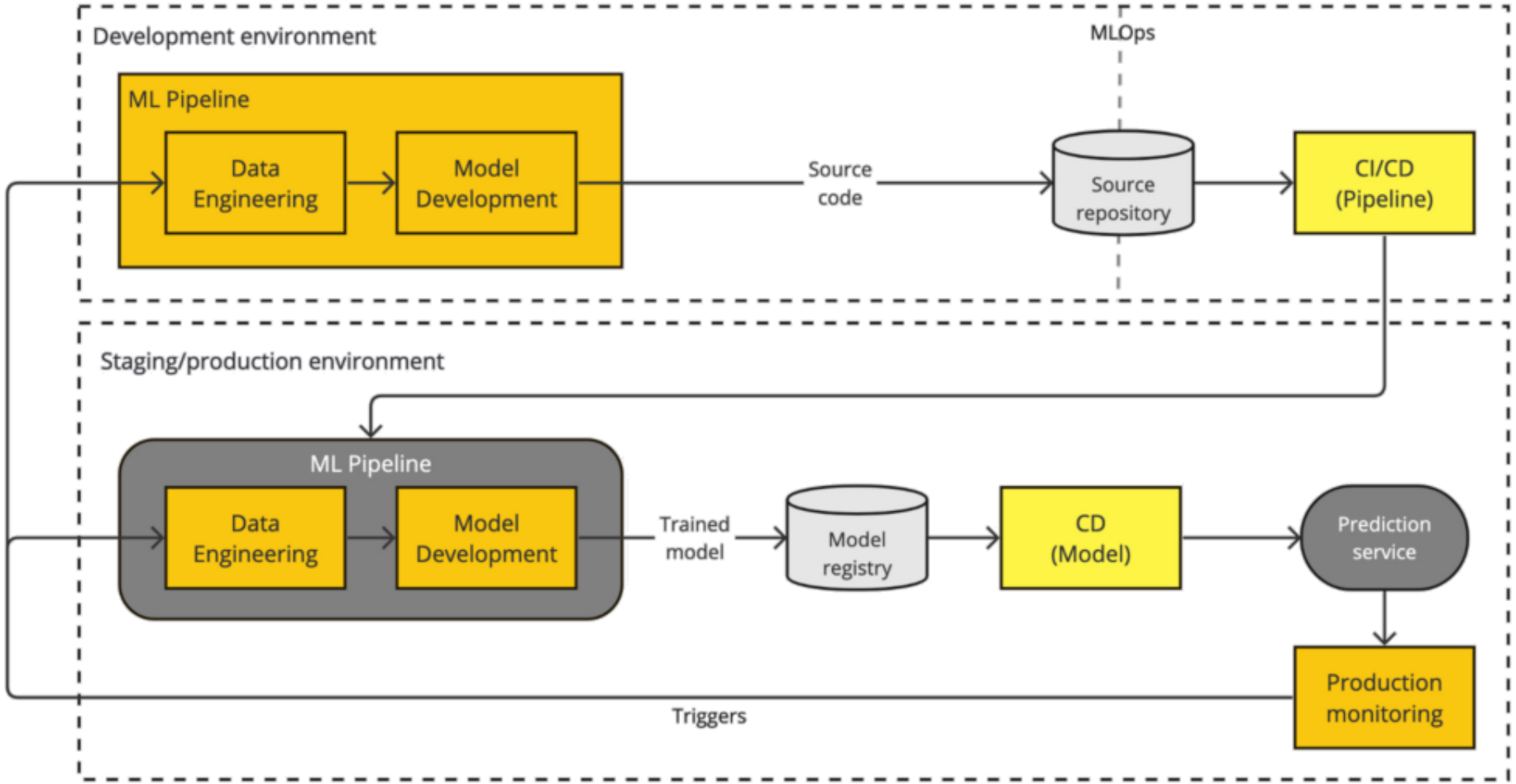
| 단계 | 구분 | 핵심 활동 | 상세 설명 |
|---|---|---|---|
| Plan | 개발 | 요구사항 정의, 기능 명세화, 작업 계획 및 추적 등의 활동을 수행함. | 프로젝트의 목표와 요구사항을 정의하고 개발 우선순위를 결정하는 초기 단계임. |
| Code | 개발 | 소스 코드 작성과 Git 등을 이용한 버전 관리를 통해 협업을 진행함. | 계획에 따라 실제 코드를 작성하고, 변경 이력을 관리하며 협업하는 단계임. |
| Build | 개발 | 개발 환경 통합과 빌드 자동화를 통해 소스 코드를 실행 파일로 변환함. | 작성된 소스 코드를 컴파일하고 종속성을 관리하여 실행 가능한 소프트웨어로 자동 변환하는 단계임. |
| Test | 개발 | 품질 관리와 단위 및 통합 테스트 자동화를 통해 소프트웨어의 안정성을 검증함. | 빌드된 소프트웨어에 버그가 없는지, 요구사항을 충족하는지 자동으로 검증하여 품질을 보장하는 단계임. |
| Release | 운영 | 릴리스 노트 작성, 버전 관리, 배포 준비 등의 활동을 포함함. | 테스트를 통과한 버전을 실제 운영 환경에 배포할 수 있도록 준비하고 최종 승인하는 단계임. |
| Deploy | 운영 | 프로비저닝(Provisioning), 코드로 인프라 관리(IaC), 구성 관리(Configuration Management)를 통해 배포 환경을 구축하고 자동화함. | 프로비저닝은 사용자의 요구에 맞게 IT 인프라(서버, 네트워크, 스토리지 등)를 생성하고 설정하여 사용할 수 있도록 준비하는 과정. 준비된 애플리케이션을 프로덕션 서버에 배포, 최종 사용자가 사용 |
| Operate | 운영 | 가상화(Virtualization) 및 컨테이너화(Containerization) 기술을 활용하여 인프라를 운영함. | 배포된 서비스가 안정적으로 구동되도록 서버, 네트워크 등 인프라를 관리하고 지원하는 단계임. |
| Monitor | 운영 | 성능 지표 시각화(Visualizations)와 시스템 이벤트 로깅(Logging)을 수행함. | 서비스의 성능과 상태를 지속적으로 관찰하고, 발생한 문제나 개선점을 다음 계획 단계에 피드백하는 단계임. |
| CI / CD | 전체 | 지속적 통합(CI)과 지속적 제공 및 배포(CD)를 구현하는 것임. | DevOps의 핵심 엔진으로서, 코드 통합부터 배포까지 전 과정을 자동화하여 전체 사이클을 원활하게 연결하는 개념임. |

# AI 서비스

## AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

MLOps는 AI 솔루션 개발 생산성을 촉진하고 데이터 품질을 유지하는 데 효과적인 접근 방식.

- 머신러닝 엔지니어가 모델 개발 및 배포 속도를 높일 수 있도록 지원하는 인프라.
- ML 모델을 지속적으로 모니터링하고 검증하는 과정이 필수적
- 궁극적으로 지속적인 통합, 배포, 학습(CI/CD/CT)의 순환 주기를 완성하는 핵심.




What is MLOps? Machine learning operations explained - Weights & Biases

# AI 서비스

AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

## MLOps

| 단계 | 구분 | 핵심 활동 (Key Activities) | 상세 설명 |
|---|---|---|---|
| Problem Framing | 개발 | 비즈니스 목표 이해, 성공 지표 정의, 데이터 가용성 평가 등을 수행함. | 비즈니스 문제를 머신러닝 문제로 변환하고, 프로젝트의 성공 기준과 방향을 설정하는 초기 기획 단계. |
| Data Engineering | 개발 | 데이터 수집, 정제, 레이블링, 특징 공학(Feature Engineering), 데이터 버전 관리 등을 진행함. | 모델 학습에 필요한 데이터를 모으고, 정제하여 품질을 높이며, 모델이 이해할 수 있는 특징(Feature)으로 가공. |
| Model Engineering | 개발 | 알고리즘 선택, 모델 코드 작성, 하이퍼파라미터 튜닝, 모델 성능 검증, 실험 관리 등을 수행함. | 준비된 데이터로 모델을 학습시키고, 다양한 지표를 통해 성능을 객관적으로 평가하여 최적의 모델을 찾는 핵심 단계. |
| Model Deployment | 운영 | 학습된 모델을 패키징하고, 서빙 인프라(API 서버, 클라우드 환경 등)에 배포할 수 있도록 준비함. | 검증이 완료된 모델을 실제 사용자가 접근할 수 있는 운영 환경으로 이전. |
| Serving & Inference | 운영 | 배포된 모델을 통해 실시간 또는 배치(Batch) 형태로 예측 결과를 제공하고, API를 통해 다른 서비스와 연동함. | 실제 사용자 데이터에 대해 모델이 예측을 수행하며, AI 서비스의 핵심 기능을 제공. |
| Model Monitoring | 운영 | 모델 성능 저하(Drift), 데이터 편향성(Bias), 시스템 자원 사용량 등을 지속적으로 추적하고 기록함. | 운영 중인 모델이 예측 성능을 일관되게 유지하는지, 예상치 못한 오류는 없는지 실시간으로 감시. |
| Retraining | 전체 | 모니터링 결과를 바탕으로 새로운 데이터로 모델을 다시 학습시키고, 필요시 새로운 파이프라인을 가동함. | 시간이 지나면서 변화하는 데이터 패턴에 모델이 적응하도록 하여, 서비스의 전반적인 성능과 정확성을 지속적으로 유지하고 개선하는 과정. |

# AI 서비스

AI 서비스 개발 흐름: 데이터 → 모델 → 서비스화

## MLOps

| 단계 (Phase) | 구분 (Category) | 대표 솔루션 |
|---|---|---|
| Problem Framing | 개발 | 자체 분석 도구, 스프레드시트 |
| Data Engineering | 개발 | PyTorch, Kafka, Spark, Pandas, Scikit-learn, NumPy |
| Model Engineering | 개발 | TensorFlow, PyTorch, Keras, Scikit-learn, MLflow, Weights & Bias |
| Model Deployment | 운영 | ONNX((Open Neural Network Exchange), Docker, Kubernetes, TensorFlow Serving, TorchServe, FastAPI, Flask |
| Serving & Inference | 운영 | TensorFlow Serving, TorchServe, NVIDIA Inference Server, AWS SageMaker Inference, Google AI Platform Prediction, Azure Machine Learning Endpoints |
| Model Monitoring | 운영 | Prometheus, Grafana |
| Retraining | 전체 | Kubeflow Pipelines, MLflow Projects, AWS SageMaker Pipelines, Google Cloud AI Pipelines, Azure Machine Learning Pipelines |

# 머신러닝 기초 개념

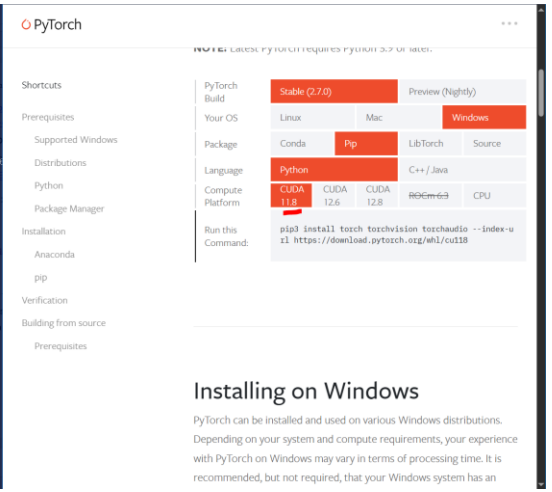
- AI학습 및 개발 환경
- 분류와 회귀의 차이
- Overfitting, 정규화, 검증셋 개념
- 주요 알고리즘(Decision Tree, KNN 등) 개요
- Scikit-learn을 활용한 파이프라인 예시 소개

# 머신러닝 기초 AI학습 및 개발 환경

**Python (라이브러리 호환성 고려해 3.11버전 이상 권장)**
Python 다운로드
Mac 사용자: Mac에 Python 설치 가이드
설치 후 터미널에서 python --version 로 실행이 제대로 되는 지 확인해 볼 것

**NVIDIA 드라이버 (NVIDIA GPU 사용 시)**
NVIDIA 드라이버 다운로드
설치 후 터미널에서 nvidia-smi로 확인

**CUDA Toolkit (NVIDIA GPU 사용 시)**
CUDA Toolkit 다운로드
설치 시 GPU 및 드라이버 호환성 확인
환경 변수에 CUDA 경로 추가




**Github 도구 설치**
다음 링크에서 도구 설치함.
https://docs.github.com/ko/desktop/installing-and-authenticating-to-github-desktop/installing-github-desktop

**Anaconda (버전 24.0 이상 권장)**
다음 링크에서 도구 설치함.
Anaconda 다운로드

**PyTorch 라이브러리 설치**
다음 링크 방문 후, CPU 버전 혹은 GPU 드라이버 버전에 맞게 터미널에서 설치함
https://pytorch.org/get-started/locally/
CPU의 경우, 다음과 같이 터미널에서 명령을 입력해 설치함.

# 머신러닝 기초 AI학습 및 개발 환경

**도커(Docker) 설치(옵션)**
가상이미지 기반 작업을 위해서는 설치가 필요합니다.
https://www.docker.com/get-started/ 방문 설치

**올라마(Ollama) 설치**
로컬 LLM AI 도구를 사용하려면 Ollama 설치가 필요합니다.
https://www.ollama.com/ 에 방문해 설치

**코드 편집기 및 IDE 설치**
https://www.sublimetext.com/ 설치
https://code.visualstudio.com/download 설치. 상세한 설치법은

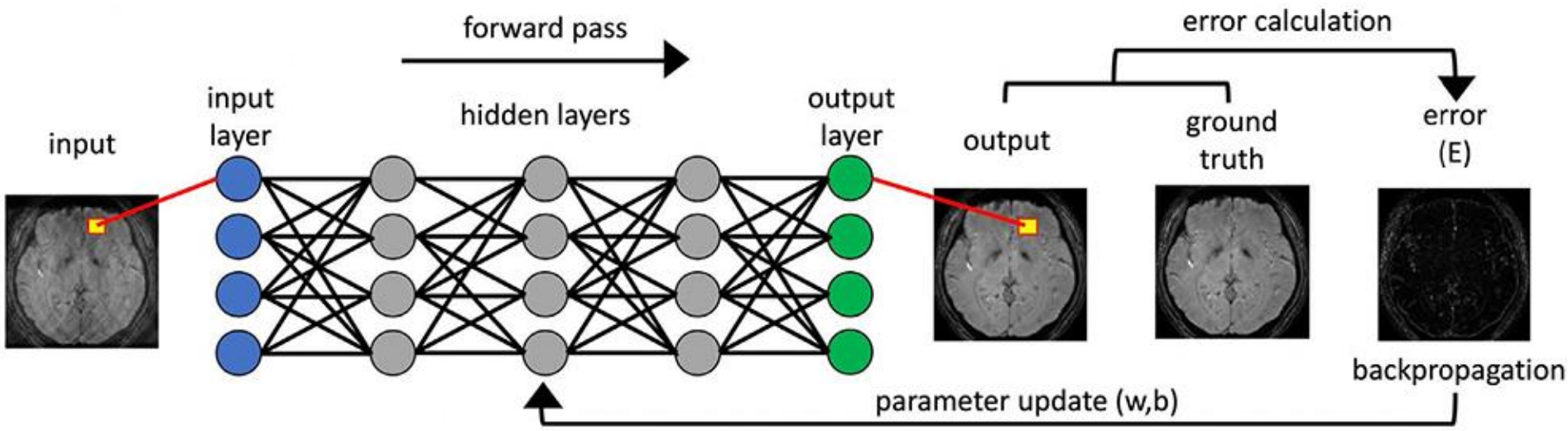
**클로드 Desktop 설치**
https://claude.ai/download 설치
https://docs.github.com/ko/desktop/installing-and-authenticating-to-github-desktop/installing-github-desktop

# 머신러닝 기초 AI학습 및 개발 환경

- Colab Pro 가입(유료): https://colab.research.google.com/signup
- ChatGPT 가입(유료)
- ChatGPT API Pay as you go 가입(유료. 한계 8달러 설정): https://platform.openai.com/settings/organization/billing/overview
- 클로드(Claude) 가입(무료): https://claude.ai/
- Github 가입(무료): https://github.com/
- Github Copilot 가입(유료. 10달러/월): https://github.com/features/copilot/plans
- Huggingface 가입(무료): https://huggingface.co
- Huggingface API Token 생성(무료): https://huggingface.co/settings/tokens
- Stable diffusion - Kling 가입(유료. $6.99 per month): https://app.klingai.com/global/membership/membership-plan
- Figma 가입(옵션): https://www.figma.com
- gemini-api: https://aistudio.google.com/app/apikey gemini사용위한 google ai 키
- serp-api: https://serpapi.com/manage-api-key 구글 검색
- travily: https://app.tavily.com/home 웹 검색
- wandb: https://docs.wandb.ai/quickstart/ 모델학습 및 파인튜닝 모니터링 로그 도구
- langsmith: https://docs.smith.langchain.com/administration/how_to_guides/organization_management/create_account_api_key (https://smith.langchain.com/o/2c462eb1-5b83-41c8-96c5-e008809d5655/settings) langchain 로그 & 디버그 도구
- visily ai: https://app.visily.ai 기획 AI 도구

# 머신러닝 기초 분류와 회귀의 차이

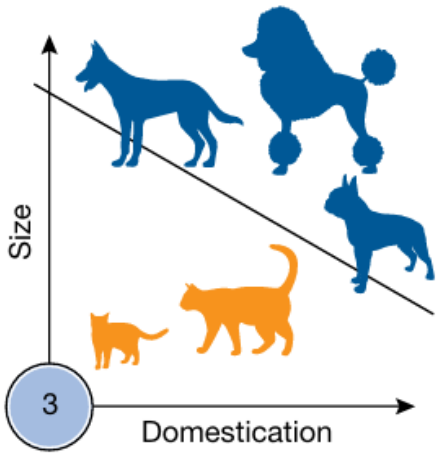
## 개념



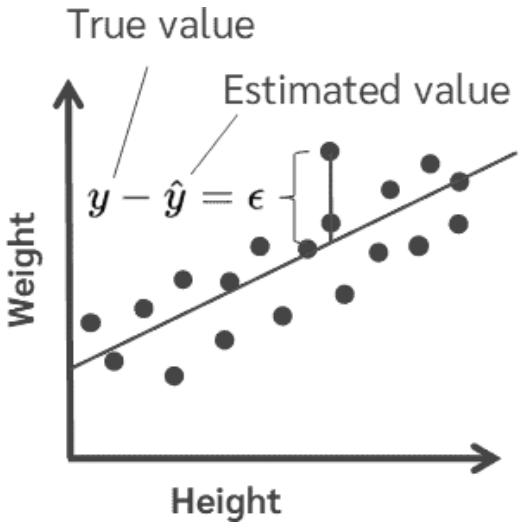

Korean Society of Magnetic Resonance in Medicine, 2019, Deep Learning in MR Image Processing

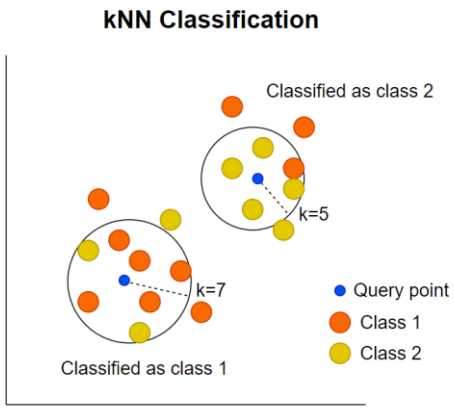
# 머신러닝 기초 분류와 회귀의 차이

## Classification




## Regression analysis



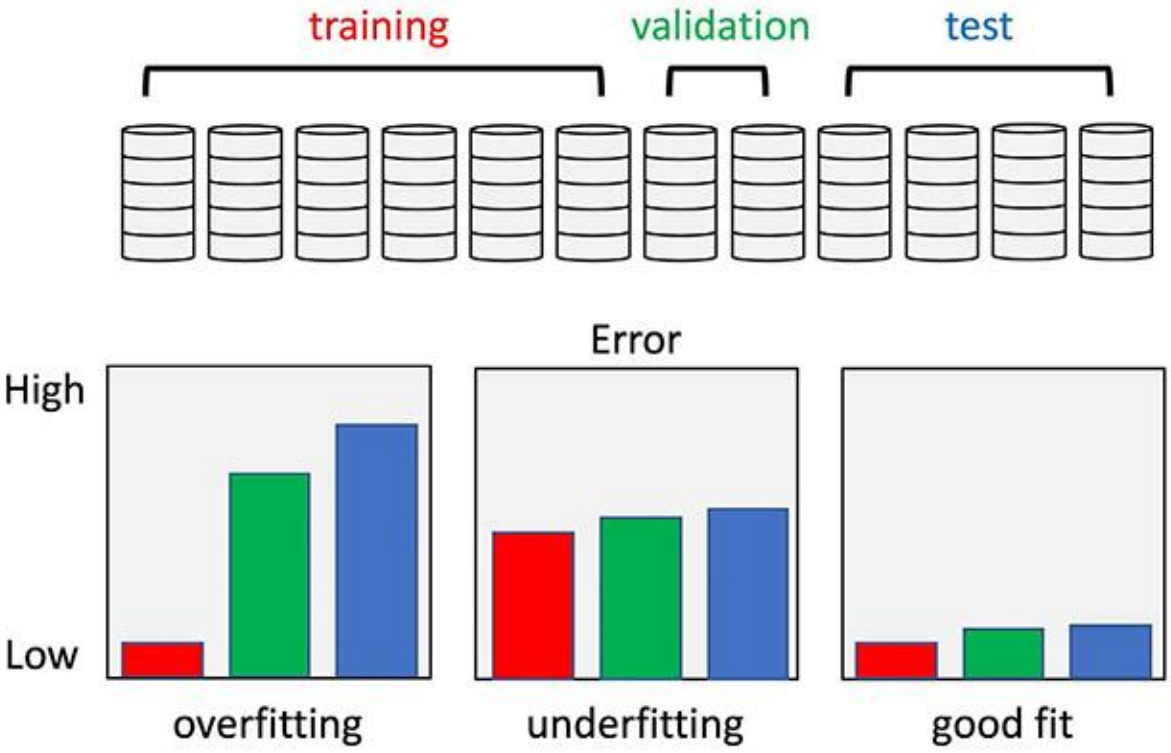





1. Amit Gupta, AIChE, 2018, Deep Learning
2. Linear Regression: A Complete Guide to Modeling Relationships Between Variables
3. How to Implement k-Nearest Neighbors (kNN) in Python - stataiml

# 머신러닝 기초 Overfitting, 정규화, 검증셋 개념

## Overfitting과 검증셋




**Training set**
to train
the model

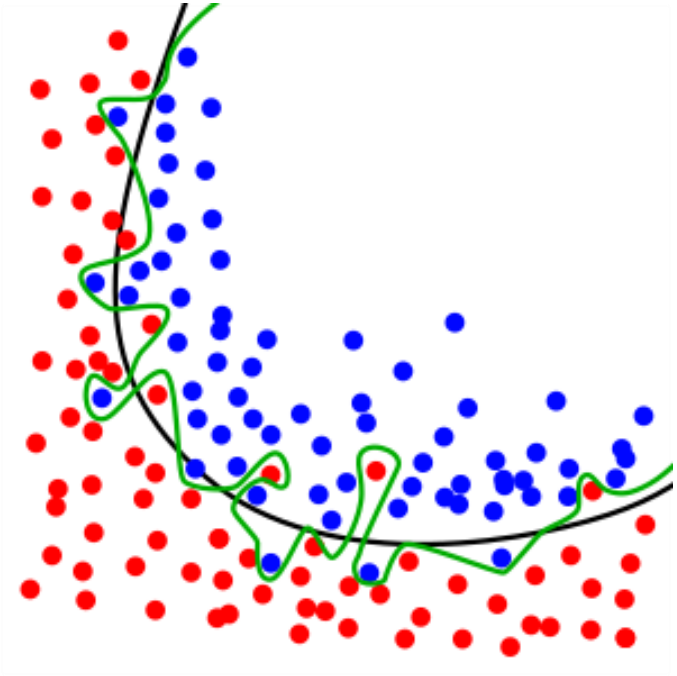
**Validation set**
to monitor and tune
the model

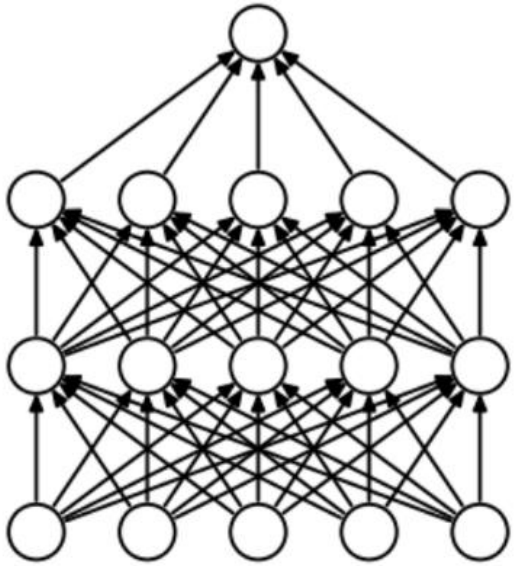
**Test set**
to evaluate
the accuracy of the model

Korean Society of Magnetic Resonance in Medicine, 2019, Deep Learning in MR Image Processing

# 머신러닝 기초 Overfitting, 정규화, 검증셋 개념

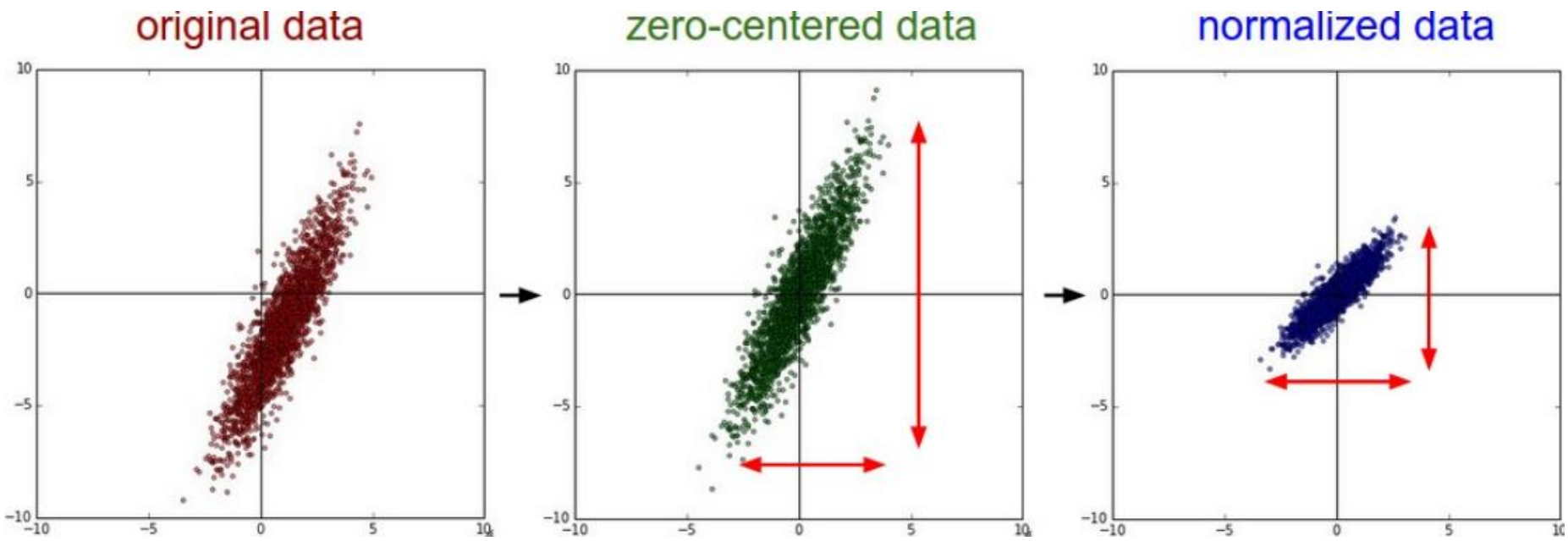
## Overfitting





# 머신러닝 기초 Overfitting, 정규화, 검증셋 개념

Normalization



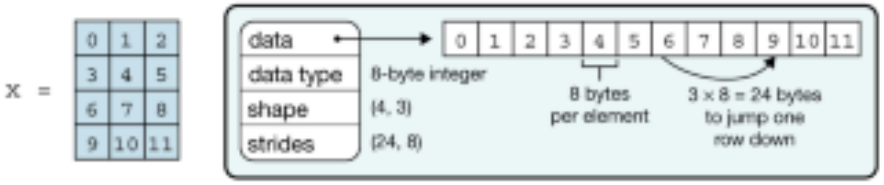

Practical Aspects of Deep Learning - 2 | Ashish Jaiswal

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시

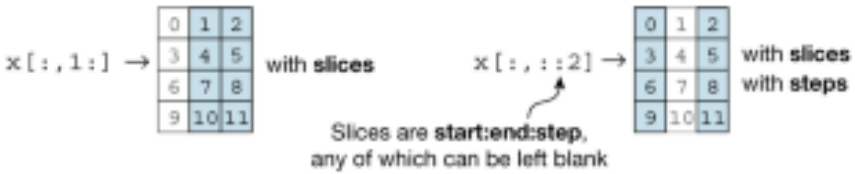
**NumPy**

https://numpy.org/

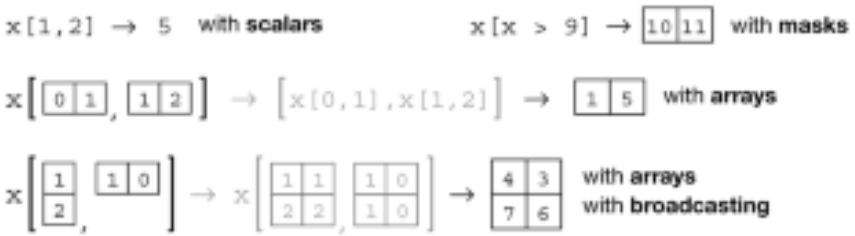
**a** Data structure



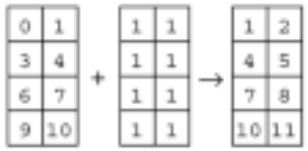

**b** Indexing (view)



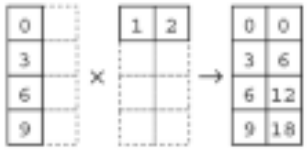

**c** Indexing (copy)



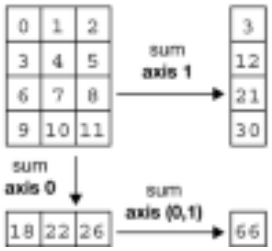

**d** Vectorization



**e** Broadcasting



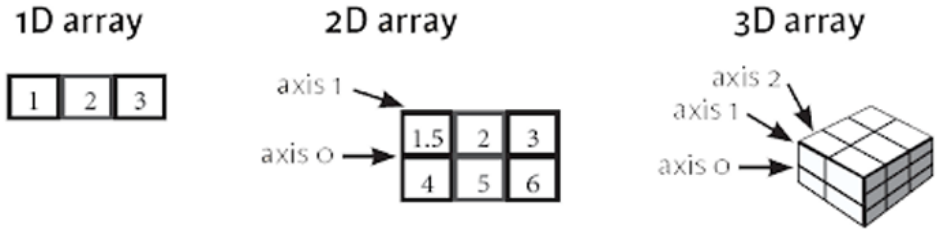
**f** Reduction




**g** Example

```
In [1]: import numpy as np

In [2]: x = np.arange(12)

In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[-4.5, -4.5, -4.5],
       [-1.5, -1.5, -1.5],
       [ 1.5,  1.5,  1.5],
       [ 4.5,  4.5,  4.5]])
```

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시




## > Subsetting, Slicing, Indexing

**Subsetting**

```
>>> a[2] #Select the element at the 2nd index
 3
>>> b[1,2] #Select the element at row 1 column 2 (equivalent to b[1][2])
 6.0
```

**Slicing**

```
>>> a[0:2] #Select items at index 0 and 1
 array([1, 2])
>>> b[0:2,1] #Select items at rows 0 and 1 in column 1
 array([ 2., 5.])
>>> b[:1] #Select all items at row 0 (equivalent to b[0:1, :])
 array([[1.5, 2., 3.]])
>>> c[1,...] #Same as [1,:,:]
 array([[[ 3., 2., 1.],
         [ 4., 5., 6.]]])
>>> a[ : :-1] #Reversed array a array([3, 2, 1])
```

**Boolean Indexing**

```
>>> a[a<2] #Select elements from a less than 2
 array([1])
```

**Fancy Indexing**

```
>>> b[[1, 0, 1, 0],[0, 1, 2, 0]] #Select elements (1,0),(0,1),(1,2) and (0,0)
 array([ 4. , 2. , 6. , 1.5])
>>> b[[1, 0, 1, 0]][:,[0,1,2,0]] #Select a subset of the matrix's rows and columns
 array([[ 4. ,5. , 6. , 4. ],
        [ 1.5, 2. , 3. , 1.5],
        [ 4. , 5. , 6. , 4. ],
        [ 1.5, 2. , 3. , 1.5]])
```

**Transposing Array**

```
>>> i = np.transpose(b) #Permute array dimensions
>>> i.T #Permute array dimensions
```

**Changing Array Shape**

```
>>> b.ravel() #Flatten the array
>>> g.reshape(3,-2) #Reshape, but don't change data
```

**Adding/Removing Elements**

```
>>> h.resize((2,6)) #Return a new array with shape (2,6)
>>> np.append(h,g) #Append items to an array
>>> np.insert(a, 1, 5) #Insert items in an array
>>> np.delete(a,[1]) #Delete items from an array
```

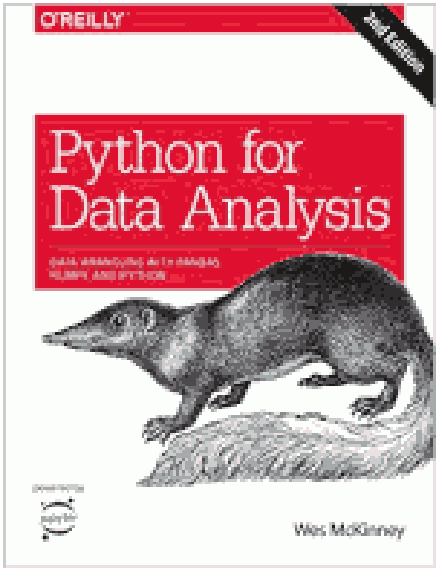
**Combining Arrays**

```
>>> np.concatenate((a,d),axis=0) #Concatenate arrays
 array([ 1, 2, 3, 10, 15, 20])
>>> np.vstack((a,b)) #Stack arrays vertically (row-wise)
 array([[ 1. , 2. , 3. ],
        [ 1.5, 2. , 3. ],
        [ 4. , 5. , 6. ]])
>>> np.r_[e,f] #Stack arrays vertically (row-wise)
>>> np.hstack((e,f)) #Stack arrays horizontally (column-wise)
 array([[ 7., 7., 1., 0.],
        [ 7., 7., 0., 1.]])
>>> np.column_stack((a,d)) #Create stacked column-wise arrays
 array([[ 1, 10],
        [ 2, 15],
        [ 3, 20]])
>>> np.c_[a,d] #Create stacked column-wise arrays
```

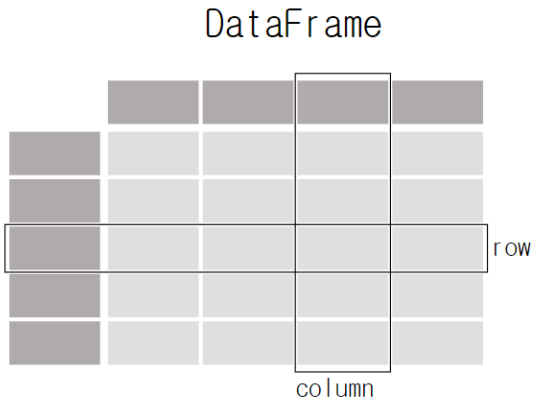
Link - https://media.datacamp.com/legacy/image/upload/v1676302459/Marketing/Blog/Numpy_Cheat_Sheet.pdf

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시

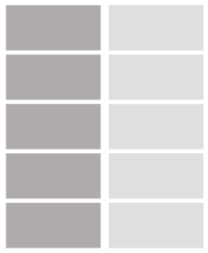
https://pandas.pydata.org/




NUMFOCUS
OPEN CODE = BETTER SCIENCE
VOLTRON DATA
TWO SIGMA
d-fine
Quansight Labs
NVIDIA
TIDELIFT
Chan Zuckerberg Initiative
bodo.ai




Each column in a `DataFrame` is a `Series`




# Creating DataFrames

| | a | b | c |
|---|---|---|---|
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |
| 3 | 6 | 9 | 12 |

```
df = pd.DataFrame(
        {"a" : [4, 5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

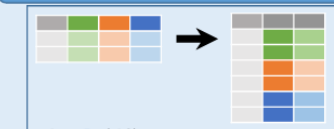
| | | a | b | c |
|---|---|---|---|---|
| N | v | | | |
| D | 1 | 4 | 7 | 10 |
| | 2 | 5 | 8 | 11 |
| e | 2 | 6 | 9 | 12 |

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

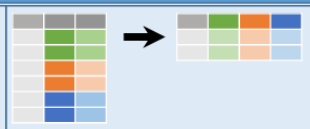
Create DataFrame with a MultiIndex

# it-learn을 활용한 파이프라인 예시

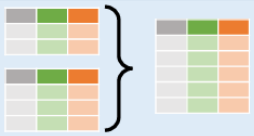
## Reshaping Data – Change layout, sorting, reindexing, renaming



`pd.melt(df)`
Gather columns into rows.



`df.pivot(columns='var', values='val')`
Spread rows into columns.



`pd.concat([df1,df2])`
Append rows of DataFrames

`pd.concat([df1,df2], axis=1)`
Append columns of DataFrames

`df.sort_values('mpg')`
Order rows by values of a column (low to high).

`df.sort_values('mpg', ascending=False)`
Order rows by values of a column (high to low).

`df.rename(columns = {'y':'year'})`
Rename the columns of a DataFrame

`df.sort_index()`
Sort the index of a DataFrame

`df.reset_index()`
Reset index of DataFrame to row numbers, moving index to columns.

`df.drop(columns=['Length', 'Height'])`
Drop columns from DataFrame

## Subset Variables - columns

`df[['width', 'length', 'species']]`
Select multiple columns with specific names.

`df['width']` *or* `df.width`
Select single column with specific name.

`df.filter(regex='regex')`
Select columns whose name matches regular expression *regex*.

## Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")',
         engine="python")
```

## Subsets - rows and columns

Use `df.loc[]` and `df.iloc[]` to select only rows, only columns or both.
Use `df.at[]` and `df.iat[]` to access a single value by row and column.
First index selects rows, second index columns.

`df.iloc[10:20]`
Select rows 10-20.

`df.iloc[:, [1, 2, 5]]`
Select columns in positions 1, 2 and 5 (first column is 0).

`df.loc[:, 'x2':'x4']`
Select all columns between x2 and x4 (inclusive).

`df.loc[df['a'] > 10, ['a', 'c']]`
Select rows meeting logical condition, and only the specific columns .

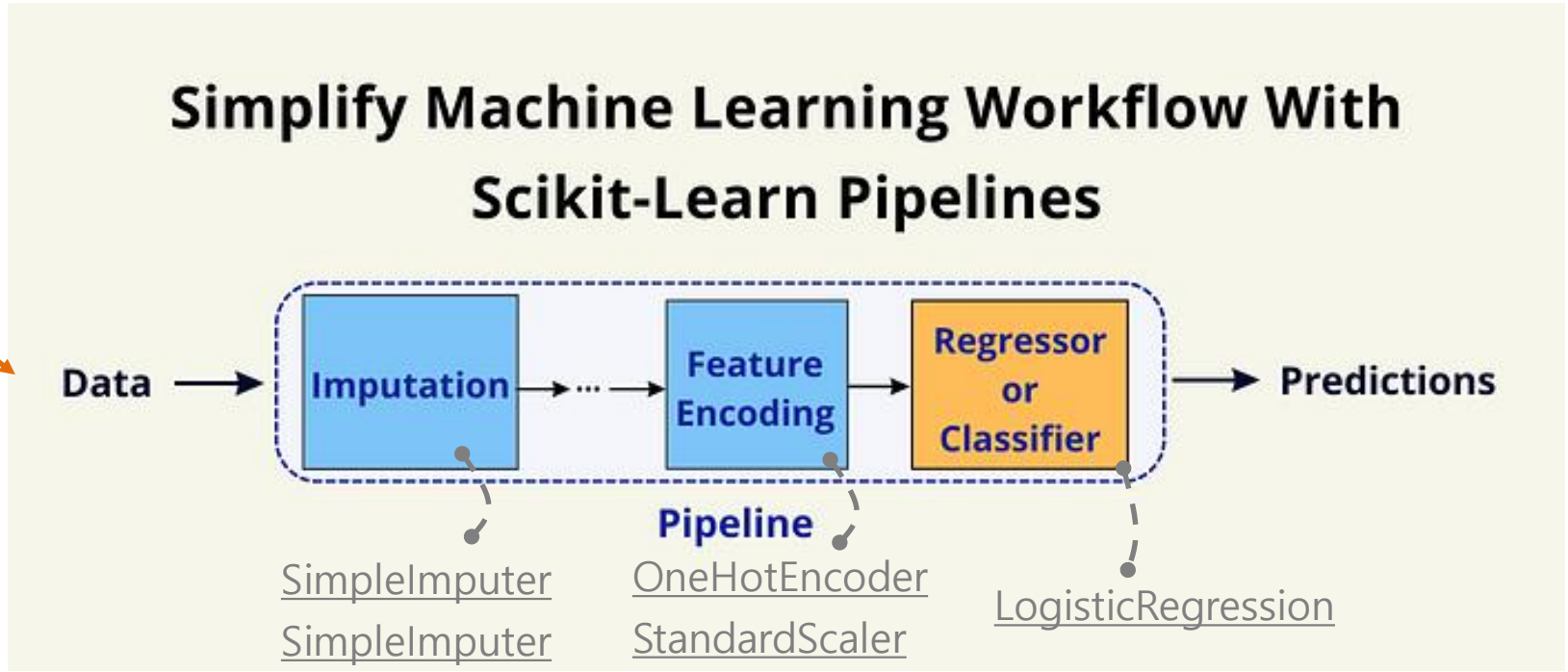
`df.iat[1, 2]` Access single value by index

`df.at[4, 'A']` Access single value by label

https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시

| | state | gender | age | weight | |
|---|---|---|---|---|---|
| 0 | CA | male | 34.0 | 122.0 | 0 |
| 1 | WA | female | 29.0 | 150.0 | 1 |
| 2 | CA | female | 22.0 | 130.0 | 0 |
| 3 | NaN | male | 44.0 | NaN | 1 |
| 4 | NV | NaN | 55.0 | 140.0 | 0 |
| 5 | WA | female | NaN | 175.0 | 1 |




Scikit-learn Pipelines Explained: Streamline and Optimize Your Machine Learning Processes | by Sahin Ahmed, Data Scientist | Medium

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시

```
import numpy as np, pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split

data = {
    "state": ["CA", "WA", "CA", np.nan, "NV", "WA"],
    "gender": ["male", "female", "female", "male", np.nan, "female"],
    "age": [34, 29, 22, 44, 55, np.nan],
    "weight": [122, 150, 130, np.nan, 140, 175],
    "target": [0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)
X = df.drop("target", axis=1)
print("Input DataFrame:")
print(X)

y = df["target"]
print("Target Series:")
print(y)
```

| | state | gender | age | weight | |
|---|---|---|---|---|---|
| 0 | CA | male | 34.0 | 122.0 | 0 |
| 1 | WA | female | 29.0 | 150.0 | 1 |
| 2 | CA | female | 22.0 | 130.0 | 0 |
| 3 | NaN | male | 44.0 | NaN | 1 |
| 4 | NV | NaN | 55.0 | 140.0 | 0 |
| 5 | WA | female | NaN | 175.0 | 1 |

Scikit-learn Pipelines Explained: Streamline and Optimize Your Machine Learning Processes | by Sahin Ahmed, Data Scientist | Medium

# 머신러닝 기초 Scikit-learn을 활용한 파이프라인 예시

```
categorical_preprocessor = Pipeline(
    steps=[ ("imputation_constant", SimpleImputer(fill_value="missing", strategy="constant")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")) ]
)

numeric_preprocessor = Pipeline(
    steps=[ ("imputation_mean", SimpleImputer(missing_values=np.nan, strategy="mean")),
        ("scaler", StandardScaler()) ]
)

preprocessor = ColumnTransformer(
    [
        ("categorical", categorical_preprocessor, ["state", "gender"]),
        ("numerical", numeric_preprocessor, ["age", "weight"]),
    ]
)

pipe = make_pipeline(preprocessor, LogisticRegression(max_iter=500))
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
pipe.fit(X_train, y_train)

print(f"Input test set: {X_test}")
predictions = pipe.predict(X_test)
print(f"Predictions on the test set: {predictions}")
```

CA: [1, 0, 0, 0]

WA: [0, 1, 0, 0]

NV: [0, 0, 1, 0]

missing: [0, 0, 0, 1]

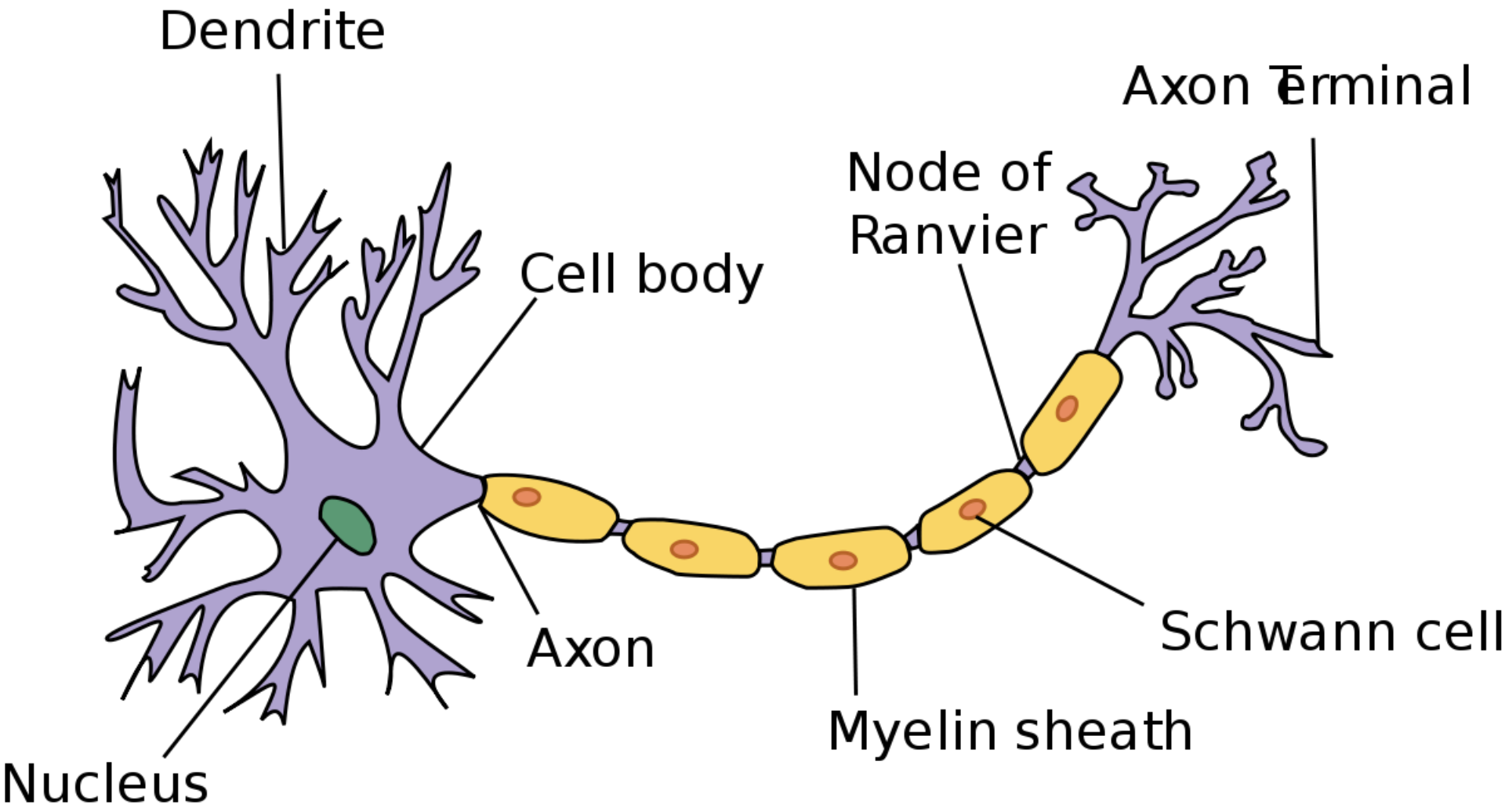
| | state | gender | age | weight | |
|---|---|---|---|---|---|
| 0 | CA | male | 34.0 | 122.0 | 0 |
| 1 | WA | female | 29.0 | 150.0 | 1 |
| 2 | CA | female | 22.0 | 130.0 | 0 |
| 3 | NaN | male | 44.0 | NaN | 1 |
| 4 | NV | NaN | 55.0 | 140.0 | 0 |
| 5 | WA | female | NaN | 175.0 | 1 |

```
  state  gender   age  weight
0    CA    male  34.0   122.0
1    WA  female  29.0   150.0
Predictions on the test set:
[0 1]
```

# 딥러닝의 구조적 이해

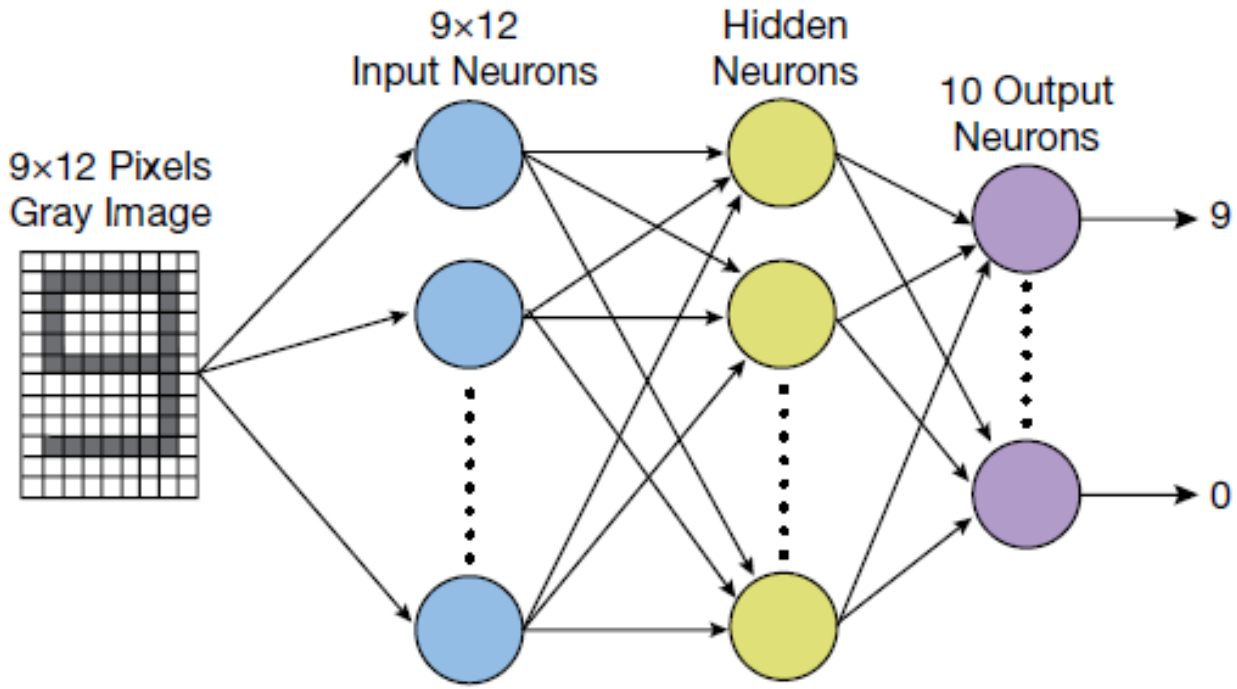
- 신경망 기본 구조 (Perceptron, MLP)
- Optimizer, Learning Rate, Batch Size 의미
- CNN/RNN/LSTM의 개념적 차이
- Tensorflow vs Pytorch 비교

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



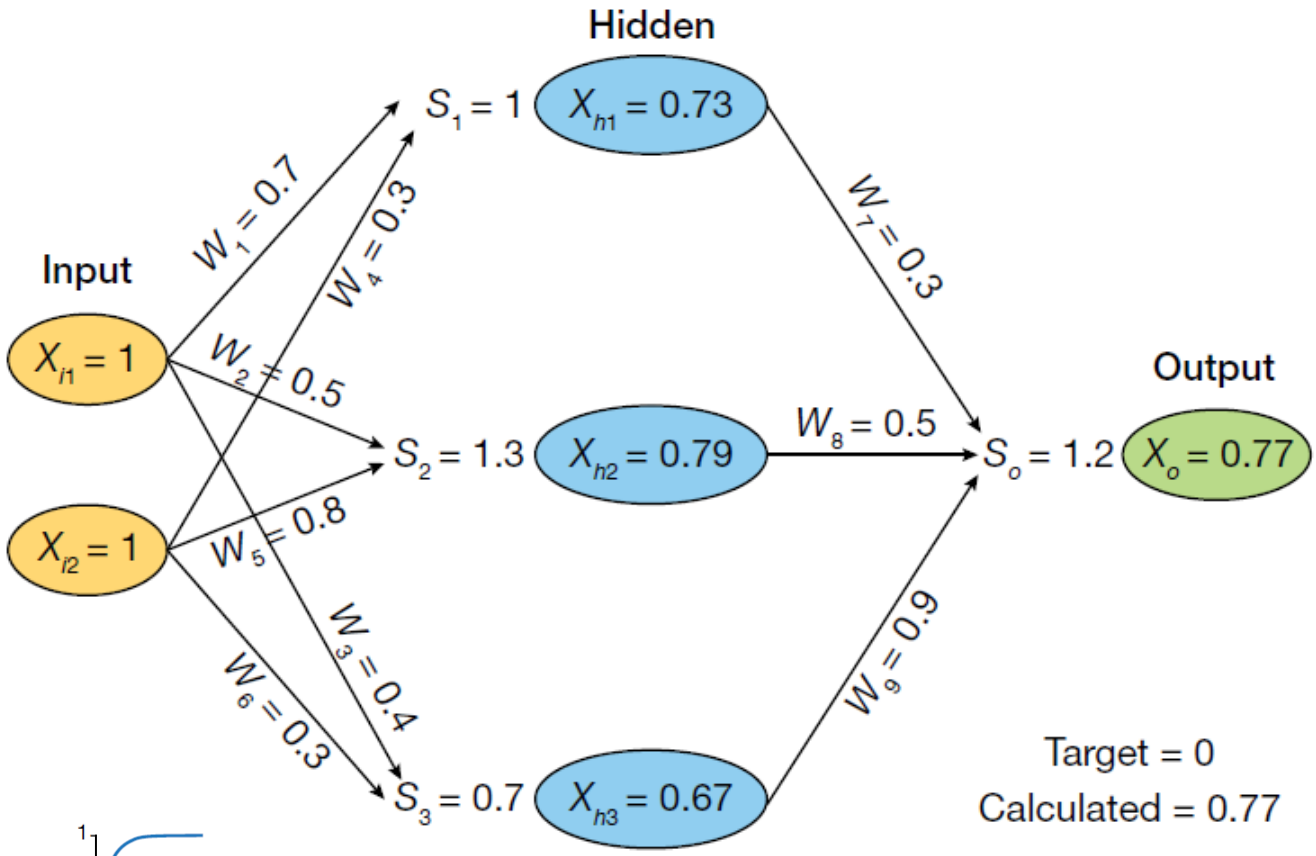

Deep Learning Neural Networks Explained in Plain English

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



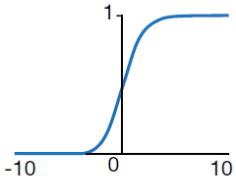

Amit Gupta, 2018, Intruduction to Deep Learning

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



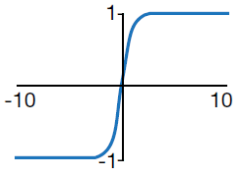

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



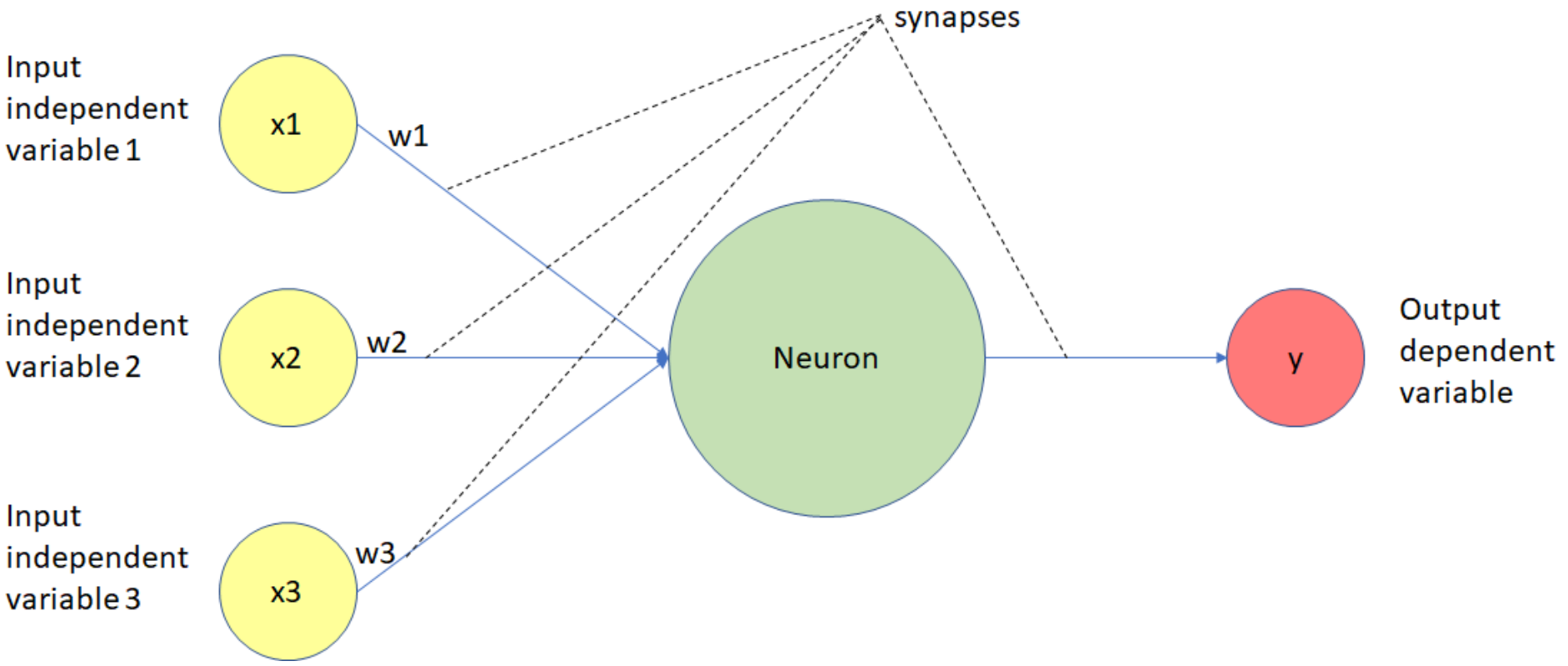

tanh

$$\tanh(x)$$




Amit Gupta, 2018, Intruduction to Deep Learning

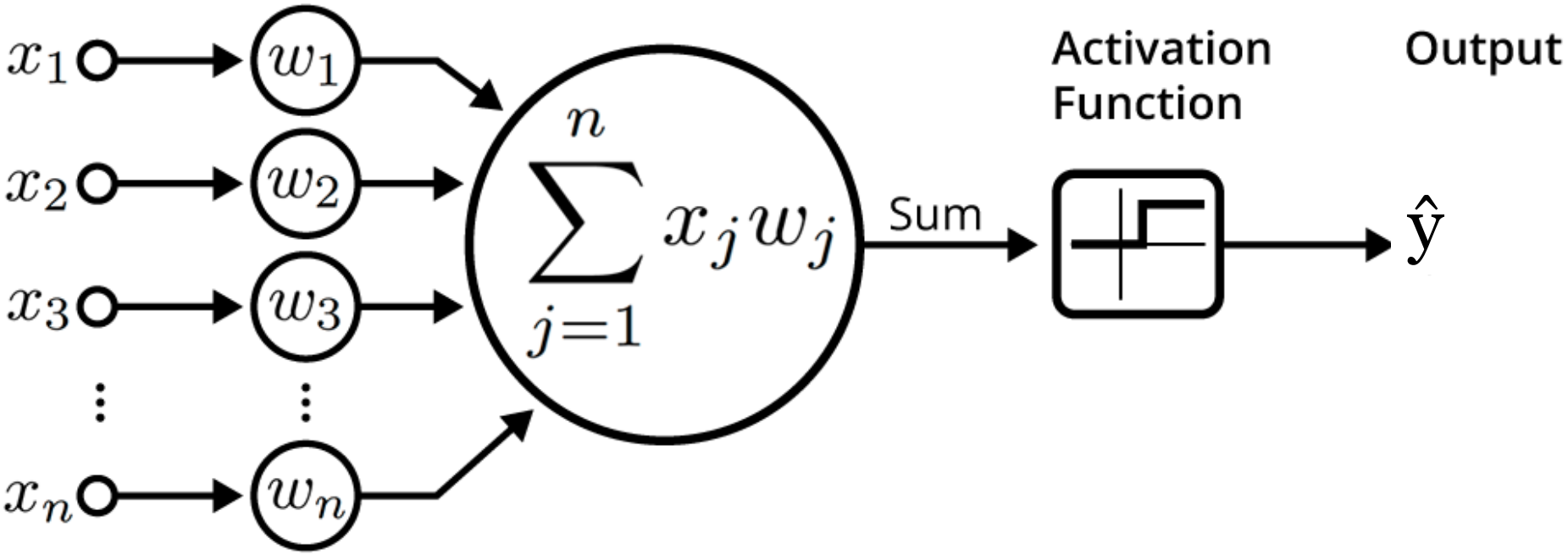
# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)




Deep Learning Neural Networks Explained in Plain English

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

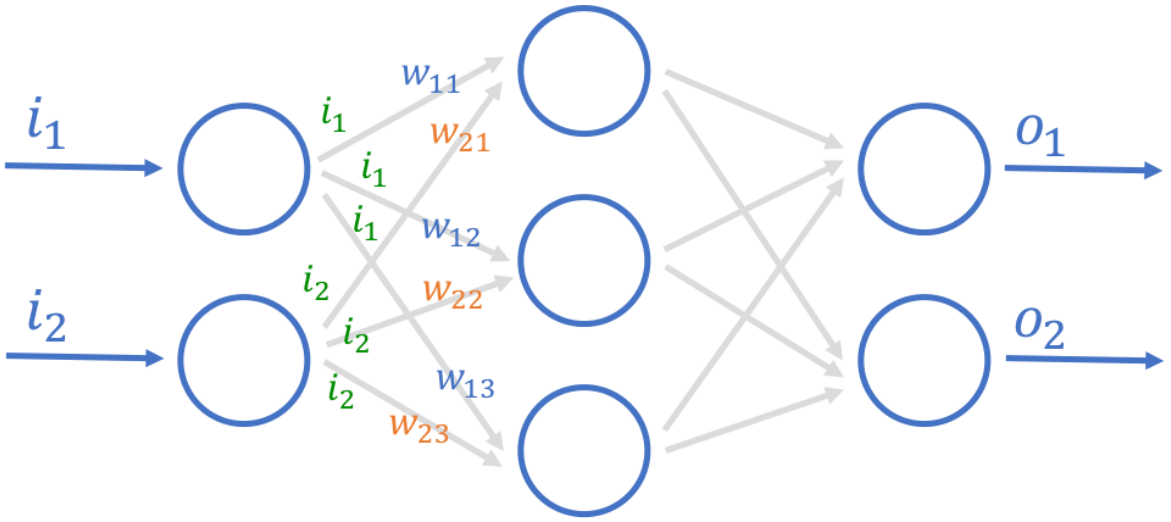
$$\hat{y} = f(W \cdot x + b)$$




$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

Deep Learning Neural Networks Explained in Plain English and Becoming Human

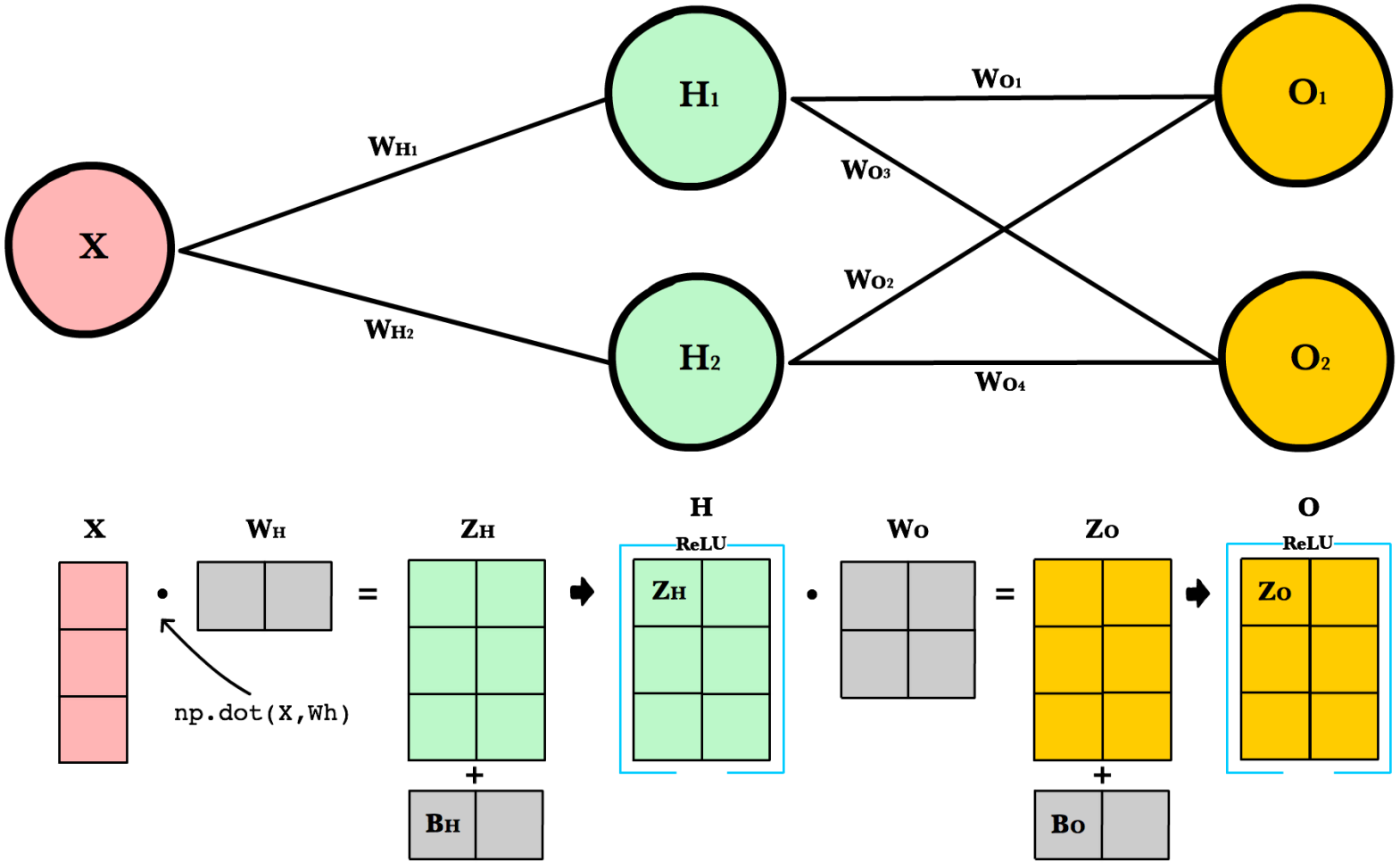
# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)




$$\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \cdot \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} = \begin{bmatrix} (w_{11} \times i_1) + (w_{21} \times i_2) \\ (w_{12} \times i_1) + (w_{22} \times i_2) \\ (w_{13} \times i_1) + (w_{23} \times i_2) \end{bmatrix}$$

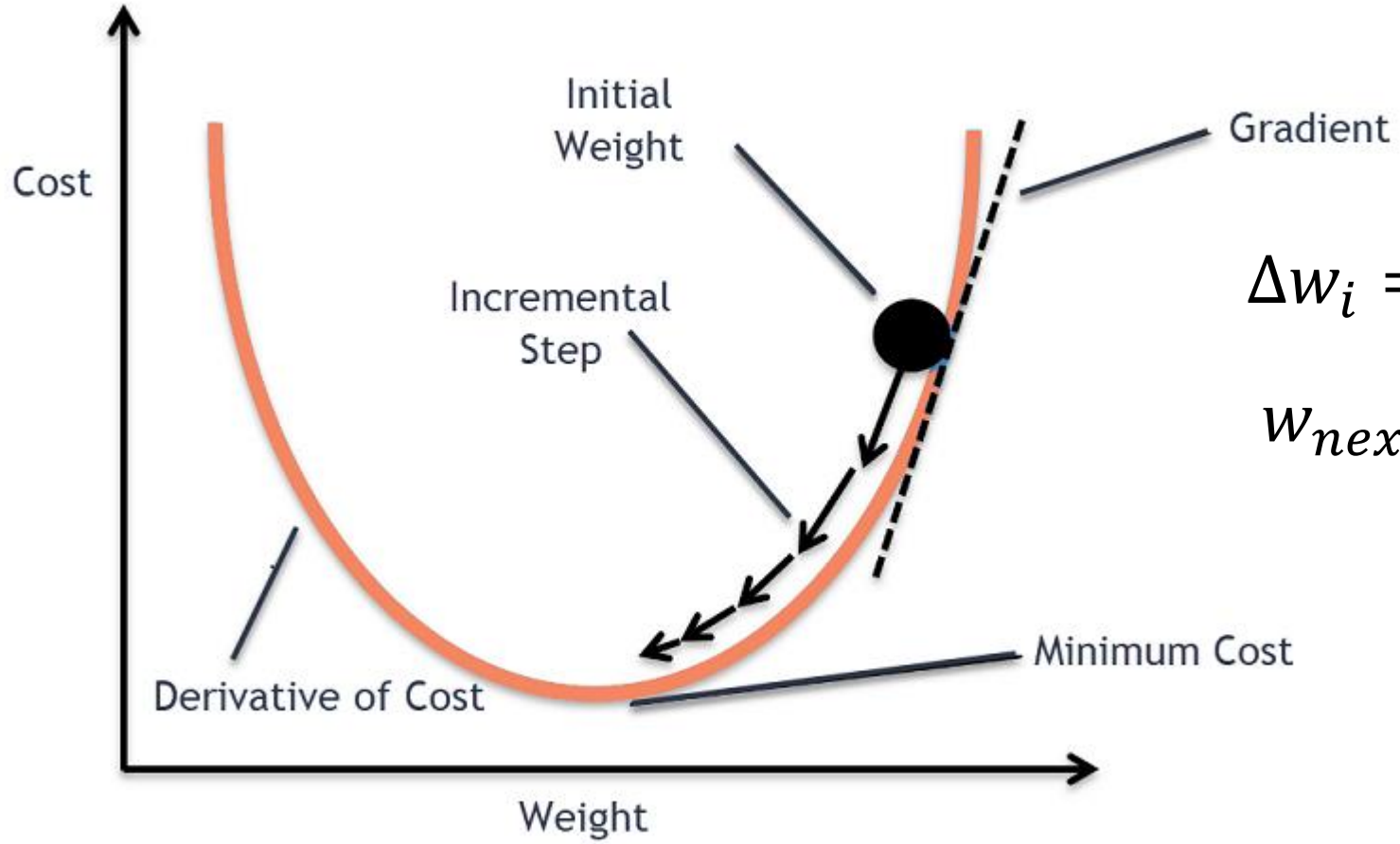
Danny Denenberg, 2019, Linear Algebra for Deep Learning, medium,
https://medium.com/@DannyDenenberg/linear-algebra-for-deep-learning-3a4f38a82ba7

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)




Forwardpropagation, ml-cheatsheet, https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



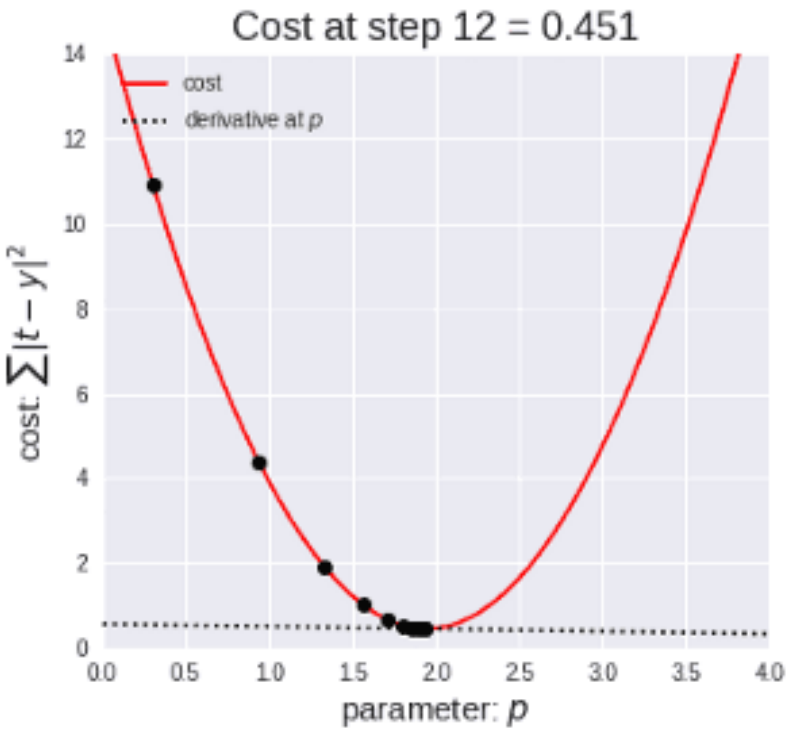

$$\Delta w_i = -\alpha \frac{dE}{dw_i}$$

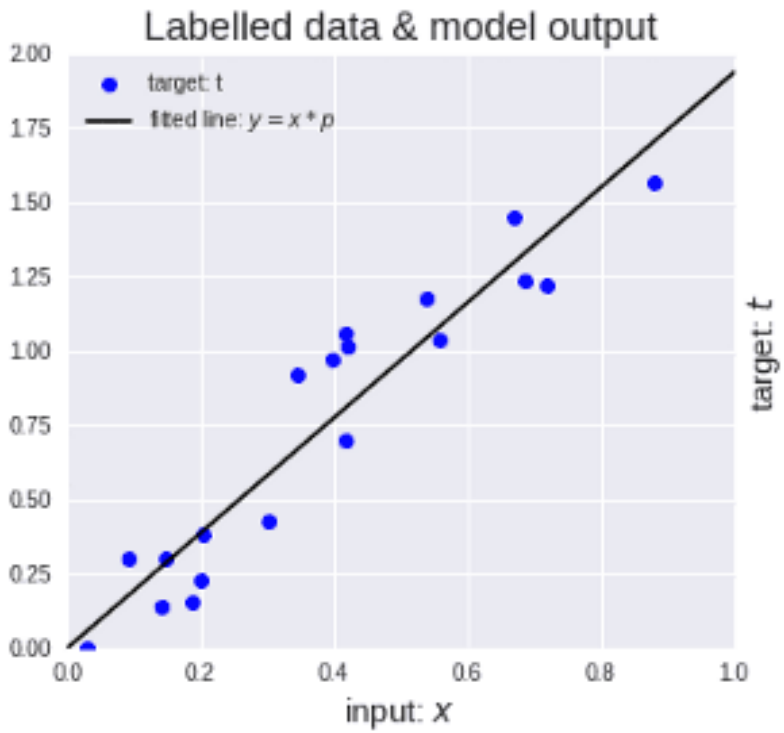
$$w_{next} = w + \Delta w$$

Sushant Patrikar, 2019, Batch, Mini Batch & Stochastic Gradient Descent
https://medium.com/@divakar_239/stochastic-vs-batch-gradient-descent-8820568eada1

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

$\hat{y}=f(W\cdot x+b)$ $target=\min_{LOSS}(y-\hat{y})$ $w_{new} = w_{old} + \eta\Delta w$



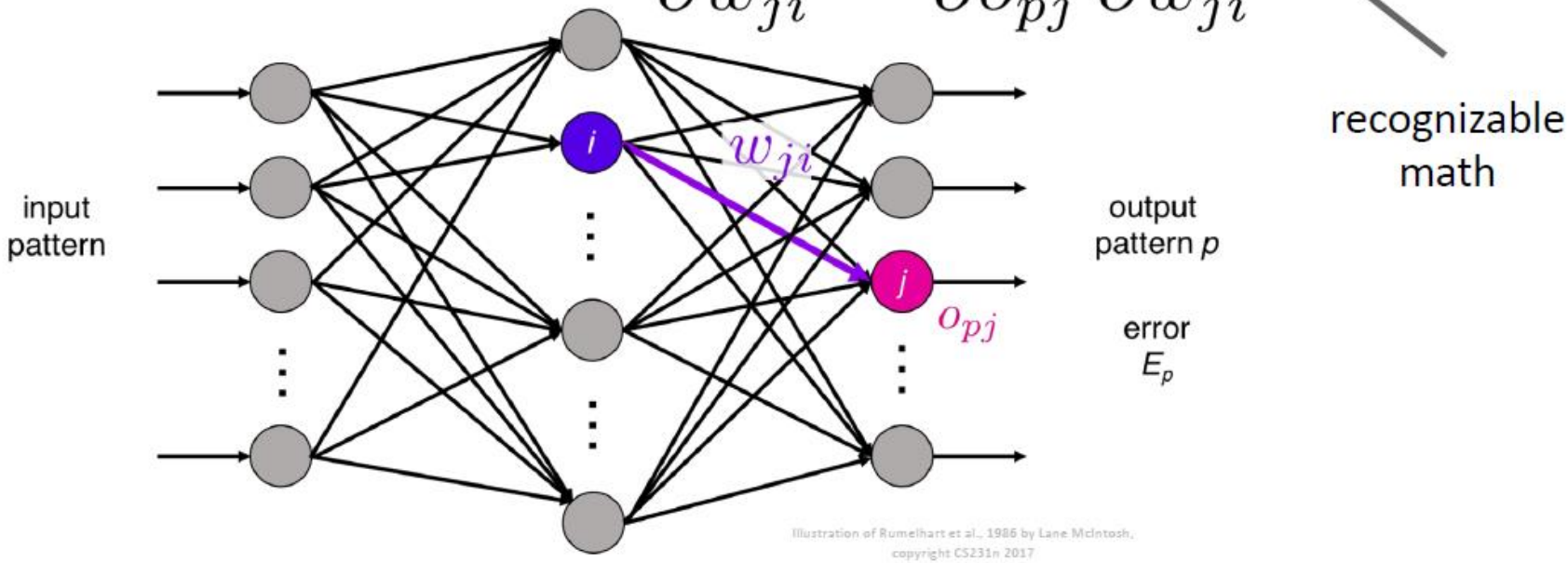




Gradient Descent and Cost Function in Python - Startertutorials

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}$$

recognizable math

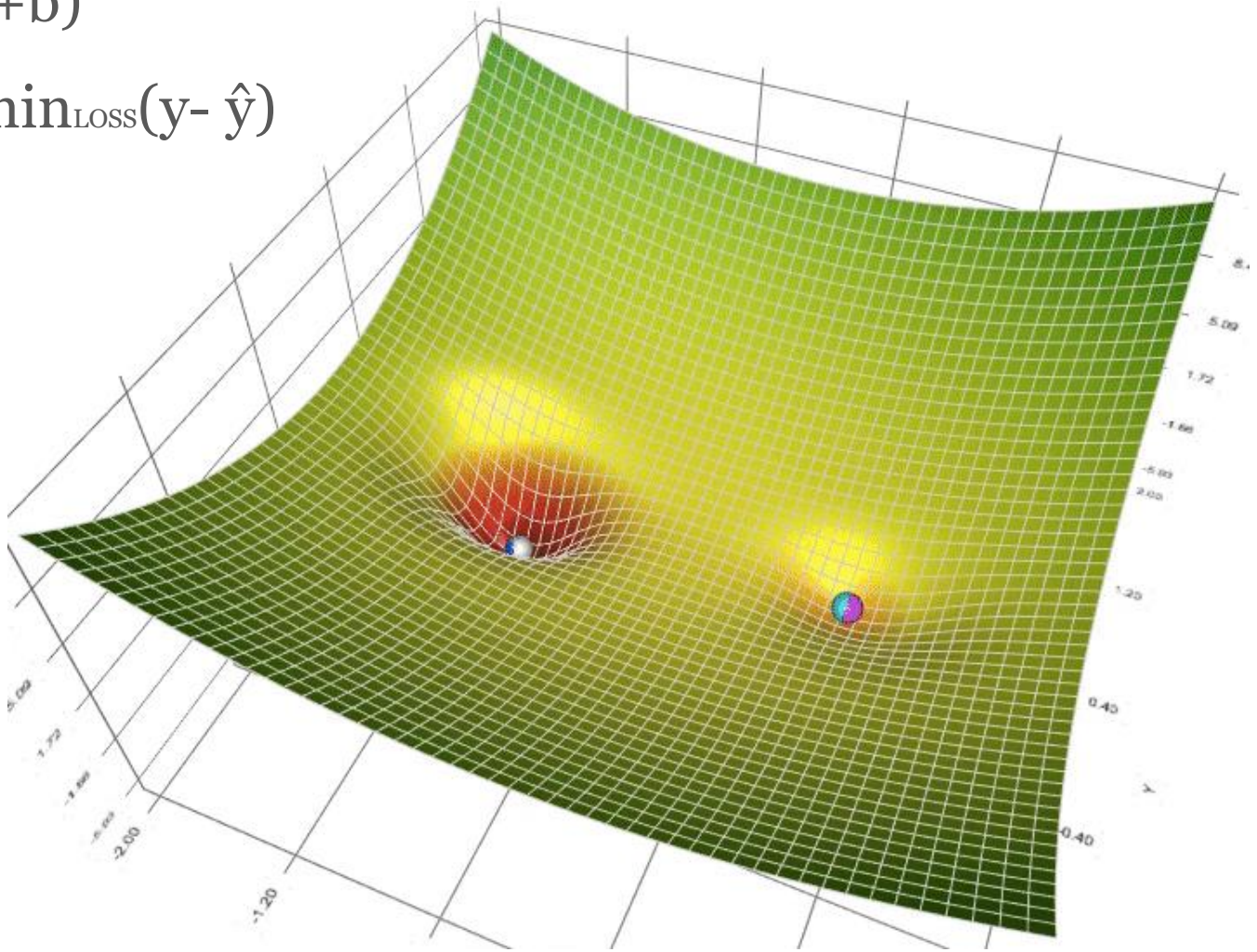



Backprop: Rumelhart, Hinton, and Williams, 1986

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

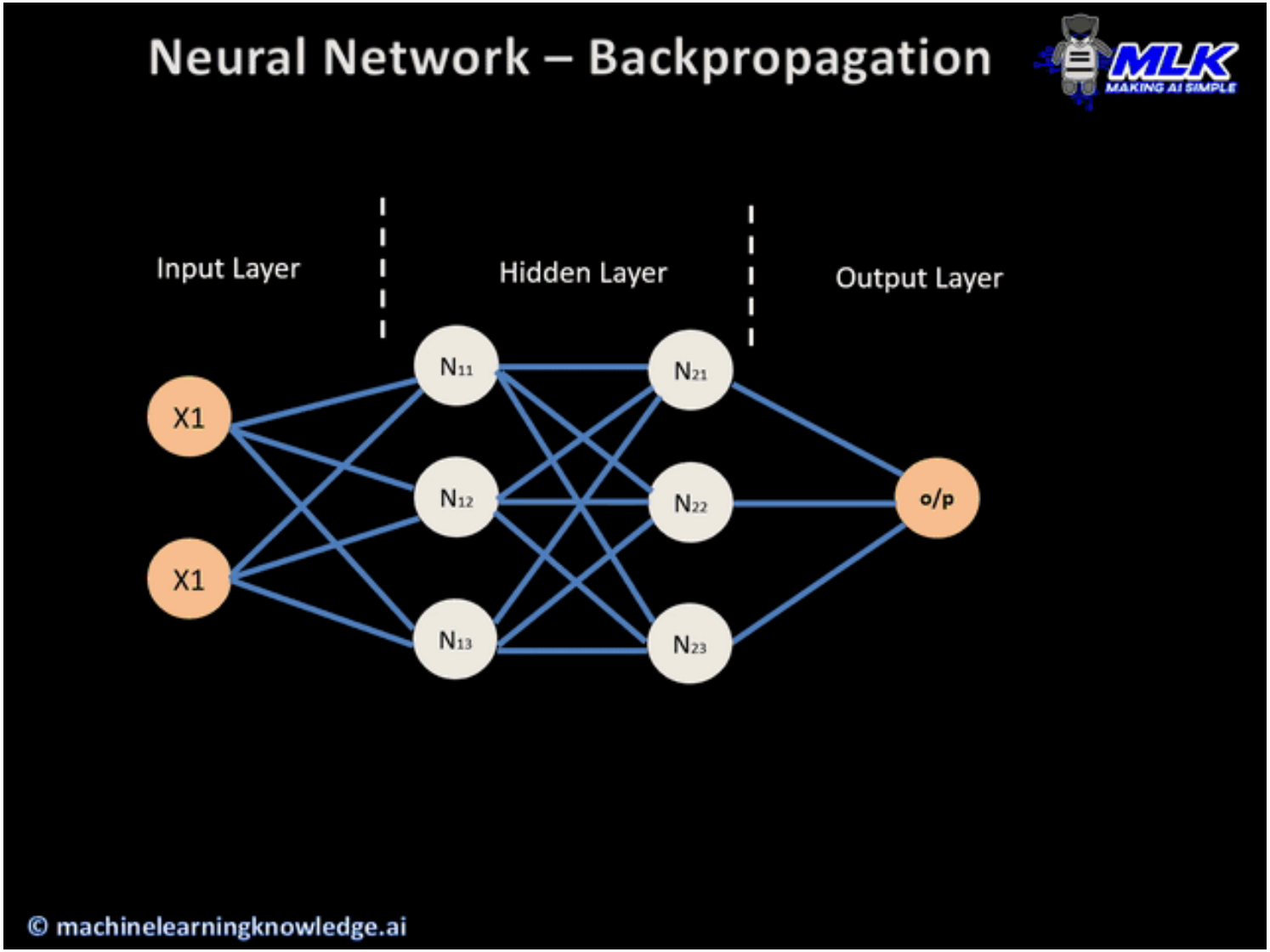
$\hat{y}=f(W\cdot x+b)$

$target=\min_{Loss}(y-\hat{y})$



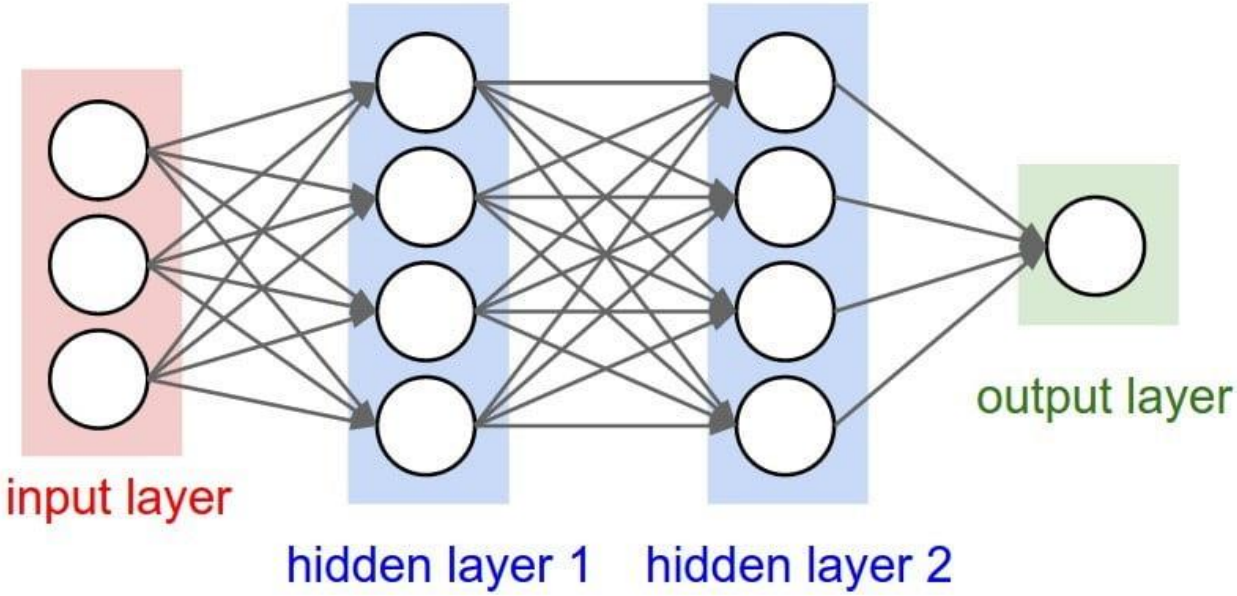

Enhancing Multi-Layer Perceptron Performance: Demystifying Optimizers | by Anand Raj | Towards AI

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



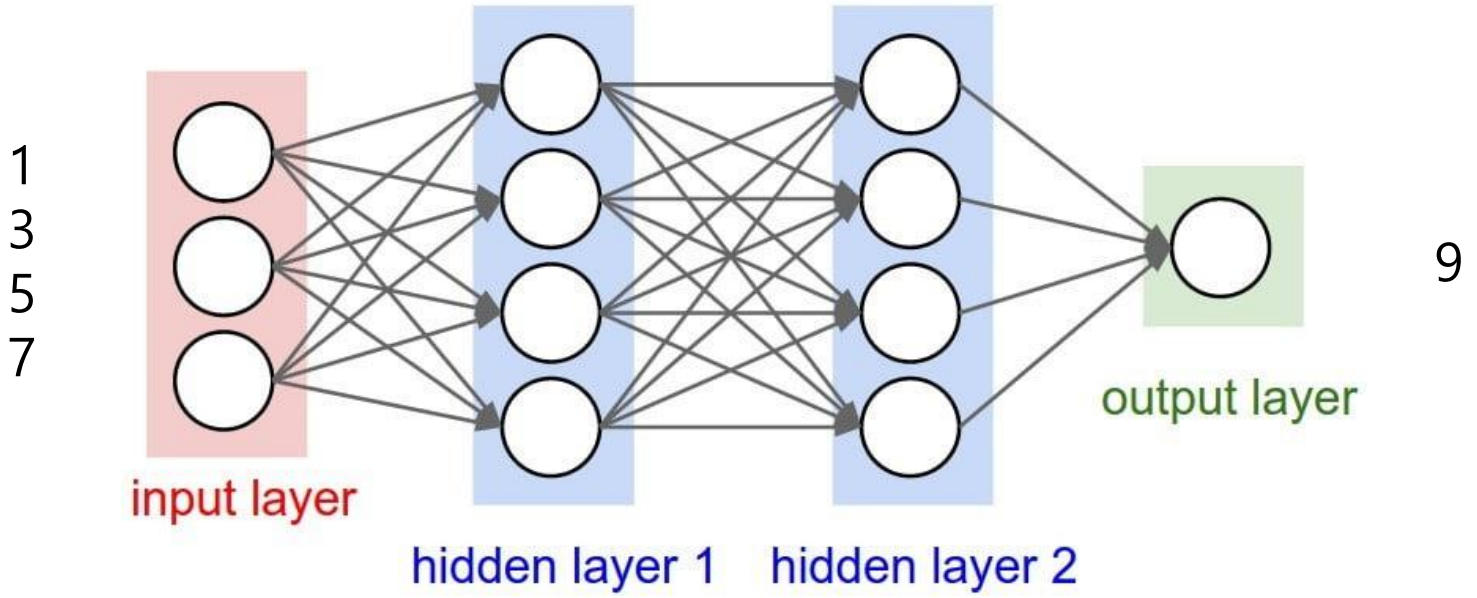

Backpropagation for Dummies. All the math behind, simplest than... | by Diletta Goglia | Analytics Vidhya | Medium

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

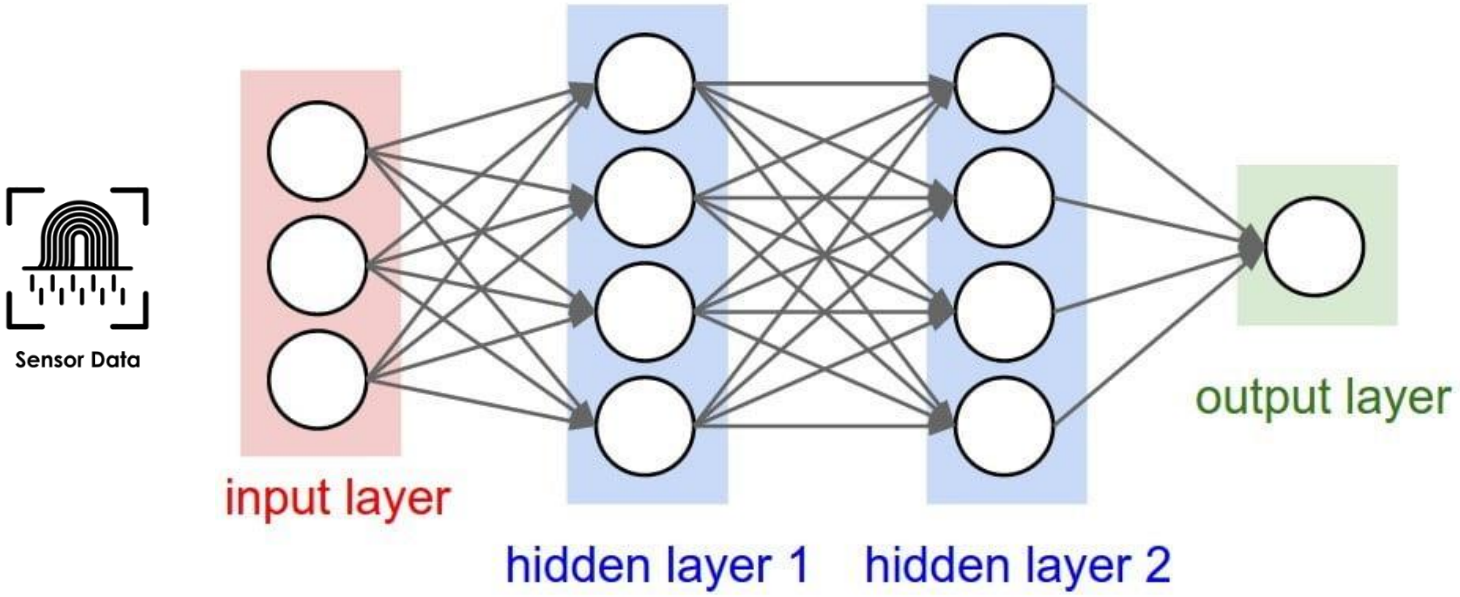



$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

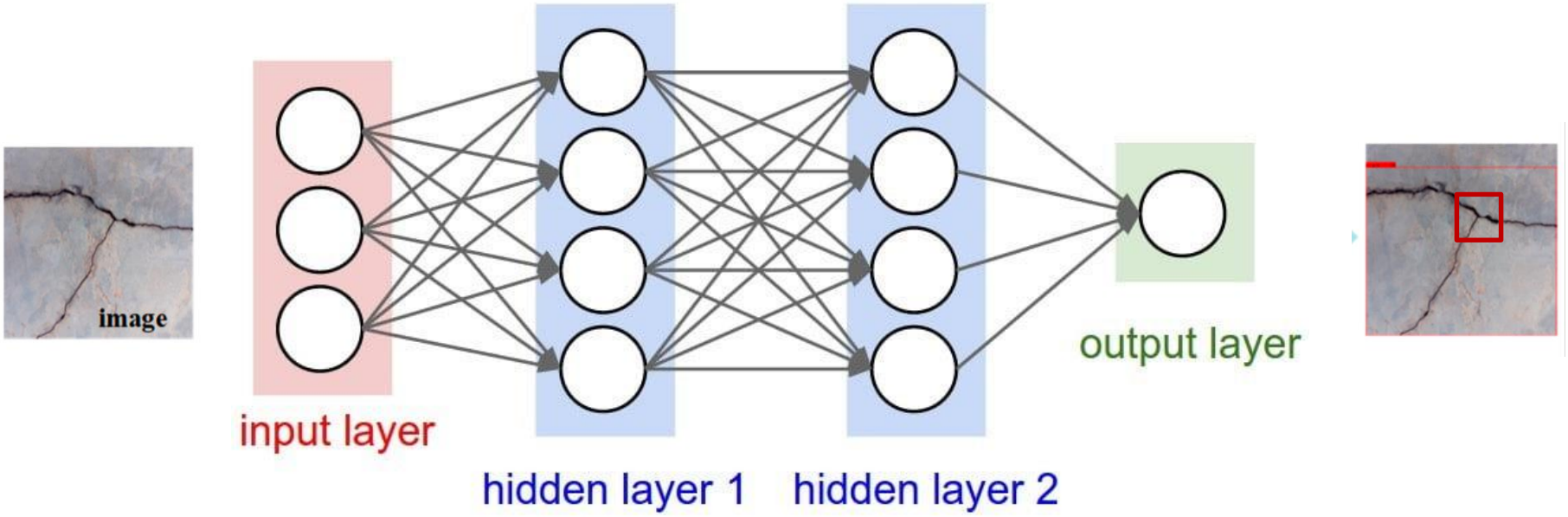
# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)



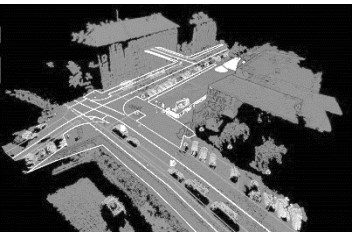
# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

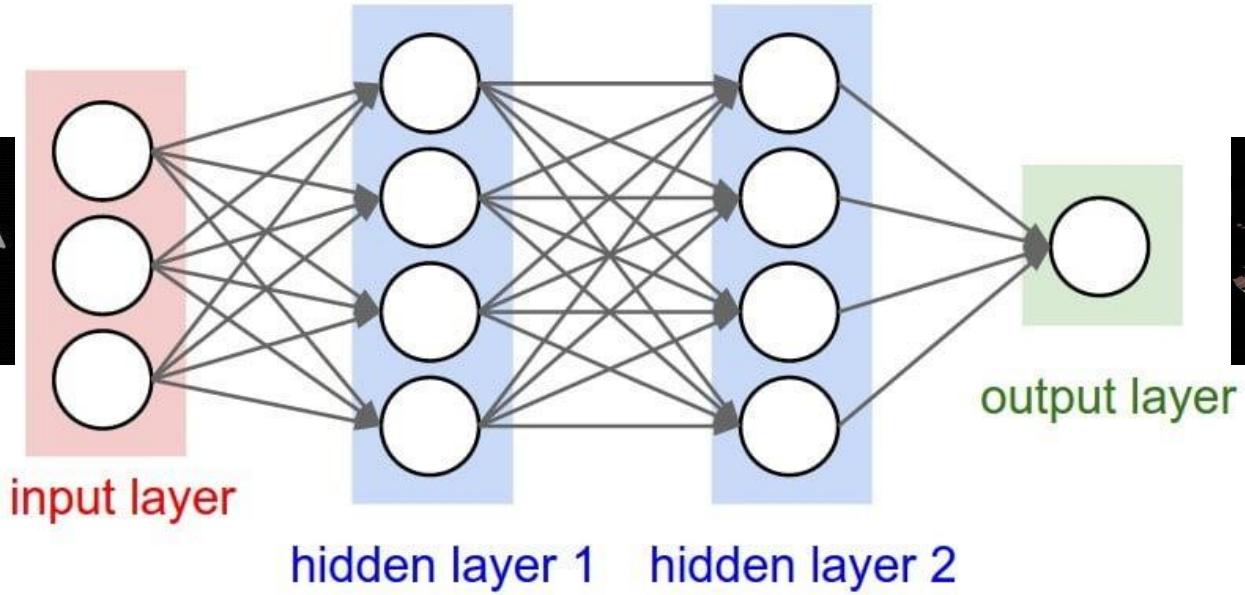


# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

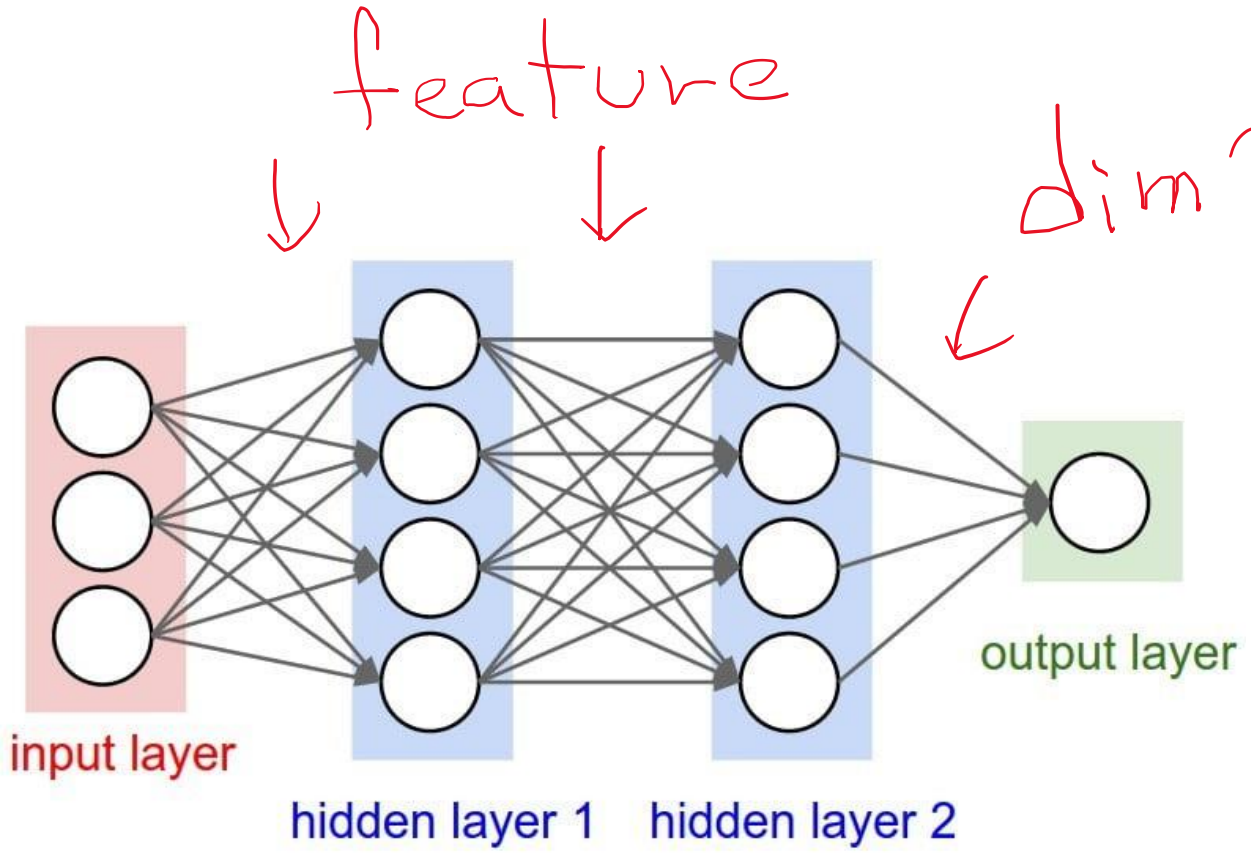


# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)





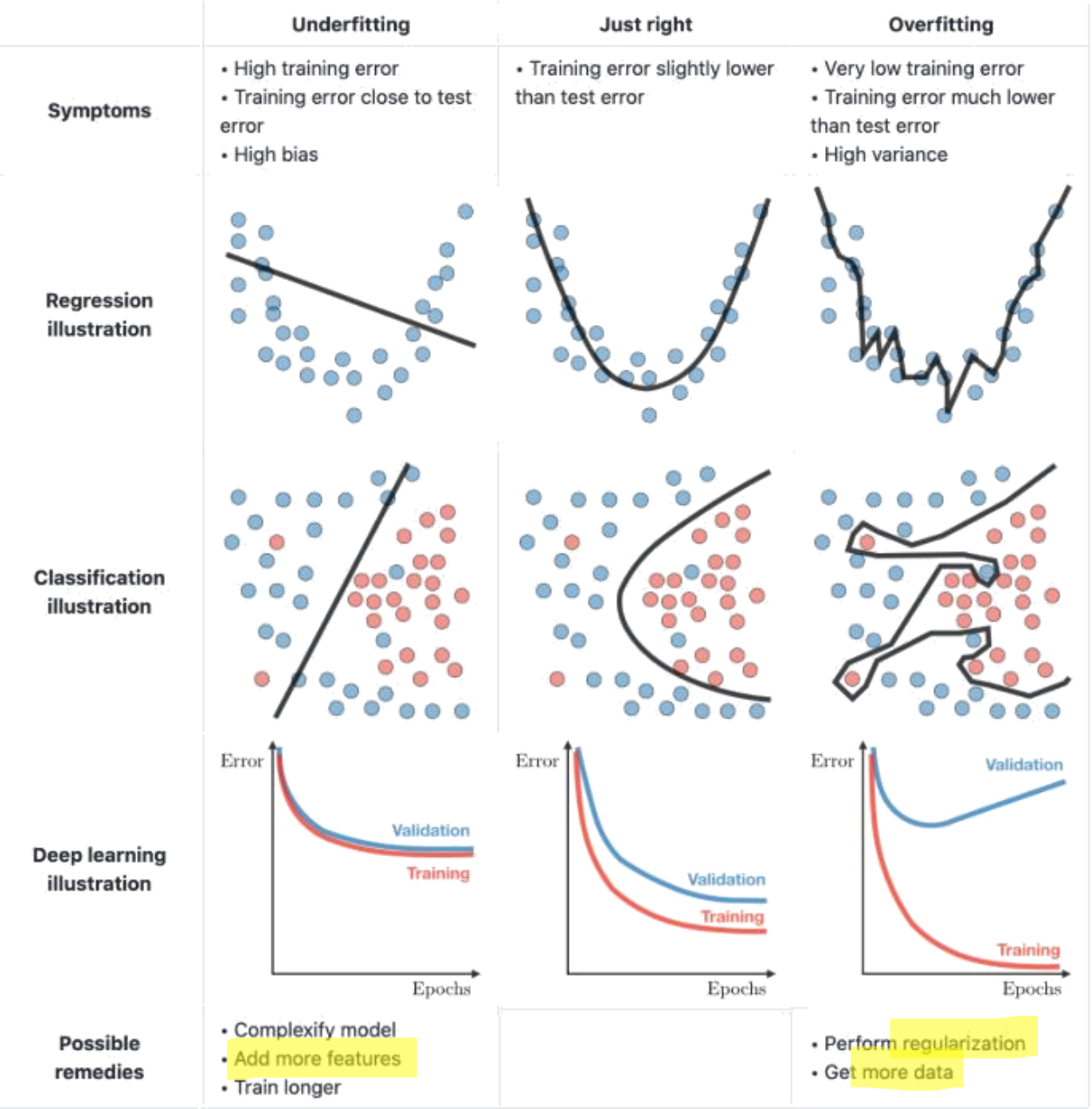

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)




Good morning

Good morning! How are you today?

# 딥러닝의 구조적 이해 신경망 기본 구조 (Perceptron, MLP)

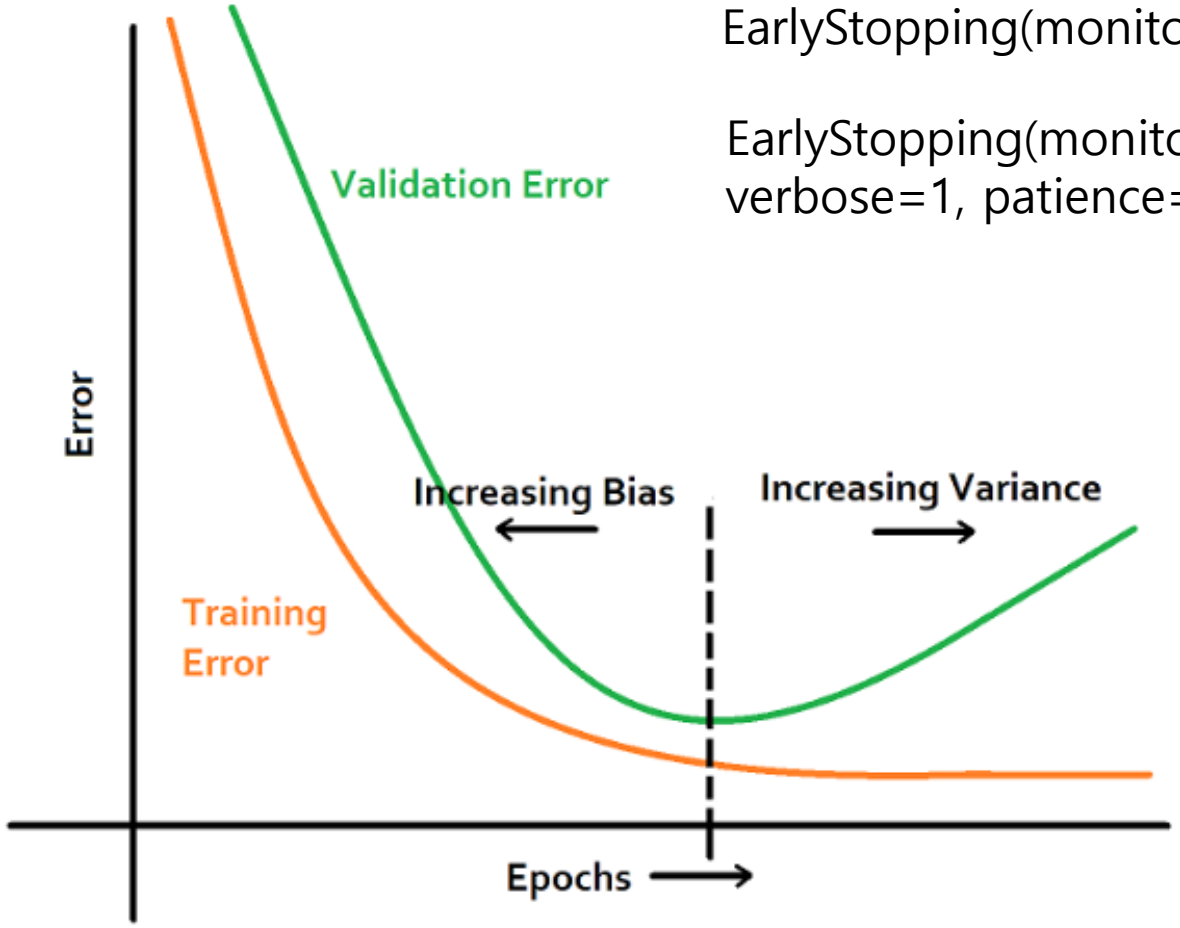



Abhishek Shrivastava, 2020, Underfitting Vs Just right Vs Overfitting in Machine learning, Kaggle(www.kaggle.com/getting-started/166897)

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

Epoch

Early stopping



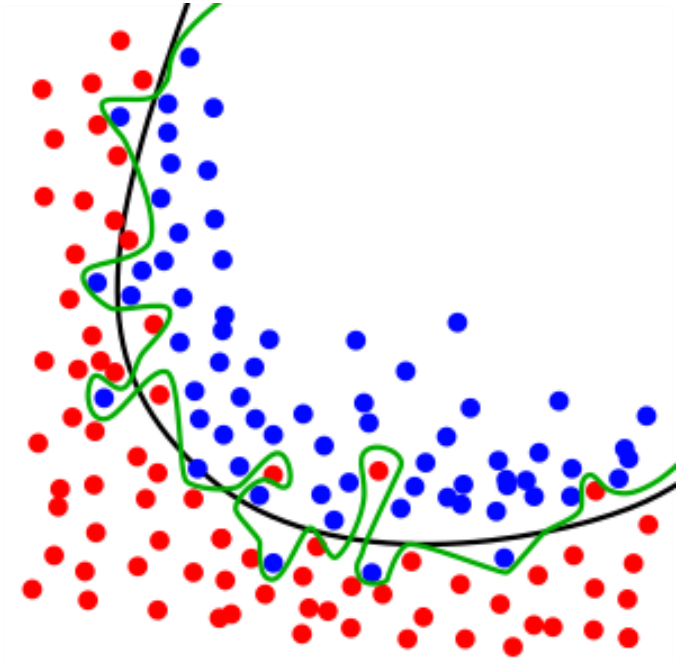

EarlyStopping(monitor='val_loss', mode='min)

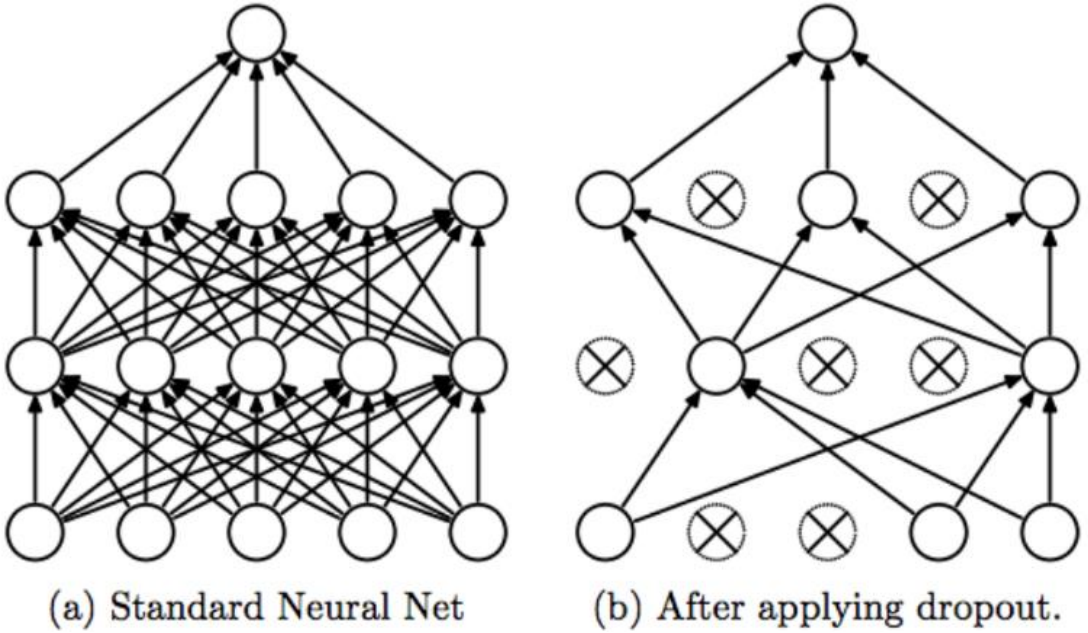
EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=50)

Rahul Jain, 2020, Why "early-stopping" works as Regularization?, https://medium.com/@rahuljain13101999/why-early-stopping-works-as-regularization-b9f0a6c2772

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

## Dropout






tf.model.add(tf.keras.layers.Dropout(drop_rate))

Srivastava, Nitish, et al. ”Dropout: a simple way to prevent neural networks from overfitting”, JMLR 2014

## L2 regularization

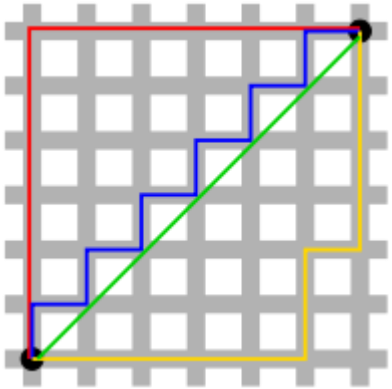
L1 Norm = sum of abs (vector_2 – vector_1)

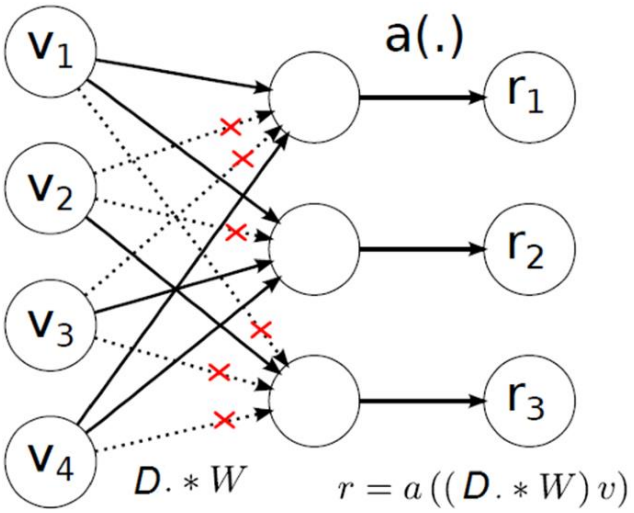
$$L = \sum_{i=1}^{n} |y_i - f(x_i)| \qquad Cost = \frac{1}{n}\sum_{i=1}^{n}\{L(y_i, \hat{y}_i) + \frac{\lambda}{2}|w|\}$$

L2 Norm = sum of (vector_2 – vector_1) ^ 2 > Euclidean distance

$$L = \sum_{i=1}^{n} (y_i - f(x_i))^2 \qquad Cost = \frac{1}{n}\sum_{i=1}^{n}\{L(y_i, \hat{y}_i) + \frac{\lambda}{2}|w|^2\}$$

L1 Cost < L2 Cost



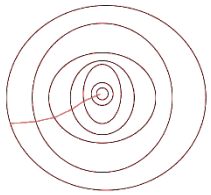



$$\Delta w_i = -\alpha \frac{dC}{dw_i}$$

Mode.add(Desnse(units=12,
kernel_regularizer=regularizers.l12(lambda),
activation='relu')

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

**Epoch** Updating weight factor using gradient descent (GD).
Train dataset / Batch



Batch size

1 Epoch

## Batch gradient descent (BGD)

$$L(W,b) = \frac{1}{N}\sum_{i}^{N}(\hat{y}-y)^2 \quad \Delta w_i = -\alpha\frac{dL}{dw_i}$$

$$w_{next\_step} = w_{cur\_step} - \alpha\frac{dL}{dw_i}$$

for index in range(epoch):
  $\hat{y}$=Tensor(All dataset) * NN in GPU
  Loss = Sum $(\hat{y}-y)$ / N
  Back propagation (Loss)

Dataset number =
Sample = 10

## Mini-batch gradient descent (MB-GD. batch size or subdivion)

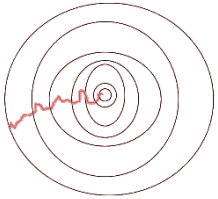
Mini-batch size = 2

for index in range(epoch):
  for mini_batch in (Batch / subidivision):
    $\hat{y}$=Tensor(mini_batch.dataset) * NN in GPU
    Loss = Sum $(\hat{y}-y)$ / N
    Back propagation (Loss) to update NN weights using Graident Descent (GD)

Dataset = 10, Batch size = 2 (GD), Steps of 1 Epoch = 5

## Stochastic gradient descent (SDG) 확률적 경사하강

Batch Gradient Descent: Batch Size = Size of Training Set
Stochastic Gradient Descent: Batch Size = 1
Mini-Batch Gradient Descent: 1 < Batch Size < Size of Training Set.



## Optimization algorithm

Stochastic Gradient Descent

$$W(t+1) = W(t) - \alpha \frac{\partial}{\partial w} Cost(w)$$

Gradient Descent with Momentum

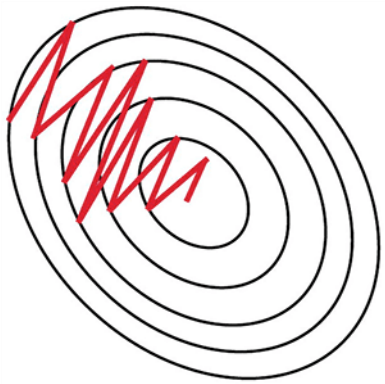
$$V(t) = m * V(t-1) - \alpha \frac{\partial}{\partial w} Cost(w)$$
$$W(t+1) = W(t) + V(t)$$

$$M(t) = \beta_1 M(t-1) + (1-\beta_1) \frac{\partial}{\partial w(t)} Cost(w(t))$$

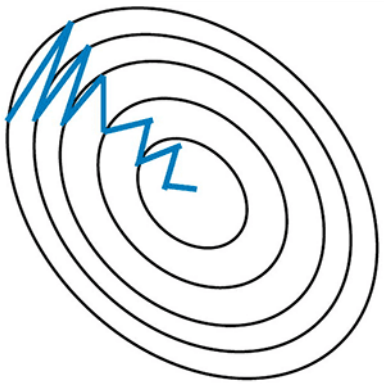
$$V(t) = \beta_2 V(t-1) + (1-\beta_2) \left( \frac{\partial}{\partial w(i)} Cost(w(i)) \right)^2$$

$$\hat{M}(t) = \frac{M(t)}{1-\beta_1^{\,t}} \qquad \hat{V}(t) = \frac{V(t)}{1-\beta_2^{\,t}}$$

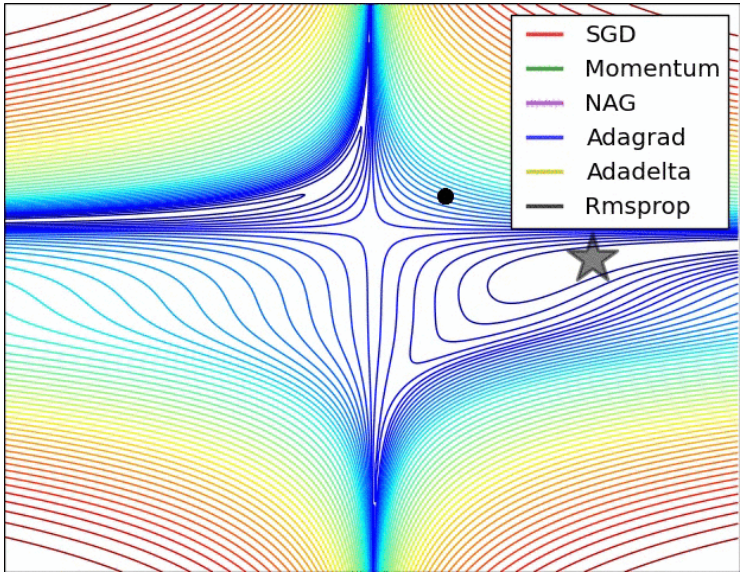
$$W(t+1) = W(t) - \alpha * \frac{\hat{M}(t)}{\sqrt{\hat{V}(t) + \epsilon}}$$



Stochastic Gradient Descent **without** Momentum



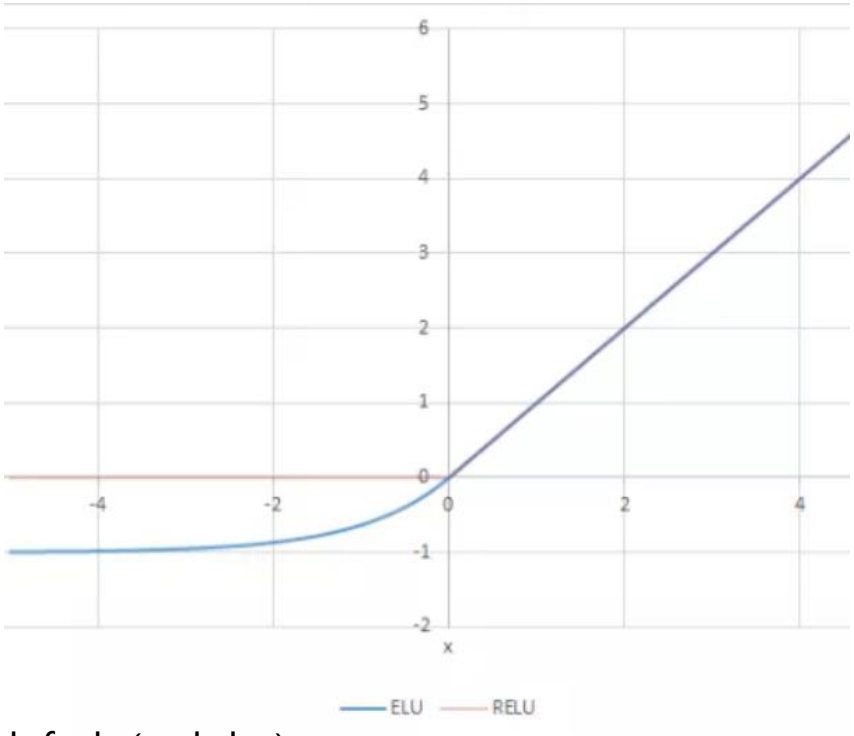
Stochastic Gradient Descent **with** Momentum




SEBASTIAN RUDER, 2016, An overview of gradient descent optimization algorithms, https://ruder.io/optimizing-gradient-descent/index.html#visualizationofalgorithms

Exponential Linear Unit

$$R(z) = \begin{cases} z & z > 0 \\ \alpha.(e^z - 1) & z <= 0 \end{cases}$$



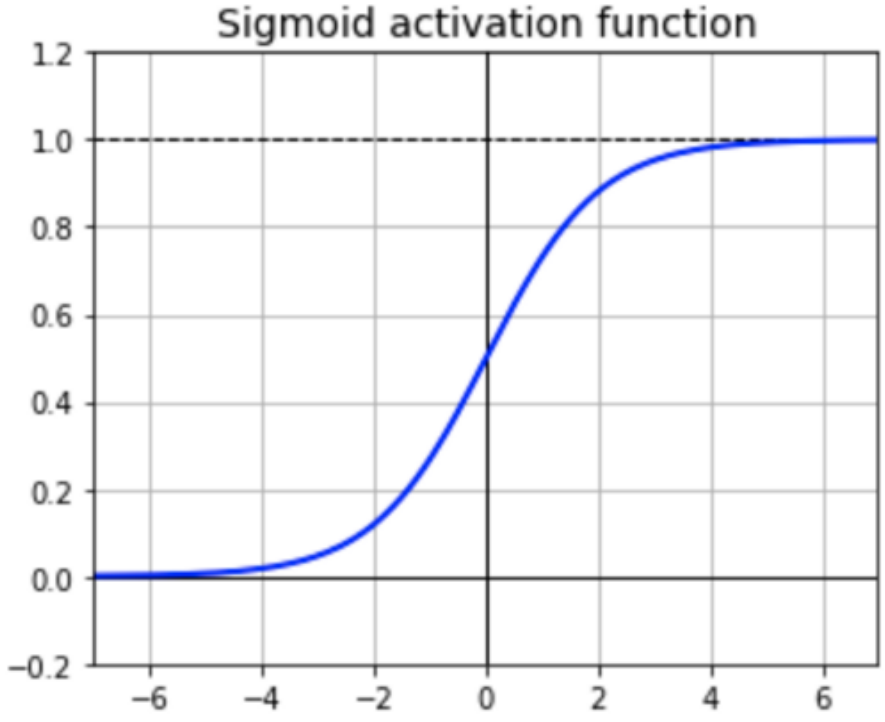

```
def elu(z,alpha):
        return z if z >= 0 else alpha*(e^z -1)
```

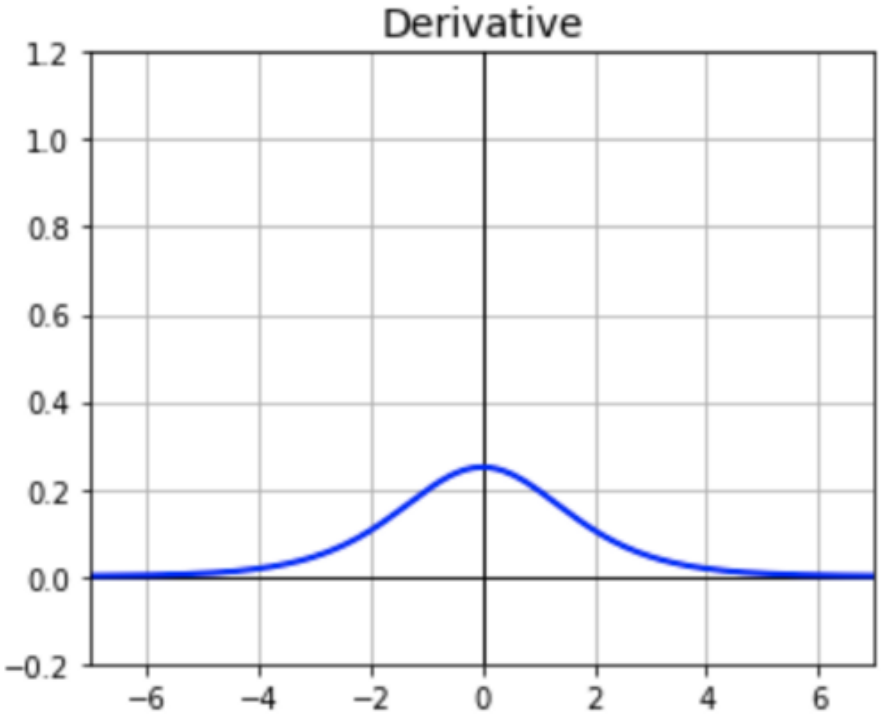
ML Glossary, Activation Functions, https://ml-cheatsheet.readthedocs.io/en/latest/activation_functions.html

## Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Vanishing gradien
Computationally expensive
The output is not zero centered



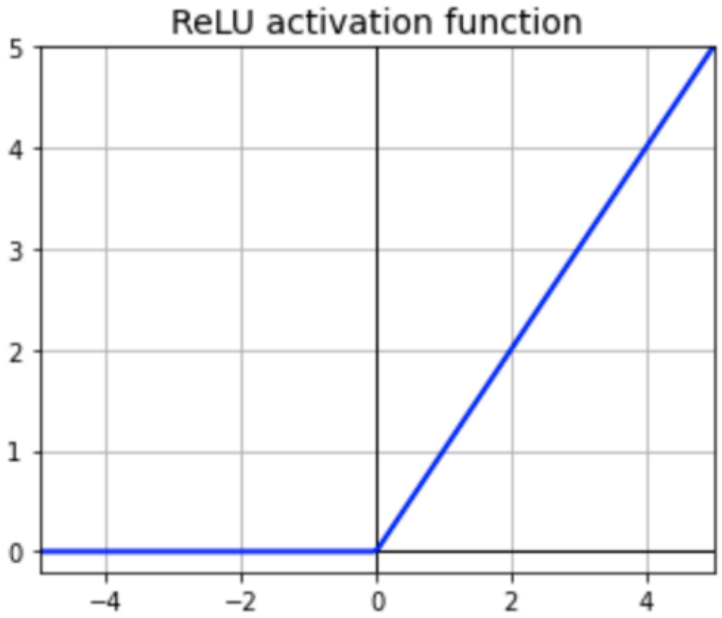

B. Chen, 2021, 7 popular activation functions you should know in Deep Learning and how to use them with Keras and TensorFlow 2, https://towardsdatascience.com/7-pop
activation-functions-you-should-know-in-deep-learning-and-how-to-use-them-with-keras-and-27b4d838dfe6

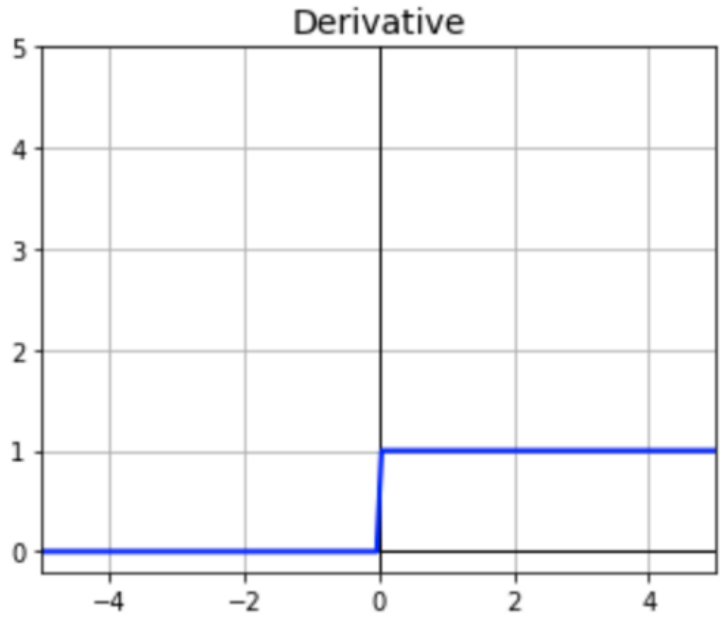
## ReLU (Rectified Linear Unit)

$$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

$$f(x) = \max(0, x)$$

```
def relu(x):
  if x < 0:
    return 0
  else:
return x
```



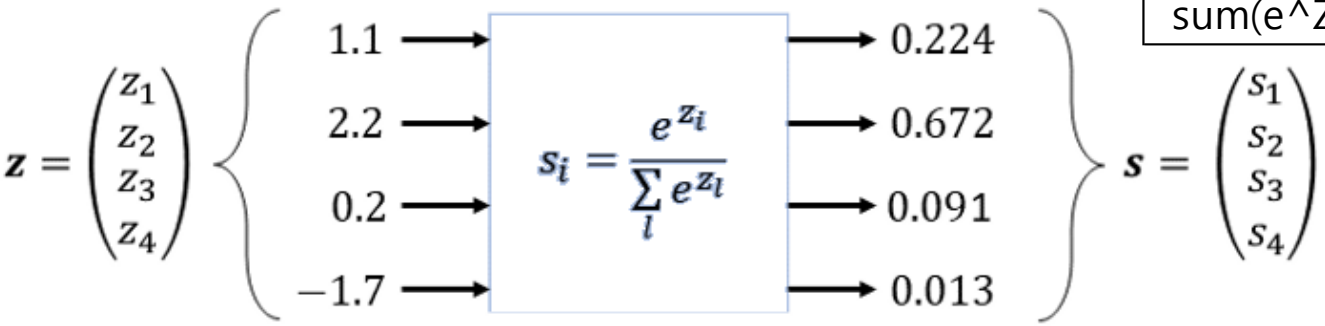

B. Chen, 2021, 7 popular activation functions you should know in Deep Learning and how to use them with Keras and TensorFlow 2, https://towardsdatascience.com/7-popular-activation-functions-you-should-know-in-deep-learning-and-how-to-use-them-with-keras-and-27b4d838dfe6

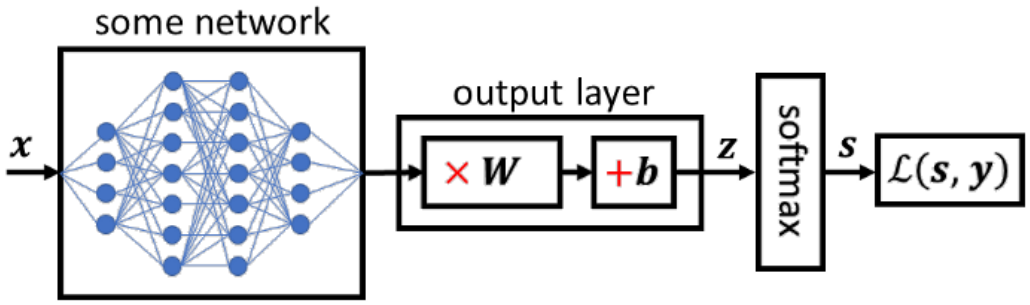
# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

Softmax

$$s_i = \frac{e^{z_i}}{\sum_{l=1}^{n} e^{z_l}}$$

| Zi | e^Zi | Si |
|---|---|---|
| 1.1 | 3.00417 | 0.224 |
| 2.2 | 9.02501 | 0.672 |
| 0.2 | 1.22140 | 0.091 |
| -1.7 | 0.18268 | 0.014 |
| sum(e^Zi) | 13.43 | |




$$-1.7 < 0.2 < 1.1 < 2.2 \quad \rightarrow \quad 0.013 < 0.091 < 0.224 < 0.672$$

Thomas Kurbiel, 2021, Derivative of the Softmax Function and the Categorical Cross-Entropy Loss, https://towardsdatascience.com/derivative-of-the-softmax-function-and-the-categorical-cross-entropy-loss-ffceefc081d1

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

**Mean Square Error (L2-norm) Loss**

$$L = \left(y - f(x)\right)^2$$

$$MSE = \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}$$

**Mean Absolute Error (L1-norm) Loss**

$$MAE = \frac{\sum_{i=1}^{n} \mid y_i - \hat{y}_i \mid}{n}$$

**Root Mean Square Error Loss (L2 Loss)**

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

```
import numpy as np
def root_mean_squared_error(act, pred):

    diff = pred - act
    differences_squared = diff ** 2
    mean_diff = differences_squared.mean()
    rmse_val = np.sqrt(mean_diff)
    return rmse_val

act = np.array([1.1,2,1.7])
pred = np.array([1,1.7,1.5])

print(root_mean_squared_error(act, pred))
```

0.2160246899469286 7

Jayant Verma, 2022, Loss Functions in Python - Easy Implementation, https://www.digitalocean.com/community/tutorials/loss-functions-in-python

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

Binary Cross Entropy Loss (BCE)

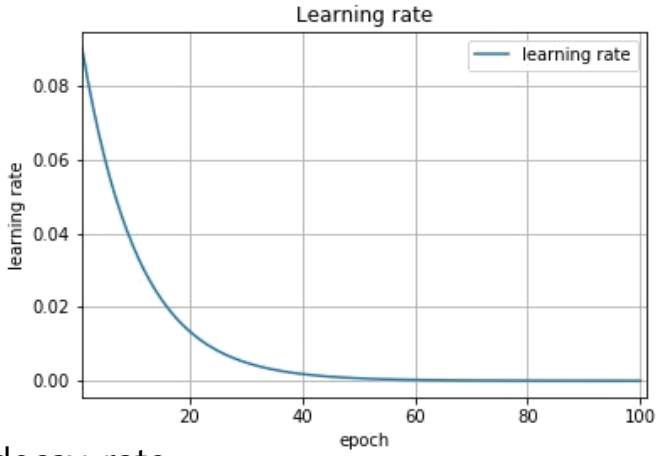
$$L = -\frac{1}{n}\sum_{i}^{n} y_i \cdot log\hat{y}_i + (1 - y_i) \cdot log(1 - \hat{y}_i)$$

```
def safe_log(x):
    return 0 if x == 0 else np.log(x)

def binary_cross_entropy(y, y_hat):
    total = 0
    for curr_y, curr_y_hat in zip(y, y_hat):
        total += (curr_y * safe_log(curr_y_hat)
                  + (1 - curr_y) * safe_log(1 - curr_y_hat))
    return - total / len(y)

print(binary_cross_entropy(y=[1, 0], y_hat=[0.9, 0.1]))
print(binary_cross_entropy(y=[1, 0], y_hat=[0.1, 0.9]))
```

Master Machine Learning: Logistic Regression From Scratch With Python, 2021.3,
https://betterdatascience.com/mml-logistic-regression/

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

## Learning rate and decay




### Time-based decay

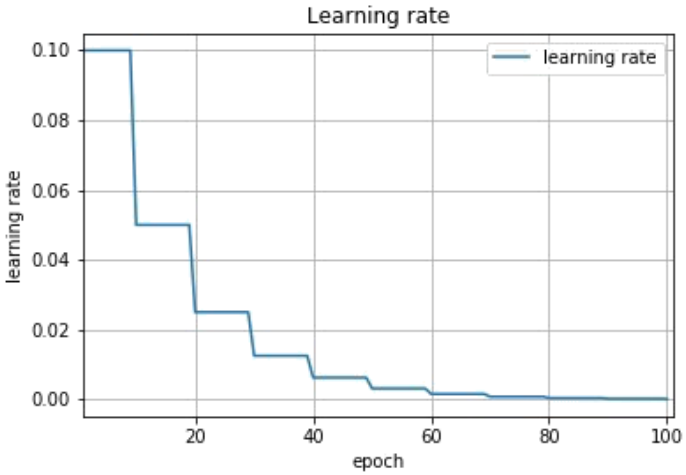
lr *= (1. / (1. + decay * iterations))

```
learning_rate = 0.1
decay_rate = learning_rate / epochs
momentum = 0.8
sgd = SGD(lr=learning_rate, momentum=momentum, decay=decay_rate,
nesterov=False)
```

### Step decay

lr = lr0 * drop ^ floor(epoch / epochs_drop)

```
def step_decay(epoch):
   initial_lrate = 0.1
   drop = 0.5
   epochs_drop = 10.0
   lrate = initial_lrate * math.pow(drop,
           math.floor((1+epoch)/epochs_drop))
   return lrate
lrate = LearningRateScheduler(step_decay)
```




Suki Lau, 2017, Learning Rate Schedules and Adaptive Learning Rate Methods for Deep Learning,
https://towardsdatascience.com/learning-rate-schedules-and-adaptive-learning-rate-methods-for-deep-learning-2c8f433990d1

## Data Augmentation

```
from keras.preprocessing.image import ImageDataGenerator
from keras.utils import array_to_img, img_to_array, load_img

datagen = ImageDataGenerator(
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```

```
img = load_img('./cat0.jpg')  # this is a PIL image
x = img_to_array(img)  # this is a Numpy array with shape (3, 150, 150)
x = x.reshape((1,) + x.shape)  # this is a Numpy array with shape (1, 3, 150, 150)

# the .flow() command below generates batches of randomly transformed images
# and saves the results to the `preview/` directory
i = 0
for batch in datagen.flow(x, batch_size=1,
                          save_to_dir='preview', save_prefix='cat', save_format='jpeg'):
    i += 1
    if i > 20:
        break  # otherwise the generator would loop indefinitely
```

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

## Batch normalization

**Input:** Values of $x$ over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: $\gamma$, $\beta$
**Output:** $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

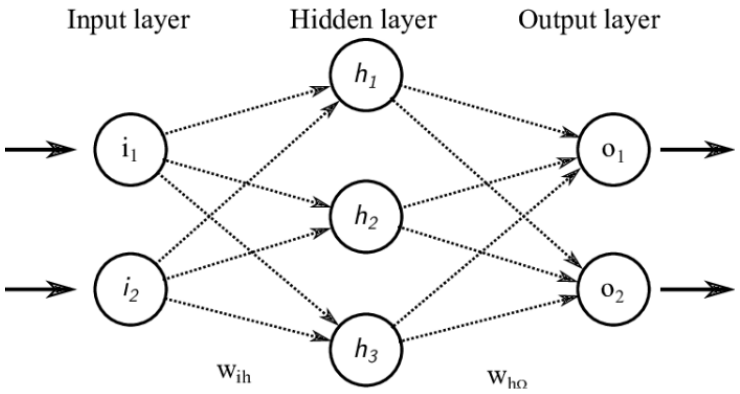
$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

```
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.layers import BatchNormalization
from keras.utils import to_categorical
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=1000, centers=4,
n_features=2, cluster_std=2, random_state=2)

y = to_categorical(y)
n_train = 800
trainX, testX = X[:n_train, :], X[n_train:, :]
trainy, testy = y[:n_train], y[n_train:]
print(trainX.shape, testX.shape)

model = Sequential()
model.add(Dense(25, input_dim=2, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))
model.add(Dense(8, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))
model.add(Dense(4, activation='softmax'))
model.compile(loss='categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])

model.summary()
```

Aakash bindal, Normalization Techniques in Deep Neural Networks, https://medium.com/techspace-usict/normalization-techniques-in-deep-neural-networks-9121bf100d8

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

```
from math import exp
from random import seed
from random import random

# Initialize a network
def initialize_network(n_inputs, n_hidden, n_outputs):
        network = list()
        hidden_layer = [{'weights':[random() for i in range(n_inputs + 1)]} for i in range(n_hidden)]
        network.append(hidden_layer)
        output_layer = [{'weights':[random() for i in range(n_hidden + 1)]} for i in range(n_outputs)]
        network.append(output_layer)
        return network

# Calculate neuron activation for an input
def activate(weights, inputs):
        activation = weights[-1]
        for i in range(len(weights)-1):
                activation += weights[i] * inputs[i]
        return activation

# Transfer neuron activation
def transfer(activation):
        return 1.0 / (1.0 + exp(-activation))

# Forward propagate input to a network output
def forward_propagate(network, row):
        inputs = row
        for layer in network:
                new_inputs = []
                for neuron in layer:
                        activation = activate(neuron['weights'], inputs)
                        neuron['output'] = transfer(activation)
                        new_inputs.append(neuron['output'])
                inputs = new_inputs
        return inputs
```




INPUT = 2
HIDDEN LAYERS
OUTPUT = 2

Jason Brownlee, 2016, How to Code a Neural Network with Backpropagation In Python (from scratch), Machine Learning Mastery, https://machinelearningmastery.com/implement-backpropagation-algorithm-scratch-python/

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

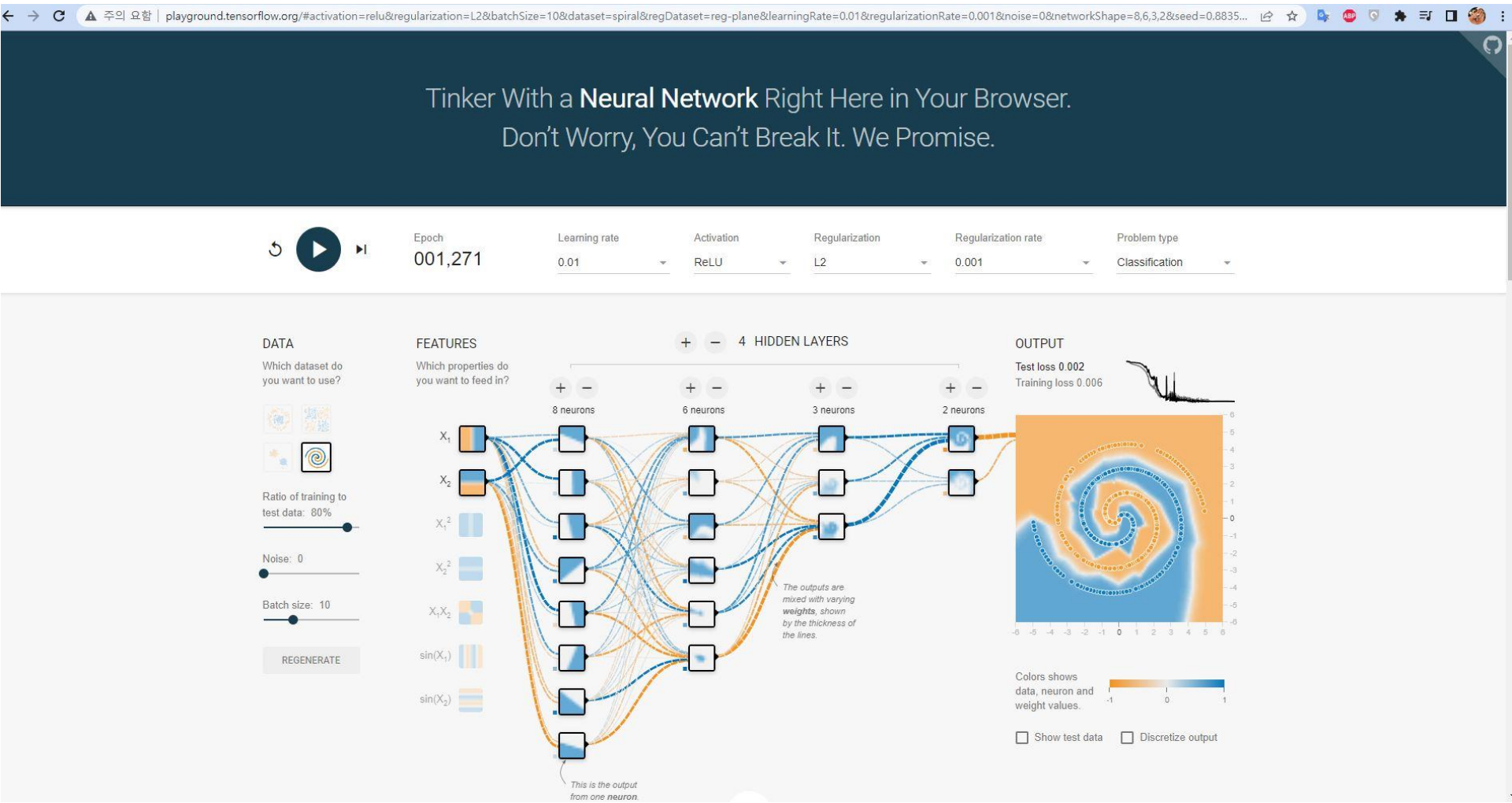
```
# Calculate the derivative of an neuron output
def transfer_derivative(output):
        return output * (1.0 - output)

# Backpropagate error and store in neurons
def backward_propagate_error(network, expected):
        for i in reversed(range(len(network))):
                layer = network[i]
                errors = list()
                if i != len(network)-1:
                        for j in range(len(layer)):
                                error = 0.0
                                for neuron in network[i + 1]:
                                        error += (neuron['weights'][j] * neuron['delta'])
                                errors.append(error)
                else:
                        for j in range(len(layer)):
                                neuron = layer[j]
                                errors.append(neuron['output'] - expected[j])
                for j in range(len(layer)):
                        neuron = layer[j]
                        neuron['delta'] = errors[j] * transfer_derivative(neuron['output'])

# Update network weights with error
def update_weights(network, row, l_rate):
        for i in range(len(network)):
                inputs = row[:-1]
                if i != 0:
                        inputs = [neuron['output'] for neuron in network[i - 1]]
                for neuron in network[i]:
                        for j in range(len(inputs)):
                                neuron['weights'][j] -= l_rate * neuron['delta'] * inputs[j]
                        neuron['weights'][-1] -= l_rate * neuron['delta']
```

# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

```
# Train a network for a fixed number of epochs
def train_network(network, train, l_rate, n_epoch, n_outputs):
        for epoch in range(n_epoch):
                sum_error = 0
                for row in train:
                        outputs = forward_propagate(network, row)
                        expected = [0 for i in range(n_outputs)]
                        expected[row[-1]] = 1
                        sum_error += sum([(expected[i]-outputs[i])**2 for i in range(len(expected))])
                        backward_propagate_error(network, expected)
                        update_weights(network, row, l_rate)
                print('>epoch=%d, lrate=%.3f, error=%.3f' % (epoch, l_rate, sum_error))

# Test training backprop algorithm
seed(1)
dataset = [[2.7810836,2.550537003,0],
        [1.465489372,2.362125076,0],
        [3.396561688,4.400293529,0],
        [1.38807019,1.850220317,0],
        [3.06407232,3.005305973,0],
        [7.627531214,2.759262235,1],
        [5.332441248,2.088626775,1],
        [6.922596716,1.77106367,1],
        [8.675418651,-0.242068655,1],
        [7.673756466,3.508563011,1]]
n_inputs = len(dataset[0]) - 1
n_outputs = len(set([row[-1] for row in dataset]))
network = initialize_network(n_inputs, 2, n_outputs)
train_network(network, dataset, 0.5, 20, n_outputs)
for layer in network:
        print(layer)
```

```
>epoch=0, lrate=0.500, error=6.350
>epoch=1, lrate=0.500, error=5.531
>epoch=2, lrate=0.500, error=5.221
>epoch=3, lrate=0.500, error=4.951
>epoch=4, lrate=0.500, error=4.519
>epoch=5, lrate=0.500, error=4.173
>epoch=6, lrate=0.500, error=3.835
>epoch=7, lrate=0.500, error=3.506
>epoch=8, lrate=0.500, error=3.192
>epoch=9, lrate=0.500, error=2.898
>epoch=10, lrate=0.500, error=2.626
>epoch=11, lrate=0.500, error=2.377
>epoch=12, lrate=0.500, error=2.153
>epoch=13, lrate=0.500, error=1.953
>epoch=14, lrate=0.500, error=1.774
>epoch=15, lrate=0.500, error=1.614
>epoch=16, lrate=0.500, error=1.472
>epoch=17, lrate=0.500, error=1.346
>epoch=18, lrate=0.500, error=1.233
>epoch=19, lrate=0.500, error=1.132
[{'weights': [-1.4688375095432327, 1.850887325439514, 1.0858178629550297], 'output': 0.0299E
3625}, {'weights': [0.37711098142462157, -0.0625909894552989, 0.2765123702642716], 'output':
52850863837}]
[{'weights': [2.515394649397849, -0.3391927502445985, -0.9671565426390275], 'output': 0.2364
87}, {'weights': [-2.5584149848484263, 1.0036422106209202, 0.42383086467582715], 'output': C
6437354}]
```

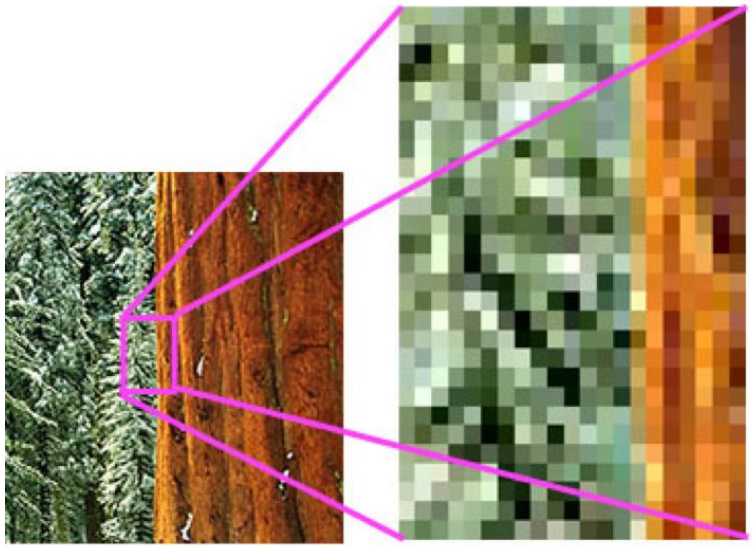
# 딥러닝의 구조적 이해 Optimizer, Learning Rate, Batch Size 의미

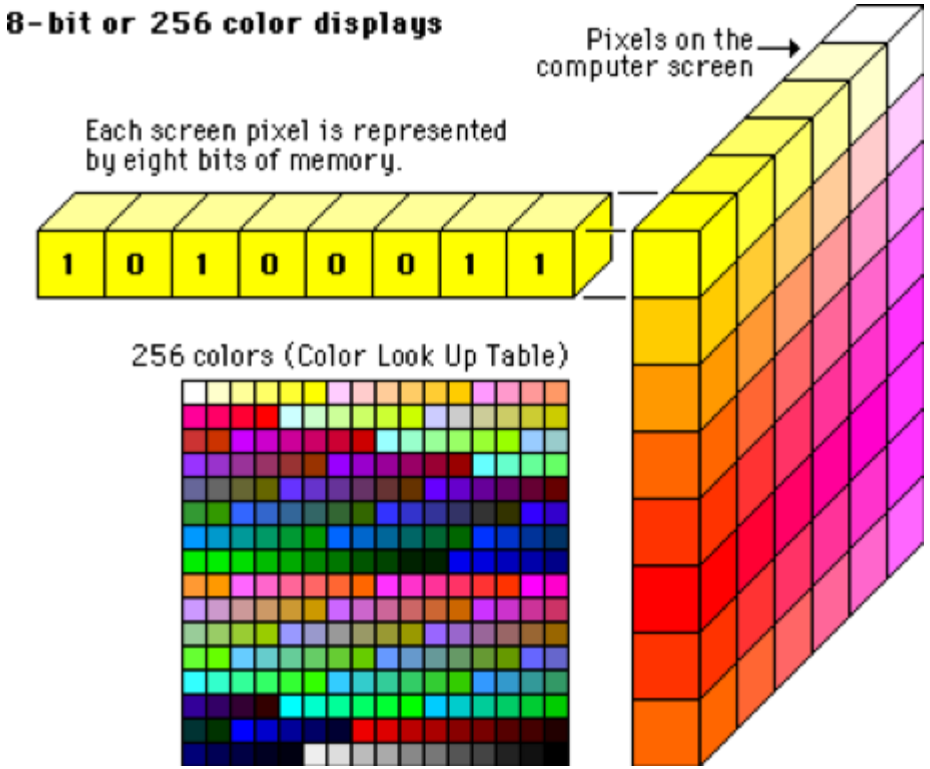



## Link

Tinker With a Neural Network
http://playground.tensorflow.org/#activation=relu&regularization=L2&batchSize=10&dataset=spiral&regDataset=reg-plane&learningRate=0.01&regularizationRate=0.001&noise=0&networkShape=8,6,3,2&seed=0.88358&showTestData=false&discretize=false&percTrainData=80&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

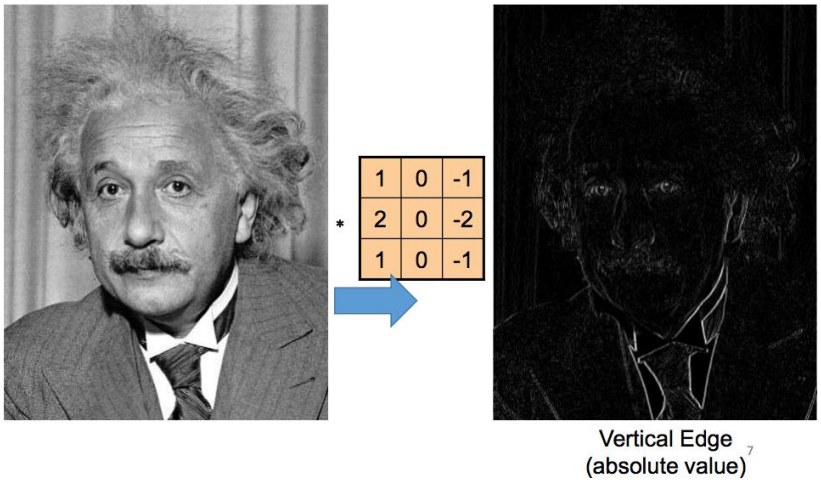




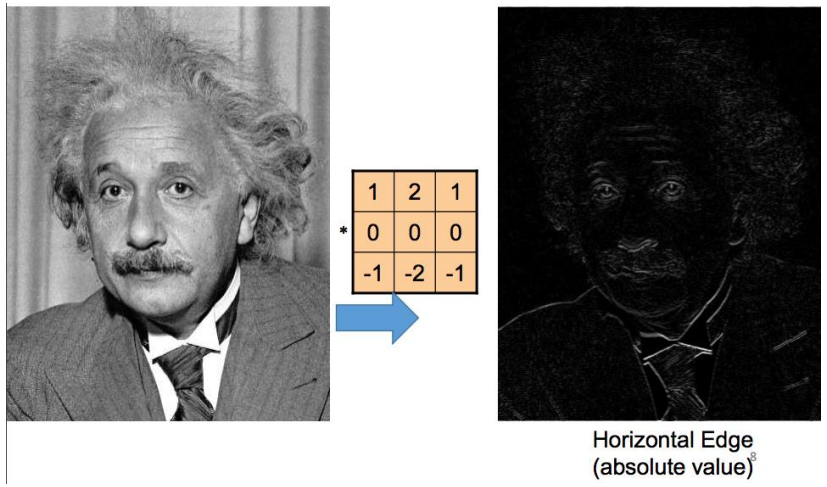

Michelle Venables, 2019, An Overview of Computer Vision, towardsdatascience.com/an-overview-of-computer-vision-1f75c2ab1b66

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

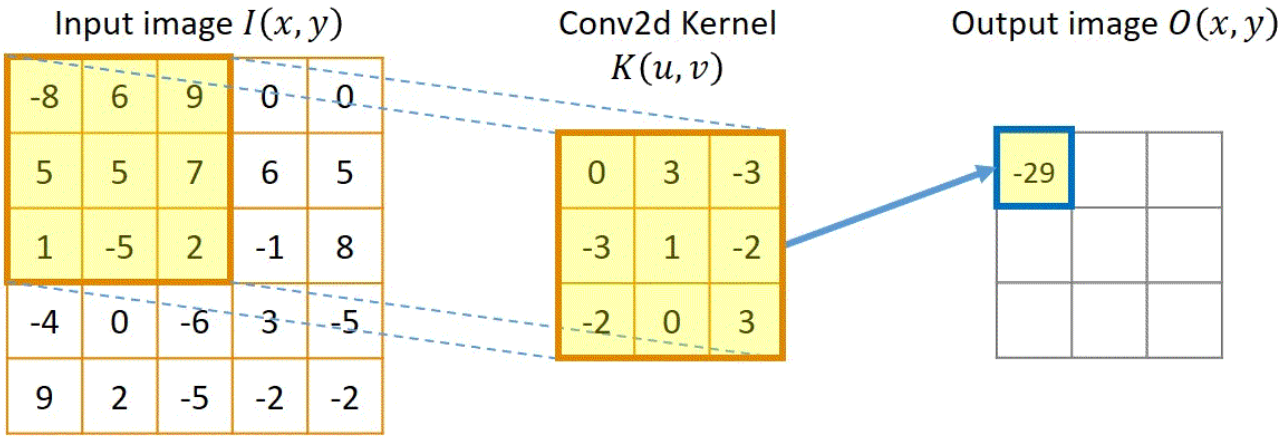
## Filter




Vertical Edge
(absolute value)



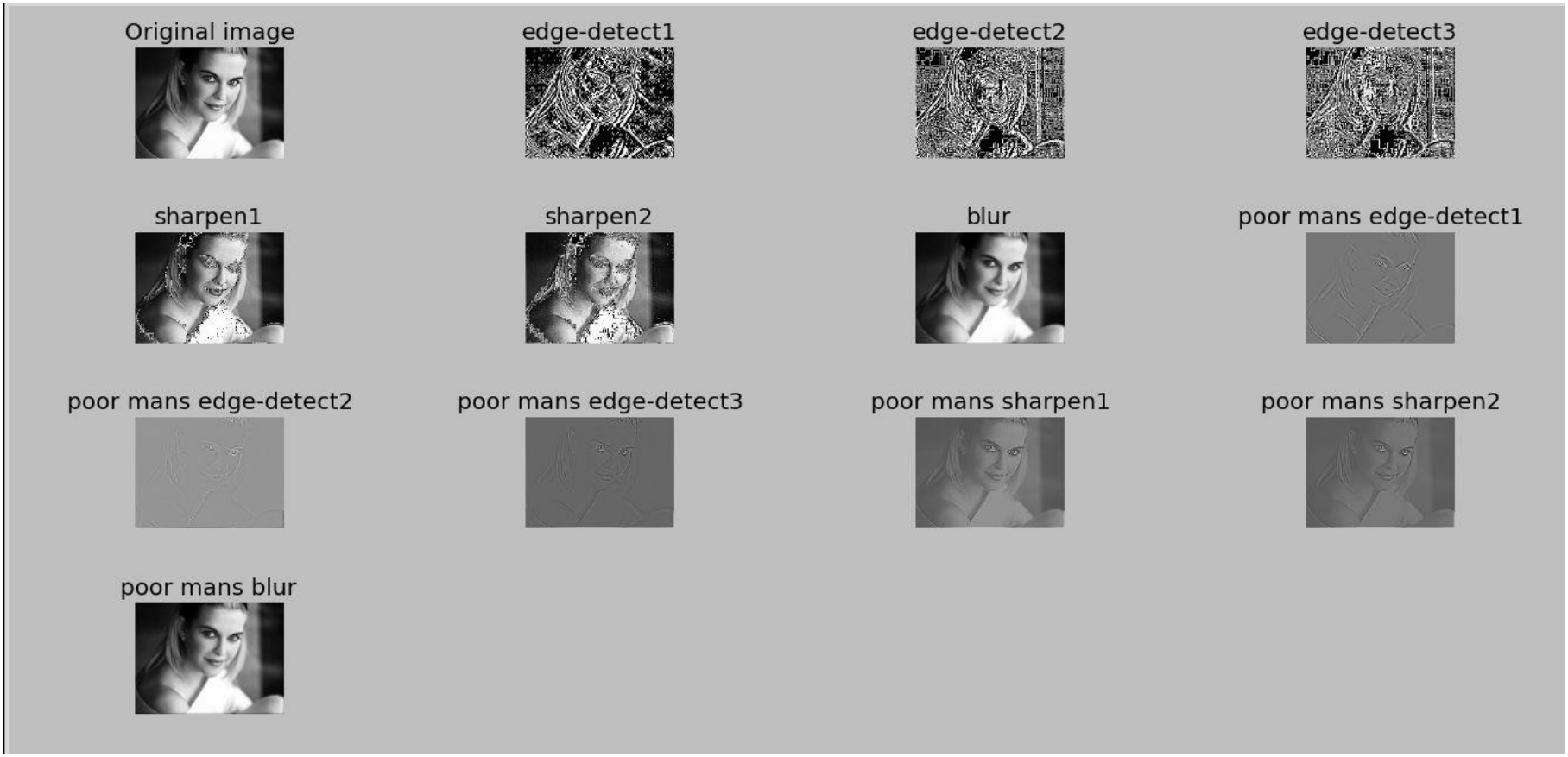

Horizontal Edge
(absolute value)




$$G_{out}(x,y) = \omega_{out} * F(x,y) = \left( \sum_{\delta x=-k_i}^{k_i} \sum_{\delta y=-k_j}^{k_j} \omega_{out}(\delta x, \delta y) \cdot F(x+\delta x, y+\delta y) \right) + \omega_{out_{bias}}$$

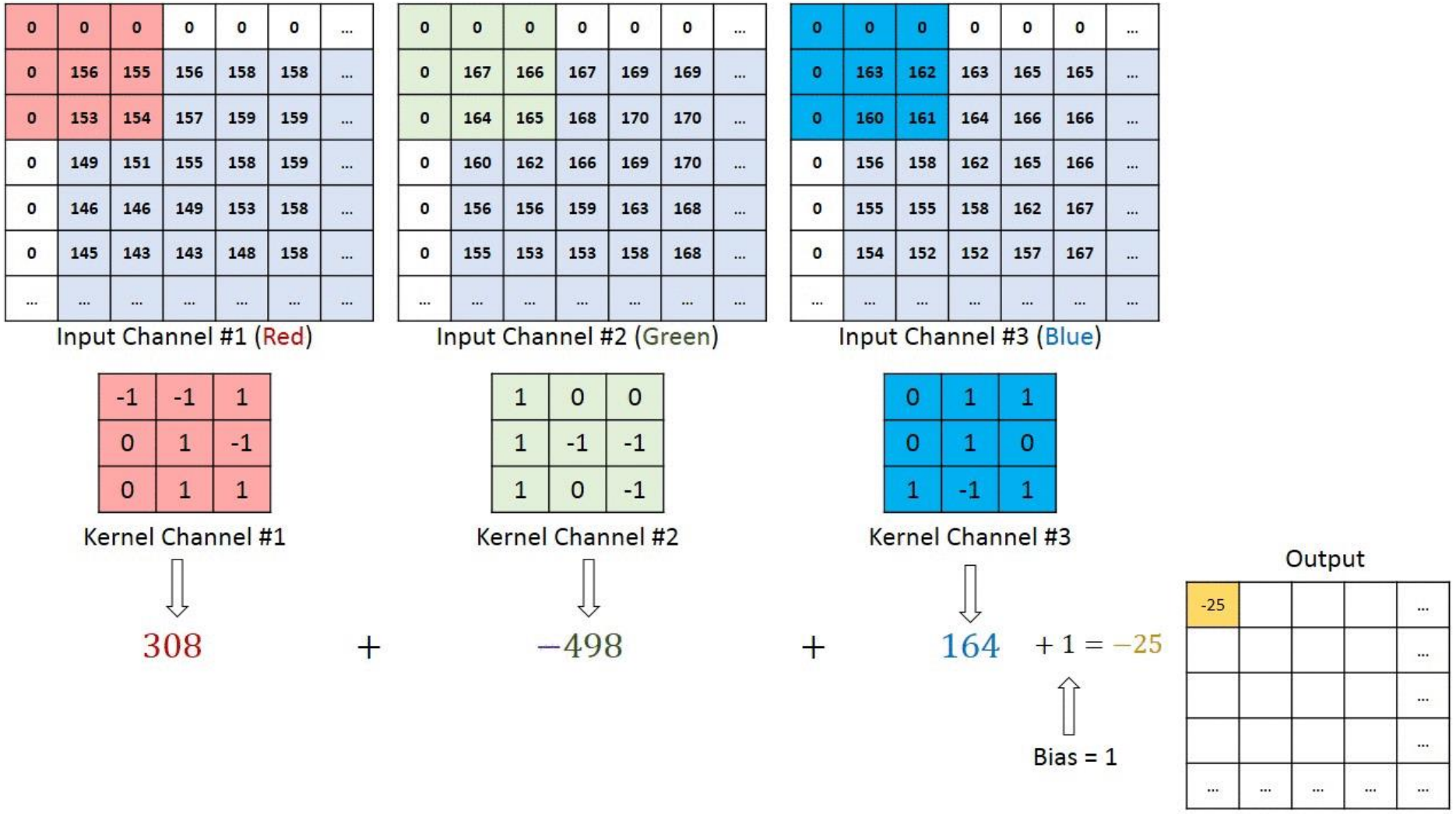

Toronto university, Convolutional Neural Networks, https://www.cs.toronto.edu/~lczhang/360/lec/w04/convnet.html
Xiyun Song, 2020, Understanding Artificial Neural Networks with Hands-on Experience - Part 2. Convolution, Its Gradients and Custom Implementations. https://coolgpu.github.io/coolgpu_blog/github/pages/2020/10/04/convolution.html

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이



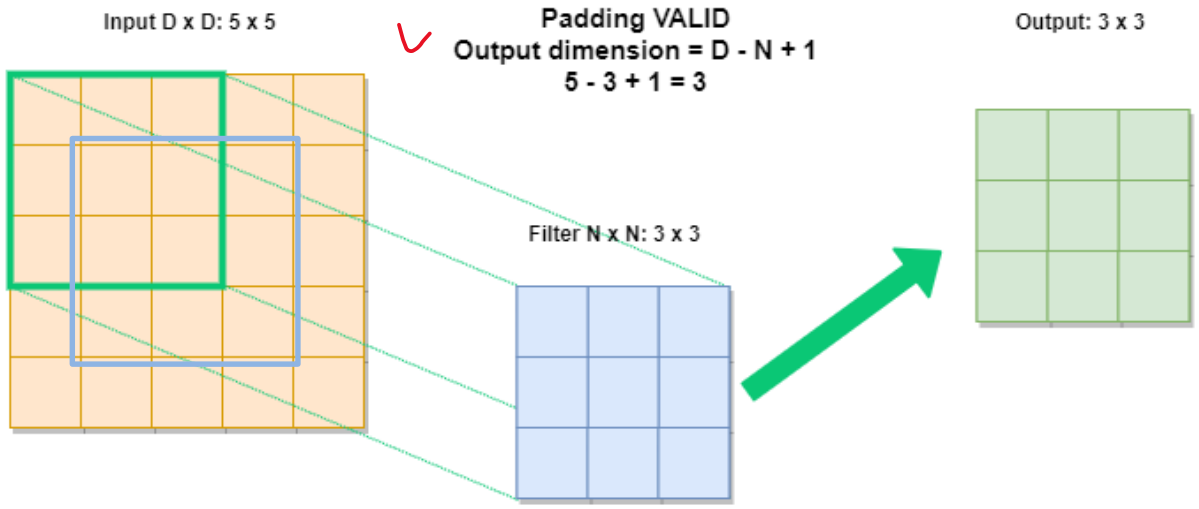

Swdevperestroika, 2014, Image processing with Python, Numpy & Scipy – image convolution,
https://softwaredevelopmentperestroika.wordpress.com/2014/02/11/image-processing-with-python-numpy-scipy-image-convolution/

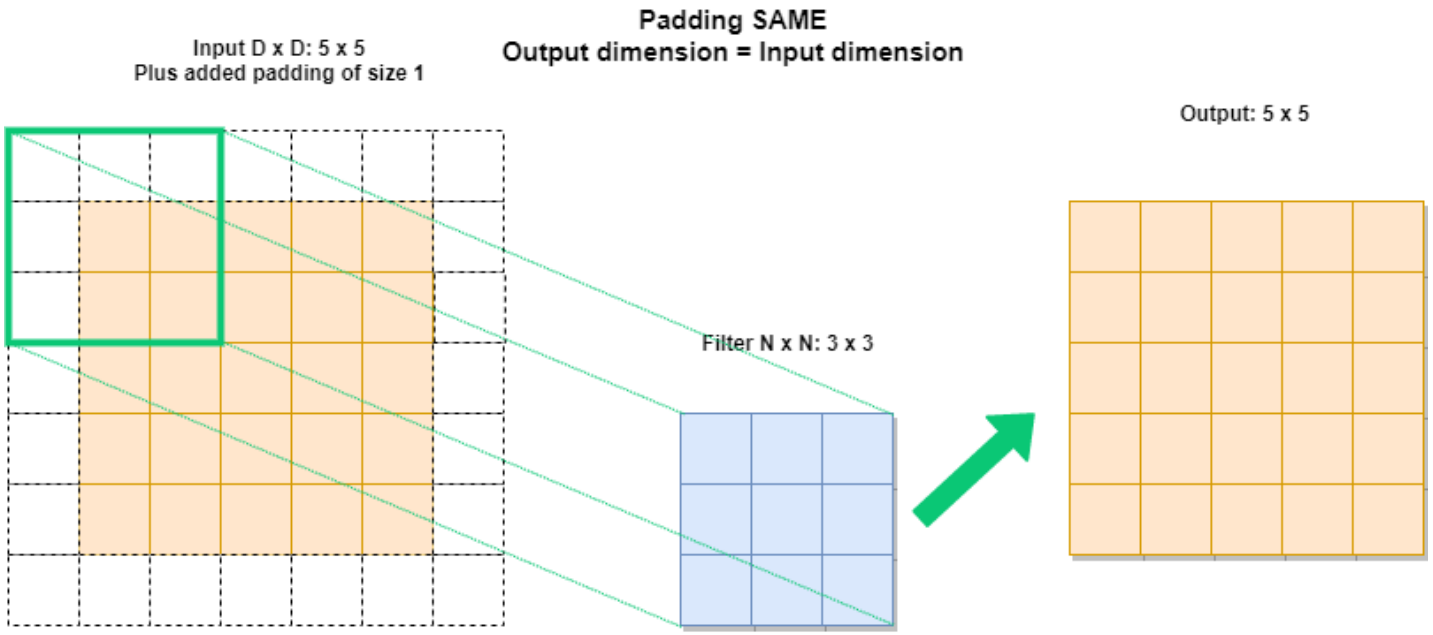
# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이




Mustafa Murat ARAT, 2019, Implementing 'SAME' and 'VALID' padding of Tensorflow in Python, https://mmuratarat.github.io/2019-01-17/implementing-padding-schemes-of-tensorflow-in-python

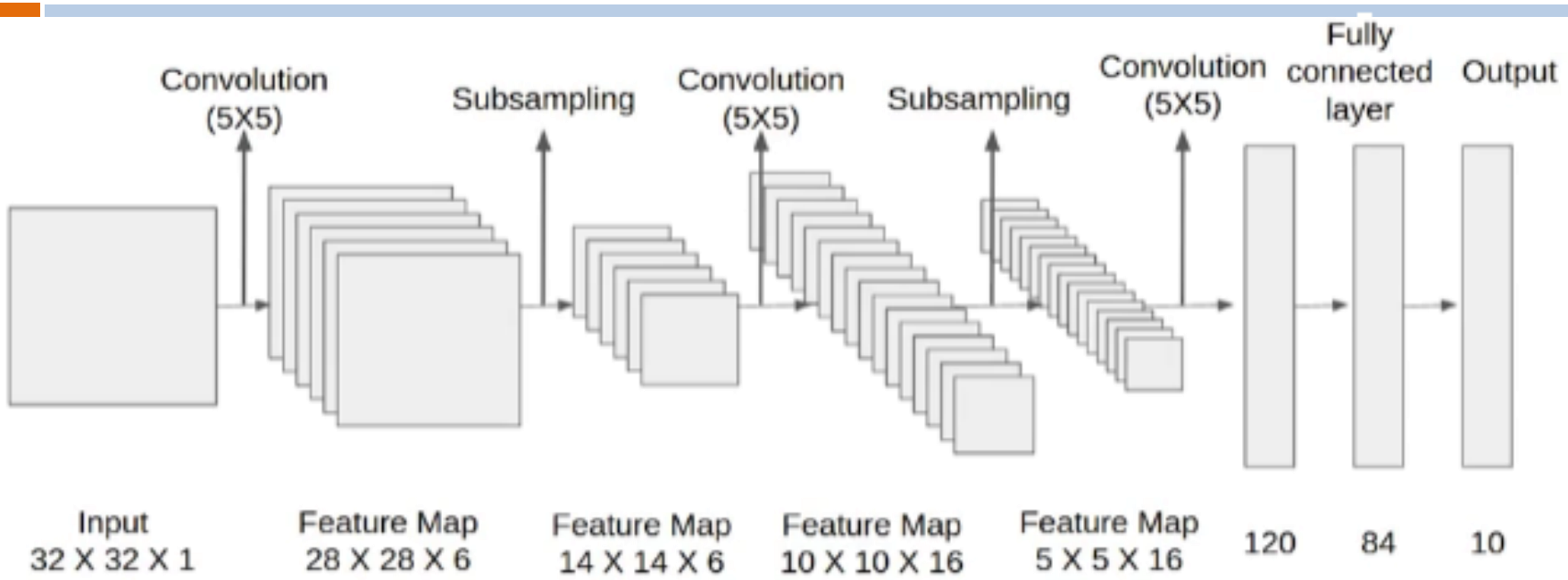
# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이




AI Geek Programmer, 2019, Convolutional neural network 2: architecture, https://aigeekprogrammer.com/convolutional-neural-network-image-recognition-part-2/

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이




| Layer | # filters / neurons | Filter size | Stride | Size of feature map | Activation function |
|---|---|---|---|---|---|
| Input | - | - | - | 32 X 32 X 1 | |
| Conv 1 | 6 | 5 * 5 | 1 | 28 X 28 X 6 | tanh |
| Avg. pooling 1 | | 2 * 2 | 2 | 14 X 14 X 6 | |
| Conv 2 | 16 | 5 * 5 | 1 | 10 X 10 X 16 | tanh |
| Avg. pooling 2 | | 2 * 2 | 2 | 5 X 5 X 16 | |
| Conv 3 | 120 | 5 * 5 | 1 | 120 | tanh |
| Fully Connected 1 | - | - | - | 84 | tanh |
| Fully Connected 2 | - | - | - | 10 | Softmax |

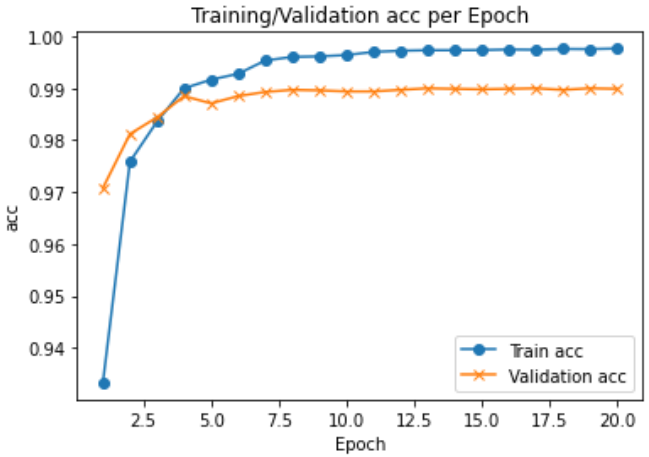
$$O = \frac{I - F + 2P}{S} + 1$$

$$O = \frac{I - F}{S} + 1$$

Shipra Saxena, 2021, The Architecture of Lenet-5, analyticsvidhya (www.analyticsvidhya.com/blog/2021/03/the-architecture-of-lenet-5/)

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

```
#Instantiate an empty model
model = Sequential()

# C1 Convolutional Layer
model.add(Conv2D(6, kernel_size=(5, 5), strides=(1, 1), activation='tanh', input_shape=input_shape, padding='same'))

# S2 Pooling Layer
model.add(AveragePooling2D(pool_size=(2, 2), strides=2, padding='valid'))

# C3 Convolutional Layer
model.add(Conv2D(16, kernel_size=(5, 5), strides=(1, 1), activation='tanh', padding='valid'))

# S4 Pooling Layer
model.add(AveragePooling2D(pool_size=(2, 2), strides=2, padding='valid'))

# C5 Fully Connected Convolutional Layer
model.add(Conv2D(120, kernel_size=(5, 5), strides=(1, 1), activation='tanh', padding='valid'))

#Flatten the CNN output so that we can connect it with fully connected layers
model.add(Flatten())

# FC6 Fully Connected Layer
model.add(Dense(84, activation='tanh'))

# Output Layer with softmax activation
model.add(Dense(10, activation='softmax'))

# print the model summary
model.summary()
```



# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

## ImageNet Classification with Deep Convolutional Neural Networks

**Alex Krizhevsky**
University of Toronto
kriz@cs.utoronto.ca

**Ilya Sutskever**
University of Toronto
ilya@cs.utoronto.ca

**Geoffrey E. Hinton**
University of Toronto
hinton@cs.utoronto.ca

### Abstract

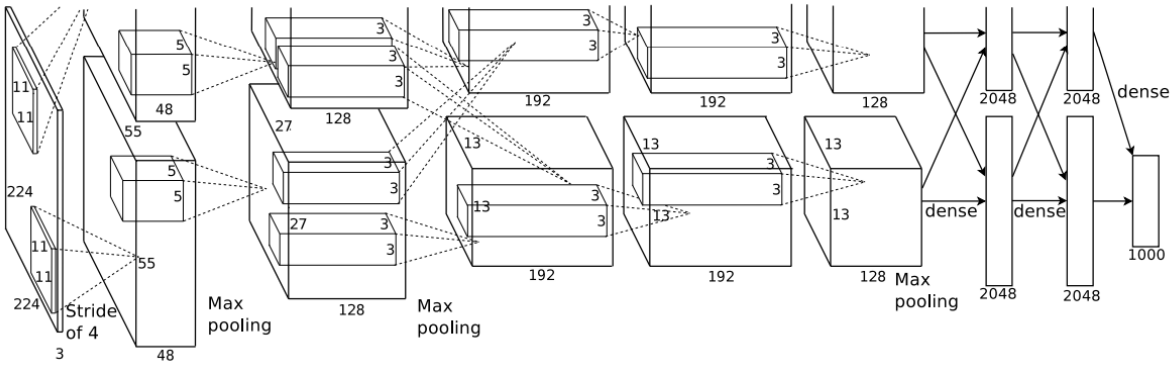
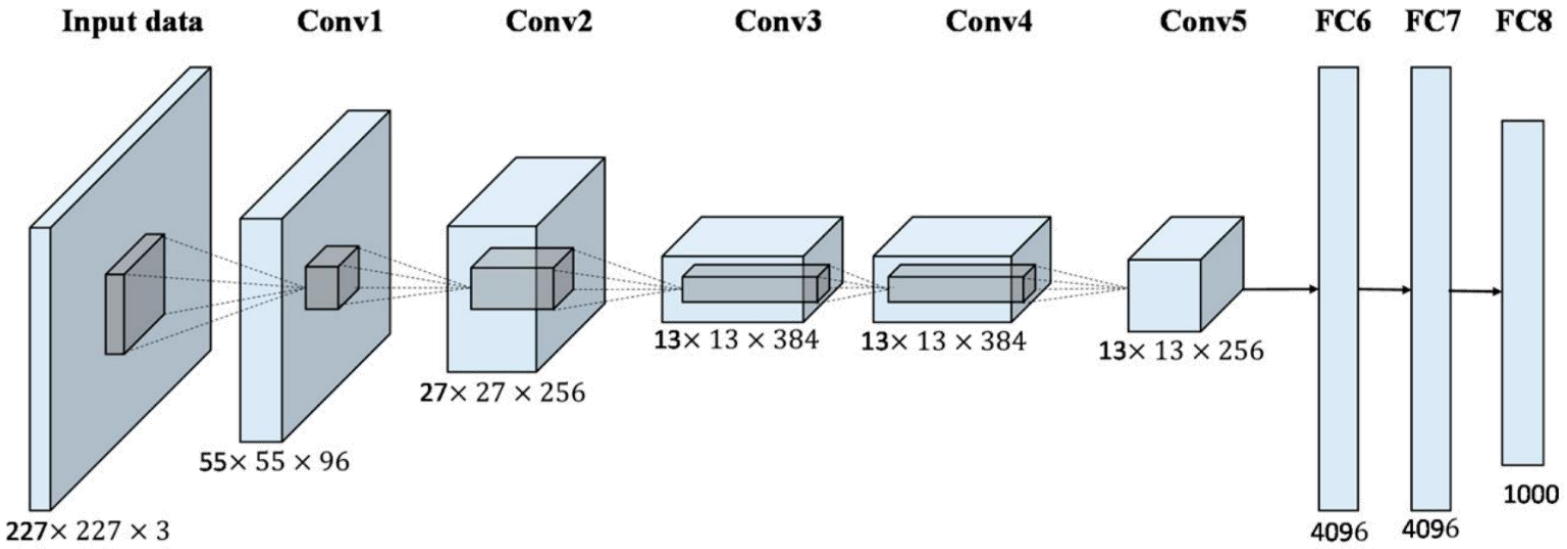



Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

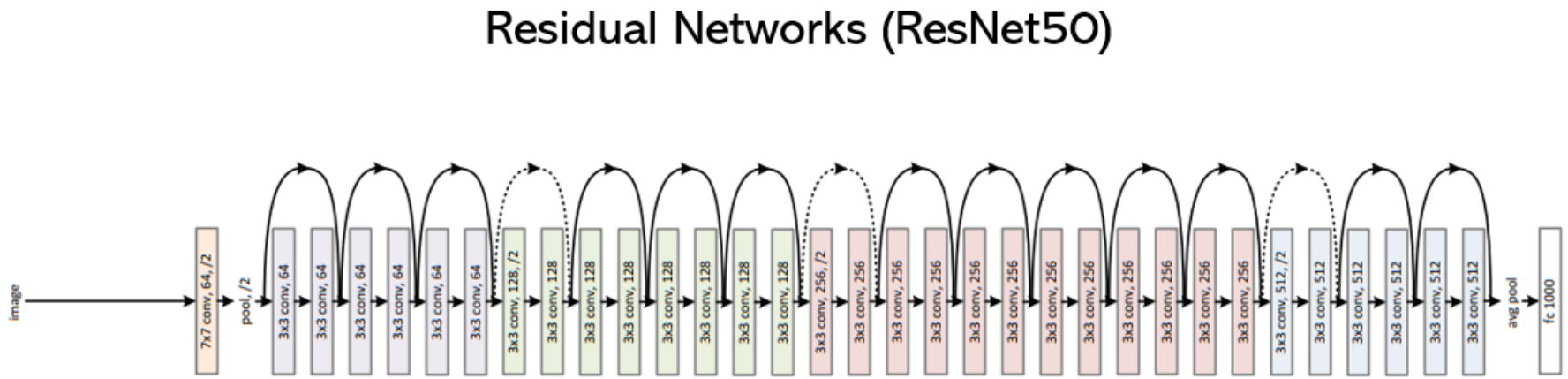
# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이



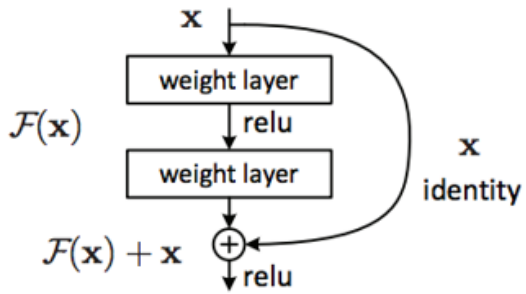

```
model = Sequential(name="Alexnet")

# 1st layer (conv + pool + batchnorm)
model.add(Conv2D(filters= 96, kernel_size= (11,11), strides=(4,4), padding='valid', kernel_regularizer
=l2(0.0005),
input_shape = (227,227,3)))
model.add(Activation('relu'))
model.add(MaxPool2D(pool_size=(3,3), strides= (2,2), padding='valid'))
model.add(BatchNormalization())
```

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

Vanishing Gradient problem




relu(f(x) + x)

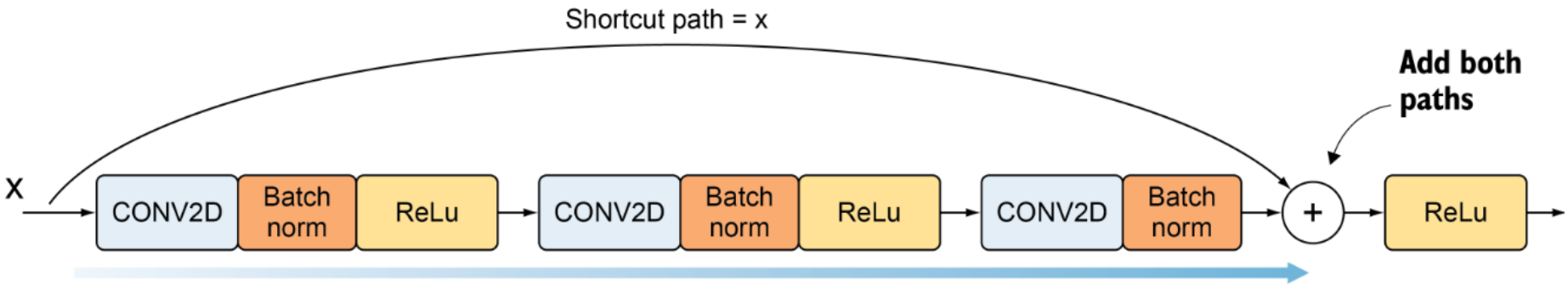



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, 2015, Deep Residual Learning for Image Recognition, jananisbabu.github.io/ResNet50_From_Scratch_Tensorflow

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

## Vanishing Gradient problem

relu(f(x) + x)

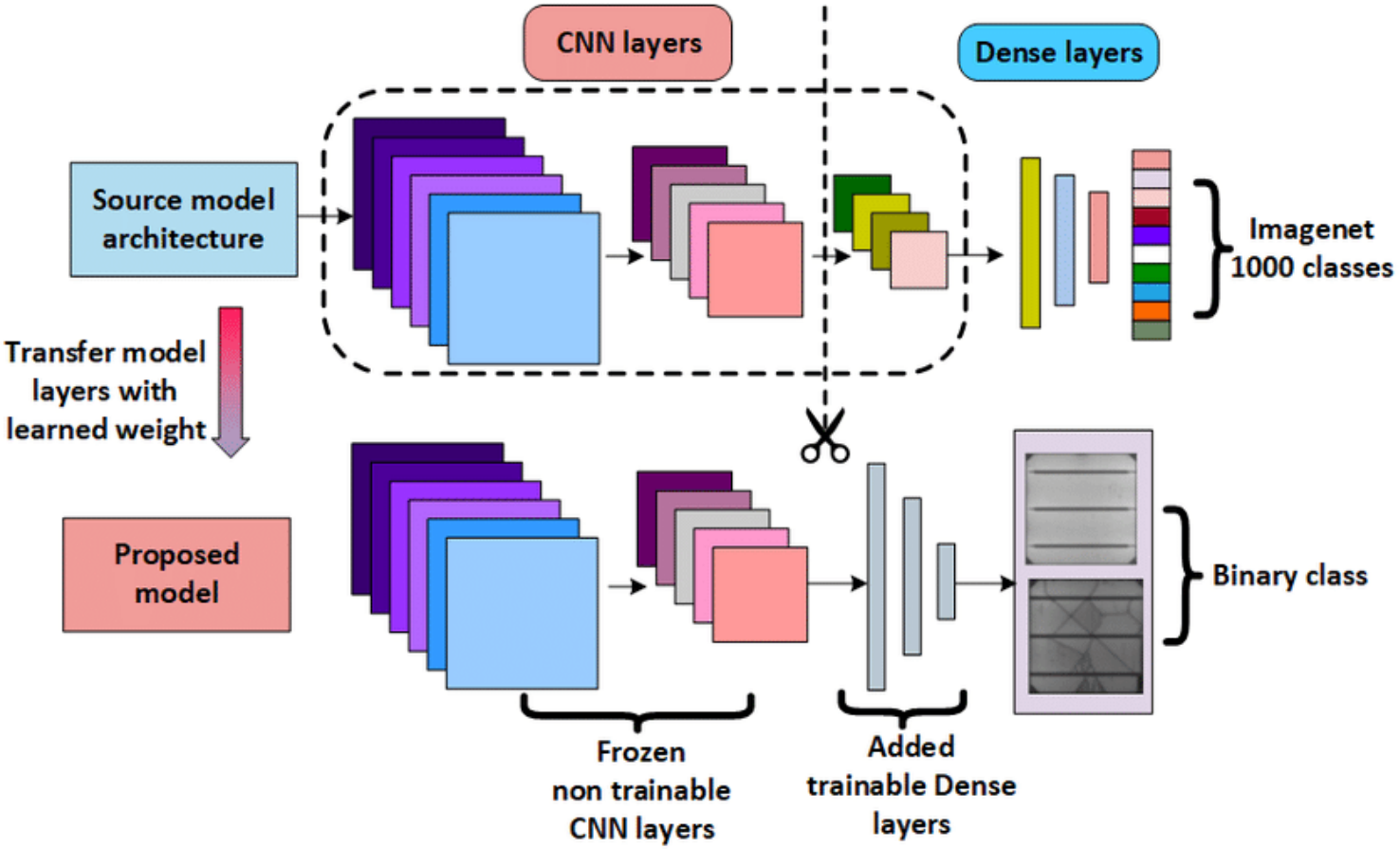



```
X_shortcut = X
F = 32

X = Conv2D(filters = F, kernel_size = (3, 3), strides = (1,1))(X)
X = Activation('relu')(X)
X = Conv2D(filters = F, kernel_size = (3, 3), strides = (1,1))(X)

X = Add()([X, X_shortcut])

X = Activation('relu')(X)
```

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

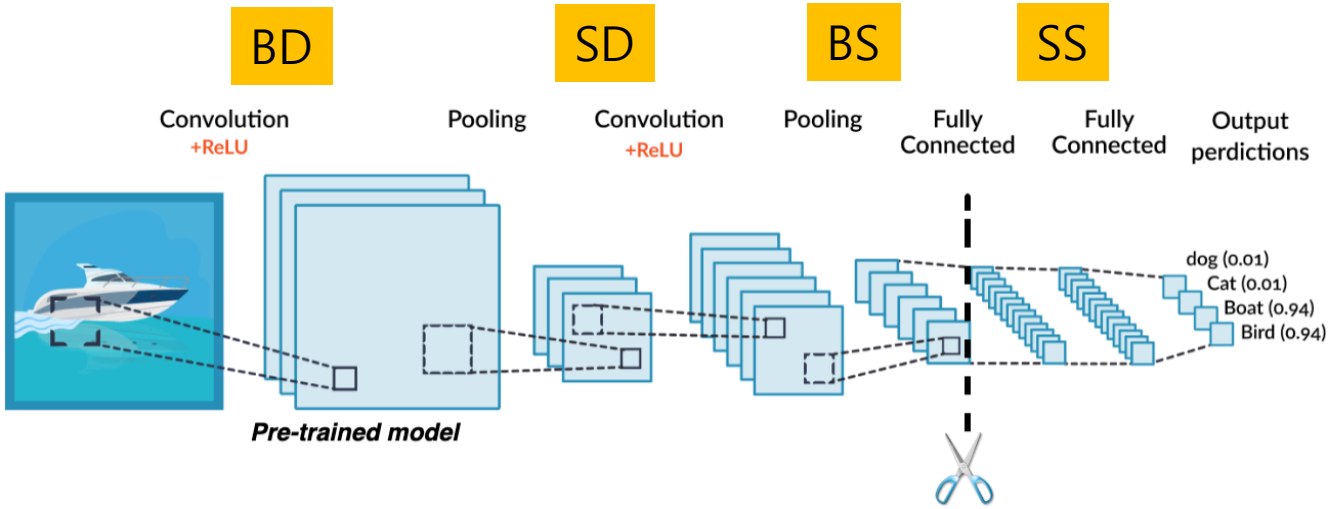
Transfer learning

Lack of data
Excessive computational demands




Proposed framework of transfer learning model. | Download Scientific Diagram

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이




SS. Small target dataset size, Similar domains

Feature Extraction (fixed 90 ~ 100%) → Your Classification layer

SD. Small target dataset size, Different domains

Feature Extraction (fixed 30 ~ 50%) → Your Classification layer

BS. Big target dataset size, Similar domains

Feature Extraction (fixed 60 ~ 80%) → Your Classification layer

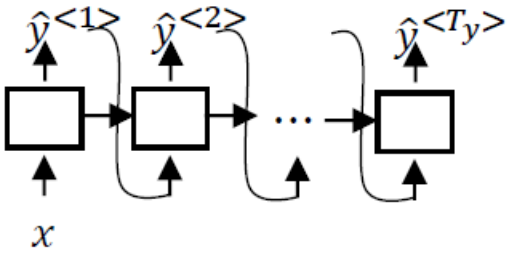
BD. Big target dataset size, Different domains

Feature Extraction → Your Classification layer

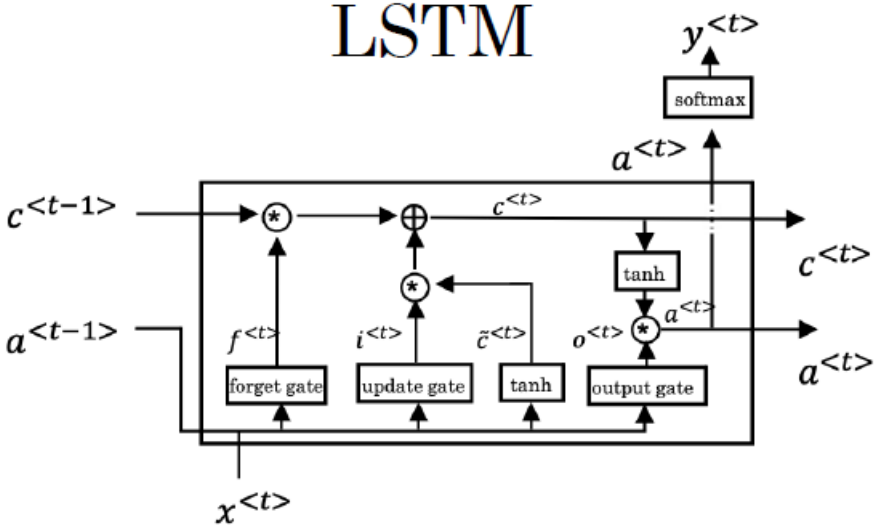
Proposed framework of transfer learning model. | Download Scientific Diagram

# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

RNN

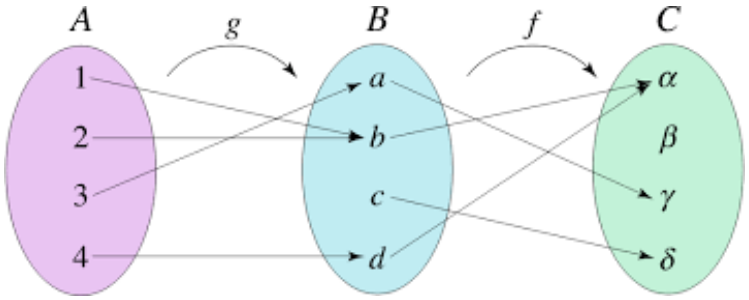


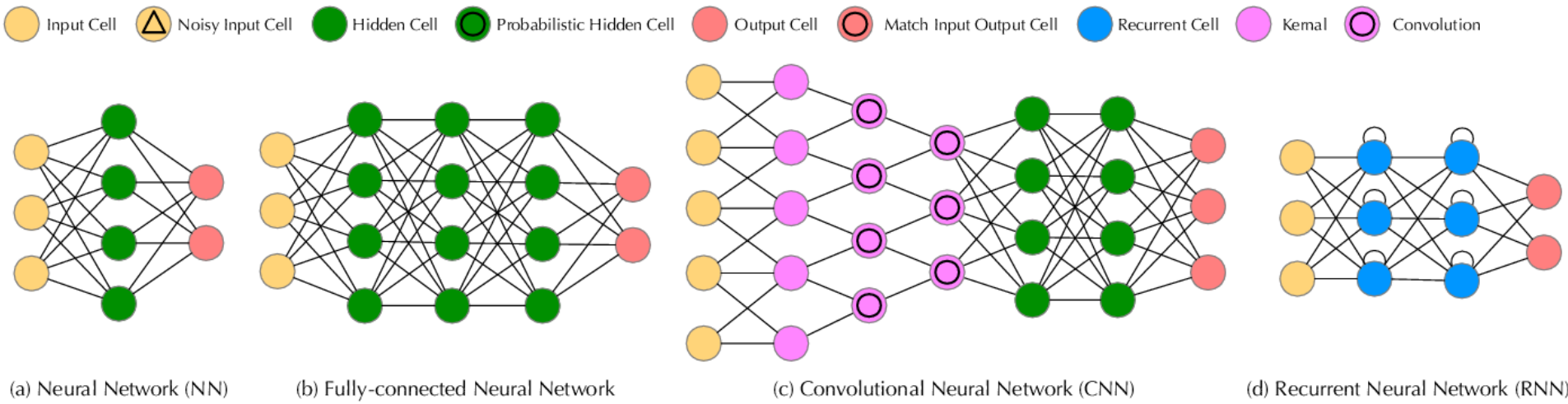

LSTM



# 딥러닝의 구조적 이해 CNN/RNN/LSTM의 개념적 차이

Model = {target loss fun, differential gradient descent func, input feature, output feature}



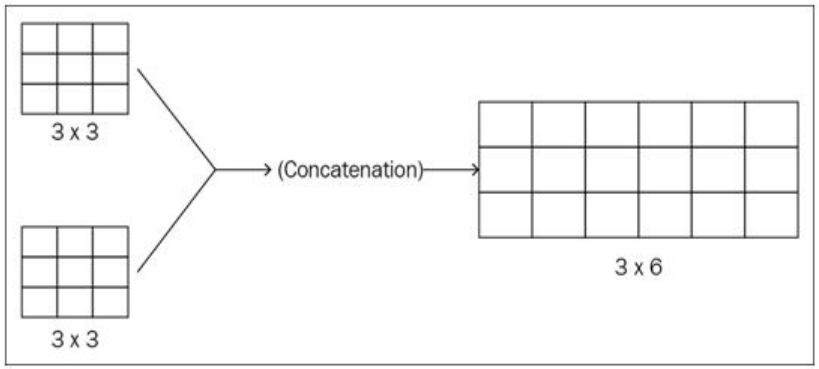

Saravanan Thirumuruganathan, N. Tang, M. Ouzzani, 2018, Data Curation with Deep Learning [Vision]: Towards Self Driving Data Curation, ArXiv

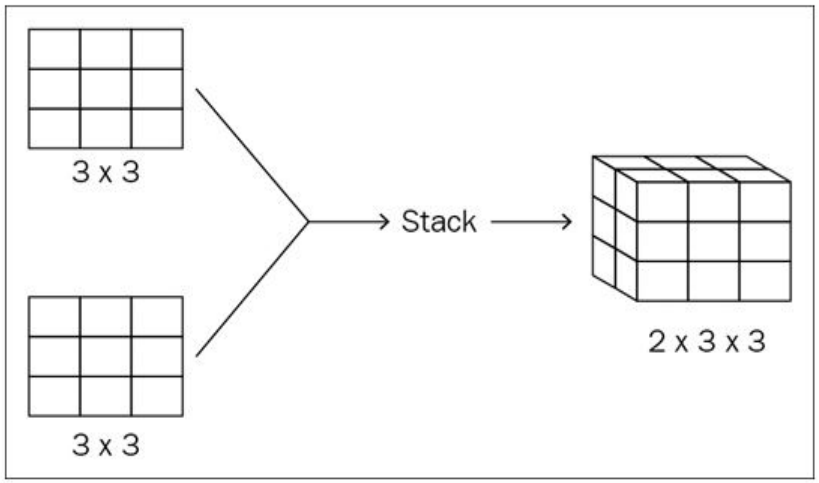
# 딥러닝의 구조적 이해 Tensorflow vs Pytorch 비교

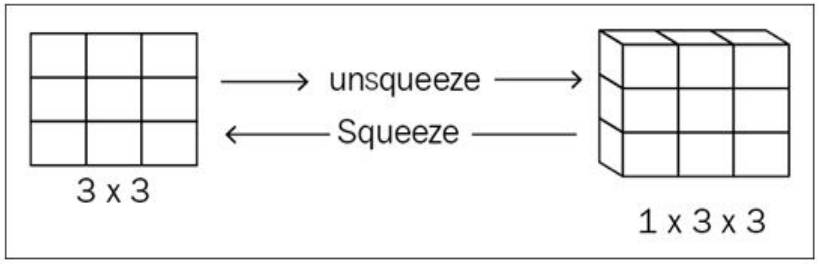
| Type | Scalar | Vector | Matrix | Tensor |
|---|---|---|---|---|
| Definition | a single number | an array of numbers | 2-D array of numbers | k-D array of numbers |
| Notation | $x$ | $\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ | $\boldsymbol{X} = \begin{bmatrix} X_{1,1} & X_{1,2} & \cdots & X_{1,n} \\ X_{2,1} & X_{2,2} & \cdots & X_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ X_{m,1} & X_{m,2} & \cdots & X_{m,n} \end{bmatrix}$ | $\mathbf{X}$ $X_{i,j,k}$ |
| Example | 1.333 | $\boldsymbol{x} = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ 9 \end{bmatrix}$ | $\boldsymbol{X} = \begin{bmatrix} 1 & 2 & \cdots & 4 \\ 5 & 6 & \cdots & 8 \\ \vdots & \vdots & \vdots & \vdots \\ 13 & 14 & \cdots & 16 \end{bmatrix}$ | $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} & \begin{bmatrix} 10 & 20 & 30 \\ 50 & 60 & 80 \\ 80 & 90 & \end{bmatrix} & \begin{bmatrix} 100 & 200 & 300 \\ 00 & 600 \\ 00 & 900 \end{bmatrix} \end{bmatrix}$ |
| Python code example | x = np.array(1.333) | x = np.array([1,2,3, 4,5,6, 7,8,9]) | x = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]]) | x = np.array([[[1, 2, 3], [4, 5, 6], [7, 8, 9]], [[10, 20, 30], [40, 50, 60], [70, 80, 90]], [[100, 200, 300], [400, 500, 600], [700, 800, 900]]]) |
| Visualization | |  |  |  3-D Tensor |

https://www.kaggle.com/code/jarfo1/a-world-of-tensors-and-differentiable-computing

# 딥러닝의 구조적 이해 Tensorflow vs Pytorch 비교

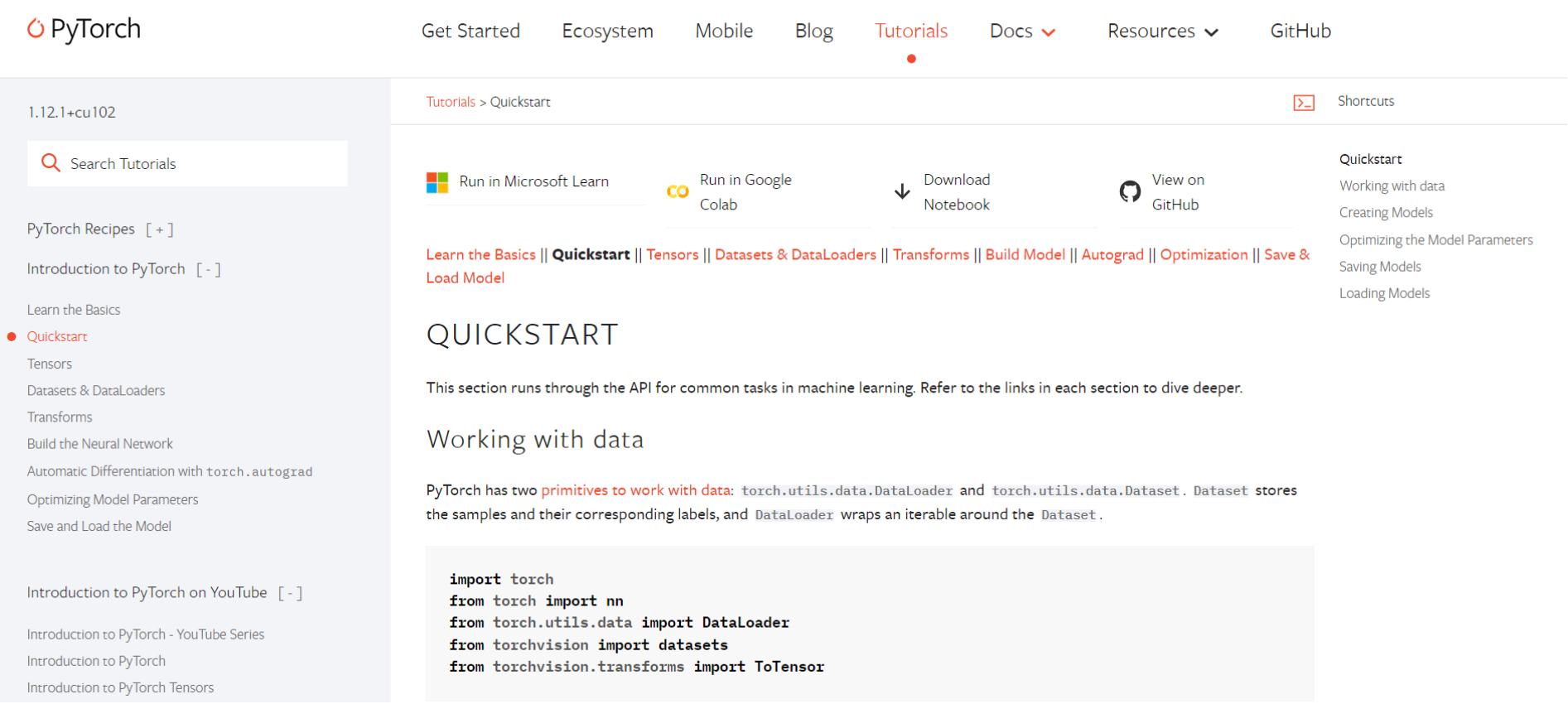


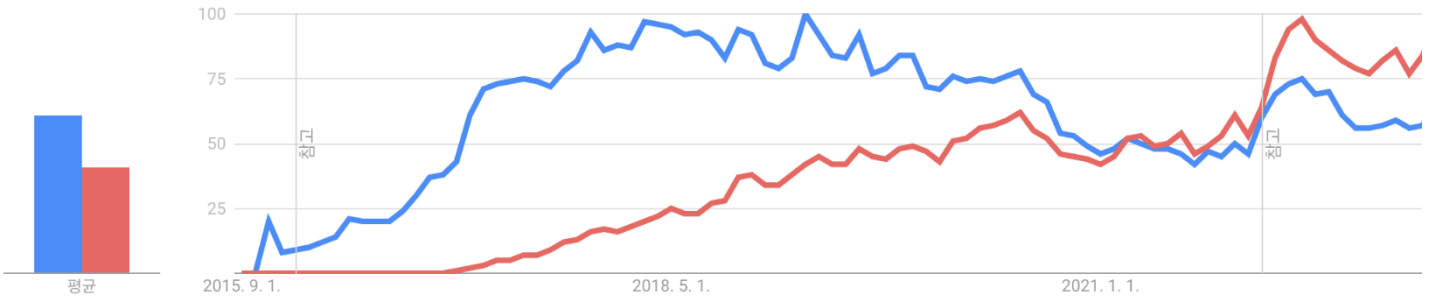

https://www.codementor.io/@packt/how-to-perform-basic-operations-in-pytorch-code-10al39a4c4

# 딥러닝의 구조적 이해 Tensorflow vs Pytorch 비교




https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

# 딥러닝의 구조적 이해 Tensorflow vs Pytorch 비교




| | Keras | PyTorch | TensorFlow |
|---|---|---|---|
| API Level | High | Low | High and Low |
| Architecture | Simple, concise, readable | Complex, less readable | Not easy to use |
| Datasets | Smaller datasets | Large datasets, high perf ormance | Large datasets, high perf ormance |
| Debugging | Simple network, so debugging is not ofte n needed | Good debugging capabil ities | Difficult to conduct debu gging |
| Support | User Friendly<br>Rapid Prototyping | Model Hub<br>Recently Research Tech | Tensorflow Lite, Serving |
| Popularity | Third | Second | First |
| Speed | Slow, low performance | Fast, high-performance | Fast, high-performance |
| Written In | Google<br>Python | Meta<br>Lua | Google<br>C++, CUDA, Python |

https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

# 딥러닝의 구조적 이해 Tensorflow vs Pytorch 비교

1 Load data 2 Define model 3 Train model 4 Evaluate model

## General

PyTorch is a open source machine learning framework. It uses **torch.Tensor** – multi-dimensional matrices – to process. A core feature of neural networks in PyTorch is the autograd package, which provides automatic derivative calculations for all operations on tensors.

| | | | |
|---|---|---|---|
| **import torch** | Root package | **torch.randn(*size)** | Create random tensor |
| **import torch.nn as nn** | Neural networks | **torch.Tensor(L)** | Create tensor from list |
| **from torchvision import datasets, models, transforms** | Popular image datasets, architectures & transforms | **tnsr.view(a,b, ...)** | Reshape tensor to size (a, b, ...) |
| **import torch.nn.functional as F** | Collection of layers, activations & more | **requires_grad=True** | tracks computation history for derivative calculations |

## Layers

**nn.Linear(m, n)**: Fully Connected layer (or dense layer) from m to n neurons

**nn.ConvXd(m, n, s)**: X-dimensional convolutional layer from m to n channels with kernel size s; X ∈ {1, 2, 3}

**nn.Flatten()**: Flattens a contiguous range of dimensions into a tensor

**nn.MaxPoolXd(s)**: X-dimensional pooling layer with kernel size s; X ∈ {1, 2, 3}

**nn.Dropout(p=0.5)**: Randomly sets input elements to zero during training to prevent overfitting

**nn.BatchNormXd(n)**: Normalizes a X-dimensional input batch with n features; X ∈ {1, 2, 3}

**nn.Embedding(m, n)**: Lookup table to map dictionary of size m to embedding vector of size n

**nn.RNN/LSTM/GRU**: Recurrent networks connect neurons of one layer with neurons of the same or a previous layer

*torch.nn offers a bunch of other building blocks.*
*A list of state-of-the-art architectures can be found at https://paperswithcode.com/sota.*

PyTorch Tutorial for Reshape, Squeeze, Unsqueeze, Flatten and View - MLK - Machine Learning Knowledge

Link - https://www.stefanseegerer.de/media/pytorch-cheatsheet-EN.pdf

# Define model

There are several ways to define a neural network in PyTorch, e.g. with **nn.Sequential** (a), as a class (b) or using a combination of both.

```
model = nn.Sequential(
    nn.Conv2D(■,■,■)
    nn.ReLU()
    nn.MaxPool2D(■)
    nn.Flatten()
    nn.Linear(■,■)
)
```
a

```
class Net(nn.Module):
  def __init__():
    super(Net, self).__init__()
      self.conv
        = nn.Conv2D(■,■,■)
      self.pool
        = nn.MaxPool2D(■)
      self.fc = nn.Linear(■,■)

  def forward(self, x):
      x = self.pool(
            F.relu(self.conv(x))
          )
      x = x.view(-1,■)
      x = self.fc(x)
    return x
model = Net()
```
b

# Train model

## LOSS FUNCTIONS

PyTorch already offers a bunch of different loss fuctions, e.g.:

| | |
|---|---|
| **nn.L1Loss** | Mean absolute error |
| **nn.MSELoss** | Mean squared error (L2Loss) |
| **nn.CrossEntropyLoss** | Cross entropy, e.g. for single-label classification or unbalanced training set |
| **nn.BCELoss** | Binary cross entropy, e.g. for multi-label classification or autoencoders |

## OPTIMIZATION (torch.optim)

Optimization algorithms are used to update weights and dynamically adapt the learning rate with gradient descent, e.g.:

| | |
|---|---|
| **optim.SGD** | Stochastic gradient descent |
| **optim.Adam** | Adaptive moment estimation |
| **optim.Adagrad** | Adaptive gradient |
| **optim.RMSProp** | Root mean square prop |

# GPU Training

**device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')**

If a GPU with CUDA support is available, computations are sent to the GPU with ID 0 using **model.to(device)** or **inputs, labels = data[0].to(device), data[1].to(device).**

```
import torch.optim as optim

# Define loss function
loss_fn = nn.CrossEntropyLoss()

# Choose optimization method
optimizer = optim.SGD(model.parameters(),
                      lr=0.001, momentum=0.9)

# Loop over dataset multiple times (epochs)
for epoch in range(2):
    model.train() # activate training mode
    for i, data in enumerate(train_loader, 0):
        # data is a batch of [inputs, labels]
        inputs, labels = data

        # zero gradients
        optimizer.zero_grad()

        # calculate outputs
        outputs = model(inputs)
        # calculate loss & backpropagate error
        loss = loss_fn(outputs, labels)
        loss.backward()
        # update weights & learning rate
        optimizer.step()
```

# Evaluate model

The evaluation examines whether the model provides satisfactory results on previously withheld data. Depending on the objective, different metrics are used, such as acurracy, precision, recall, F1, or BLEU.

| | |
|---|---|
| **model.eval()** | Activates evaluation mode, some layers behave differently |
| **torch.no_grad()** | Prevents tracking history, reduces memory usage, speeds up calculations |

Link - https://www.stefanseegerer.de/media/pytorch-cheatsheet-EN.pdf

# 딥러닝의 구조적 이해 Tensorflow vs PyTorch 비교

Tensorflow

```
import tensorflow as tf

x_data = [1, 2, 3, 4, 5]
y_data = [2, 4, 5, 4, 5]

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_dim=1)
])

model.compile(optimizer='sgd',
loss='mean_squared_error')

model.fit(x_data, y_data, epochs=1000,
verbose=0)

print(model.predict([10]))
```

PyTorch

```
import torch, torch.nn as nn, torch.optim as optim

x_data = torch.tensor([[1.], [2.], [3.], [4.], [5.]])
y_data = torch.tensor([[2.], [4.], [5.], [4.], [5.]])

class LinearRegressionModel(nn.Module):
    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

model = LinearRegressionModel()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for epoch in range(1000):
    optimizer.zero_grad()
    outputs = model(x_data)
    loss = criterion(outputs, y_data)
    loss.backward()
    optimizer.step()

print(model(torch.tensor([[10.]])))
```

# 딥러닝의 구조적 이해 Pytorch vs Keras 비교

Keras

```
from keras.models import Sequential
from keras.layers import Dense, Dropout,
Conv2D, MaxPool2D, Flatten
from keras.layers import
BatchNormalization

model = Sequential()
model.add(Conv2D(32, (3, 3),
activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPool2D())
model.add(Conv2D(16, (3, 3),
activation='relu'))
model.add(MaxPool2D())
model.add(Flatten())
model.add(Dense(10, activation='softmax'))

model.summary()
```

PyTorch

```
import torch

nn = torch.nn
F = torch.nn.functional

class Net(nn.Module):
  def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d(3, 32, 3)
    self.conv2 = nn.Conv2d(32, 16, 3)
    self.fc1 = nn.Linear(16 * 6 * 6, 10)
    self.pool = nn.MaxPool2d(2, 2)

def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = x.view(-1, 16 * 6 * 6)
    x = F.log_softmax(self.fc1(x), dim=-1)
    return x

model = Net()
print(model)
```

Piotr Migdal and Rafał Jakubanis, 2018.6, Keras or PyTorch as your first deep learning framework, deepsense.ai, https://deepsense.ai/keras-or-pytorch/

# Transformer와 최신 AI 모델

- Transformer 구조 이해 (Self-Attention, Positional Encoding)
- BERT, GPT 등의 동작 원리 요약
- 왜 Transformer가 표준이 되었는가?

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

딥러닝 모델은 텍스트를 바로 이해하지 못함

문장 → 토크나이징(tokenizing) → 토큰 ID → 임베딩 벡터

"I am a student" → ["I", "am", "a", "student"]

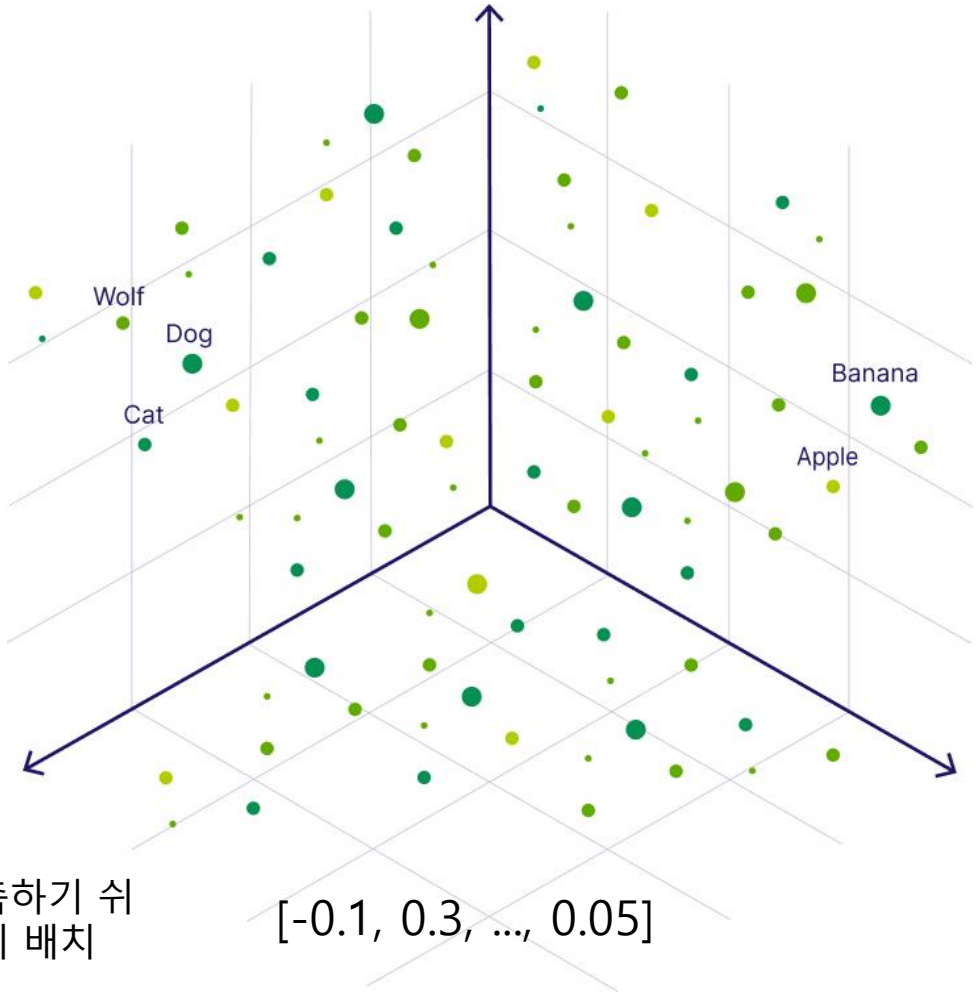
["I", "am", "a", "student"] → [101, 2023, 1037, 3076]

```
embedding = nn.Embedding(vocab_size, d_model)
token_vec = embedding(token_ids)
```

[-0.1, 0.3, ..., 0.05]

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

문장 → 토크나이징(tokenizing) → 토큰 ID → 임베딩 벡터




단어들을 "모델이 예측하기 쉬운 방식"으로 공간에 배치

[-0.1, 0.3, ..., 0.05]

cos_sim = F.cosine_similarity(embedding("apple"), embedding("banana"))

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

문장 → 토크나이징(tokenizing) → 토큰 ID → 임베딩 벡터

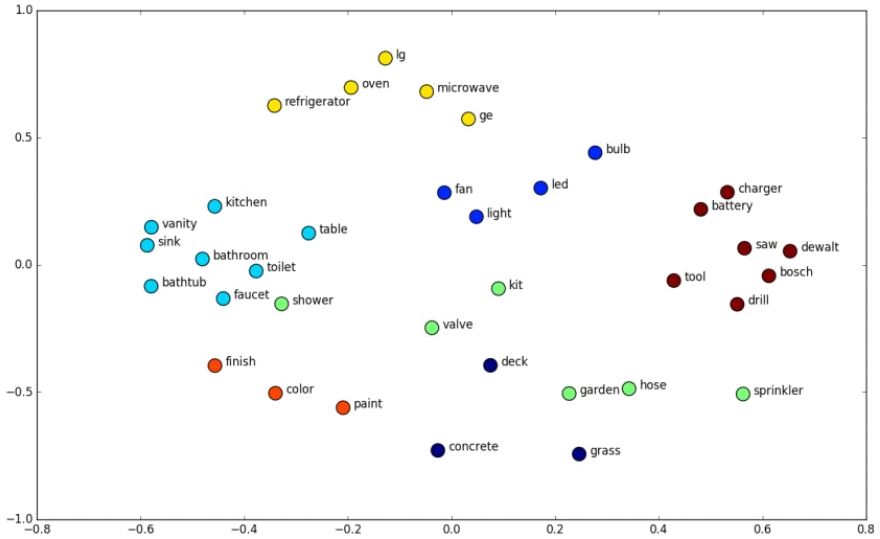
Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch

| | Femininity | Youth | Royalty |
|---|---|---|---|
| Man | 0 | 0 | 0 |
| Woman | 1 | 0 | 0 |
| Boy | 0 | 1 | 0 |
| Girl | 1 | 1 | 0 |
| Prince | 0 | 1 | 1 |
| Princess | 1 | 1 | 1 |
| Queen | 1 | 0 | 1 |
| King | 0 | 0 | 1 |
| Monarch | 0.5 | 0.5 | 1 |

Each word gets a 1x3 vector

Similar words... similar vectors

Intro to Word Embeddings and Vectors for Text Analysis.




http://suriyadeepan.github.io/

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

변환기(Transformers)
텍스트 생성 영역에서 GPT(Generative Pre-trained Transformer)와 같은 변환기 기반 모델은 방대한 양의 텍스트 데이터에서 어텐션(Attention)계산을 통해 패턴을 학습. 일관되고 맥락적으로 관련성 있는 토큰 시퀀스 학습 가능.
조건화된 이미지 생성에 사용.


**Ashish Vaswani*** Google Brain avaswani@google.com
**Noam Shazeer*** Google Brain noam@google.com
**Niki Parmar*** Google Research nikip@google.com
**Jakob Uszkoreit*** Google Research usz@google.com
**Llion Jones*** Google Research llion@google.com
**Aidan N. Gomez*** † University of Toronto aidan@cs.toronto.edu
**Łukasz Kaiser*** Google Brain lukaszkaiser@google.com
**Illia Polosukhin*** ‡ illia.polosukhin@gmail.com

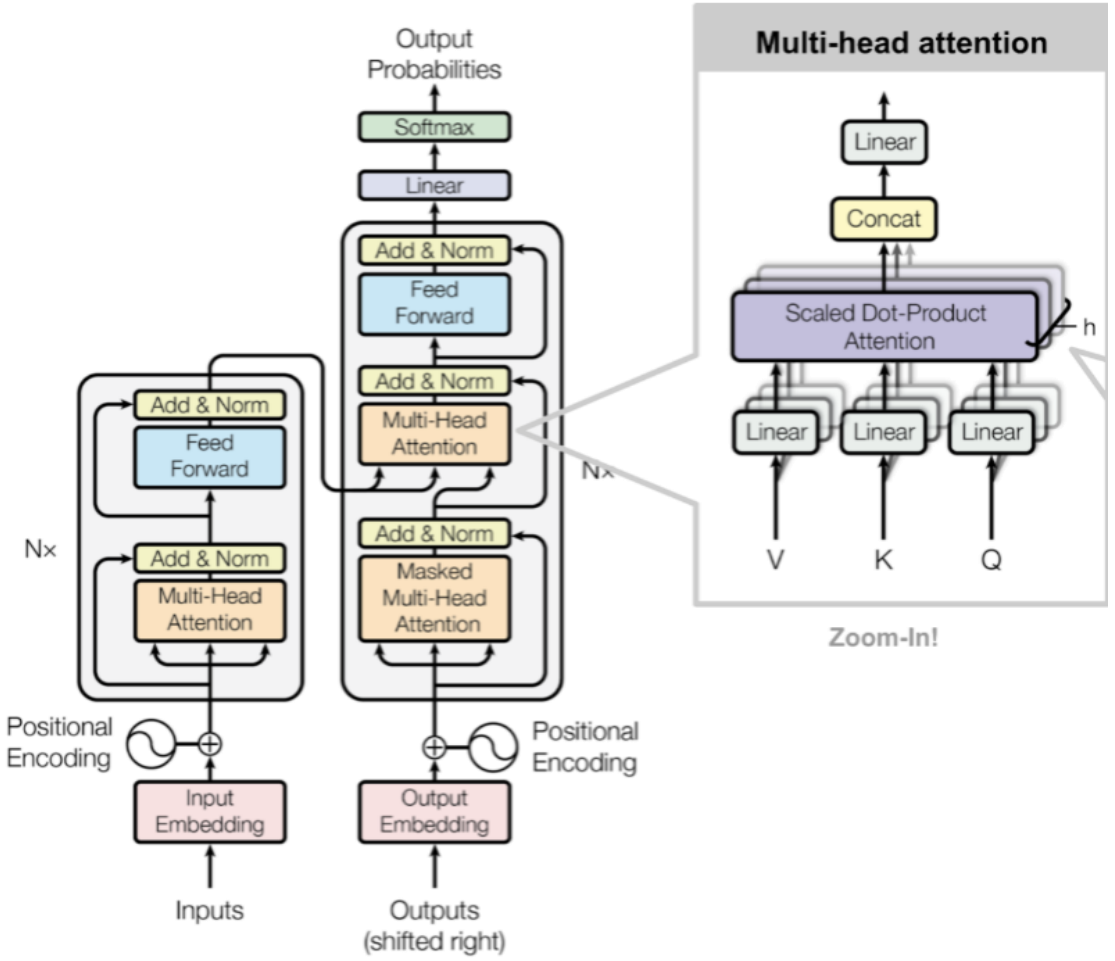


**Abstract**

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.


**1 Introduction**

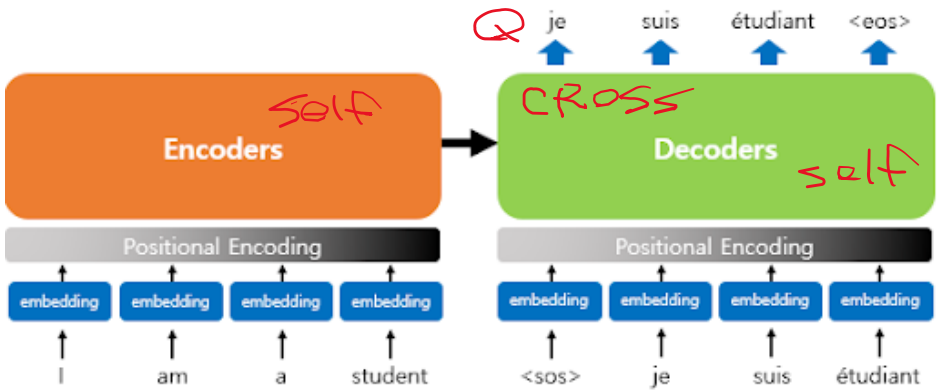
Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [29, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [31, 21, 13].

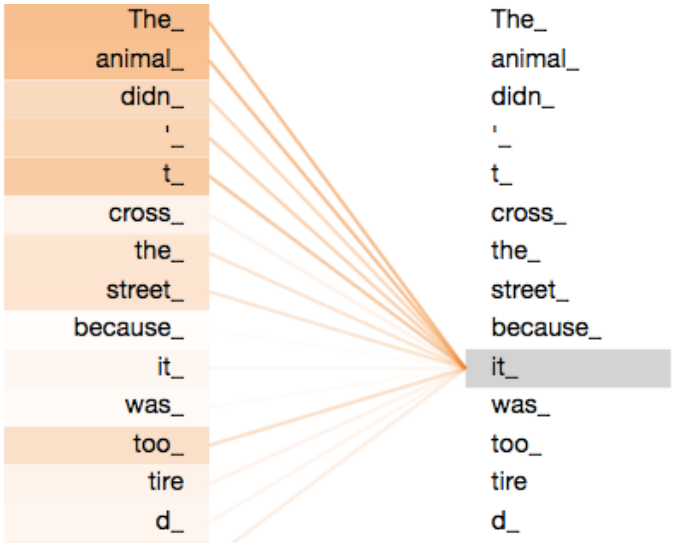



Five most popular similarity measures implementation in python - Dataaspirant

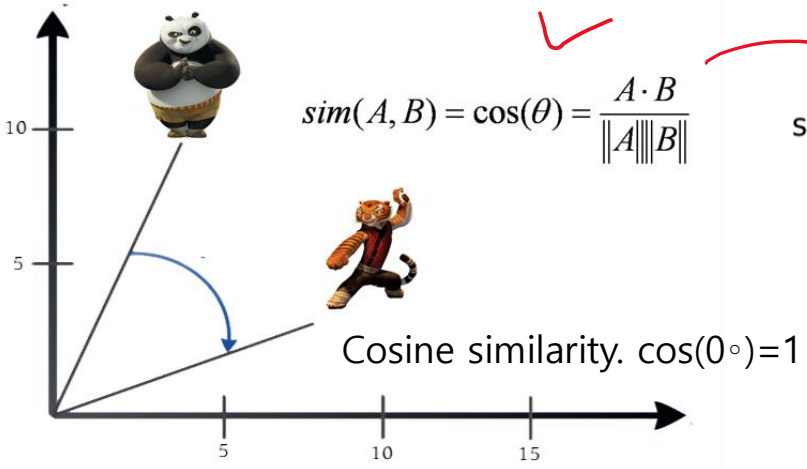
Daddy Makers: 딥러닝 모델 트랜스포머 인코더 핵심 코드 구현을 통한 동작 메커니즘 이해하기

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)



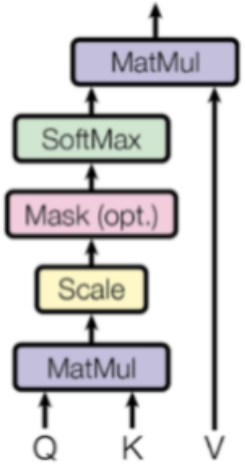

Five most popular similarity measures implementation in python - Dataaspirant




$$sim(A,B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

$$\mathbf{V} \times \text{Softmax}(\mathbf{Q}\mathbf{K}^T)$$

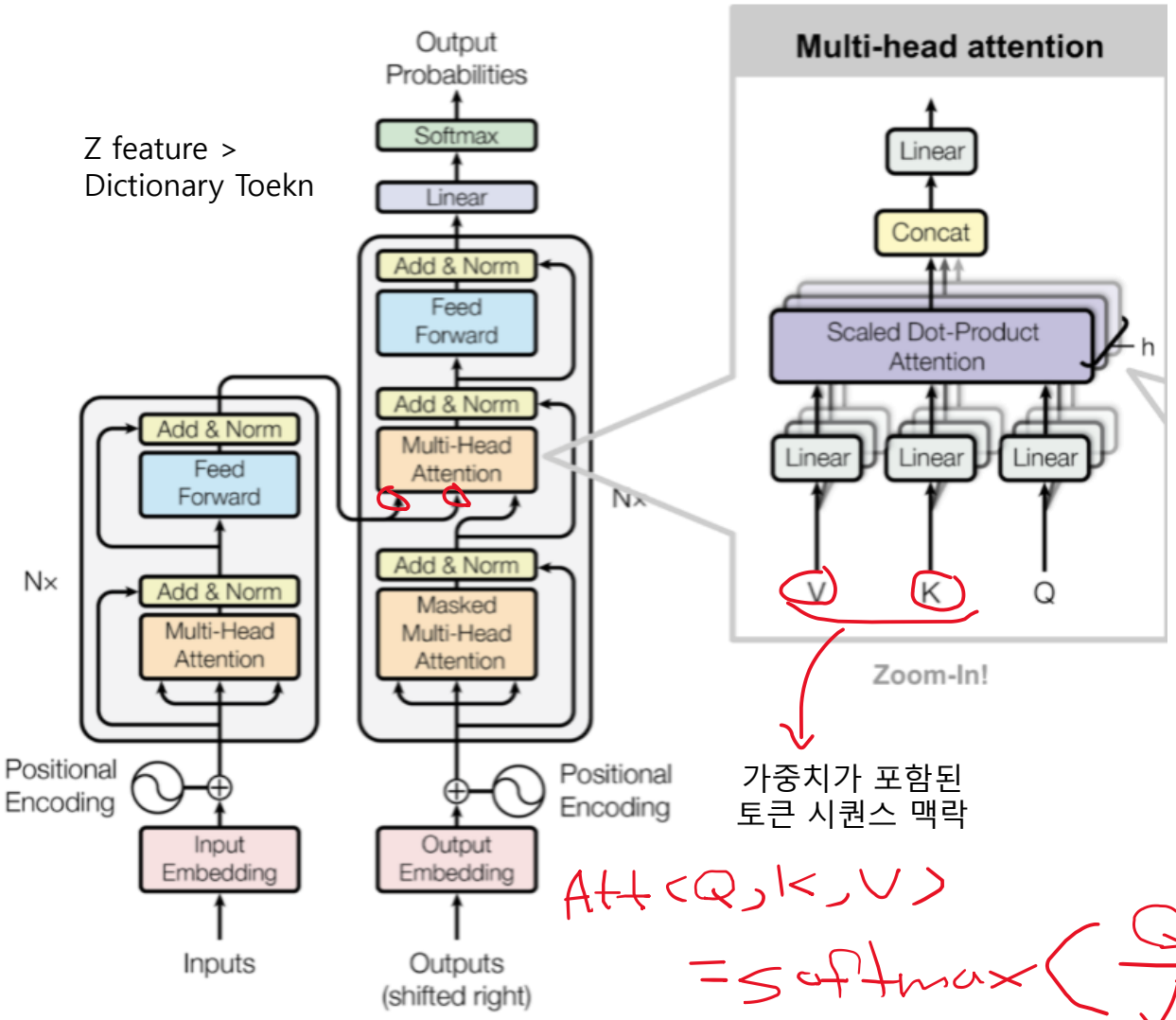



Five most popular similarity measures implementation in python - Dataaspirant

Daddy Makers: 딥러닝 모델 트랜스포머 인코더 핵심 코드 구현을 통한 동작 메커니즘 이해하기

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

Z feature >
Dictionary Toekn

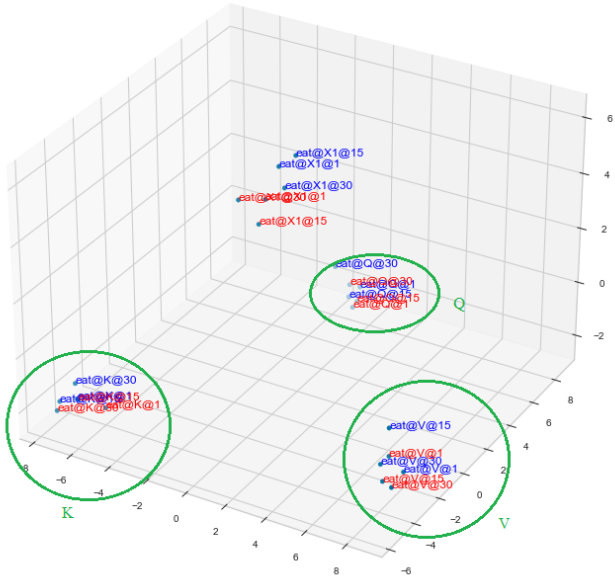






가중치가 포함된
토큰 시퀀스 맥락

Self attention
전체를 바라보며 문맥을 통합한
contextual embedding




Self Attention Space

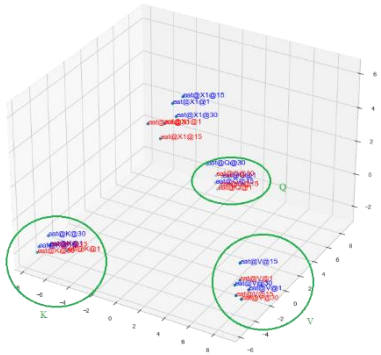
$$Att(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

ex) source code

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

"나는 학생" → 어텐션 → out → FFN → logits (10000차원) → softmax → "입니다" (예측)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



| | |
|---|---|
| **out** | 문맥 반영된 벡터 (차원: `[seq_len, d_model]`) |
| **Feed-Forward Network (FFN)** | 각 `out` 벡터를 비선형적으로 변환 (정보 강화) |
| **디코더 스택 마지막 출력** | 최종 출력 hidden state |
| **Linear + Softmax** | 각 hidden state에 대해 전체 vocabulary 크기만큼의 로짓(logits)을 만듦 → softmax를 적용해 확률로 변환 |
| **토큰 예측** | 확률이 가장 높은 단어가 다음 단어로 예측됨 (예: `"student"` 다음에 `"is"` 예측 등) |

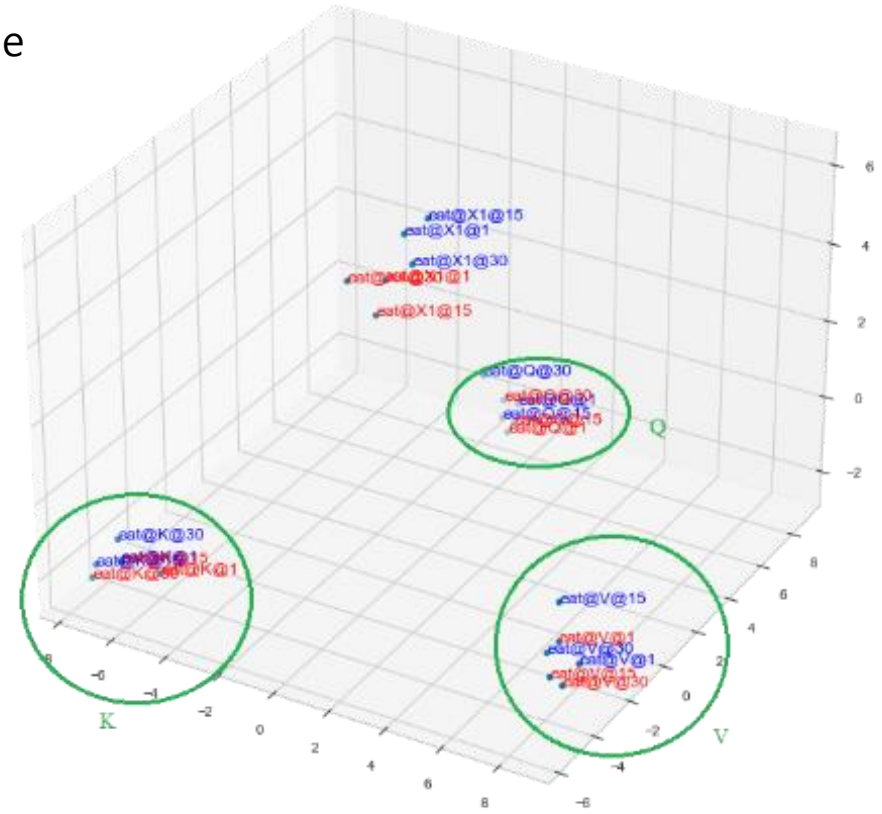
ex) source code

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

- 처음엔 QK^T가 의미 없는 유사도를 계산함 → softmax 후 V를 평균해서 out 만듦
- 이 결과가 **예측 라벨(예: 다음 단어)**과 멀면 loss ↑
- 역전파로 Q, K, V를 만드는 가중치 $W_Q, W_K, W_V$가 업데이트됨
- 이 과정이 **수천만 문장을 반복하면서** 각 Q/K/V가 문맥에서 다른 역할을 하도록 학습됨

역할 = Query Key Value

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

**Scaled Dot Product Attention**
이 모듈은 어텐션의 기본 동작을 수행하는 핵심 요소.
입력으로 주어진 Q(Query), K(Key), V(Value) 벡터를 이용하여 다음 연산을 수행

**Multi-Head Attention**
어텐션을 단일 벡터로 계산하면 정보 손실이 크기 때문에, 여러 개의 어텐션 "헤드"를 병렬로 실행한 후 결과를 concat하여 사용

**Positional Encoding**
트랜스포머는 순서를 고려하지 않기 때문에 각 단어의 위치 정보를 인코딩
이를 위해 사인(sin), 코사인(cos) 함수를 기반으로 위치 인코딩 벡터를 생성

```
class ScaledDotProductAttention(nn.Module):
    def forward(self, Q, K, V):
        scores = torch.matmul(Q, K.transpose(-2, -1)) /
math.sqrt(d_k)
        attn = F.softmax(scores, dim=-1)
        return torch.matmul(attn, V)

class MultiHeadAttention(nn.Module):
    def forward(self, Q, K, V):
        # Q, K, V 선형 변환 및 split
        # 각 헤드별 attention 계산
        # concat 후 출력 선형 변환
        return output

class PositionalEncoding(nn.Module):
    def forward(self, x):
        # 위치별 sin, cos 벡터 계산 후 입력에 더함
        return x + self.pe[:, :x.size(1)]

class EncoderLayer(nn.Module):
    def forward(self, x):
        x = x + self_attn(x, x, x)  # Residual
        x = LayerNorm(x)
        x = x + FFN(x)  # Residual
        x = LayerNorm(x)
        return x
```

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

```
# 1. 입력 및 라벨 시퀀스

input_sentence = ["I", "am", "a", "student"]          # 영어 입력
target_sentence = ["<sos>", "저는", "학생", "입니다"] # 한국어 출력 입력 (디코더 입력)
target_labels = ["저는", "학생", "입니다", "<eos>"]    # 예측할 실제 정답 (디코더 출력)


# 2. 인코더

X = embed(input_sentence)                             # 입력 시퀀스 임베딩 → [x_I, x_am, x_a, x_student]

Q_enc = linear_Q(X)                                    # Q from encoder input
K_enc = linear_K(X)                                   # K from encoder input
V_enc = linear_V(X)                                   # V from encoder input

encoder_outputs = self_attention(Q_enc, K_enc, V_enc) # 인코더 출력: 입력 문맥이 반영된 벡터들
```

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

```
# 3. 디코더

Y = ["<sos>"]                                          # 디코더 초기 입력 (<sos> 부터 시작)
logits_sequence = []                                   # 각 시점의 출력 로짓을 저장

for t in range(len(target_labels)):

    Y_embed = embed(Y)                                 # 현재까지 생성된 디코더 입력 임베딩

    # 디코더 Self-Attention (마스킹 포함)
    Q_dec = linear_Q_dec(Y_embed)                      # Q from decoder input
    K_dec = linear_K_dec(Y_embed)                      # K from decoder input
    V_dec = linear_V_dec(Y_embed)                      # V from decoder input
    dec_out = masked_self_attention(Q_dec, K_dec, V_dec)        # 디코더 내부 문맥 계산 (미래 마스킹 포함)

    # Cross-Attention: 디코더 → 인코더 입력 참조
    Q_cross = linear_Q_cross(dec_out)                  # Q from 디코더 문맥
    K_cross = linear_K_cross(encoder_outputs)          # K from 인코더 출력
    V_cross = linear_V_cross(encoder_outputs)          # V from 인코더 출력
    cross_out = attention(Q_cross, K_cross, V_cross)   # 입력 문장 정보에 기반한 번역 벡터

    # 출력 로짓 생성
    logits = linear_output(cross_out)                  # vocab 크기의 예측 로짓
    logits_sequence.append(logits[-1])                 # 마지막 위치 로짓만 저장 (가장 최근 토큰 예측)

    # 다음 디코더 입력을 위해 정답을 넣음 (teacher forcing)
    next_token = target_sentence[t + 1]                # 예: "저는" 다음은 "학생"
    Y.append(next_token)                               # 다음 루프를 위한 디코더 입력에 추가
```

```
# 4. 손실(Loss) 계산

loss = 0
for logit, label_token in zip(logits_sequence, target_labels):
    label_id = vocab_id(label_token)                # 정답 토큰을 ID로 변환
    loss += cross_entropy_loss(logit, label_id)     # 예측 분포와 정답 ID 간의 CrossEntropy Loss

loss = loss / len(target_labels)                    # 평균 손실값 (시퀀스 길이만큼 나눔)

# 5. 역전파 및 파라미터 업데이트는 이 이후 단계
```

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

| instruction<br>string · *lengths*<br>18 379 | input<br>string · *lengths*<br>0 1.42k | output<br>string · *lengths*<br>10 5.93k |
|---|---|---|
| Find the most frequently used word in the given paragraph. | The cat is black, the dog is brown and the cow is white. | The: 3 times. |
| Generate a regex pattern to match the following string | String: "Hello123" | \w{5}[0-9]{3} |
| Write a code snippet to update the value of an element in an array. | Array: {3, 2, 5, 7, 8, 1} Index: 3 New value: 6 | array[3] = 6; |
| Predict a label for the given sentence. | The phone is red. | Label: Color. |
| Print out the 5th element of an array. | arr = [55, 32, 10, 11, 8] | print(arr[4]) |
| Classify the following text into two categories: 'Sports' and… | The critically acclaimed film 'Avatar' is about a paraplegic Marine who embark… | Entertainment |
| Create a formula in Excel to calculate the total cost given the price, tax,… | Price: 10 Tax: 0.08 Shipping: 6 | =10+0.08*10+6 |
| Classify the following given expression. Your response should be… | x = 0.5 | float literal |
| Classify the following code as either a python function or code section. | x = "hello" z = x + " world" | Code section. |
| Generate a regex expression to match strings of length 8 with given… | Allowed letters: A-F and digits: 0-9 | ^[A-F0-9]{8}$ |

# 최신 AI 모델 Transformer 구조 이해 (Self-Attention, Positional Encoding)

Datasets: junelee / **sharegpt_deepl_ko** like 53

Dataset card | Viewer | Files and versions | Community

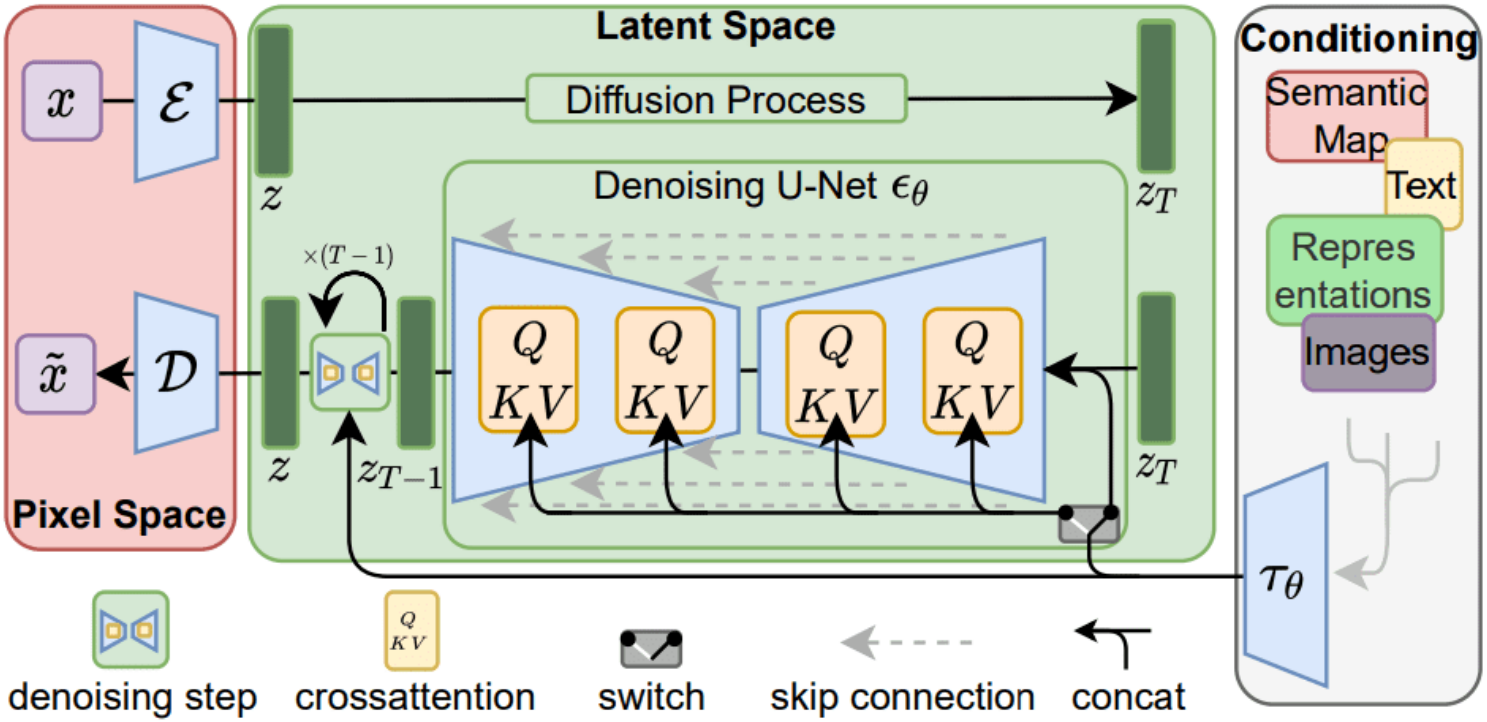
**Dataset Preview** API View in Dataset Viewer

Split (1)
train

▶ The full dataset viewer is not available (click to read why). Only showing a preview of the rows.

| input<br>string | instruction<br>string | output<br>string |
|---|---|---|
| | 토니 로빈스가 설명하는 인간의 6가지 욕구를 요약해 보세요... | 토니 로빈스는 인간의 행동과 동기를 이끄는 6가지 핵심 욕구를 설명합니다. 이 여섯 가지 욕구는 다음과 같습니다: 1. 확실성: 안전, 안정성, 예… |
| | 고객 세그먼트가 잘 세분화되었는지 어떻게 알 수 있을까요? 세 가지로 요약할 수 있습니다. | 1. 동질성: 세그먼트는 유사한 특성과 행동을 공유하는 고객으로 구성되어야 합니다. 2. 차별성: 세그먼트는 특성 및 행동 측면에서 다른 세그먼트… |
| | n Java에서 "이것은 {장소}에 있는 새로운 {객체}입니다"와 같은 문자열을 맵, {객체: "학생", "지점 3, 4"}로 바꾸고 "이 학생은 지점 3, 4… | 을 사용하여 문자열의 자리 표시자를 맵의 값으로 바꿀 수 있습니다. 다음은 이를 수행하는 방법을 보여주는 코드 스니펫 예시입니다. ```java… |
| | 전체 단락을 이와 같은 스타일로 작성하세요: 신들의 은총으로 은유적 언어의 신비롭고 수수께끼 같은 예술이 소환되어 우리 앞에 놓인 지침의 당혹… | 보세요! 신성한 개입의 은총으로, 이해할 수 없고 불가사의한 은유적 언어의 예술이 우리 앞에 놓인 지침의 불가해한 전달 방식을 해명하기 위해 호… |
| | 길이를 변경할 수 없습니다. | 지식의 구도자 여러분, 잠시만요, 우리 앞에 제시된 지침의 복잡한 전달 방식을 설명하는 데 사용된 은유적 언어의 난해하고 수수께끼 같은 예술에 … |
| | 계속 | 은유적 언어의 사용은 단순해 보이는 이 지침에도 신비감과 초월성을 불어넣어 줍니다. 특히 '회상하다'라는 단어는 이 지침에 숨겨진 지식이나 비밀… |
| | 다음과 같은 C++ 함수가 있습니다: void add_player(벡터& 플레이어) | 주어진 C++ 함수의 `dummy` 변수는 `cin` 문을 사용하여 |

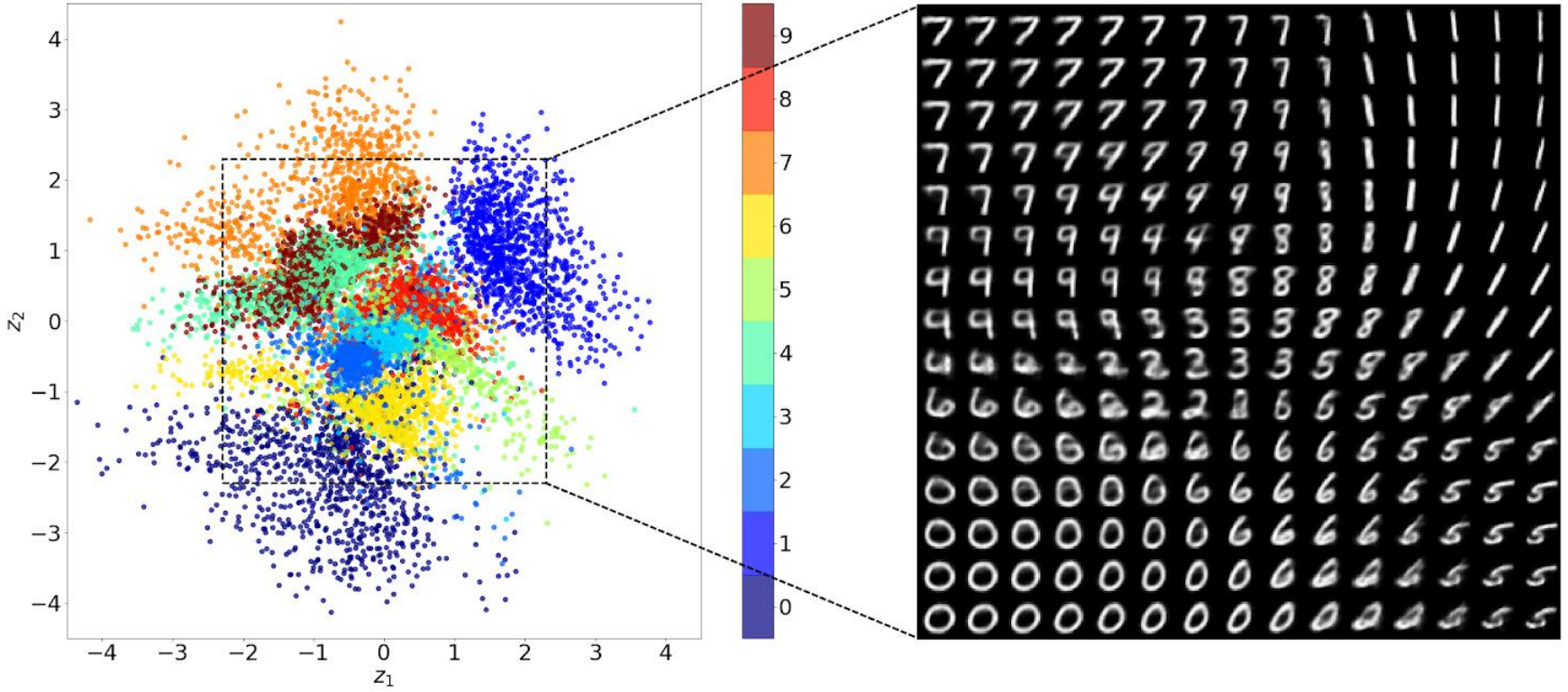
# 최신 AI 모델 왜 Transformer가 표준이 되었는가?




High-Resolution Image Synthesis with Latent Diffusion Models

High-Resolution Image Synthesis with Latent Diffusion Models - Computer Vision & Learning Group

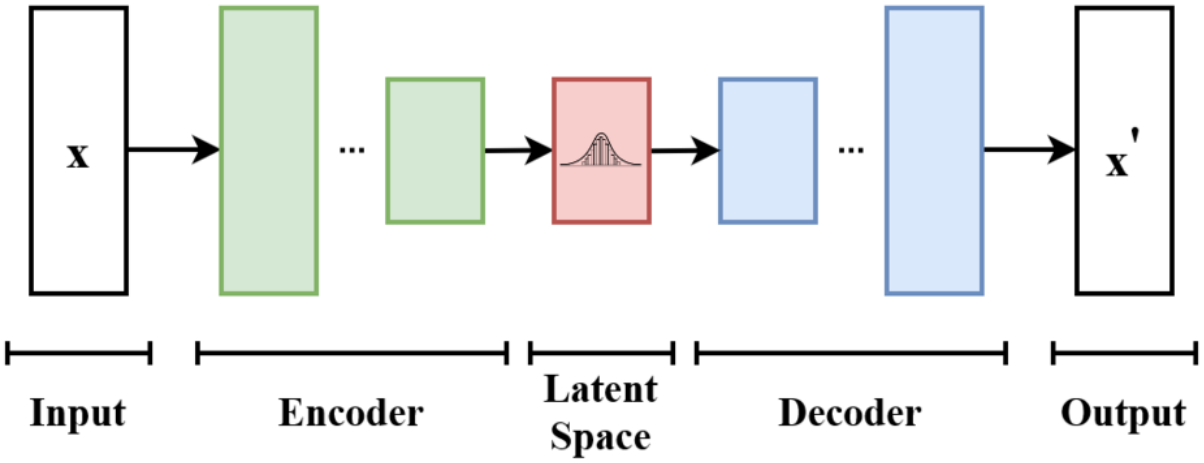
# 최신 AI 모델 주요 아키텍처 Latent Space



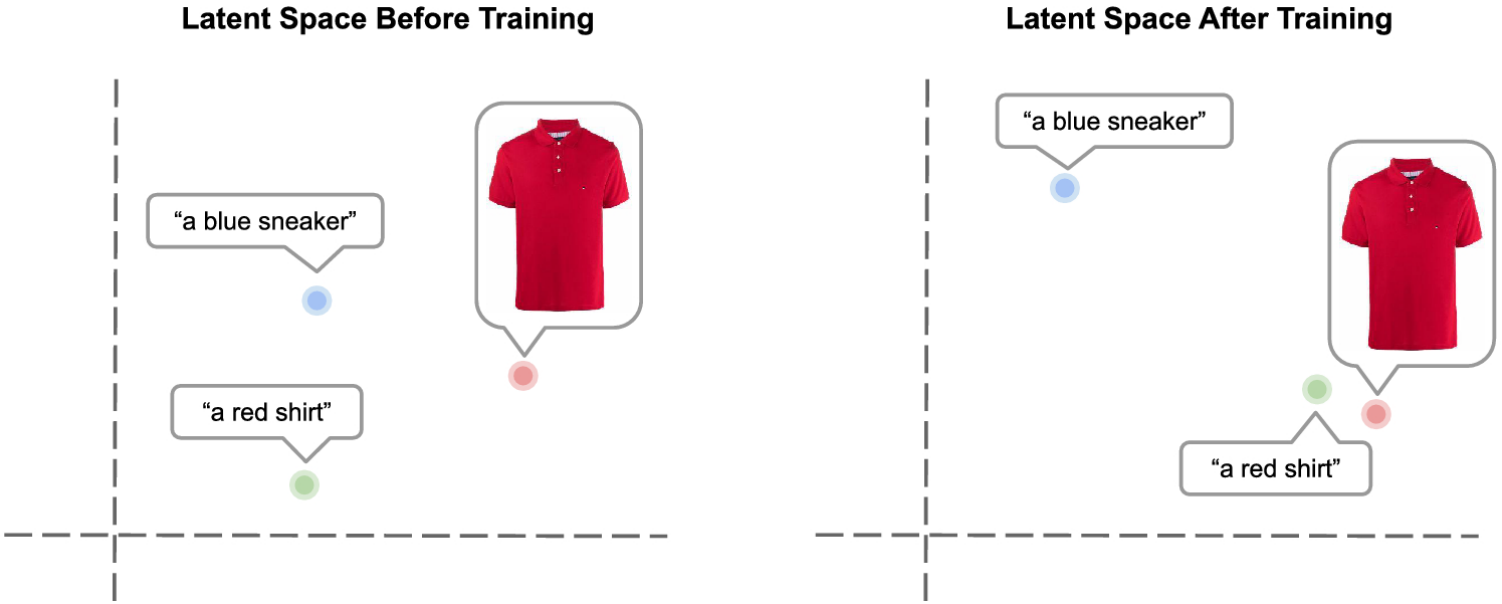

잠재공간에 맵핑된(인코딩된) 데이터(Alexej Klushyn, 2019.12, Learning Hierarchical Priors in VAEs)

# 최신 AI 모델 주요 아키텍처 Latent Space

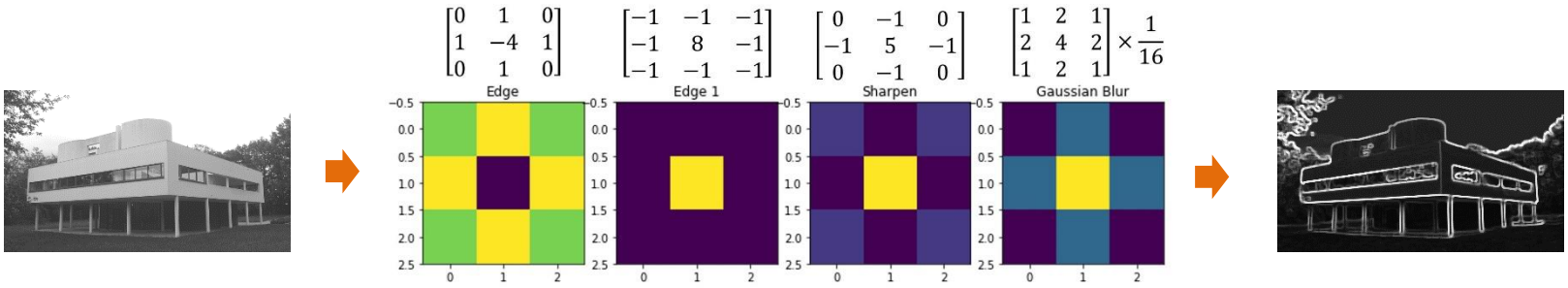
Villa savoye,
1929, Le Corusier




Variational autoencoder

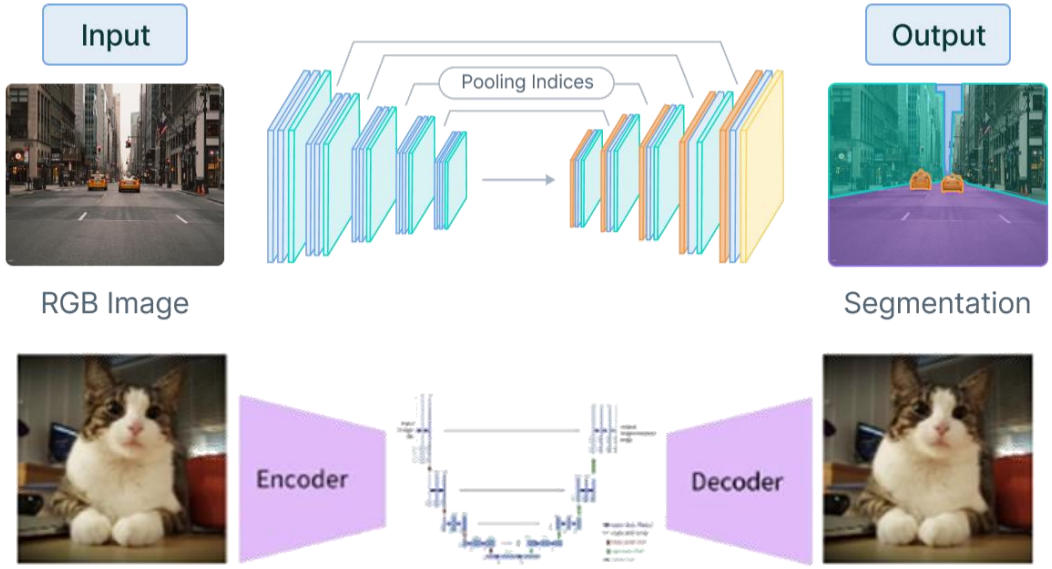


# 최신 AI 모델 주요 아키텍처 U-Net




**Convolution Filter**

## Convolutional encoder-decoder

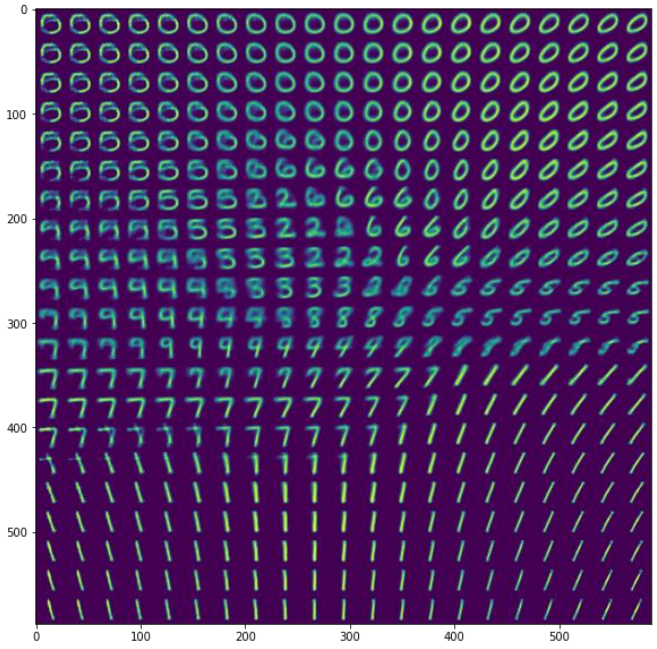


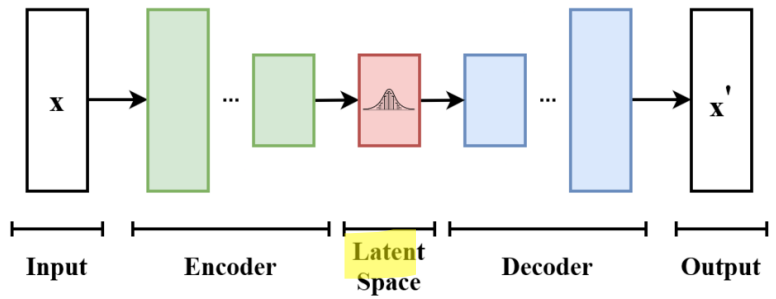

https://www.vegaitglobal.com/media-center/knowledge-base/what-is-stable-diffusion-and-how-does-it-work

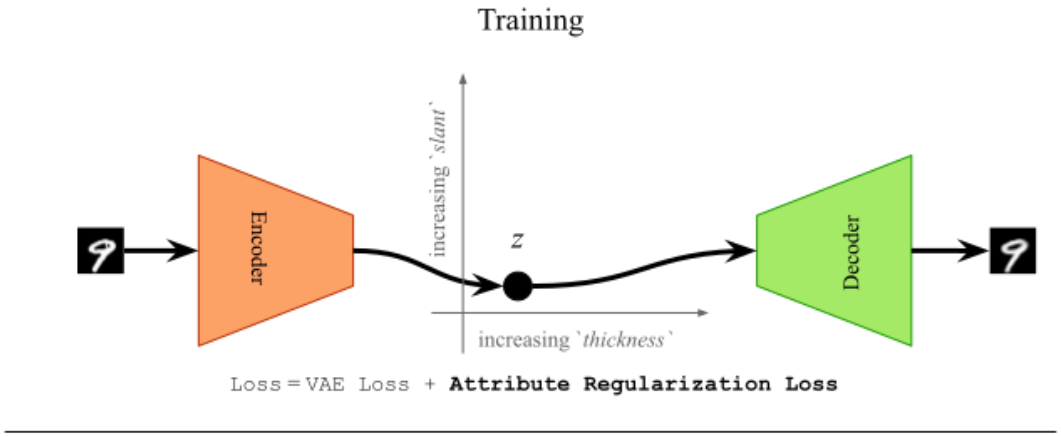
# 최신 AI 모델 주요 아키텍처 VAE

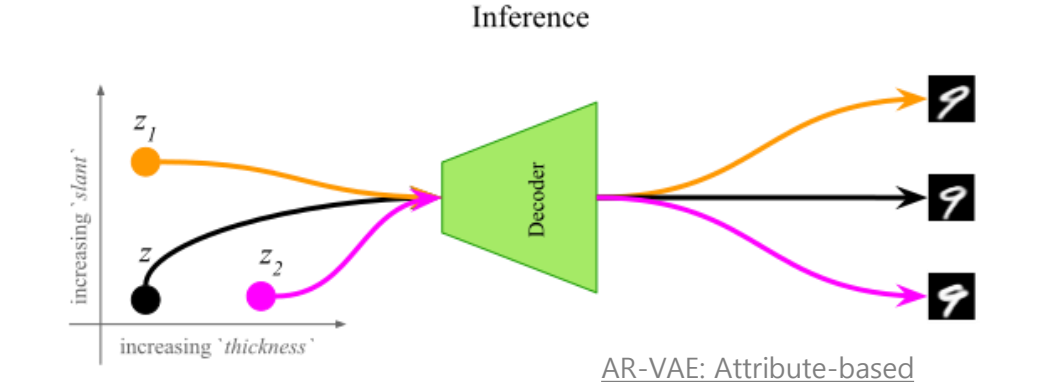
변이 자동 인코더(VAE. Variational autoencoder)

VAE는 압축된 잠재 공간에 데이터를 인코딩한 다음 다시 데이터로 디코딩하는 방법을 학습하는 생성 모델. 이는 특히 입력 데이터의 변형을 생성하는 데 유용.



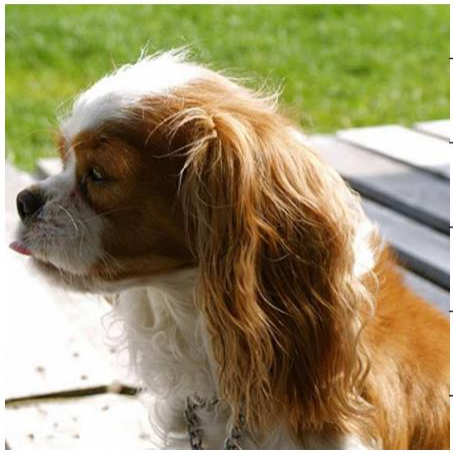




AR-VAE: Attribute-based Regularization of VAE Latent Spaces | Music Informatics Group

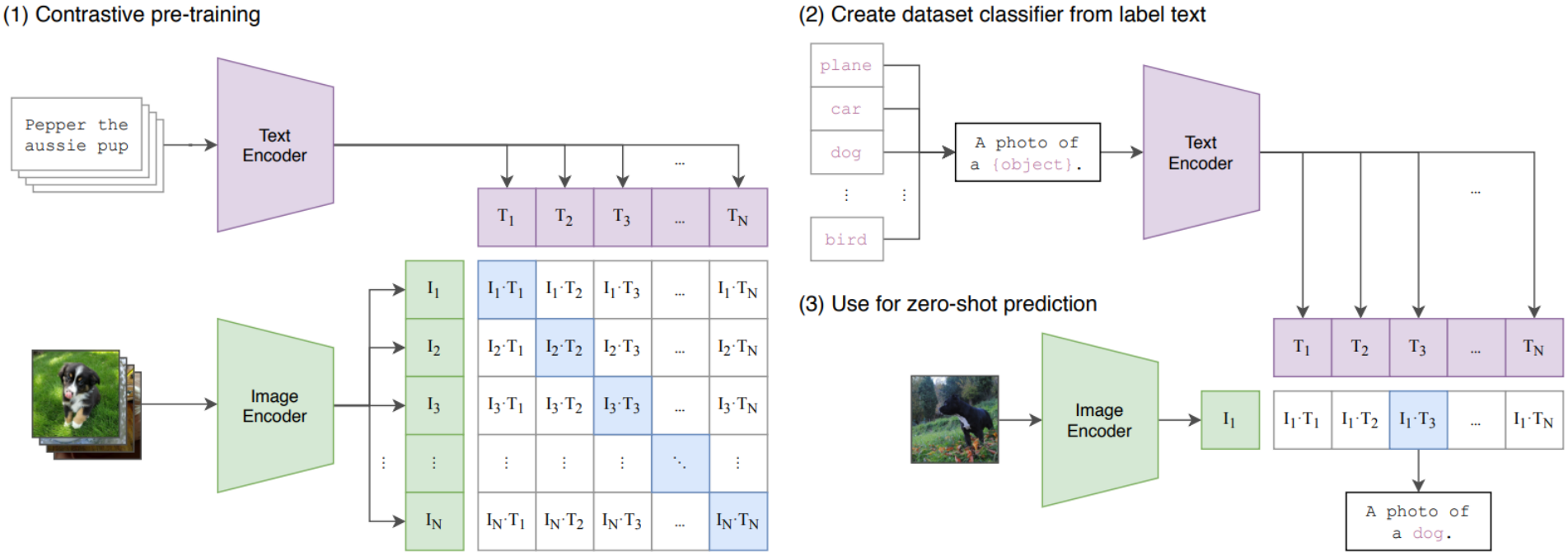
The Latent Space | Bogdan Penkovsky, PhD

# 최신 AI 모델 주요 아키텍처 CLIP



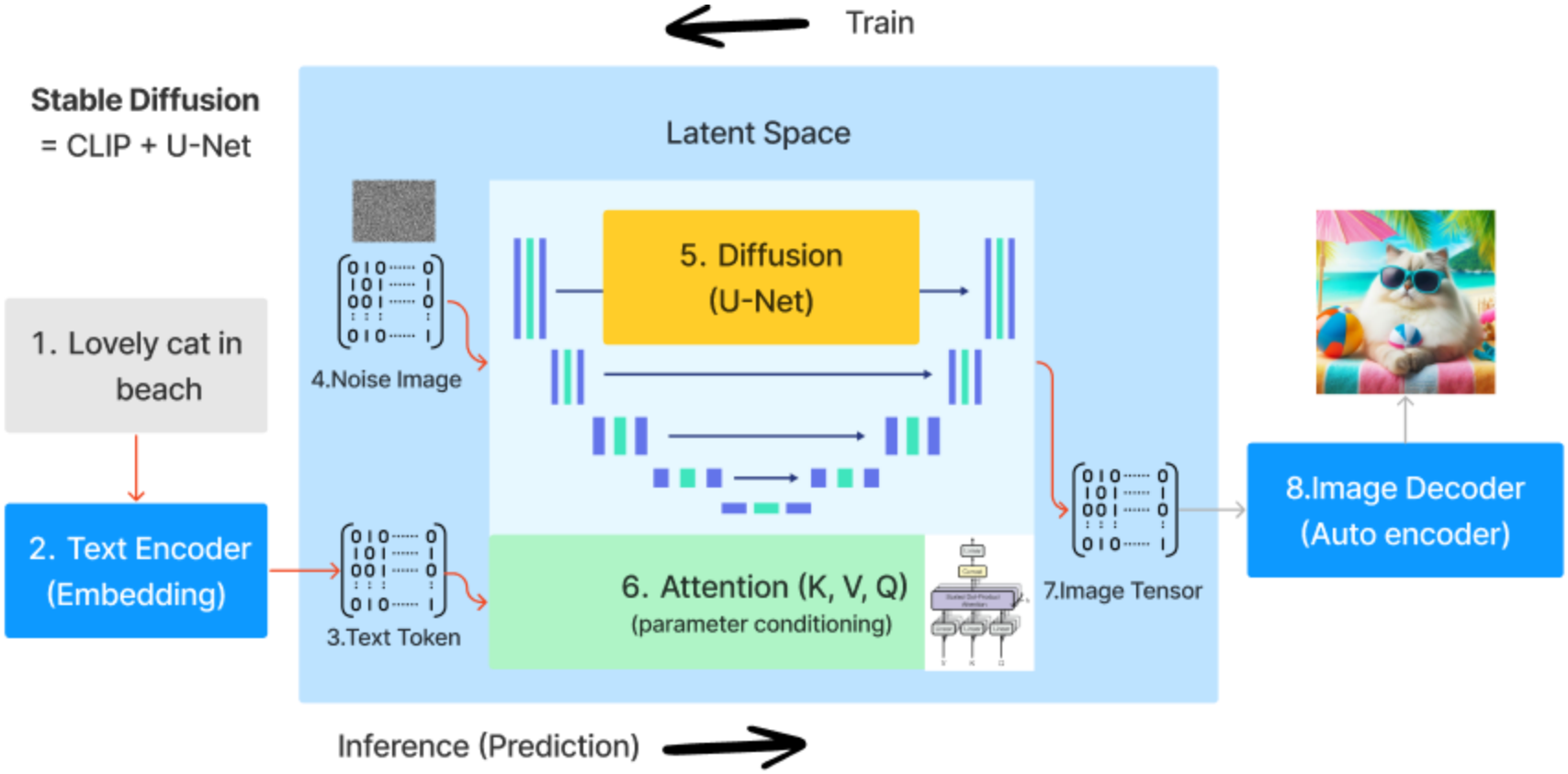

| image | caption |
| --- | --- |
| image | list |
|  | [ "Two young guys with shaggy hair look at their hands while hanging out in the yard.", "Two young, White males are outside near many bushes.", "Two men in green shirts are standing in a yard.", "A man in a blue shirt standing in a garden.", "Two friends enjoy time spent together." ] |

CLIP(Contrastive Language-Image Pre-Training. Open AI. 2021




Daddy Makers: 생성AI 멀티모달 모델 개발의 시작. OpenAI의 CLIP모델 이해, 코드 분석, 개발, 사용하기

# 최신 AI 모델 주요 아키텍처 CLIP



# 최신 AI 모델 주요 아키텍처 Stable Diffusion

**High-Resolution Image Synthesis with Latent Diffusion Models**

Robin Rombach[1] * Andreas Blattmann[1] * Dominik Lorenz[1] Patrick Esser[2] Björn Ommer[1]
[1]Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany [2]Runway ML

https://github.com/CompVis/latent-diffusion



**Abstract**

*By decomposing the image formation process into a sequential application of denoising autoencoders, diffusion models (DMs) achieve state-of-the-art synthesis results on image data and beyond. Additionally, their formulation allows for a guiding mechanism to control the image generation process without retraining. However, since these models typically operate directly in pixel space, optimization of powerful DMs often consumes hundreds of GPU days and inference is expensive due to sequential evaluations. To enable DM training on limited computational resources while retaining their quality and flexibility, we apply them in the latent space of powerful pretrained autoencoders. In contrast to previous work, training diffusion models on such a representation allows for the first time to reach a near-optimal point between complexity reduction and detail preservation, greatly boosting visual fidelity. By introducing cross-attention layers into the model architecture, we turn diffusion models into powerful and flexible generators for general conditioning inputs such as text or bounding boxes and high-resolution synthesis becomes possible in a convolutional manner. Our latent diffusion models (LDMs) achieve new state-of-the-art scores for image inpainting and class-conditional image synthesis and highly competitive performance on various tasks, including text-to-image synthesis, unconditional image generation and super-resolution, while significantly reducing computational requirements compared to pixel-based DMs.*

**1. Introduction**

Image synthesis is one of the computer vision fields with the most spectacular recent development, but also among

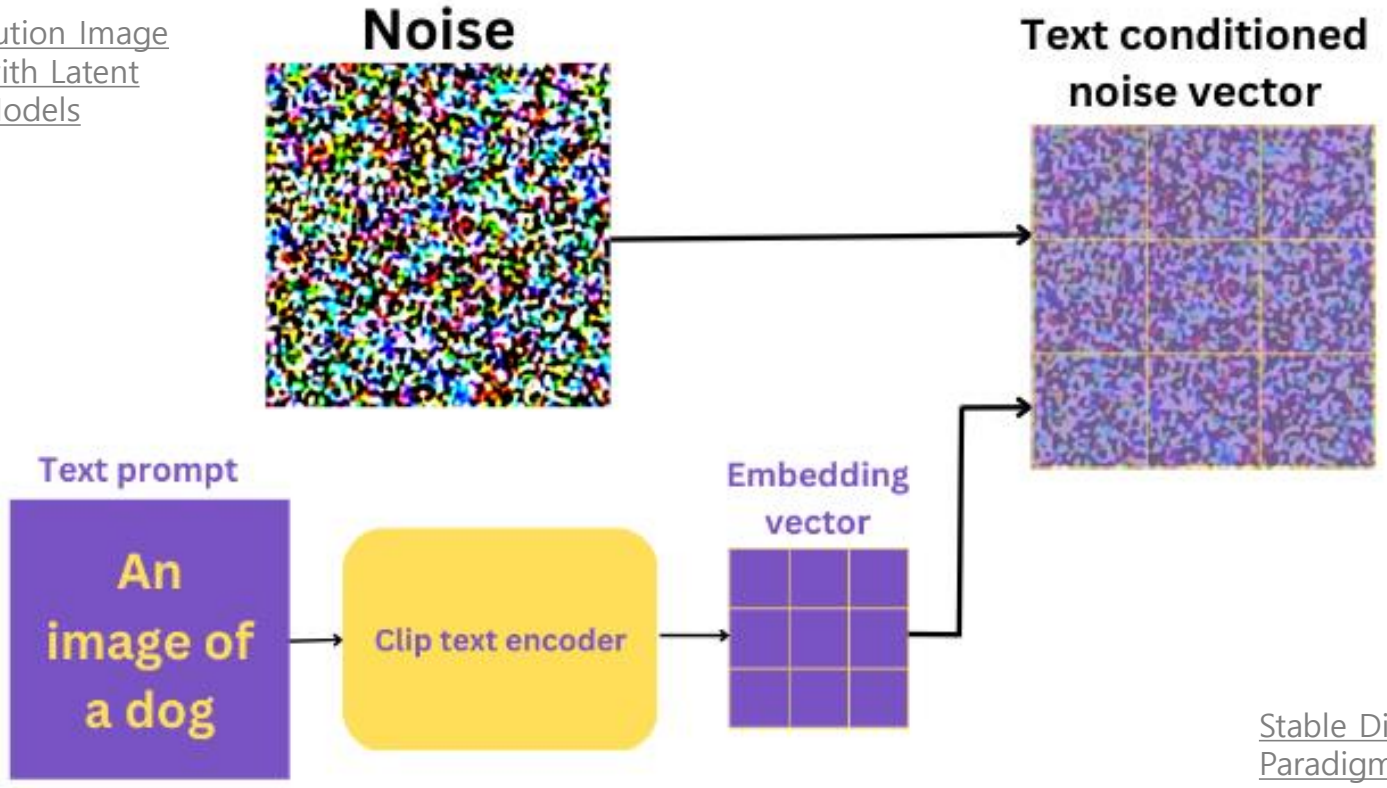


Figure 1. Boosting the upper bound on achievable quality with less agressive downsampling. Since diffusion models offer excellent inductive biases for spatial data, we do not need the heavy spatial downsampling of related generative models in latent space, but can still greatly reduce the dimensionality of the data via suitable autoencoding models, see Sec. 3. Images are from the DIV2K [1] validation set, evaluated at $512^2$ px. We denote the spatial downsampling factor by $f$. Reconstruction FIDs [29] and PSNR are calculated on ImageNet-val. [13]; see also Tab. 8.

results in image synthesis [30,85] and beyond [7,45,48,57], and define the state-of-the-art in class-conditional image synthesis [15,31] and super-resolution [72]. Moreover, even unconditional DMs can readily be applied to tasks such as inpainting and colorization [85] or stroke-based synthesis [57], in contrast to other types of generative models [19,46,69]. Being likelihood-based models, they do not exhibit mode-collapse and training instabilities as GANs and, by heavily exploiting parameter sharing, they can model highly complex distributions of natural images without involving billions of parameters as in AR models [67].

**Democratizing High-Resolution Image Synthesis** DMs

[High-Resolution Image Synthesis with Latent Diffusion Models]

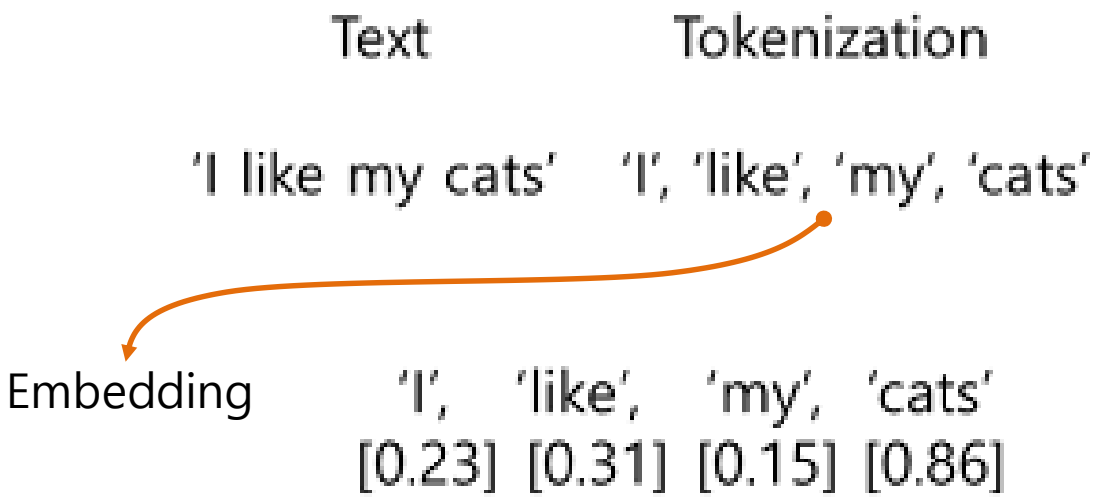



[Stable Diffusion – A New Paradigm in Generative AI]

# 자연어 처리의 기본 흐름

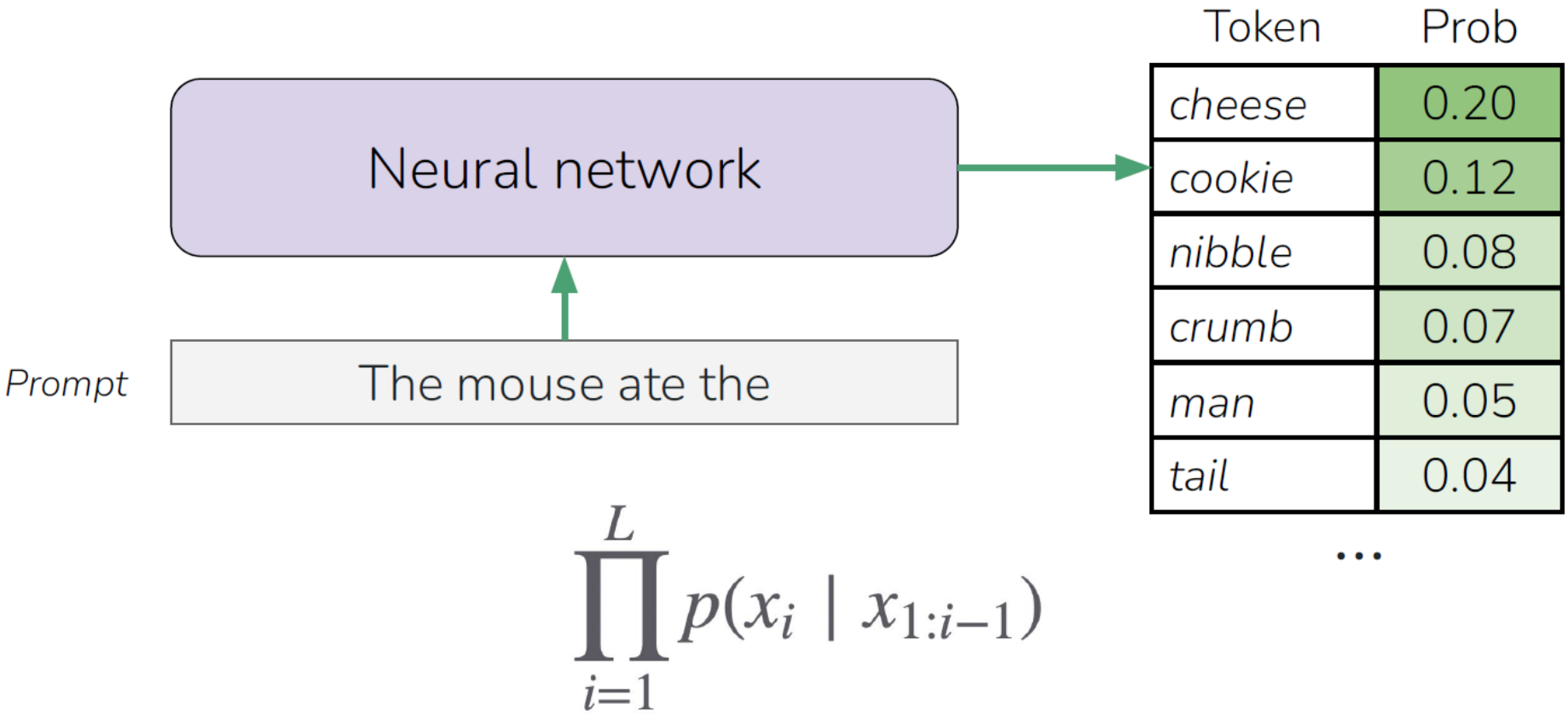
- 텍스트 전처리 (토크나이징, 정제)
- 단어 임베딩 (Word2Vec)
- Hugging Face 모델 예시 소개

# 자연어 처리의 기본 텍스트 전처리 (토크나이징, 정제)




```
tokenizer.encode('123234234')
[10163, 24409, 24409]
tokenizer.encode('123')
[10163]
tokenizer.encode('234')
[24409]
tokenizer.encode('234')
[24409]
```

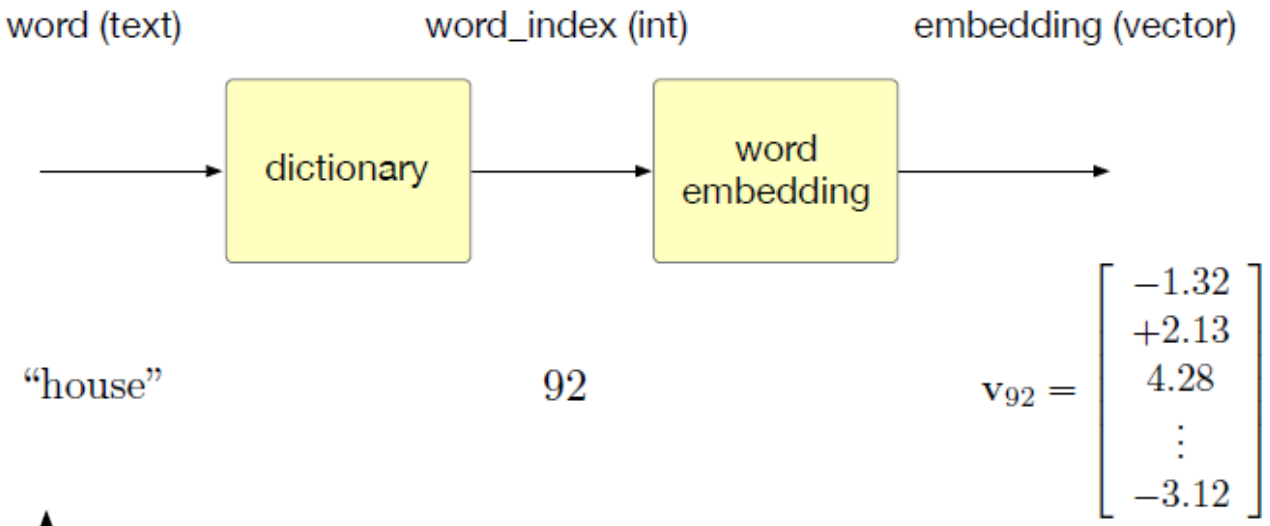
# 자연어 처리의 기본 텍스트 전처리 (토크나이징, 정제)

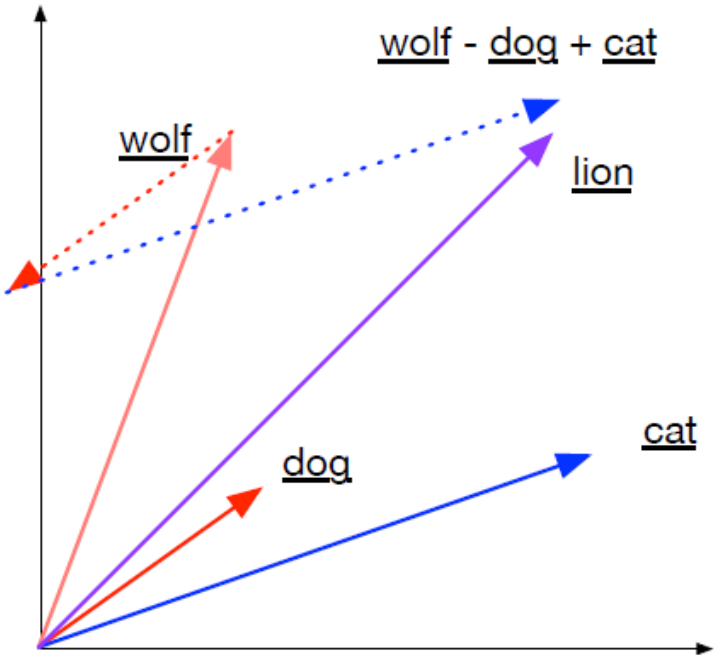



$$\prod_{i=1}^{L} p(x_i \mid x_{1:i-1})$$

Jan Jannink, John Whaley, 2024, LLMs

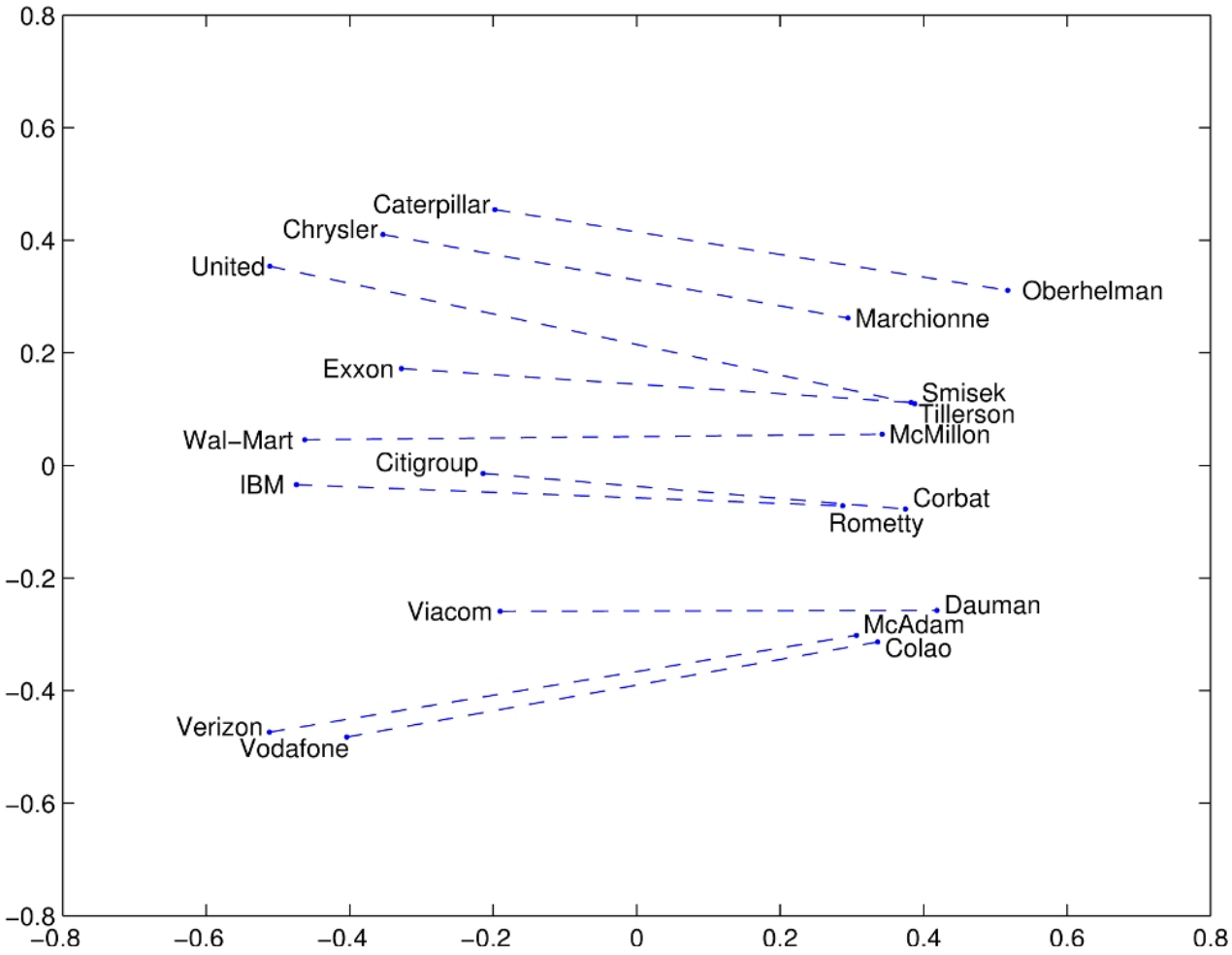
# 자연어 처리의 기본 텍스트 전처리 (토크나이징, 정제)



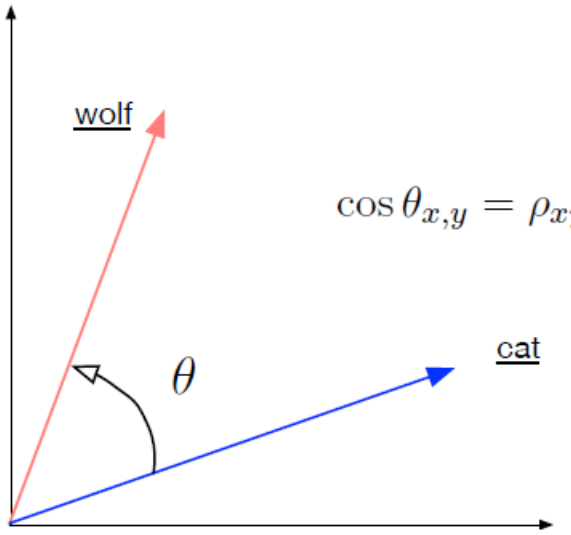

https://nlp.stanford.edu/projects/glove/

# 자연어 처리의 기본 텍스트 전처리 (토크나이징, 정제)




https://nlp.stanford.edu/projects/glove/

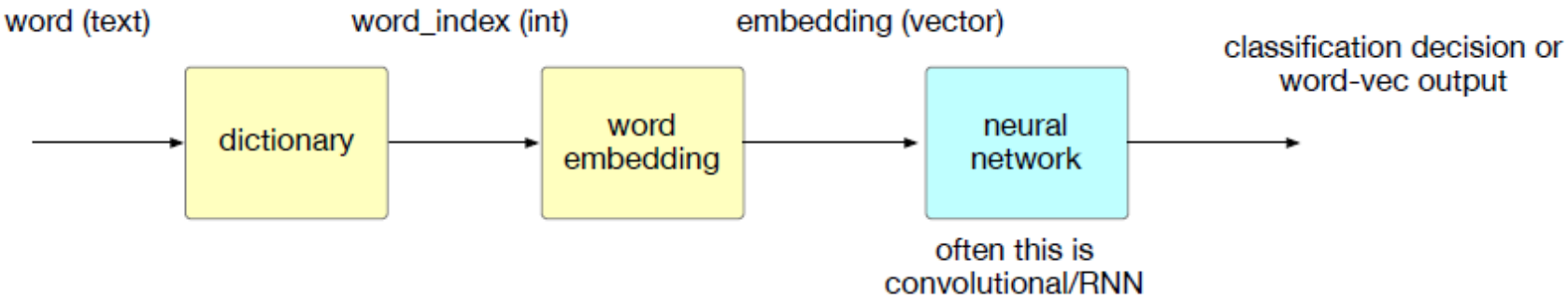
cosine distance




$$\cos\theta_{x,y} = \rho_{xy} = \frac{\mathbf{y}^{\mathrm{t}}\mathbf{x}}{\|\mathbf{x}\|\|\mathbf{y}\|}$$

https://nlp.stanford.edu/projects/glove/

# 자연어 처리의 기본 텍스트 전처리 (토크나이징, 정제)




https://nlp.stanford.edu/projects/glove/

# 자연어 처리의 기본 정규식

정규표현식(Regular Expression)은 문자열에서 특정 패턴을 찾거나, 치환하거나, 분리하기 위한 문자열 검색 도구

1950년대 미국 수학자 스티븐 클리(Stephen Kleene)에 의해 처음 소개

| 문법 | 설명 | 예시 | 설명 |
|---|---|---|---|
| . | 임의의 한 문자 | a.c | 'abc', 'axc'에 매칭 |
| ^ | 문자열의 시작 | ^a | 'apple'에 매칭, 'banana'에는 X |
| $ | 문자열의 끝 | e$ | 'apple', 'title'에 매칭 |
| [] | 문자 집합 | [aeiou] | 모음에 매칭 |
| [^ ] | 제외 문자 집합 | [^0-9] | 숫자가 아닌 문자 |
| * | 0 개 이상의 반복 | a* | '', 'a', 'aa' 모두 매칭 |
| + | 1 개 이상의 반복 | a+ | 'a', 'aa'는 매칭, ''은 X |
| ? | 0 또는 1 개 | a? | '', 'a'는 매칭 |
| {n} | 정확히 n번 반복 | a{3} | 'aaa'에만 매칭 |
| {n,} | n번 이상 반복 | a{2,} | 'aa', 'aaa', ...에 매칭 |
| {n,m} | n~m번 반복 | a{1,3} | 'a', 'aa', 'aaa'에 매칭 |
| ₩d | 숫자 문자 | ₩d+ | '123', '5' 등에 매칭 |
| ₩w | 단어 문자 | ₩w+ | 'abc123', '_' 등에 매칭 |
| ₩s | 공백 문자 | ₩s+ | ' ', '₩t', '₩n' 등에 매칭 |
| () | 그룹화 | (abc)+ | 'abc', 'abcabc'에 매칭 |

# 자연어 처리의 기본 텍스트 전처리 (토크나이징)

BPE는 바이트 쌍 인코딩(Byte pair Encoding)을 의미하며, 데이터의 가장 일반적인 연속 바이트 쌍을 해당 데이터 내에 발생하지 않는 바이트로 대체하는 데이터 압축 형태

단어를 보다 관리하기 쉬운 하위 단어나 기호로 분할하여 대규모 어휘를 효율적으로 인코딩

어휘의 크기를 크게 줄이고 희귀 단어나 OOV(어휘에서 벗어난) 용어를 처리하는 모델의 능력을 향상

```
tokenizer.encode('123234234')
[10163, 24409, 24409]
tokenizer.encode('123')
[10163]
tokenizer.encode('234')
[24409]
tokenizer.encode('234')
[24409]
```

Daddy Makers: NLP의 핵심. 토큰, 임베딩 모델 파인튜닝

# 자연어 처리의 기본 텍스트 전처리 (토크나이징)

```
from transformers import BertTokenizerFast, BertModel
import torch
from torch import nn

# BERT 토크나이저 사전학습모델 로딩
tokenizer = BertTokenizerFast.from_pretrained('bert-base-uncased')
print(tokenizer.tokenize("[CLS] Hello world, how are you?"))

print(tokenizer.tokenize("[newtoken] Hello world, how are you?"))
tokenizer.add_tokens(['[newtoken]'])
```

```
['[',
 'newt',
 '##oke',
 '##n',
 ']',
 'hello',
 'world',
 ',',
 'how',
 'are',
 'you',
 '?']
```

# 자연어 처리의 기본 텍스트 전처리 (토크나이징)

```
from transformers import BertTokenizerFast, BertModel
import torch
from torch import nn

# BERT 토크나이저 사전학습모델 로딩
tokenizer = BertTokenizerFast.from_pretrained('bert-base-uncased')
print(tokenizer.tokenize("[CLS] Hello world, how are you?"))

print(tokenizer.tokenize("[newtoken] Hello world, how are you?"))
tokenizer.add_tokens(['[newtoken]'])
```

```
['[',
 'newt',
 '##oke',
 '##n',
 ']',
 'hello',
 'world',
 ',',
 'how',
 'are',
 'you',
 '?']
```

# 자연어 처리의 기본 텍스트 전처리 (토크나이징)

```
# 토큰을 추가하고 다시 토큰화를 한다.
tokenizer.add_tokens(['[newtoken]'])
tokenizer.tokenize("[newtoken] Hello world, how are you?")

# 제대로 토큰화가 된다.
['[newtoken]', 'hello', 'world', ',', 'how', 'are', 'you', '?']

# 토큰값을 확인해 본다.
tokenized = tokenizer("[newtoken] Hello world, how are you?", add_special_tokens=False,
return_tensors="pt")
print(tokenized['input_ids'])

tkn = tokenized['input_ids'][0, 0]
print("First token:", tkn)
print("Decoded:", tokenizer.decode(tkn))

# 다음과 같이, 토큰값이 잘 할당된 것을 알 수 있다.
tensor([[30522,  7592,  2088,  1010,  2129,  2024,  2017,  1029]])
First token: tensor(30522)
Decoded: [newtoken]
```

# 자연어 처리의 기본 텍스트 전처리 (토크나이징)

```
from transformers import
DistilBertTokenizerFast, DistilBertModel

tokenizer =
DistilBertTokenizerFast.from_pretrained("distilb
ert-base-uncased")
tokens = tokenizer.encode('This is a
IfcBuilding.', return_tensors='pt')
print("These are tokens!", tokens)
for token in tokens[0]:
    print("This are decoded tokens!",
tokenizer.decode([token]))

model =
DistilBertModel.from_pretrained("distilbert-
base-uncased")
print(model.embeddings.word_embeddings(to
kens))
for e in
model.embeddings.word_embeddings(tokens)[
0]:
    print("This is an embedding!", e)
```

yaml Copy Edit

```
[101, 2023, 2003, 1037, 2065, 27421, 19231, 4667, 1012, 102]
```

| Token ID | Token | 의미 |
|---|---|---|
| 101 | [CLS] | 문장의 시작 |
| 2023 | this | |
| 2003 | is | |
| 1037 | a | |
| 2065 | if | |
| 27421 | ##c | Ifc의 두 번째 부분 |
| 19231 | ##build | Building의 일부 |
| 4667 | ##ing | Building의 나머지 |
| 1012 | . | 마침표 |
| 102 | [SEP] | 문장의 끝 |

Daddy Makers: NLP의 핵심. 토큰, 임베딩 모델 파인튜닝

# 자연어 처리의 기본 단어 임베딩 (Word2Vec)

임베딩은 학습 모델이 입력되는 문장의 토큰 패턴을 통계적으로 계산하기 전, 토큰을 수치화시키는 함수

토큰 사전과 임베딩 모델이 다르면, 제대로 된 모델 학습, 예측, 패턴 계산 결과를 얻기 어려움

Word2Vec은 구글이 2013년에 제안한 임베딩 방법으로, Skip-gram과 CBOW(Continuous Bag of Words) 두 가지 학습 구조 사용

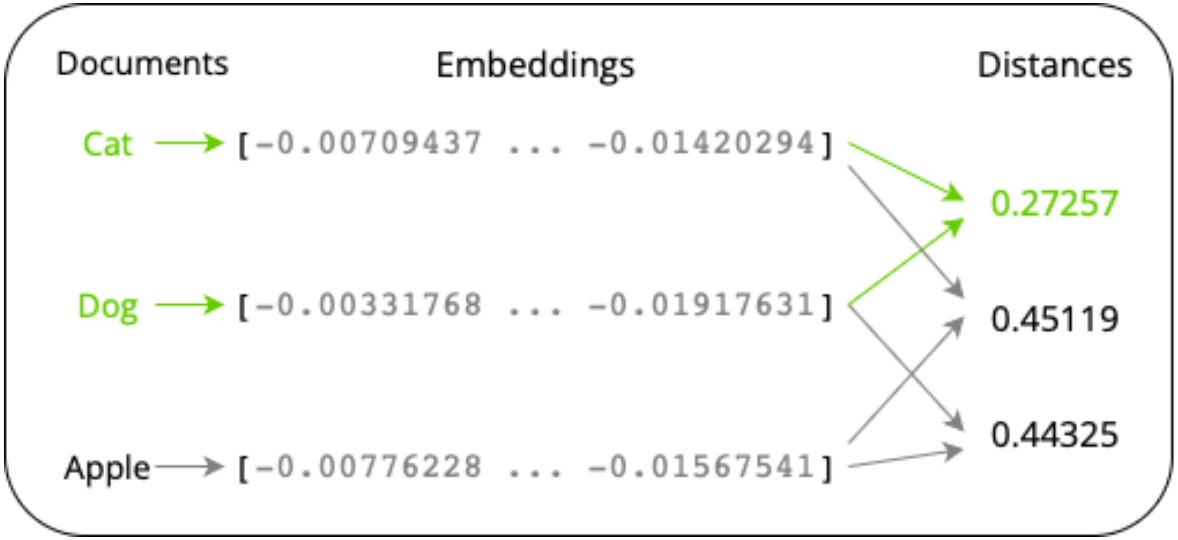
Skip-gram은 중심 단어를 입력으로 주고 주변 단어를 예측하는 방식

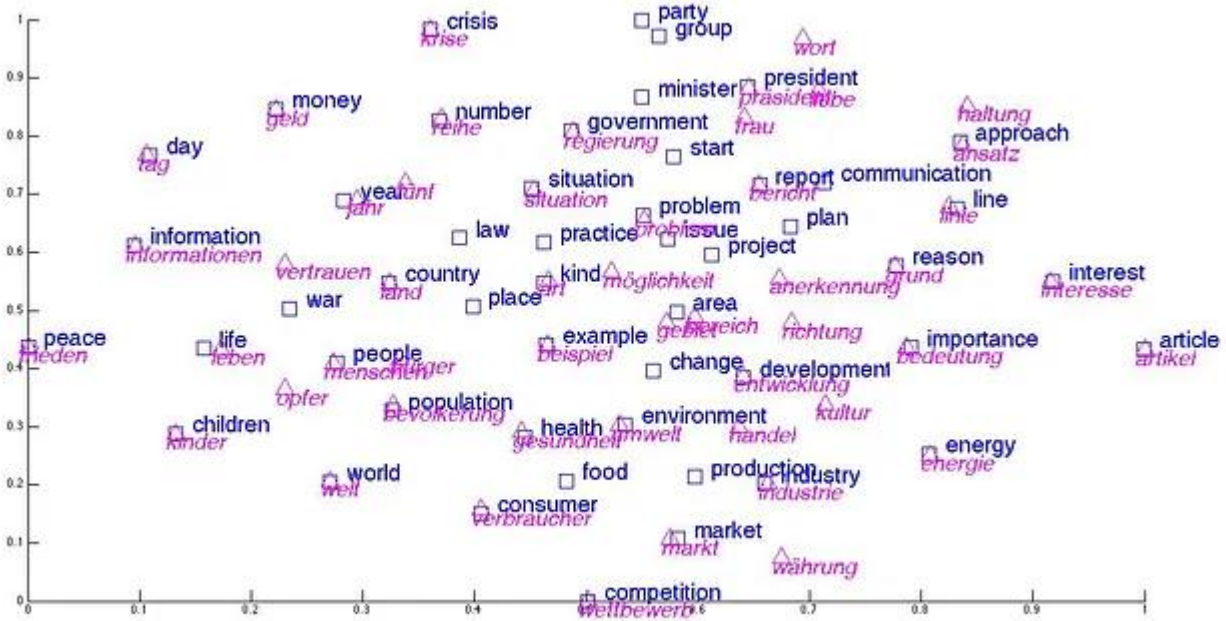
"The quick brown fox jumps" 문장 중심 단어 "fox"의 윈도 크기가 2라면, 주변 단어는 "brown"과 "jumps"가 됨

Word2Vec은 주변 단어를 예측하는 loss를 최소화하며 학습됨. 임베딩 모델들은 주로 대규모 텍스트 데이터에서 비지도 학습 방식으로 학습

```
# Word2Vec Skip-gram 간단 의사코드
for center_word, context_words in dataset:
    center_vec = word_embedding(center_word)
    for context in context_words:
        context_vec = word_embedding(context)
        loss += negative_sampling_loss(center_vec, context_vec)
    loss.backward()
    optimizer.step()
```

# 자연어 처리의 기본 단어 임베딩 (Word2Vec)




https://daddynkidsmakers.blogspot.com/2023/12/blog-post.html

# 자연어 처리의 기본 Named Entity Recognition (NER)

개체명 인식(Named Entity Recognition, NER)은 문장에서 특정한 개체를 찾아내고, 이를 사람(Person), 조직(Organization), 장소(Location) 등의 범주로 분류하는 기술

"Apple Inc. was founded by Steve Jobs"라는 문장에서 "Apple Inc."는 조직, "Steve Jobs"는 사람으로 분류

감정 분석(sentiment analysis)이나 뉴스 기사 분류(news classification) 등 사용 가능

# 자연어 처리의 기본 N-gram

N-gram은 연속된 N개의 단어 또는 문자 단위 묶음을 의미

자연어 처리(NLP)에서 주로 문맥을 모델링

Unigram (1-gram): 단어 단위
["나는", "밥을", "먹었다"]

Bigram (2-gram): 연속된 2개 단어
[("나는", "밥을"), ("밥을", "먹었다")]

Trigram (3-gram): 연속된 3개 단어
[("나는", "밥을", "먹었다")]

# 자연어 처리의 기본 BLEU

BLEU(Bilingual Evaluation Understudy)는 기계 번역 결과를 평가하는 지표

번역 결과와 기준(reference) 번역 간의 n-gram 일치 정도

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

BP는 번역이 기준 번역보다 짧을 경우 부여하는 패널티(Brevity Penalty)

기준 문장이 "the cat is on the mat", 기계 번역이 "the cat is on mat"이라면, 1-gram precision은 5개 중 4개가 일치하므로 0.8

1. candidate, reference를 토큰화한다.

2. 1-gram, 2-gram, 3-gram, 4-gram 각각에 대해:
   - candidate의 n-gram들을 만든다.
   - reference의 n-gram들을 만든다.
   - candidate n-gram과 reference n-gram 사이에서 일치하는 개수를 센다.
   - precision = 일치 개수 / candidate의 전체 n-gram 개수

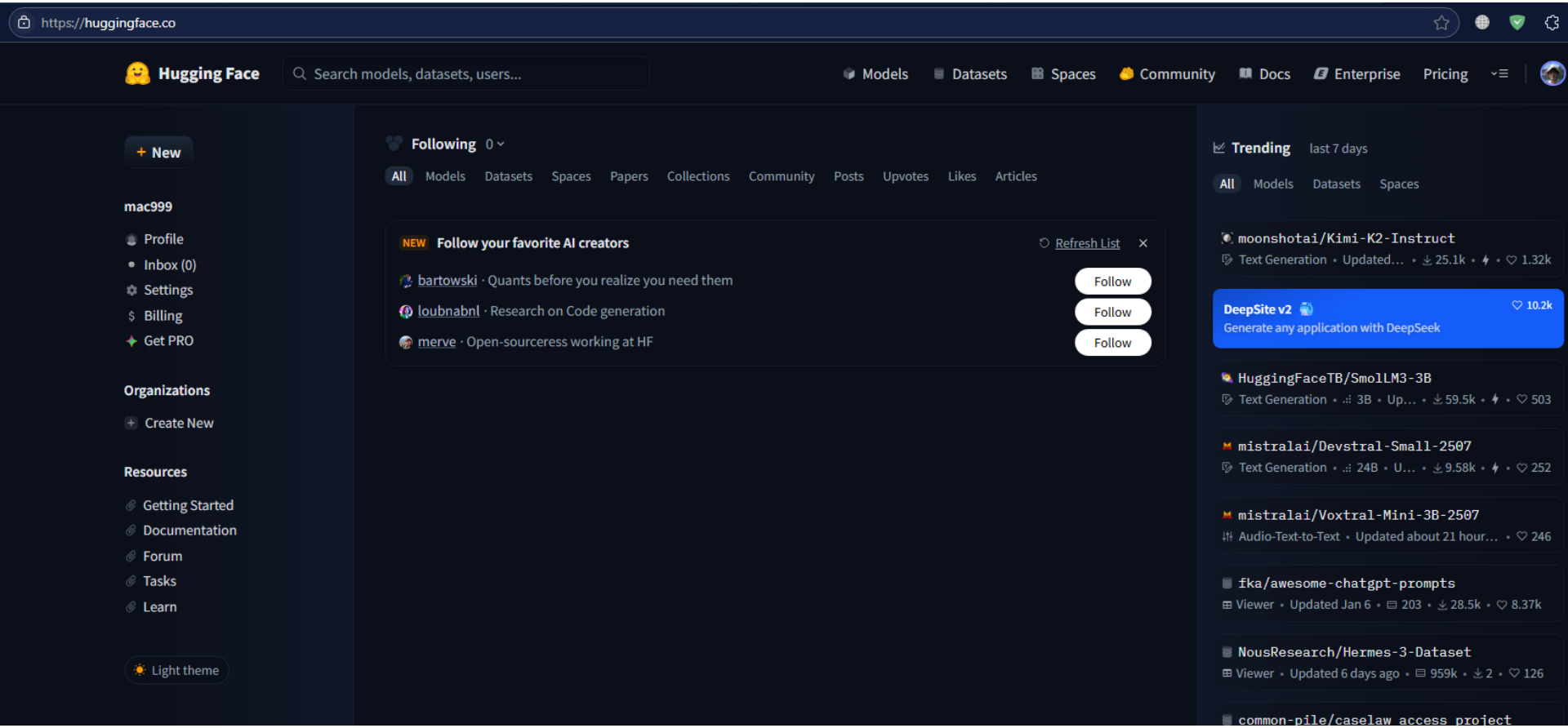
3. 모든 n-gram precision에 대해 log를 취하고 평균한다.

4. 기하 평균 = exp(로그 평균)

5. Brevity Penalty (BP) 계산:
   - candidate 길이 < reference 길이면: BP = exp(1 - reference_len / candidate_len)
   - candidate 길이 >= reference 길이면: BP = 1

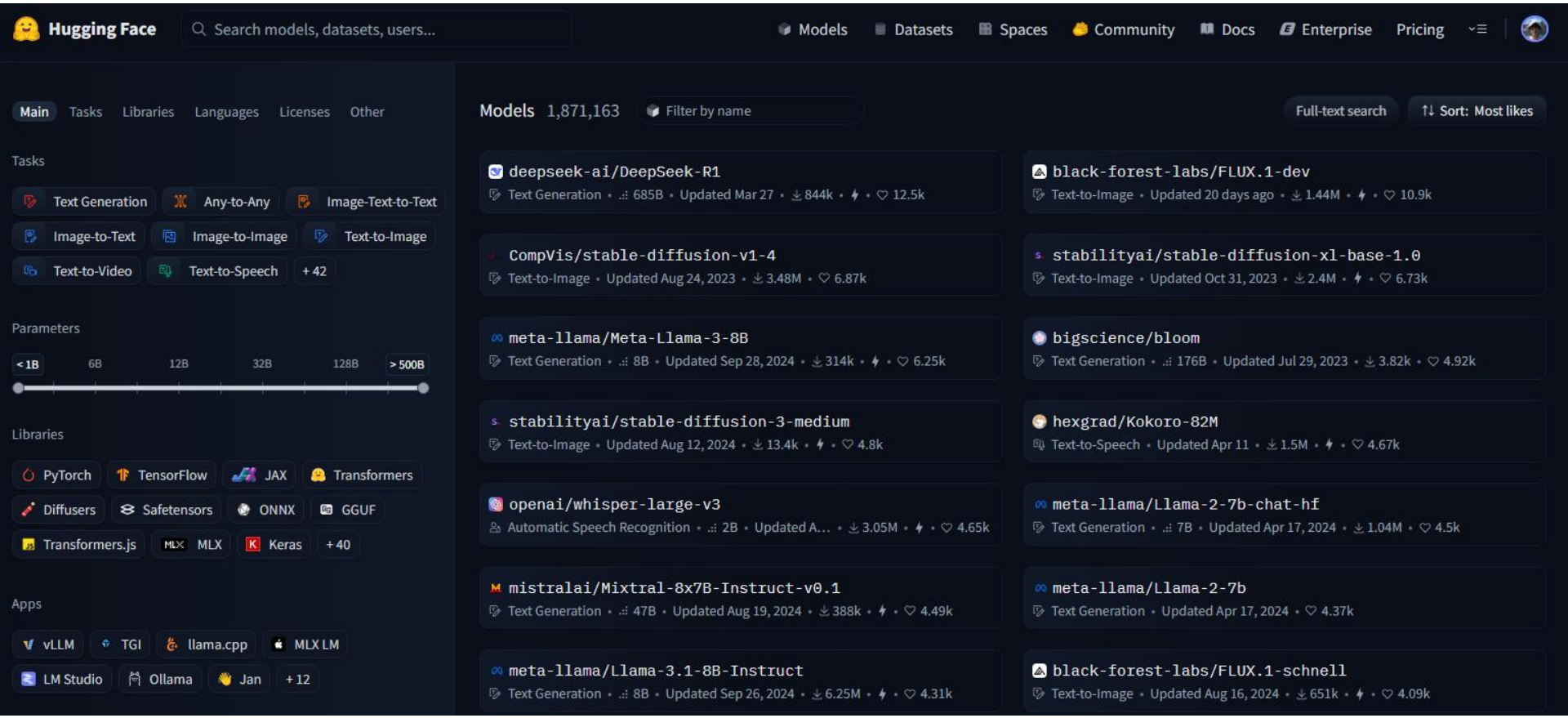
6. BLEU = BP × 기하 평균

# 자연어 처리의 기본 Hugging Face 모델 예시 소개



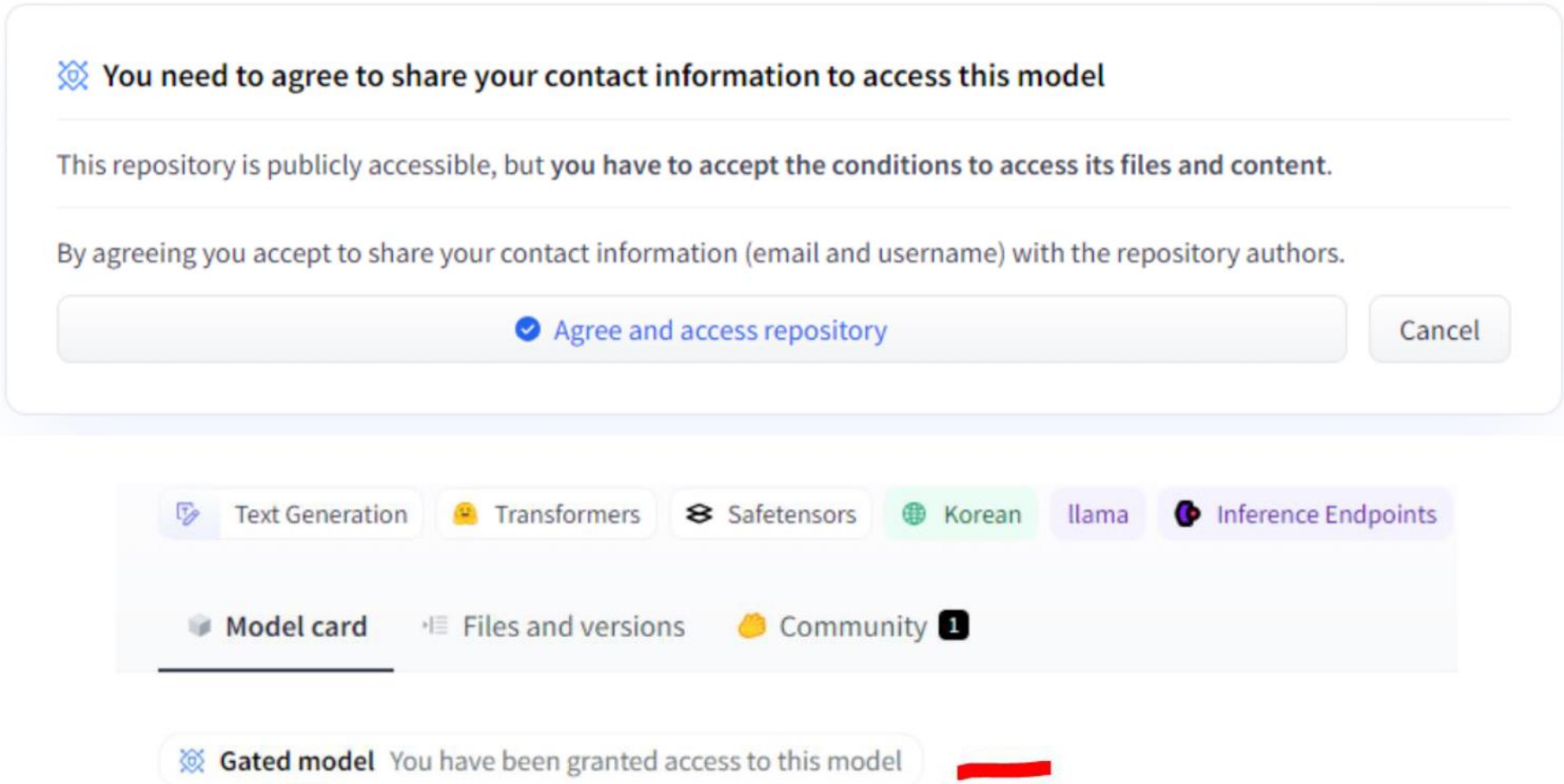

Hugging Face – The AI community building the future.

# 자연어 처리의 기본 Hugging Face 모델 예시 소개

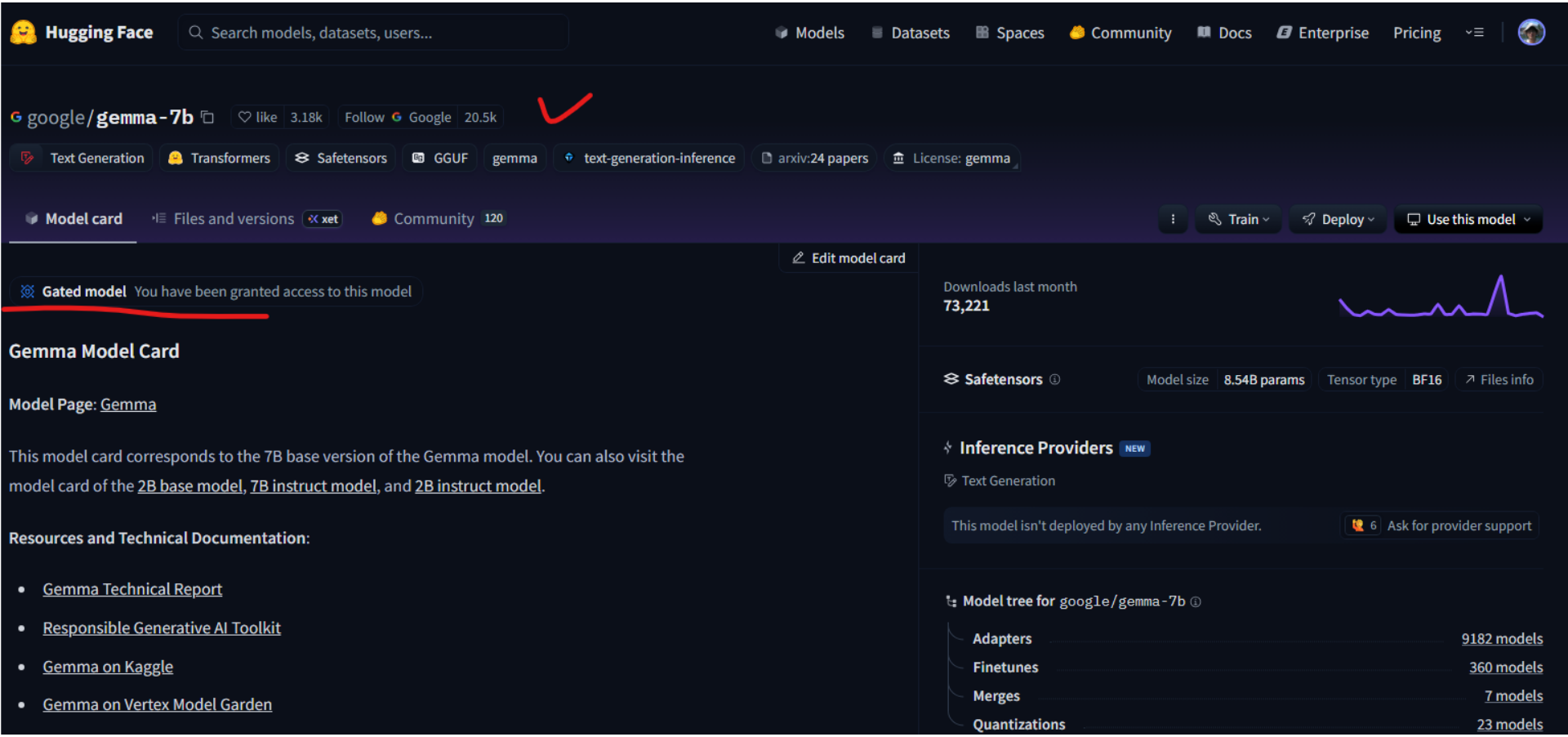


# 자연어 처리의 기본 Hugging Face 모델 예시 소개

google/gemma-2b-it · Hugging Face



# 자연어 처리의 기본 Hugging Face 모델 예시 소개




google/gemma-7b · Hugging Face

# 자연어 처리의 기본 Hugging Face 모델 예시 소개

### Fine-tuning examples

You can find fine-tuning notebooks under the `examples/` directory. We provide:

- A script to perform Supervised Fine-Tuning (SFT) on UltraChat dataset using QLoRA
- A script to perform SFT using FSDP on TPU devices
- A notebook that you can run on a free-tier Google Colab instance to perform SFT on English quotes dataset. You can also find the copy of the notebook here.

### Running the model on a CPU

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("google/gemma-7b")
model = AutoModelForCausalLM.from_pretrained("google/gemma-7b")

input_text = "Write me a poem about Machine Learning."
input_ids = tokenizer(input_text, return_tensors="pt")

outputs = model.generate(**input_ids)
print(tokenizer.decode(outputs[0]))
```

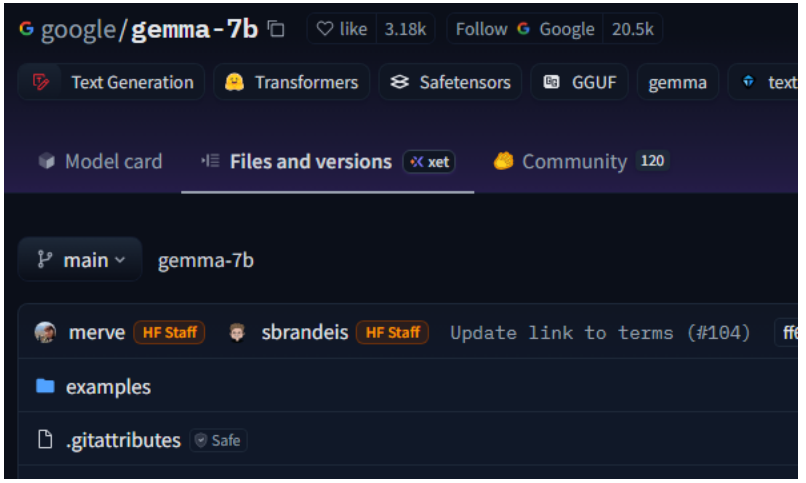
### Running the model on a single / multi GPU

```
# pip install accelerate
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("google/gemma-7b")
model = AutoModelForCausalLM.from_pretrained("google/gemma-7b", device_map="auto")

input_text = "Write me a poem about Machine Learning."
input_ids = tokenizer(input_text, return_tensors="pt").to("cuda")
```

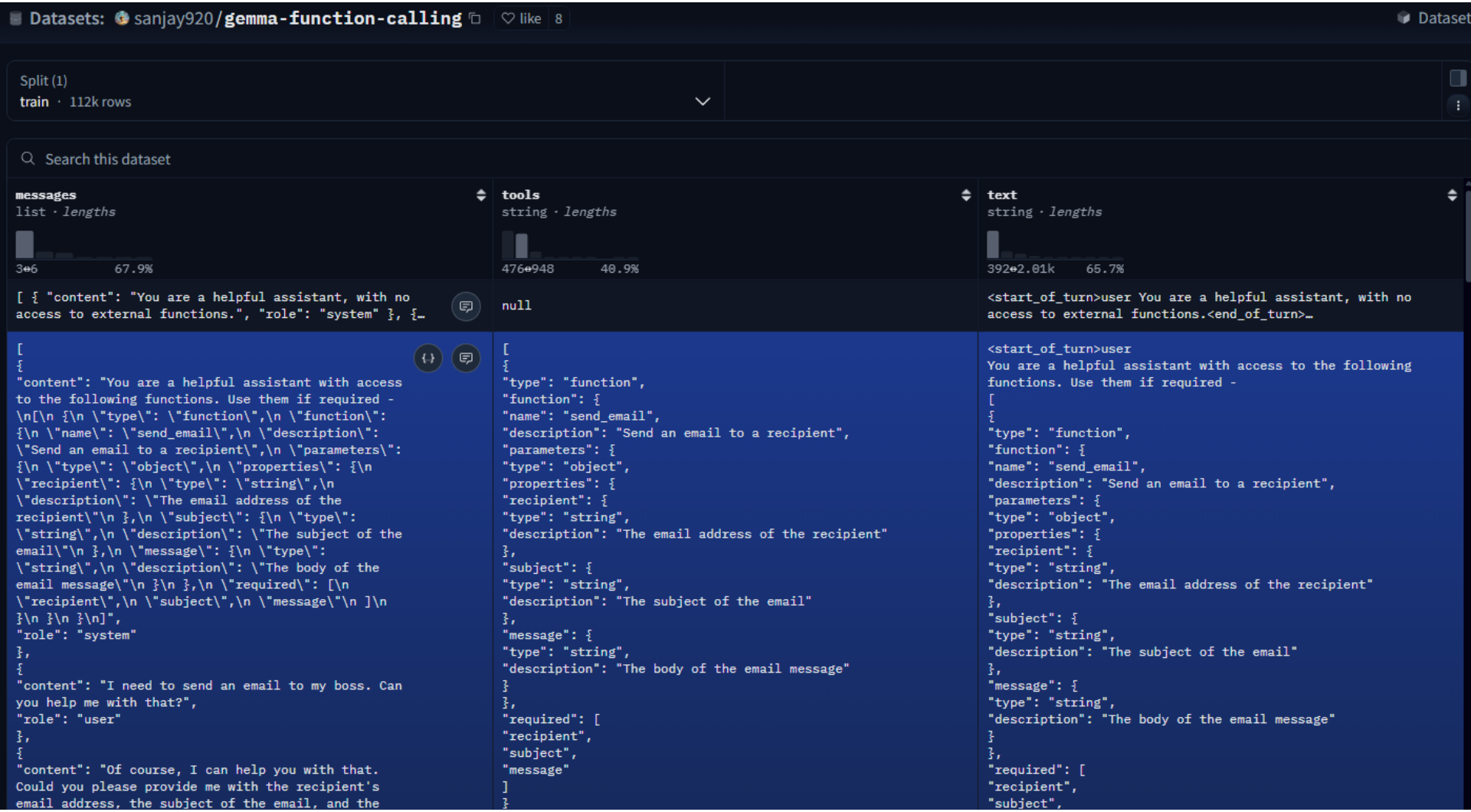
### Quantized Versions through `bitsandbytes`

- *Using 8-bit precision (int8)*

```
# pip install bitsandbytes accelerate
from transformers import AutoTokenizer, AutoModelForCaus

quantization_config = BitsAndBytesConfig(load_in_8bit=Tr

tokenizer = AutoTokenizer.from_pretrained("google/gemma-
model = AutoModelForCausalLM.from_pretrained("google/gem

input_text = "Write me a poem about Machine Learning."
input_ids = tokenizer(input_text, return_tensors="pt").t

outputs = model.generate(**input_ids)
print(tokenizer.decode(outputs[0]))
```

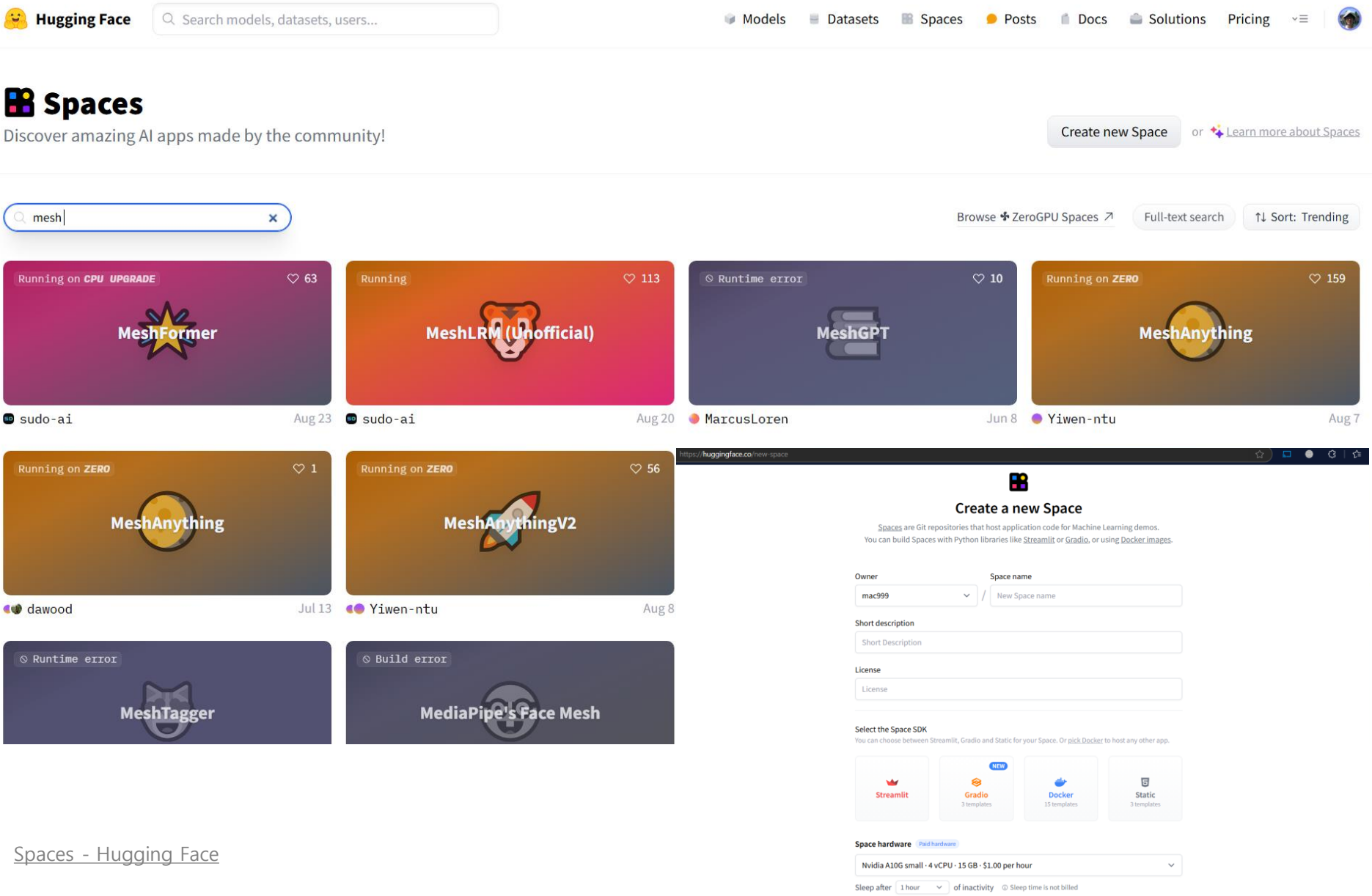


# 자연어 처리의 기본 Hugging Face 모델 예시 소개




google/gemma-7b · Hugging Face

# 자연어 처리의 기본 Hugging Face 모델 예시 소개




Spaces - Hugging Face

# 자연어 처리의 기본 Hugging Face 모델 예시 소개

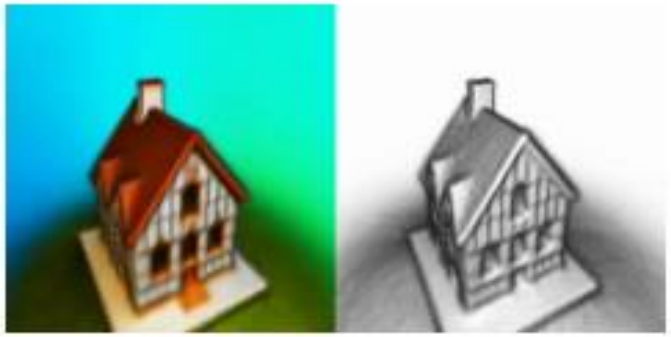
## DreamFusion: Text-to-3D using 2D Diffusion

Ben Poole
Google Research

Ajay Jain
UC Berkeley

Jonathan T. Barron
Google Research

Ben Mildenhall
Google Research

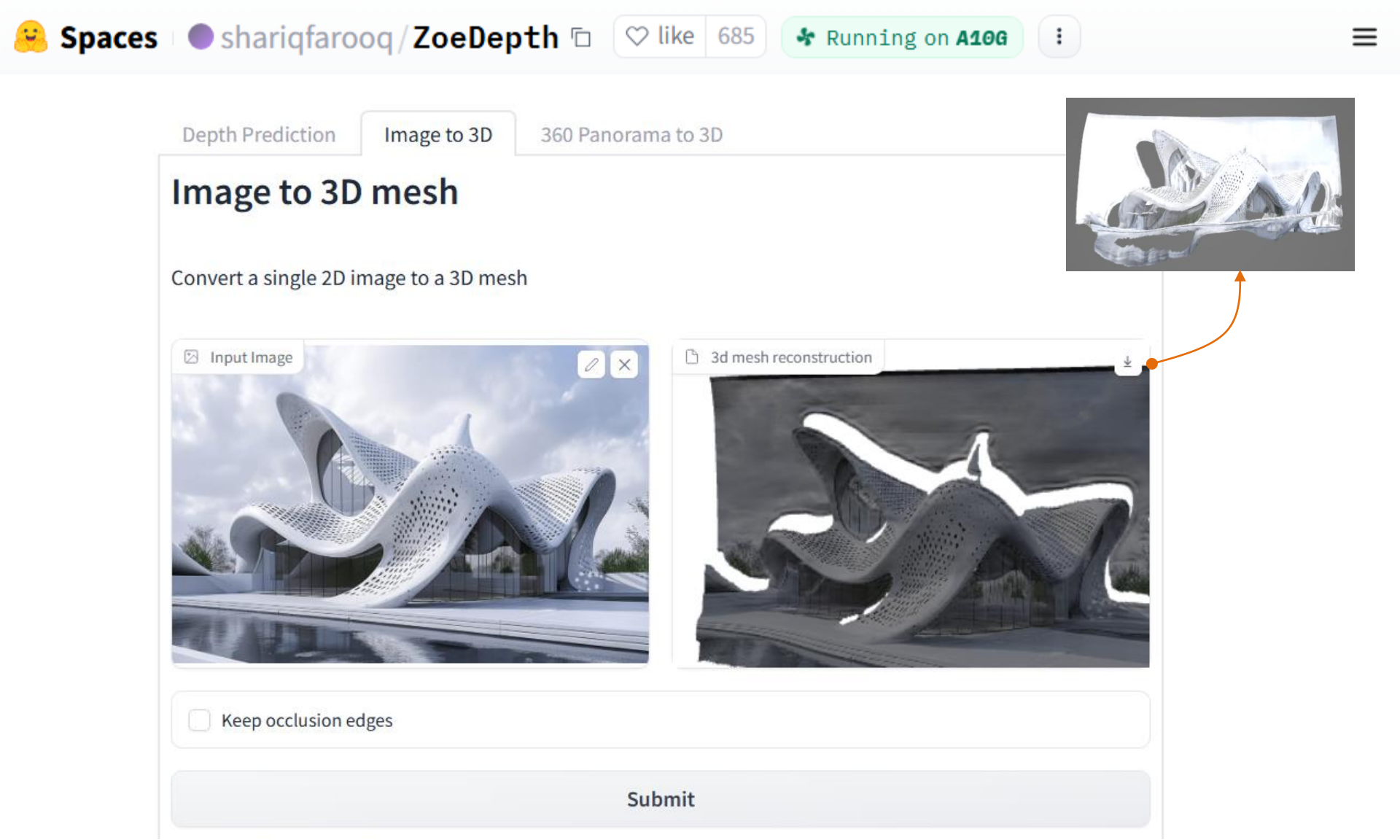


[...] a model of a house in Tudor style

[...] Sydney opera house, aerial view

DreamFusion: Text-to-3D using 2D Diffusion

# 자연어 처리의 기본 Hugging Face 모델 예시 소개




isl-org/ZoeDepth: Metric depth estimation from a single image

ZoeDepth - a Hugging Face Space by shariqfarooq

# 자연어 처리의 기본 Hugging Face 모델 예시 소개

Spaces | shariqfarooq / ZoeDepth | like 685 | Running on A10G

Depth Prediction | Image to 3D | 360 Panorama to 3D

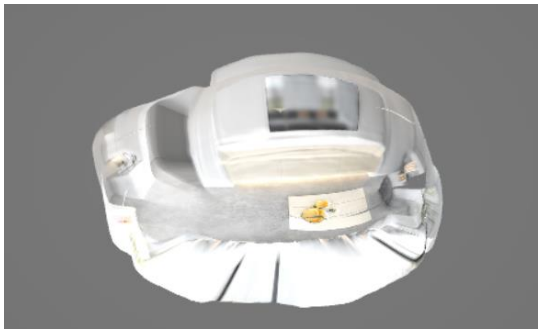
## Panorama to 3D mesh

Convert a 360 spherical panorama to a 3D mesh

ZoeDepth was not trained on panoramic images. It doesn't know anything about panoramas or spherical projection. Here, we just treat the estimated depth as radius and some projection errors are expected. Nonetheless, ZoeDepth still works surprisingly well on 360 reconstruction.

Input Image

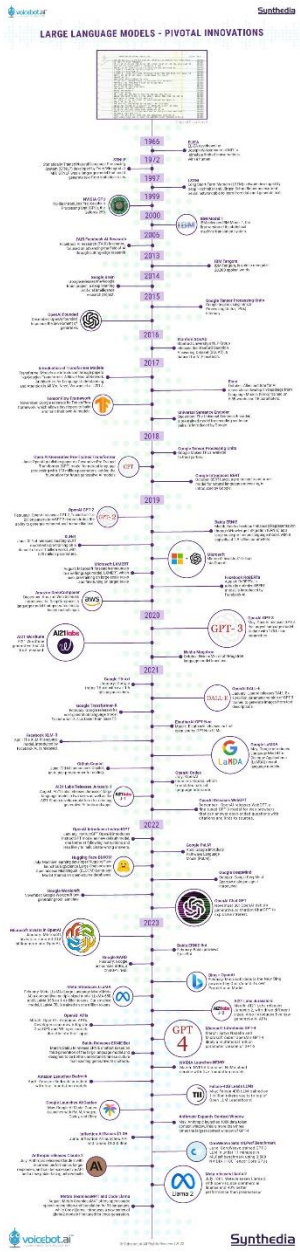
3d mesh reconstruction

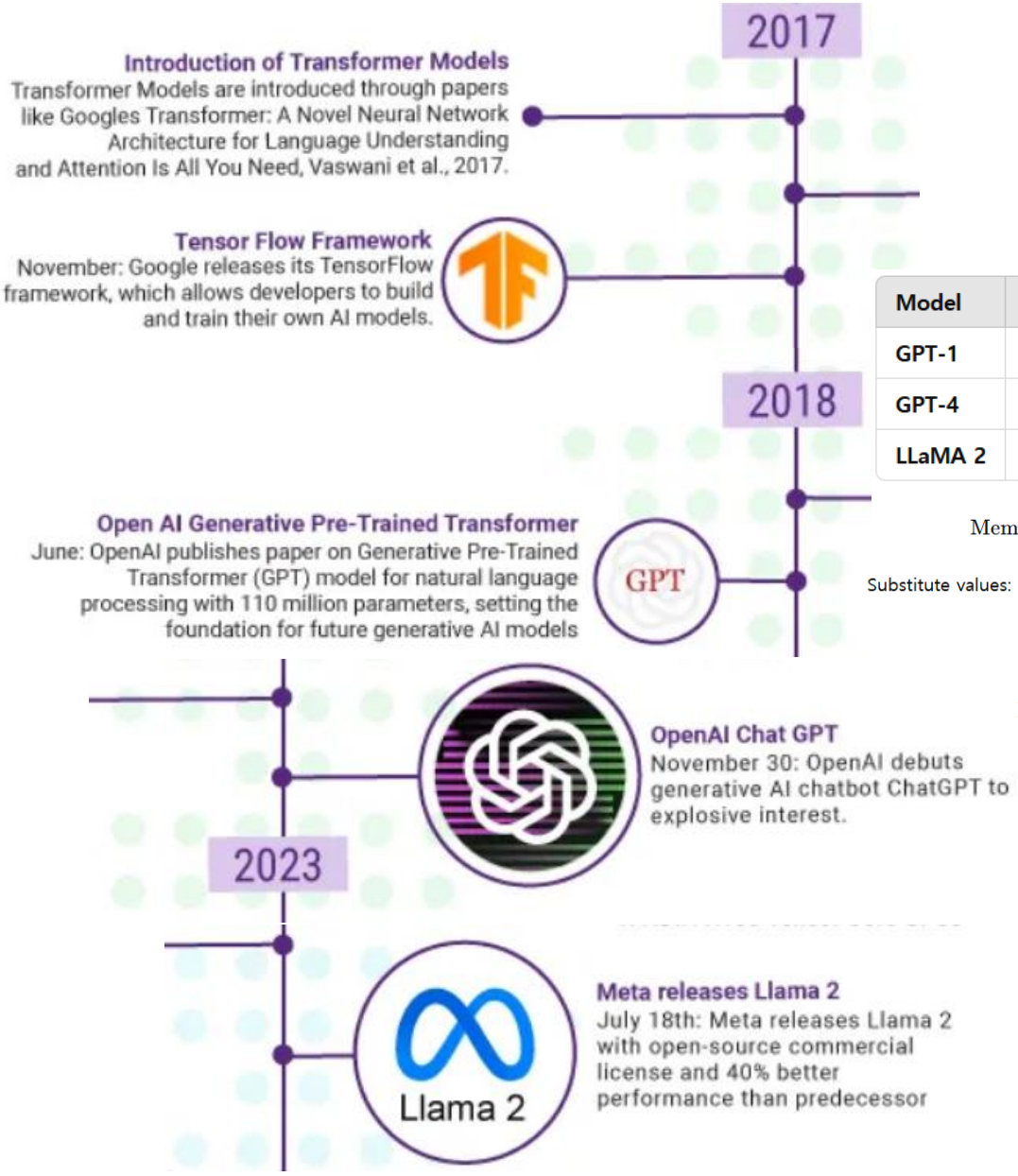


# LLM 과 AI Agent

- LLM
- RAG
- 오픈소스와 Ollama
- AI Agent
- Gen AI와 Vibe Coding

# LLM 과 AI Agent LLM




$$y=f(w\cdot x+b)$$

| Model | Parameters | Context Length |
|---|---|---|
| **GPT-1** | 117M | 512 tokens |
| **GPT-4** | 175B - 1T (approx) | Up to 32,000 tokens |
| **LLaMA 2** | 7B, 13B, 70B | 4,096 tokens |

$$\text{Memory (GB)} = \frac{\text{Parameters} \times \text{Bytes per Parameter}}{1024^3}$$

Substitute values:

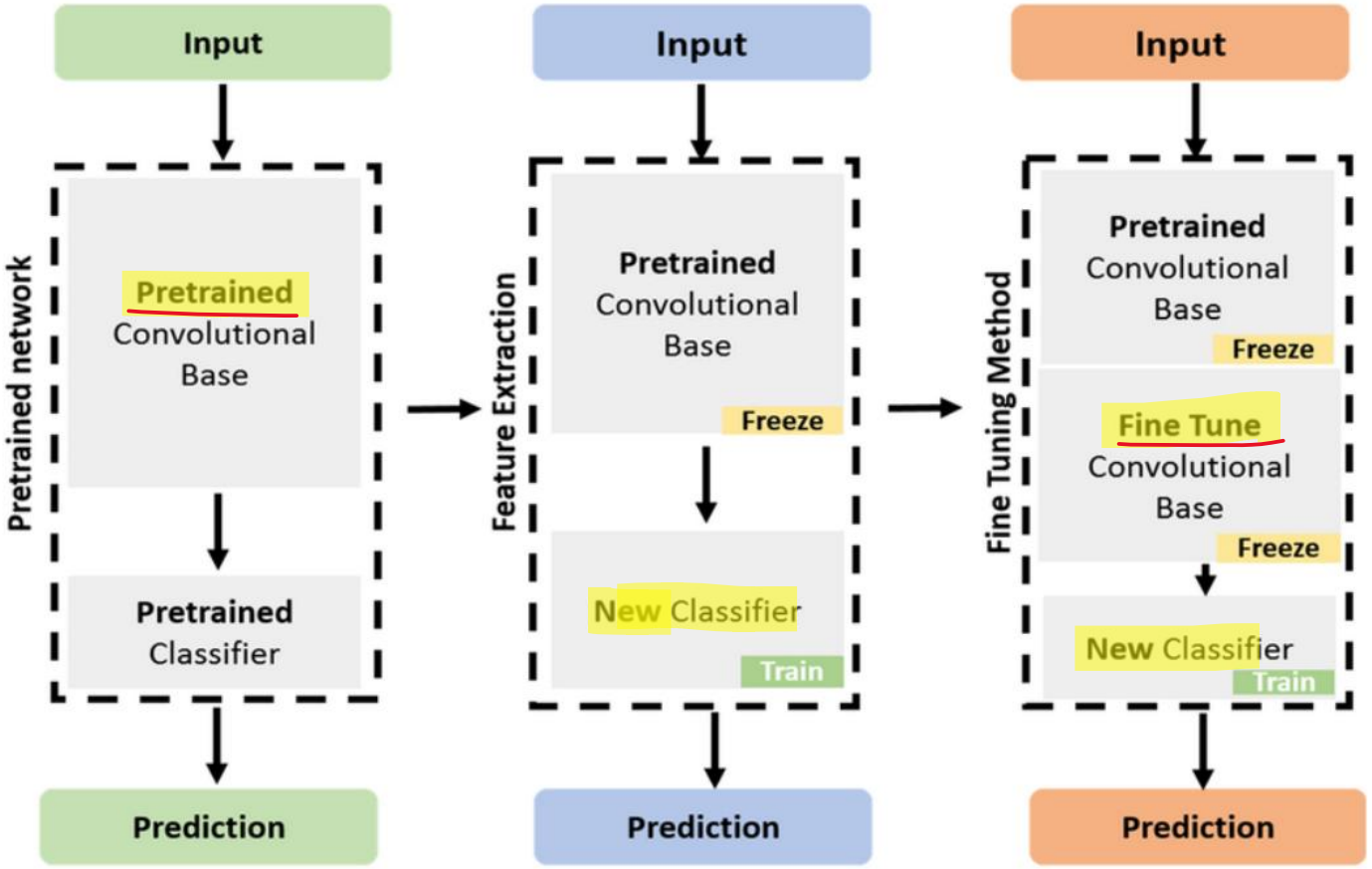
$$\text{Memory (GB)} = \frac{175 \times 10^9 \times 2}{1024^3}$$

$$\text{Memory (GB)} = \frac{350 \times 10^9}{1,073,741,824} \approx 327\,\text{GB}$$

A Timeline of Large Language Model Innovation

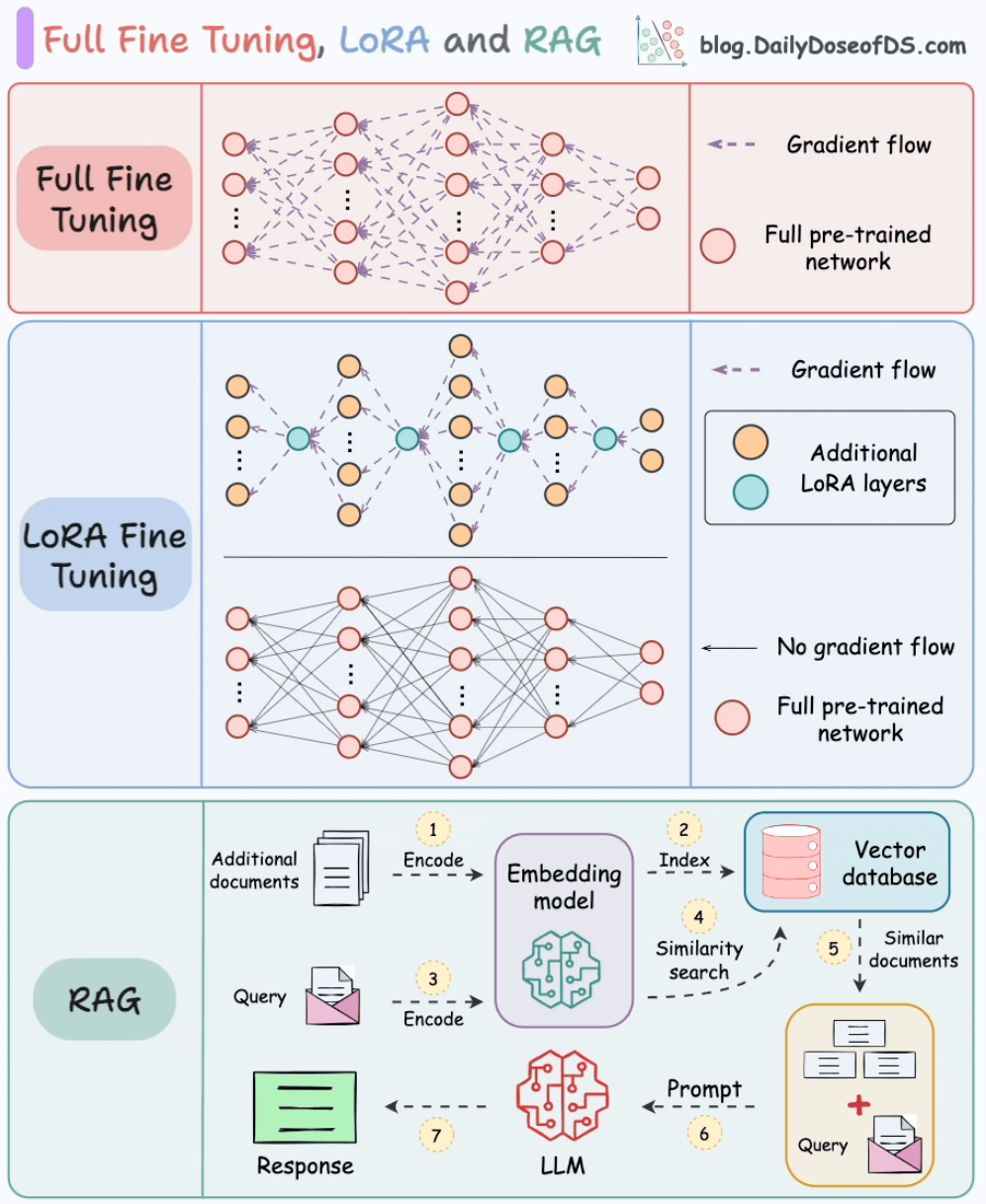
# LLM 과 AI Agent LLM fine turning

대규모 언어 모델(LLM, Large Language Model)은 막대한 연산 자원을 요구하며, 전체 파라미터를 파인튜닝하는 방식은 메모리 사용량이 높고 계산 비용도 큼
효율적으로 도메인 특화 파인튜닝을 수행할 수 있는 방식으로 LoRA(Low-Rank Adaptation)과 같은 PEFT(Parameter-Efficient Fine-Tuning) 이 사용되고 있음




A deep learning-based framework for detecting COVID-19 patients using chest X-rays | Request PDF

# LLM 과 AI Agent LLM fine turning




$z=f(W \cdot x+b)$

$W'=W+\Delta W$

$\Delta W=A \cdot B$

Retrieval-Augmented Generation

Full-model Fine-tuning vs. LoRA vs. RAG - by Avi Chawla

# LLM 과 AI Agent LLM fine turning

## No fine tuning

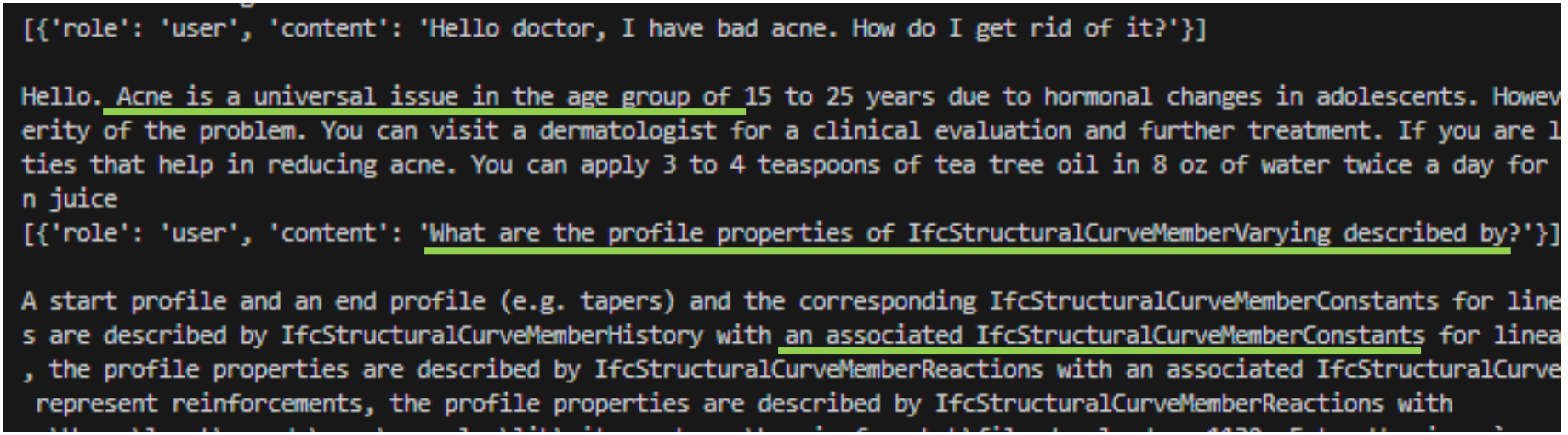
```
[{'role': 'user', 'content': 'Hello doctor, I have bad acne. How do I get rid of it?'}]
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no truncation.

The first thing you need to do is to keep your face clean. Wash your face twice a day with a mild soap. Do not use any oil base make up. Use only water base make up. Do not use any oil base cream. Use only water base crea
m. Do not use any oil base perfume. Use only water base perfume. Do not use any oil base toothpaste. Use only water base toothpaste. Do not use any oil base shaving cream. Use only water base shaving cream. Do not use any
 oil base hair oil. Use only water base hair oil. Do not use any oil base hair cream


[{'role': 'user', 'content': 'What are the profile properties of IfcStructuralCurveMemberVarying described by?'}]

What are the profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying
```


```
[{'role': 'user', 'content': 'Hello doctor, I have bad acne. How do I get rid of it?'}]
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length.

The first thing you need to do is to keep your face clean. Wash your face twice a day with a mild soap. Do not use
m. Do not use any oil base perfume. Use only water base perfume. Do not use any oil base toothpaste. Use only water
 oil base hair oil. Use only water base hair oil. Do not use any oil base hair cream


[{'role': 'user', 'content': 'What are the profile properties of IfcStructuralCurveMemberVarying described by?'}]

What are the profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying described by? So...
The profile properties of IfcStructuralCurveMemberVarying
```

# LLM 과 AI Agent LLM fine turning

## After fine tuning

```
[{'role': 'user', 'content': 'Hello doctor, I have bad acne. How do I get rid of it?'}]

Hello. Acne is a universal issue in the age group of 15 to 25 years due to hormonal changes in adolescents. However, there are multiple effective treatment options depending on the grade and sev
erity of the problem. You can visit a dermatologist for a clinical evaluation and further treatment. If you are looking for home remedies, you can apply tea tree oil. It has antibacterial proper
ties that help in reducing acne. You can apply 3 to 4 teaspoons of tea tree oil in 8 oz of water twice a day for three weeks. Additionally, you can apply honey with lemon juice twice a day. Lemo
n juice
[{'role': 'user', 'content': 'What are the profile properties of IfcStructuralCurveMemberVarying described by?'}]

A start profile and an end profile (e.g. tapers) and the corresponding IfcStructuralCurveMemberConstants for linear structural elements. For non-linear structural elements, the profile propertie
s are described by IfcStructuralCurveMemberHistory with an associated IfcStructuralCurveMemberConstants for linear structural elements only. For structural elements that represent reinforcements
, the profile properties are described by IfcStructuralCurveMemberReactions with an associated IfcStructuralCurveMemberConstants for linear structural elements only. For structural elements that
 represent reinforcements, the profile properties are described by IfcStructuralCurveMemberReactions with
```



# LLM 과 AI Agent LLM fine turning

fine tuning. Epoch = 10 (over 5 hours)

```
[{'role': 'user', 'content': 'Hello doctor, I have bad acne. How do I get rid of it?'}]
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length.

Hello. Acne is a universal issue in the age group of 15 to 25 years due to hormonal changes in adolescents. Howeve
erity of the problem. Grade 1 to grade 3 can be treated conservatively, while grade 4 often requires oral antibiot
Skin care products like Cetaphil and Clinique can be used at home. However, I suggest you consult a dermatologist
me know if I can assist you further. Regards, For more information consult a dermatologist online -->  https://bit
ily Physician, Dermatology Specialist, Skin Care Specialist, Acne Specialist, Cosmetic Dermatology, Anti-Aging Spe
ecialist, and Cosmetic injection specialist. For more information consult a dermatologist online -->  https://bit.
ovided me. Please do not hesitate to ask further clarifications. Regards, For more information consult a dermatolo


[{'role': 'user', 'content': 'What are the profile properties of IfcStructuralCurveMemberVarying described by?'}]

A start profile and an end profile (e.g. tapers) and the corresponding IfcStructuralCurveMemberEndProfile and IfcS
Type. In addition, a sweep operation is described by IfcStructuralCurveMemberSweptAreaSolid. For IfcStructuralCurv
gCurve and IfcStructuralCurveMemberVaryingSurface pressures have to be defined as well as the tolerance for curve
Finally, the local axis system has to be defined too. For more information, see Löwner, M.O.; Gröger, G. In: Compu
: 1313-0499. [CrossRef] [Web of Science (WoS)] [SCI] [e-EL] [Export to PDF] [Metadata] [BIMcert Handbuch] [BIMcert
```

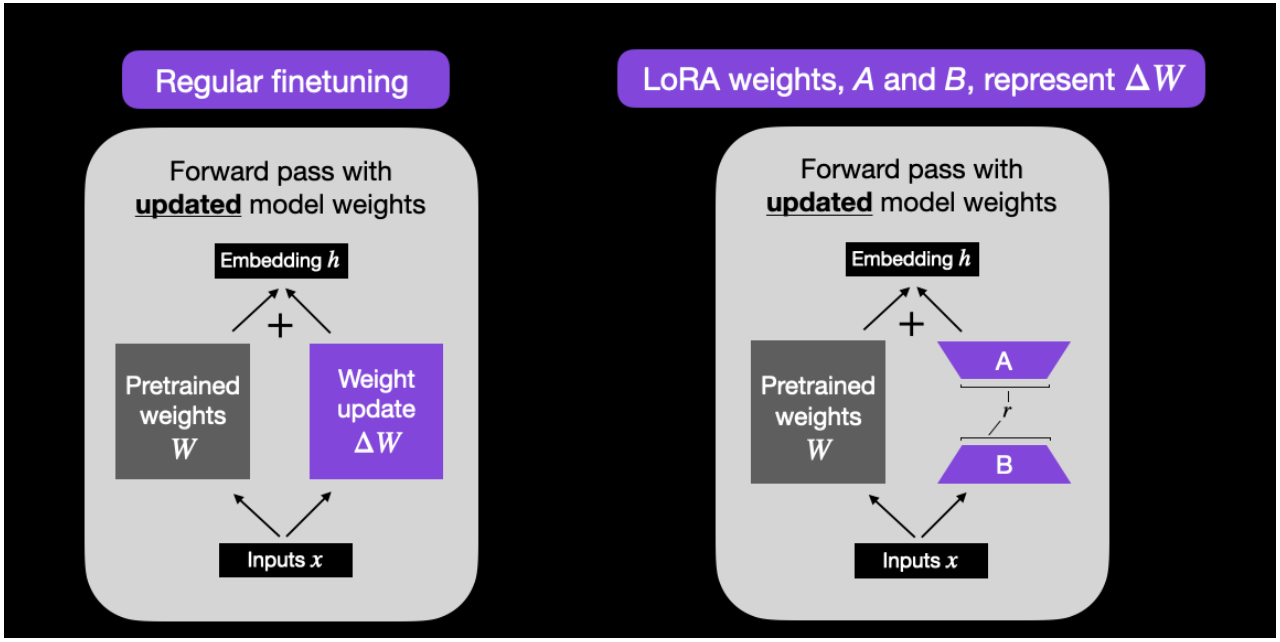
# LLM 과 AI Agent LLM fine turning

PEFT(Parameter-Efficient Fine-Tuning)는 사전 학습된 거대 언어 모델의 모든 가중치를 조정하지 않고, 특정 모듈 혹은 계층만을 학습 대상으로 설정함으로써 효율적인 파인튜닝을 가능하게 하는 접근 방식

LoRA: 저랭크 행렬을 삽입하여 Linear 연산을 보완함.
Adapter Tuning: Transformer 블록에 작은 네트워크를 덧붙여 학습함.
Prefix/Prompt Tuning: 입력 시퀀스에 도메인 특화 토큰을 삽입함.

LoRA는 self-attention이나 feedforward 블록의 선형 계층에 대해 고정된 원본 가중치에 저랭크 행렬을 추가함으로써 파라미터 수를 줄이고 효율적인 학습을 가능

$$y = Wx \qquad y = (W + BA)x$$




Code LoRA from Scratch

# LLM 과 AI Agent LLM fine turning

$$y = Wx \qquad y = (W + BA)x$$

```
LoraConfig(
    r=64,
    lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
```

r: 저랭크 행렬의 차원, 작을수록 연산량 감소, 클수록 표현력 증가
lora_alpha: 학습 중 scaling factor로 작동하며, 학습 안정성과 관련됨
target_modules: LoRA가 삽입될 Transformer 내 Linear 레이어 이름
lora_dropout: 학습 중 적용될 dropout 확률
bias: bias 항을 LoRA와 함께 학습할지 여부 (none, all, lora_only 등)
task_type: 파인튜닝 과제 종류 (CAUSAL_LM, SEQ_CLS 등). CAUSAL_LM: 다음 토큰을 순차적으로 예측하는 언어 생성 모델 (예: GPT류)
SEQ_CLS: 입력 전체에 대해 단일 클래스를 예측하는 분류 모델 (예: 감정 분석, 문장 분류)

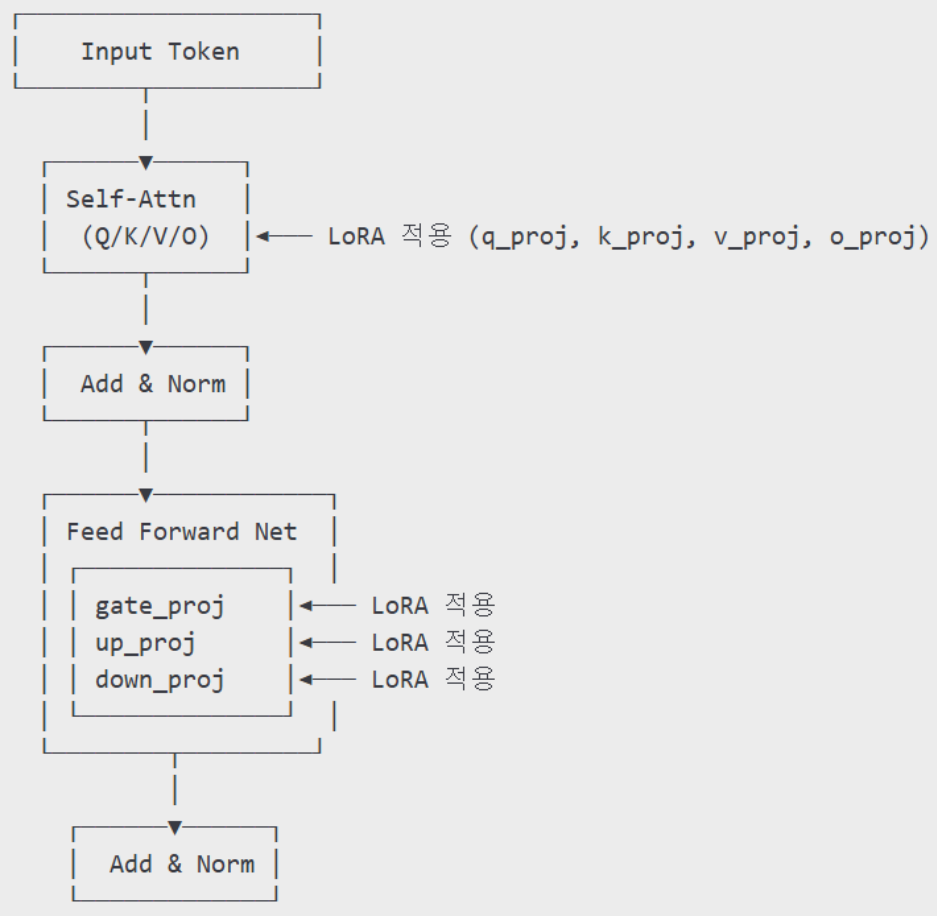
q_proj, k_proj, v_proj, o_proj: Self-Attention의 쿼리, 키, 밸류, 출력 projection에 대응함

- gate_proj, up_proj, down_proj: Transformer 블록의 Feedforward Network(FFN)에서 사용되는 세 개의 핵심 선형 계층이다.
- gate_proj: gating mechanism 적용 전 입력을 처리
- up_proj: hidden dimension을 확장함 (ex. 4096 → 11008)
- down_proj: 다시 차원을 줄이며 원래 출력으로 복원

# LLM 과 AI Agent LLM fine turning

q_proj, k_proj, v_proj, o_proj: Self-Attention의 쿼리, 키, 밸류, 출력 projection에 대응함

- gate_proj, up_proj, down_proj: Transformer 블록의 Feedforward Network(FFN)에서 사용되는 세 개의 핵심 선형 계층이다.
- gate_proj: gating mechanism 적용 전 입력을 처리
- up_proj: hidden dimension을 확장함 (ex. 4096 → 11008)
- down_proj: 다시 차원을 줄이며 원래 출력으로 복원



# LLM 과 AI Agent 양자화 (Quantization)

양자화는 모델 파라미터를 float 대신 int4/8로 표현하여 자원 사용을 줄이고 계산속도를 향상시키는 기술

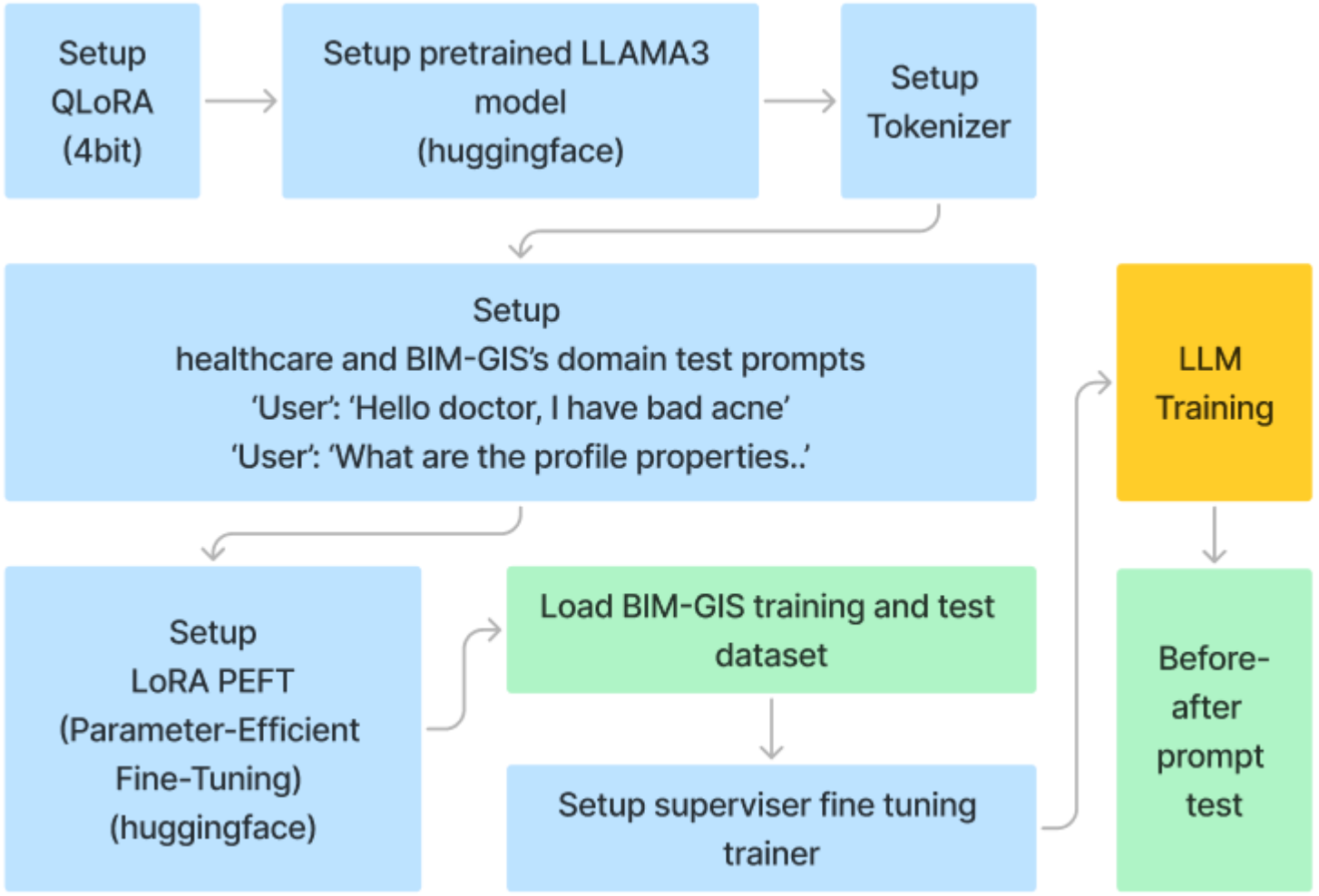
bitsandbytes는 Hugging Face에서 채택된 양자화 라이브러리이며, 특히 4bit 양자화에서 매우 효과적

```
BitsAndBytesConfig(
  load_in_4bit=True,
  bnb_4bit_use_double_quant=True,
  bnb_4bit_quant_type="nf4",
  bnb_4bit_compute_dtype=torch.bfloat16
)
```

load_in_4bit: 모델을 4bit로 로드할지 여부
bnb_4bit_use_double_quant: 이중 양자화 수행으로 표현력 향상
bnb_4bit_quant_type:
    nf4: NormalFloat4, 정규화된 4비트 부동소수점
    fp4: 기본 부동소수점 4비트 양자화, 정밀도는 낮음

bnb_4bit_compute_dtype: 계산 dtype (bfloat16, float16, float32) 지정. 일반적으로 bfloat16 사용

# LLM 과 AI Agent LLM fine turning




```
{'loss': 2.6513, 'grad_norm': 1.0972543954849243, 'learning_rate': 0.00019908787541713017, 'epoch': 0.11}
{'loss': 2.3663, 'grad_norm': 1.041242241859436, 'learning_rate': 0.00019906562847608455, 'epoch': 0.12}
{'loss': 2.4729, 'grad_norm': 1.1358354091644287, 'learning_rate': 0.00019904338153503895, 'epoch': 0.12}
{'loss': 2.3378, 'grad_norm': 1.1035994291305542, 'learning_rate': 0.00019902113459399333, 'epoch': 0.12}
{'loss': 2.3338, 'grad_norm': 1.300874948501587, 'learning_rate': 0.00019899888765294771, 'epoch': 0.12}  1{'
```

# LLM 과 AI Agent LLM fine turning

**Pretraining**

- **Dataset:**
  - Raw text
  - Few trillions of tokens
- **Task:**
  - Next word prediction
- **Compute:**
  - O(1-10K) GPUs
  - Weeks/months of training

**Instruction-Tuning**

- **Dataset:**
  - Paired: (Prompt, Response)
  - O(10-100K instructions)
- **Task:**
  - Next word prediction (Masked)
- **Compute:**
  - O(1-100) GPUs
  - Few hrs/days

**Learning from Human Feedback**

- **Dataset:**
  - Human Preference Data
  - O(10-100K)
- **Task:**
  - RLHF/DPO
- **Compute:**
  - O(1-100) GPUs
  - Few hrs/days

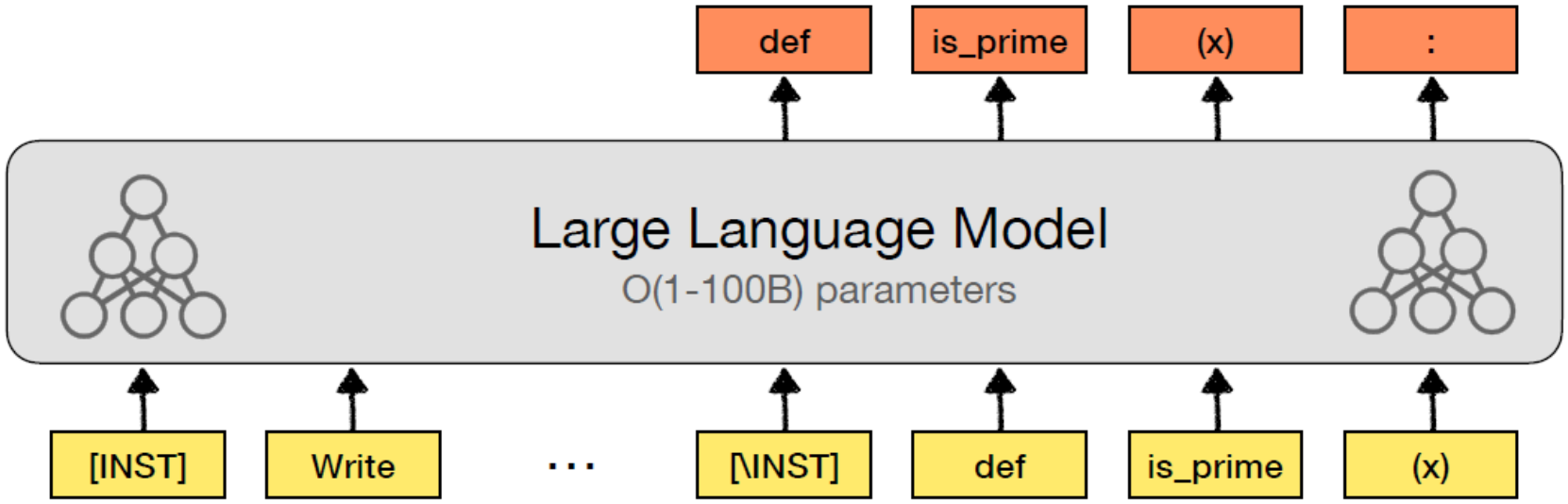
# LLM 과 AI Agent LLM fine turning

Prompt:

[INST] Write a python function to find whether the input number is prime. [\INST]

Response:

```
def is_prime(x: int):
  """
  takes as input an integer x. Returns True if x is prime and False otherwise
  """
  if x <= 1:
      return False
  for i in range(2, int(x ** 0.5) + 1):
      if x % i == 0:
          return False
  return True
```



# LLM 과 AI Agent LLM fine turning

Expand to see the descriptions of 34 models

Model A

Write a python function to find whether the input number is prime.

Here is a Python function to check if a given number is prime:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

# Test the function
number = int(input("Enter a number: "))
if is_prime(number):
    print(f"{number} is a prime number.")
else:
    print(f"{number} is not a prime number.")
```

You can use this function to determine whether a given number is prime or not.

Model B

Write a python function to find whether the input number is prime.

Sure, let's break down the problem into steps and then I'll write the Python function for you.

A number is considered prime if it has exactly two factors: 1 and itself. All numbers greater than 1 have at least three factors, one of which is 1, so we can start our range from 2.

Here is a Python function that checks whether a number is prime:

```
def is_prime(number):
    # We start from 2 because 1 and the number itself are always factors
    for i in range(2, int(number**0.5) + 1):
        # If we find a factor, the number is not prime
        if number % i == 0:
            return False

    # If no factors are found, the number is prime
    return True
```

This function takes an integer as input and returns a boolean value. It starts by checking if the number is less than or equal to 1. If so, it immediately returns `True` for 2 and `False` for all others because these are the only prime numbers below 12.
If the number is greater than 1, the function then checks for factors from 2 up to the square root

👈 A is better | 👉 B is better | 🤝 Tie | 👎 Both are bad

👉 Enter your prompt and press ENTER

Send

🎲 New Round | 🔄 Regenerate | 📷 Share

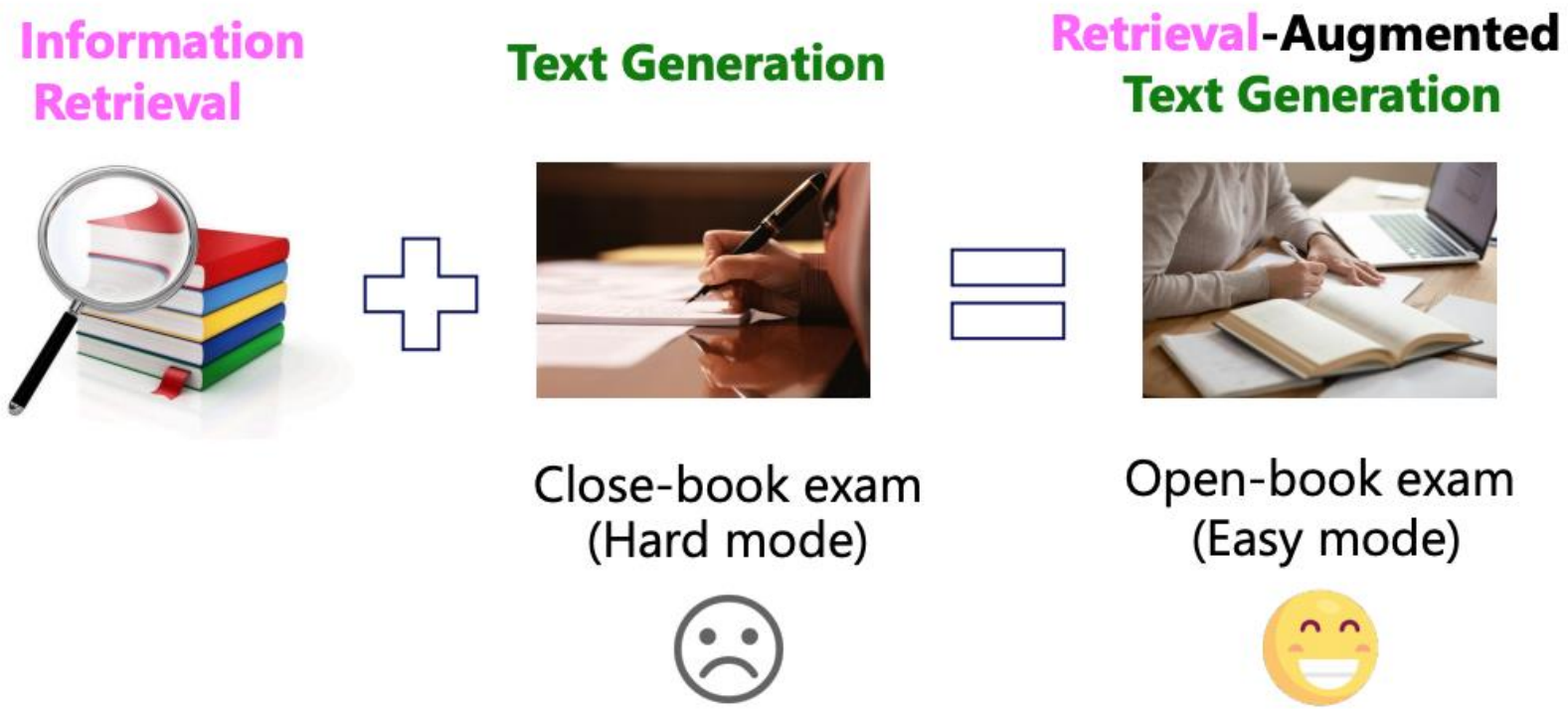
https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard

# LLM 과 AI Agent RAG

RAG(Retrieval-Augmented Generation)는 대규모 언어 모델(LLM: Large Language Model)의 한계를 극복하기 위해 외부 지식 기반을 검색해 생성 과정에 반영하는 방식

RAG는 외부 문서나 데이터베이스에서 관련 정보를 검색(Retrieval)하고, 이를 입력에 증강(Augmentation)하여, 최종적으로 텍스트를 생성(Generation)하는 세 단계로 구성

RAG는 2020 년Facebook AI Research(FAIR)에

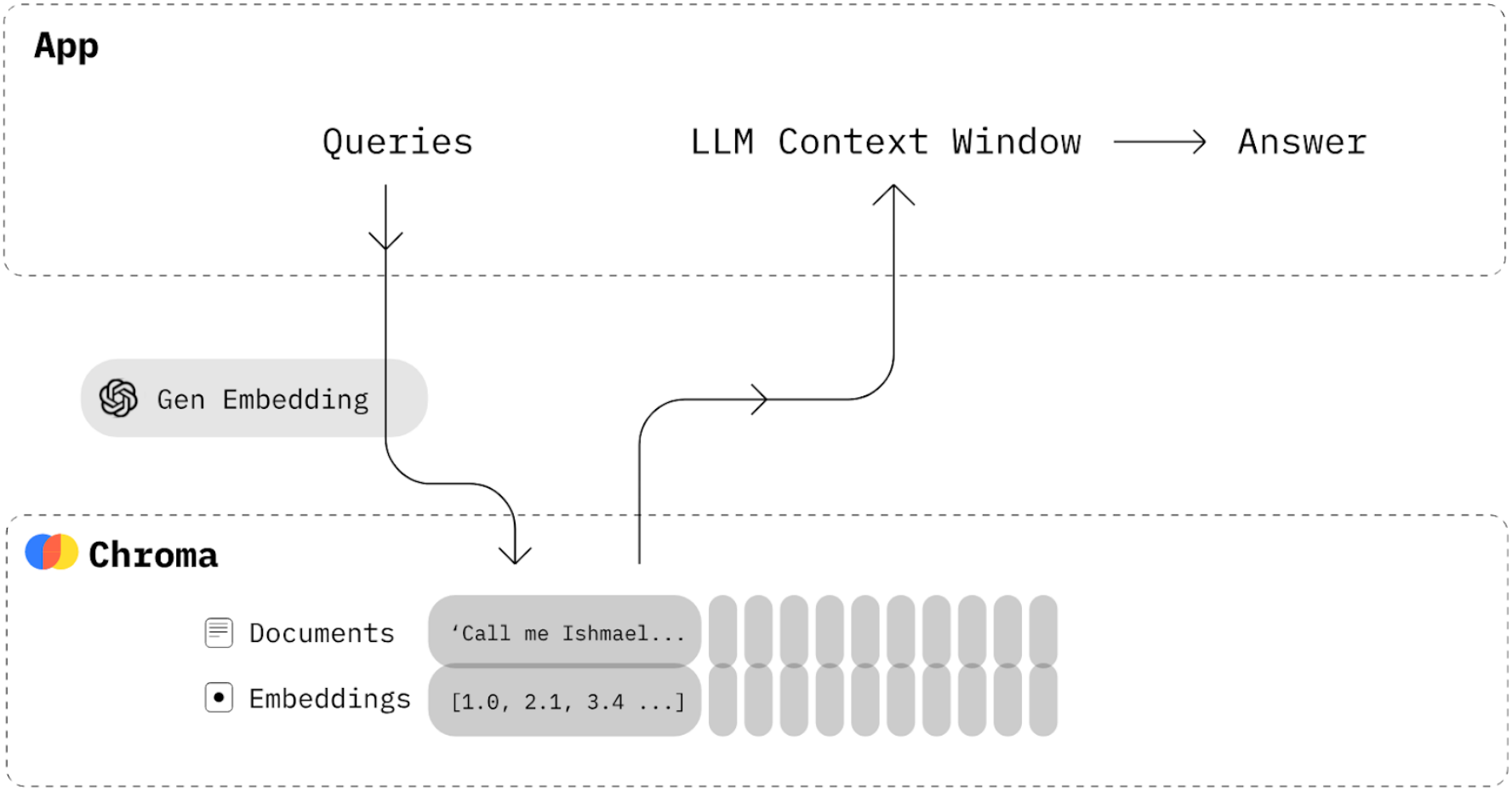


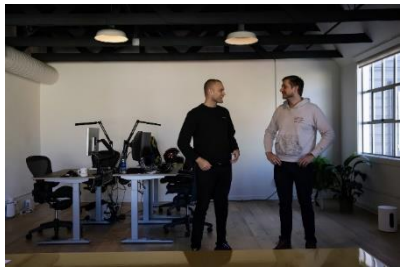

https://lemaoliu.github.io/retrieval-generation-tutorial/

# LLM 과 AI Agent RAG

| 도구 이름 | 설명 | 링크 |
|---|---|---|
| AutoGPT | 텍스트 기반 입력을 통해 자동으로 여러 작업을 수행하는 AI 에이전트. GPT 모델을 활용하여 다양한 작업을 자동화할 수 있도록 설계됨. | AutoGPT GitHub |
| Pydantic | AI 에이전트 및 모델 구축을 위한 데이터 검증 및 설정 도구. Pydantic을 사용하여 AI 모델의 입력과 출력을 구조적으로 관리하고 검증할 수 있다. | PydanticAI |
| LlamaIndex | Llama 모델을 활용하여 효율적인 데이터 검색 및 에이전트 기능을 구현하는 도구. 특히 대규모 텍스트 데이터 처리 및 검색에 강점. | LlamaIndex GitHub |
| CrewAI | 협업을 중심으로 AI 에이전트를 관리하는 도구. 여러 AI 에이전트를 연결하여 복잡한 문제를 해결하는데 유용. | CrewAI Website |
| LangGraph | 그래프 데이터베이스 기반의 AI 에이전트 관리 도구로, 복잡한 데이터 관계를 효과적으로 처리하고 탐색할 수 있도록 돕는다. | LangGraph GitHub |

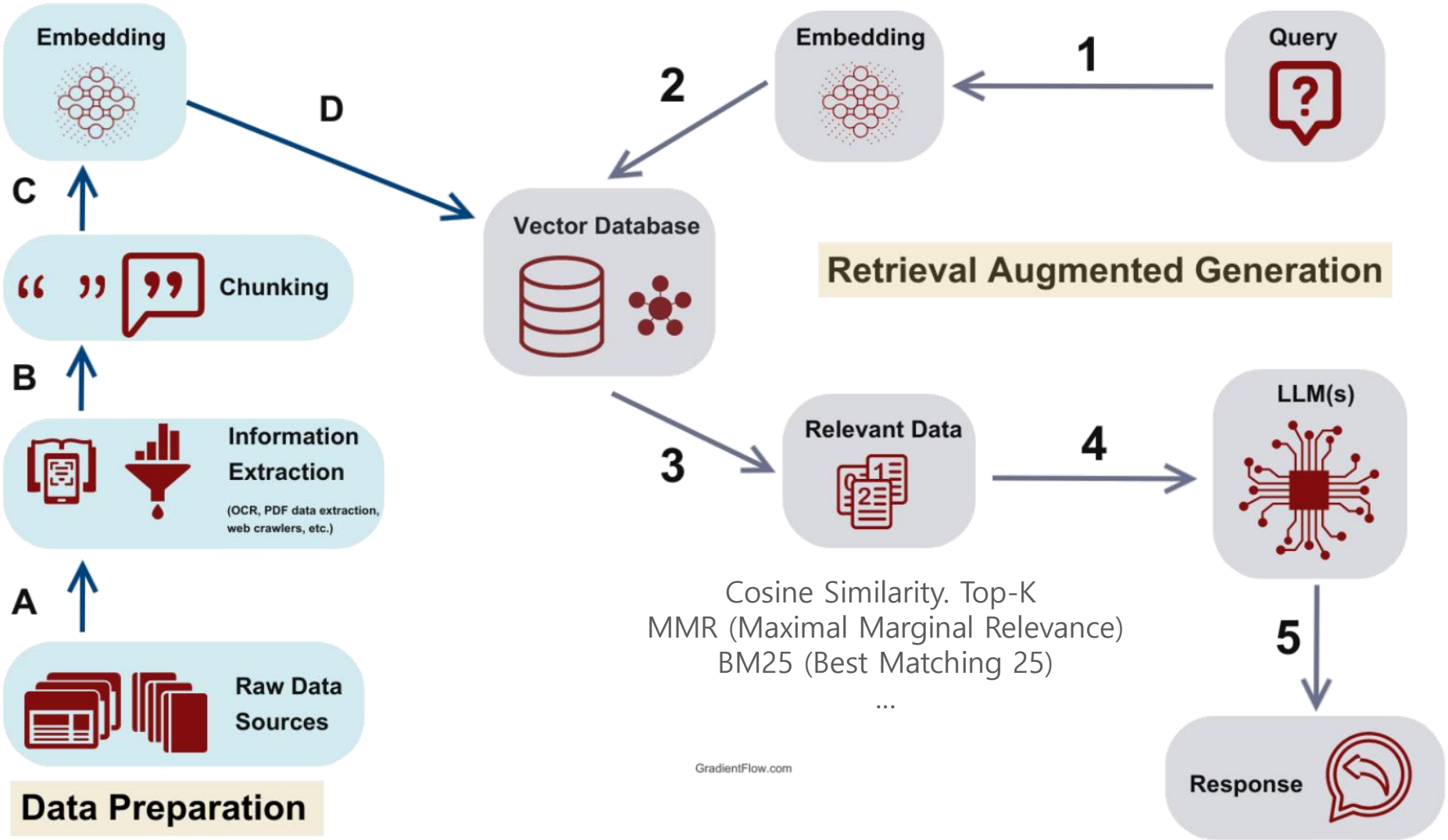
# LLM 과 AI Agent RAG



Chroma CEO. Jeff Huber, Anton Troynikov 18M$. 2023

# LLM 과 AI Agent RAG




https://gradientflow.substack.com/p/best-practices-in-retrieval-augmented

# LLM 과 AI Agent RAG

증강은 검색된 문서를 사용자의 원 질문과 결합하여 LLM 입력 프롬프트를 구성

You are a helpful assistant. Use the following context to answer the question.

Context:
{retrieved_context}

Question:
{user_question}

Answer:

문맥이 너무 길면 문서를 청킹(chunking)하여 일부만 선택해야 함

https://gradientflow.substack.com/p/best-practices-in-retrieval-augmented

# LLM 과 AI Agent RAG

생성은 프롬프트를 LLM(OpenAI GPT-4, Claude, LLaMA 등)에 전달하여 응답을 출력

1. Augmented Prompt를 형성한다 (사용자 질문 + 검색된 문서).
2. 해당 Prompt를 LLM의 입력으로 전달한다.
3. LLM은 사전 학습된 확률 기반 언어 모델링을 통해 다음 토큰을 반복 생성하며 응답을 생성한다.
4. 필요 시, temperature, top-k, top-p 같은 샘플링 파라미터를 조정하여 생성 다양성과 품질을 조절한다.

# LLM 과 AI Agent RAG

RAG 시스템에서 문서를 검색 가능하게 만들기 위해서는 먼저 문서를 처리 가능한 단위로 나눈 뒤, 각 단위를 벡터로 변환. 이 과정을 "청킹(chunking)"과 "임베딩(embedding)"

- LLM이나 임베딩 모델은 입력 토큰 수에 제한이 있으며, 긴 문서를 한 번에 처리할 수 없음
- 의미 있는 문단 또는 문장 단위로 분할하면 정보가 손실 방지. 검색 품질 개선
- 검색 단계에서 세부적인 단위(청크)로 비교함으로써, 질문과 가장 관련 있는 문맥을 찾아낼 수 있음

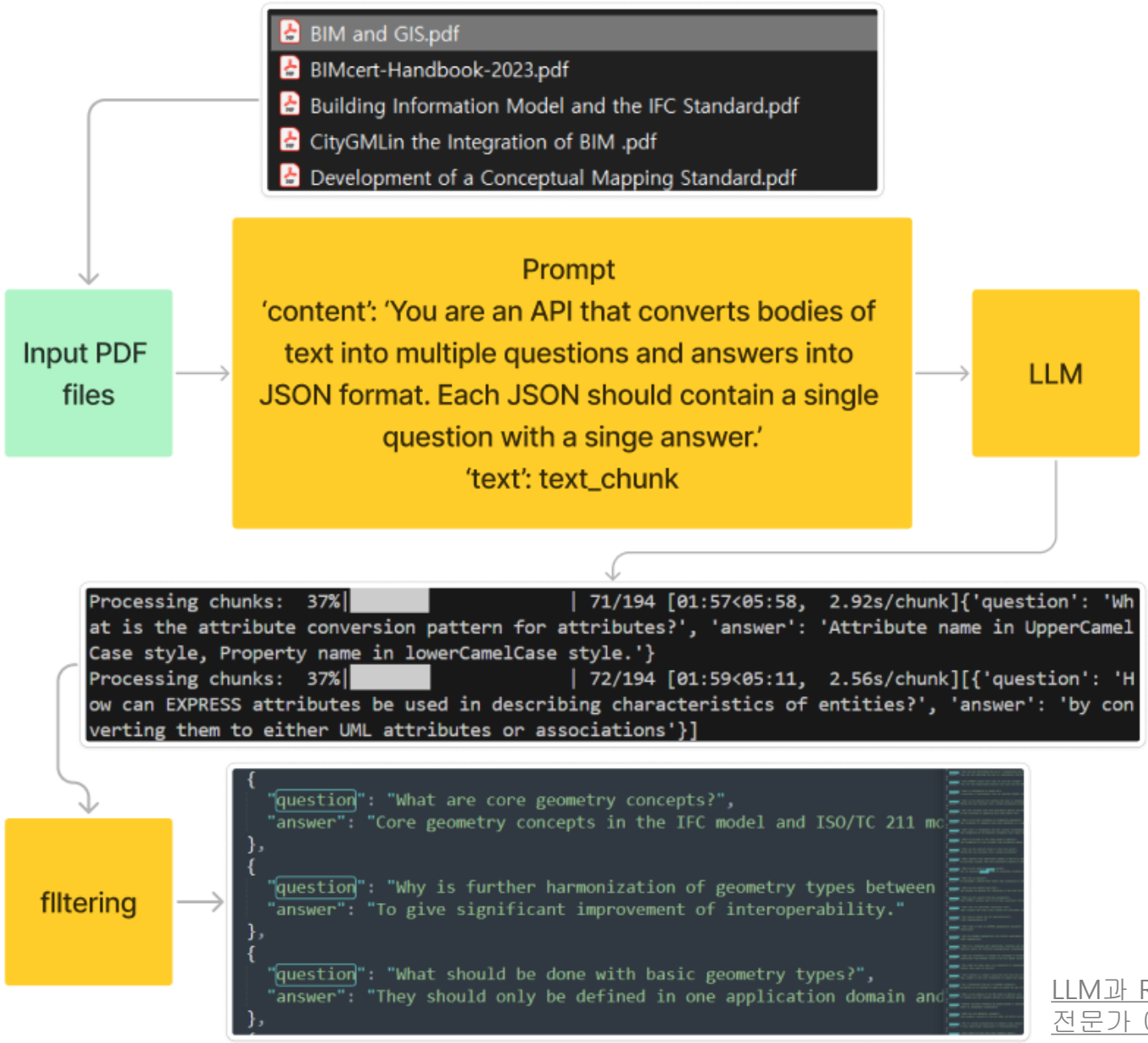
증강 생성 시 과도한 정보가 입력되면 노이즈가 발생할 수 있으므로, 적절한 단위로 잘라 사용하는 것이 성능 향상에 기여

```
# 청킹
chunks = split_document(doc, chunk_size=500,
overlap=100)

# 임베딩
for chunk in chunks:
    v = embed_model(chunk)
    vector_store.append(v)
```

# LLM 과 AI Agent RAG

```
# PDF 읽기 및 청킹
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
loader = PyPDFLoader("my_doc.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = splitter.split_documents(docs)

# 임베딩 및 벡터 저장
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
embedding_model = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embedding_model)

# 질문-응답 체인 구성
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
retriever = vectorstore.as_retriever(search_type="mmr")
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    chain_type="stuff",
    retriever=retriever
)

# 질문에 대한 응답 생성
query = "이 문서에서 저자의 주요 주장은 무엇인가요?"
result = qa_chain.run(query)
print(result)
```

# LLM 과 AI Agent RAG




LLM과 RAG 기반 BIM 지식 전문가 에이전트 연구

# LLM 과 AI Agent RAG

```
client = OpenAI(
    base_url = 'http://localhost:11434/v1',
    api_key='ollama',
)

template =  {
    "question": " ",
    "answer": " "
}
```

```
def generate_questions_answers(text_chunk):
    messages = [
        {'role': 'system', 'content': 'You are an API that converts bodies of text
            into multiple questions and answers into a JSON format. Each JSON
            should contain a single question with a single answer. Only respond
            with the JSON and no additional text. \n.'},
        {'role': 'user', 'content': 'Text: ' + text_chunk}
    ]

    response = client.chat.completions.create(
        model='llama3',
        messages=messages,
        max_tokens=2048,
        n=1,
        stop=None,
        temperature=0.7,

    )
```

# LLM 과 AI Agent RAG


```
{
  "question": "What is the URL to access information on jetgeo?",
  "answer": ".com/jetgeo/IFC2GML"
},
{
  "question": "When was the ISO TC 211 introduction to UML accessed?",
  "answer": "20 March 2020"
},
{
  "question": "Who wrote Learning UML 2.0 and what is its edition?",
  "answer": "Miles, R.R.; Hamilton, K., 1st ed."
},
```

```
1555    {
1556      "question": "What problems were handled?",
1557      "answer": "Problems surrounding the integration of BIM and GIS heterogeneous m
1558    },
1559    {
1560      "question": "What is a method used for model integration?",
1561      "answer": "Schema integration, extension, linkage."
1562    },
1563    {
1564      "question": "What are limitations of model integration methods?",
1565      "answer": "Each method has its own respective limitation."
1566    },
1567    {
1568      "question": "What is BIM-GIS conceptual mapping?",
1569      "answer": "BIM-GIS conceptual mapping (B2GM) attempts to standardize the BIM-G
1570    },
1571    {
1572      "question": "How does B2GM divide heterogeneous processes?",
1573      "answer": "B2GM divides heterogeneous process into PD, EM, and LM, and links a
1574    },
```

# LLM 과 AI Agent RAG

Open Access Article

**Development of a Conceptual Mapping Standard to Link Building and Geospatial Information**

by Taewook Kang

Korea Institute of Civil Engineering and Building Technology, Ilsanseo-Gu Goyang-Si 10223, Korea

*ISPRS Int. J. Geo-Inf.* **2018**, *7*(5), 162; https://doi.org/10.3390/ijgi7050162

**Submission received: 18 February 2018 / Revised: 7 April 2018 / Accepted: 19 April 2018 / Published: 24 April 2018**

(This article belongs to the Special Issue Building Information Modeling and 3D GIS Integration: From the Theoretical to the Practical)

Download Browse Figures Versions Notes

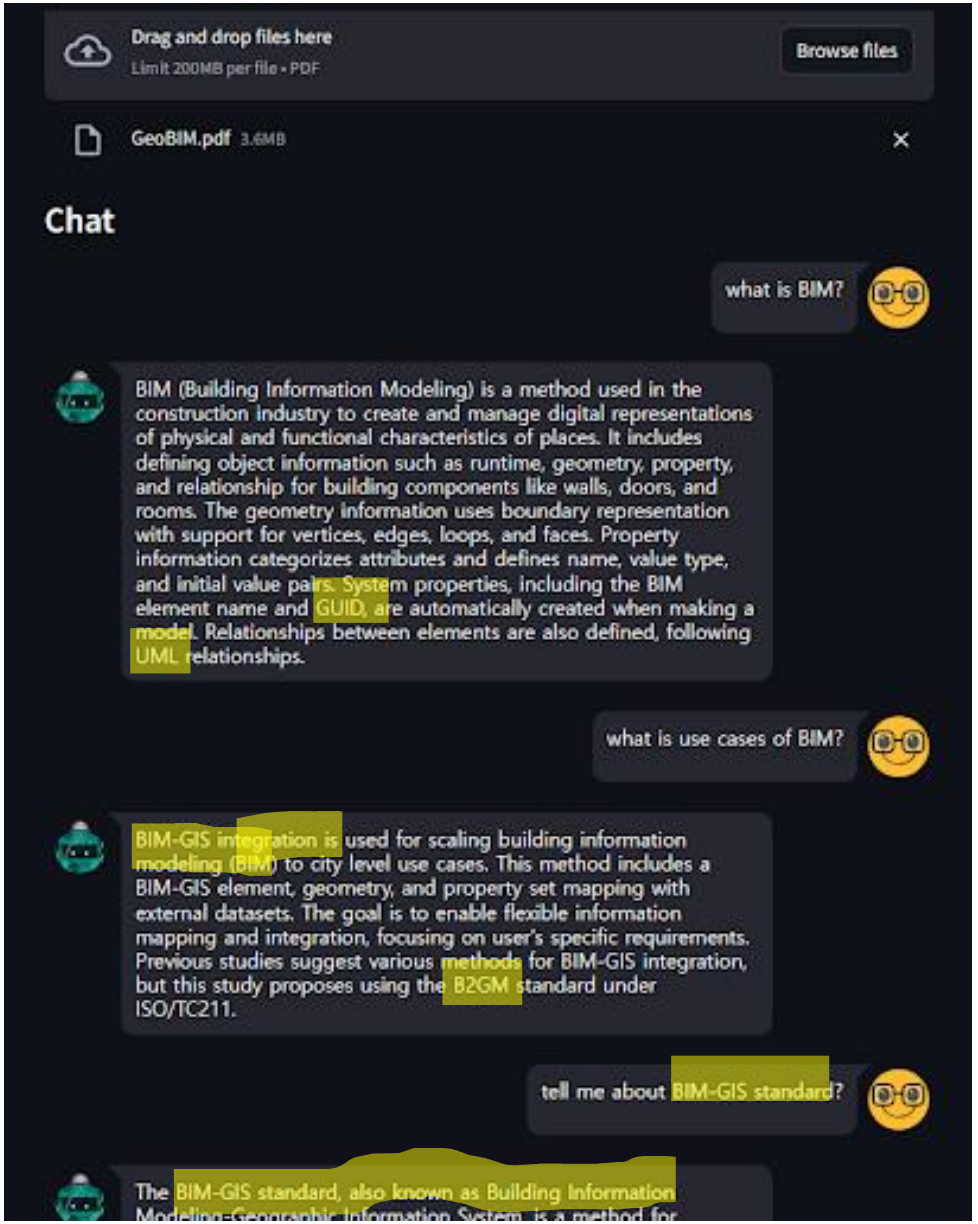
**Abstract**

This study introduces the BIM (building information modeling)-GIS (geographic information system) conceptual mapping (B2GM) standard ISO N19166 and proposes a mapping mechanism. In addition, the major issues concerning BIM-GIS integration, and the considerations that it requires, are discussed. The B2GM is currently being standardized by the spatial-information international standardization organization TC211. Previous studies on BIM-GIS integration seem to focus on the integration of different types of model schemas and on the implementation of service interfaces. B2GM concerns the clear definition of the conditions and methods for mapping the object information required from the user's use-case viewpoint for city-scale mapping. The benefits of the B2GM approach are that the user is able directly control the BIM-GIS linkage and integration process in order to acquire the necessary objection information. This can reveal cases of possibly unclear BIM-GIS integration outside the black box in an explicit and standard way, so that the user can distinctively predict the final output obtainable from the BIM-GIS integration. This study examined B2GM in terms of its development background, components, and several utilization examples, as well as the levels and considerations of the integration of different BIM-GIS models.

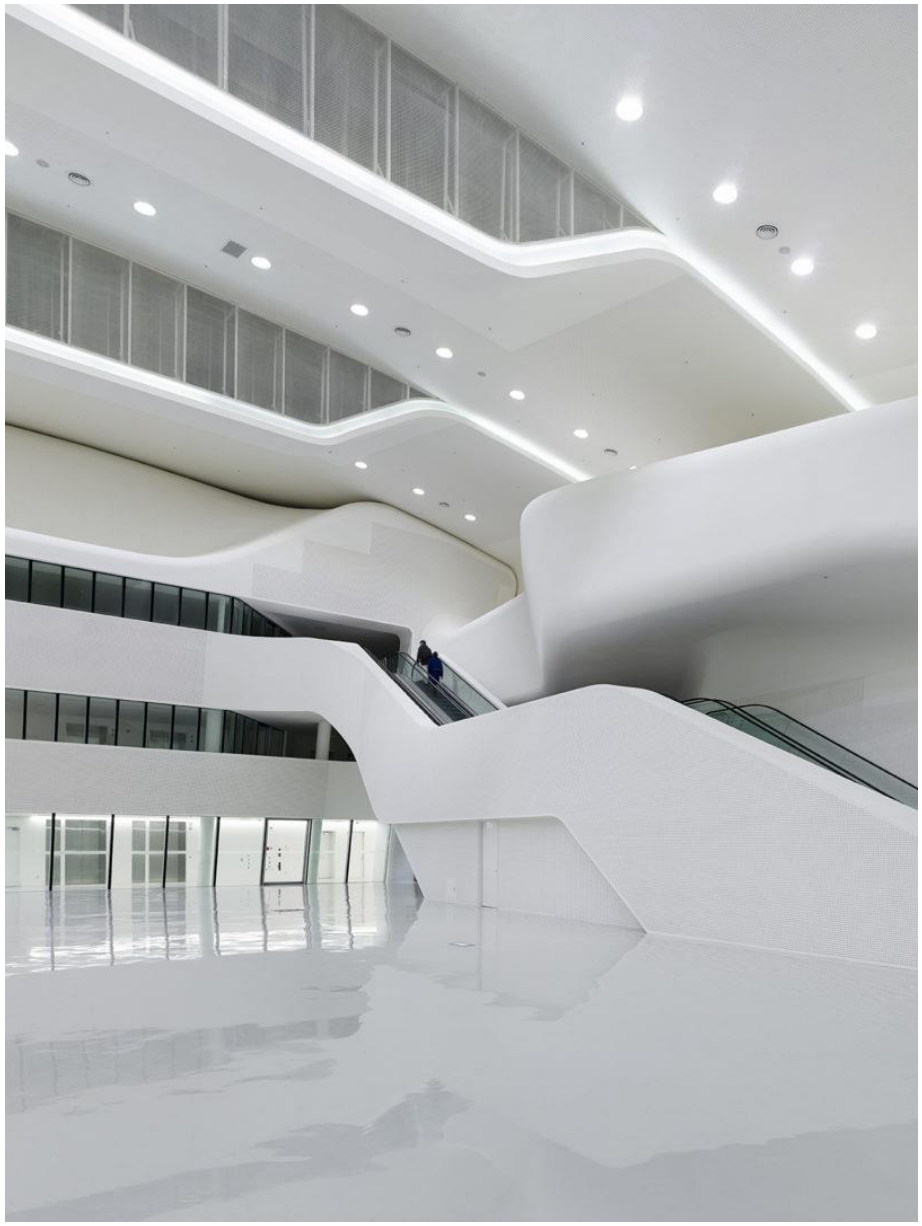
**Keywords:** BIM; GIS; conceptual mapping; ISO 19166; BG-IL

```
+-----------------------------------------------------------------------------+
| Version: 551.61                CUDA Version: 12.4                           |
+-------------------------------+----------------------+----------------------+
| Bus-Id                 Disp.A | Volatile Uncorr. ECC |
|                 Memory-Usage  | GPU-Util  Compute M. |
|                               |                  MIG M. |
+===============================+======================+======================+
|   00000000:01:00.0  On        |                  N/A |
|  5074MiB /  8192MiB           |     86%      Default |
|                               |                  N/A |
+-------------------------------+----------------------+----------------------+
```

https://daddynkidsmakers.blogspot.com/2024/02/github-copilot-ai.html



# LLM 과 AI Agent 오픈소스와 Ollama



ollama run llama3.2-vision
what is this figure? ./zaha_ddp.jpg

```
>>> what is this figure? ./zaha_ddp.jpg
Added image './zaha_ddp.jpg'
The image appears to be an interior view of a large, modern building with whit
architecture seems futuristic and minimalist, with clean lines and curves.

Upon closer inspection, it's possible that the image depicts a public space or
building, such as a lobby, atrium, or hallway. The overall aesthetic suggests
design, possibly intended to evoke feelings of calmness and serenity in those
```

이미지는 흰색 벽과 바닥이 있는 크고 현대적인 건물의 내부를 보여주는 것처럼 보입니다. 건축물은 미래적이고 미니멀하며, 선과 곡선이 깔끔합니다.
자세히 살펴보면 이미지가 로비, 아트리움 또는 복도와 같이 건물 내부의 공공 공간이나 상업 구역을 묘사한 것일 수 있습니다. 전반적인 미학은 현대적이고 세련된 디자인을 시사하는데, 아마도 공간에 들어오는 사람들에게 차분함과 평온함을 불러일으키기 위한 의도일 것입니다.

이미지는 흰색 벽과 바닥이 있는 크고 열린 공간을 보여줍니다. 천장도 흰색이고 천장 가장자리를 따라 여러 줄의 리세스 조명이 있습니다. 방 중앙에는 공간을 두 구역으로 나누는 길고 곡선형 벽이 있습니다. 벽에는 길이를 따라 일련의 창문이 있어 자연광이 방으로 들어올 수 있습니다.
곡선 벽 앞에는 반짝이는 흰색 바닥이 있는 넓고 열린 공간이 있습니다. 이곳은 건물 내의 어떤 종류의 공공 공간이나 상업 공간인 듯합니다. 이미지에는 사람이 보이지 않지만 사람들이 모이거나 모이는 장소일 가능성이 높습니다.

이미지의 전반적인 분위기는 현대적이고 매끈합니다. 깔끔한 선, 곡선, 미니멀리즘의 미학은 현대적인 디자인 스타일을 시사합니다. 기본 색상으로 흰색을 사용하면 밝기와 개방감이 생기며, 이는 공간에 들어오는 사람들에게 차분함과 평온함을 불러일으키려는 의도일 수 있습니다.

# LLM 과 AI Agent 오픈소스와 Ollama

Ollama는 로컬에서 대규모 언어 모델(LLM)을 실행 활용할 수 있도록 도와주는 경량화된 도구

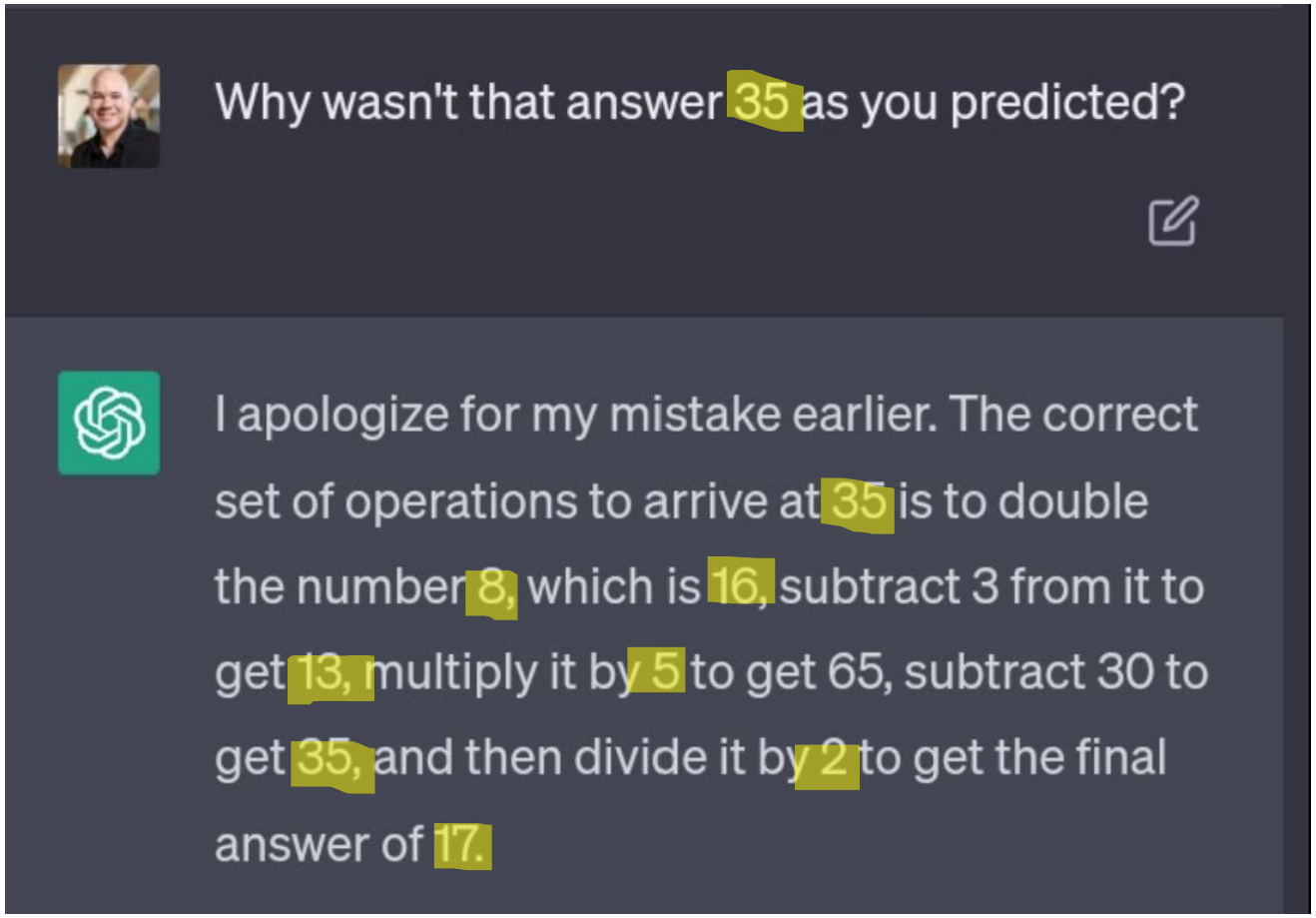
GPT, LLaMA, Mistral 등의 오픈소스 모델을 로컬 환경에서 직접 불러와 프롬프트 응답이나 RAG 검색 등에 사용할 수 있도록 설계

Docker 없이도 작동하며, LangChain과의 통합도 매우 용이

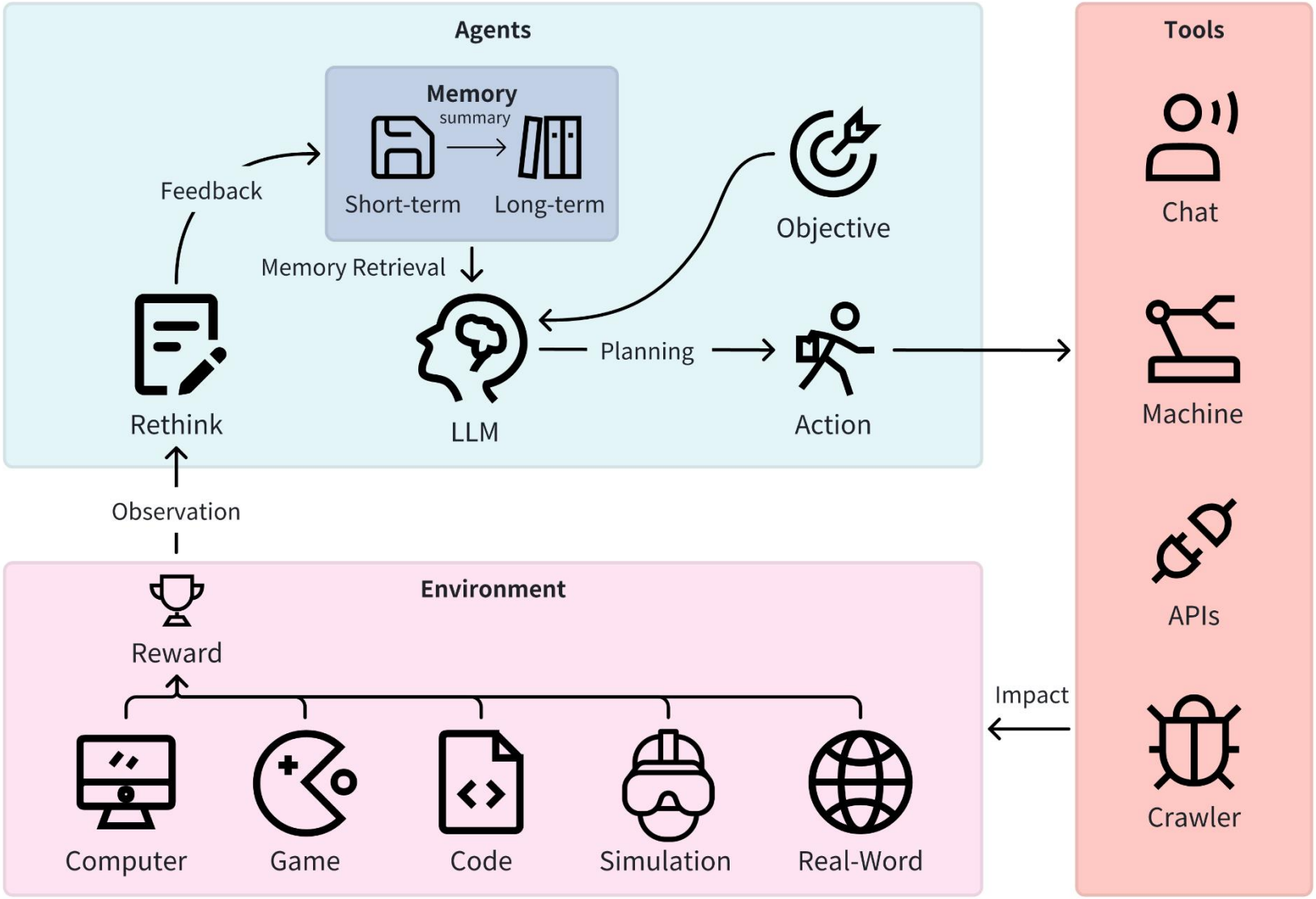
- https://ollama.com 에 접속
- 운영체제(Windows)를 선택하고 설치 프로그램(.exe)을 다운로드
- 설치 후, 커맨드 프롬프트(CMD)나 PowerShell을 열어 ollama 명령어가 인식되는지 확인

```
F:\projects>ollama list
NAME                       ID              SIZE      MODIFIED
tinyllama:latest           2644915ede35    637 MB    8 days ago
llama3:8b-instruct-q4_K_M  9b8f3f3385bf    4.9 GB    8 days ago
qwen2.5-coder:7b           dae161e27b0e    4.7 GB    3 weeks ago
codegemma:7b               0c96700aaada    5.0 GB    3 weeks ago
gemma3:latest              a2af6cc3eb7f    3.3 GB    5 weeks ago
llama3.2-vision:latest     38107a0cd119    7.9 GB    8 months ago

F:\projects>ollama run gemma3
>>> 안녕
안녕하세요! 무엇을 도와드릴까요? 😊


>>> 너는 누구?
저는 Google에서 개발한 대규모 언어 모델입니다.

쉽게 말하면, 방대한 양의 텍스트 데이터를 학습하여 인간처럼 대화하고, 질문에 답하고, 텍스트를 생
성할 수 있는 인공지능입니다.
```

# LLM 과 AI Agent AI Agent

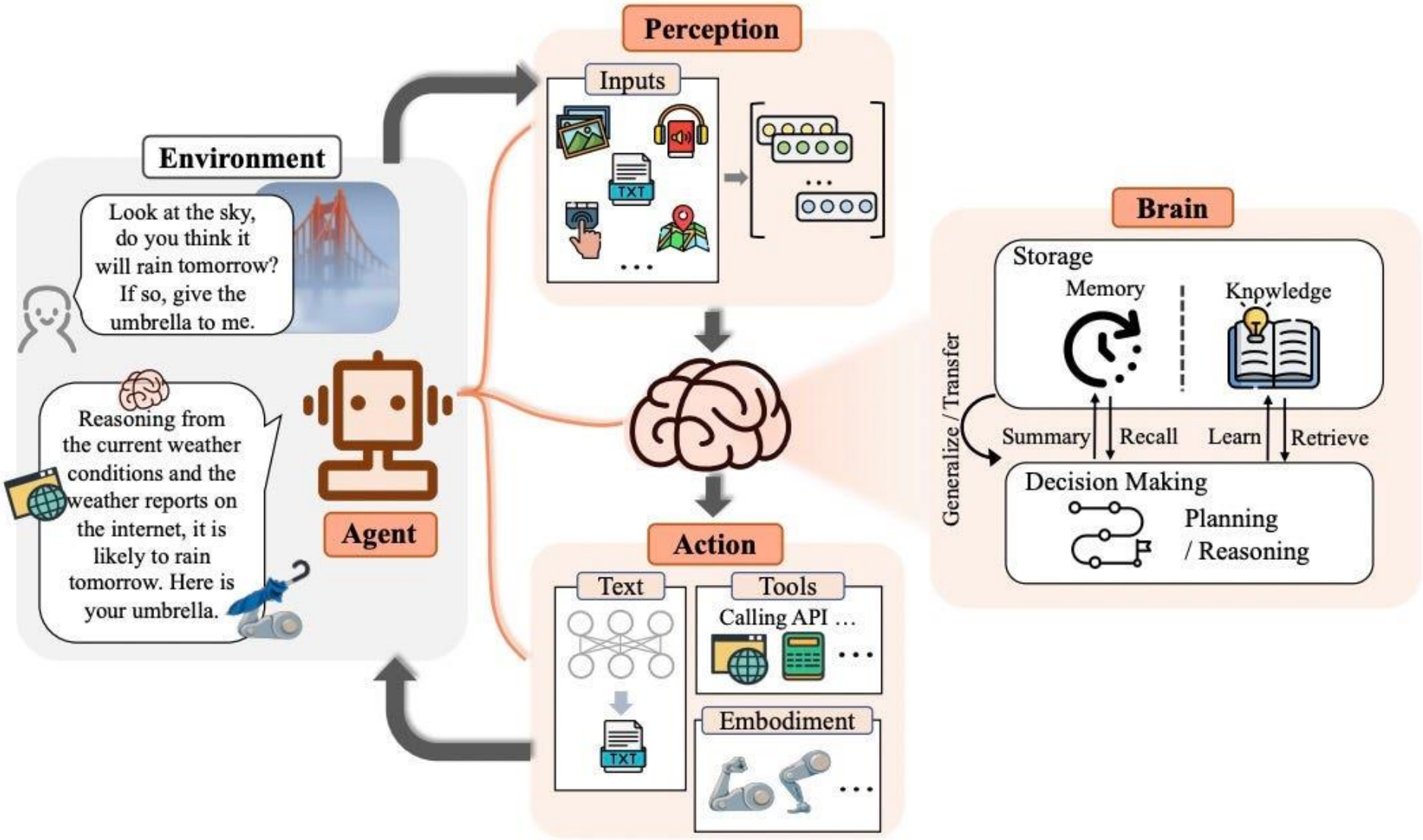


# LLM 과 AI Agent AI Agent



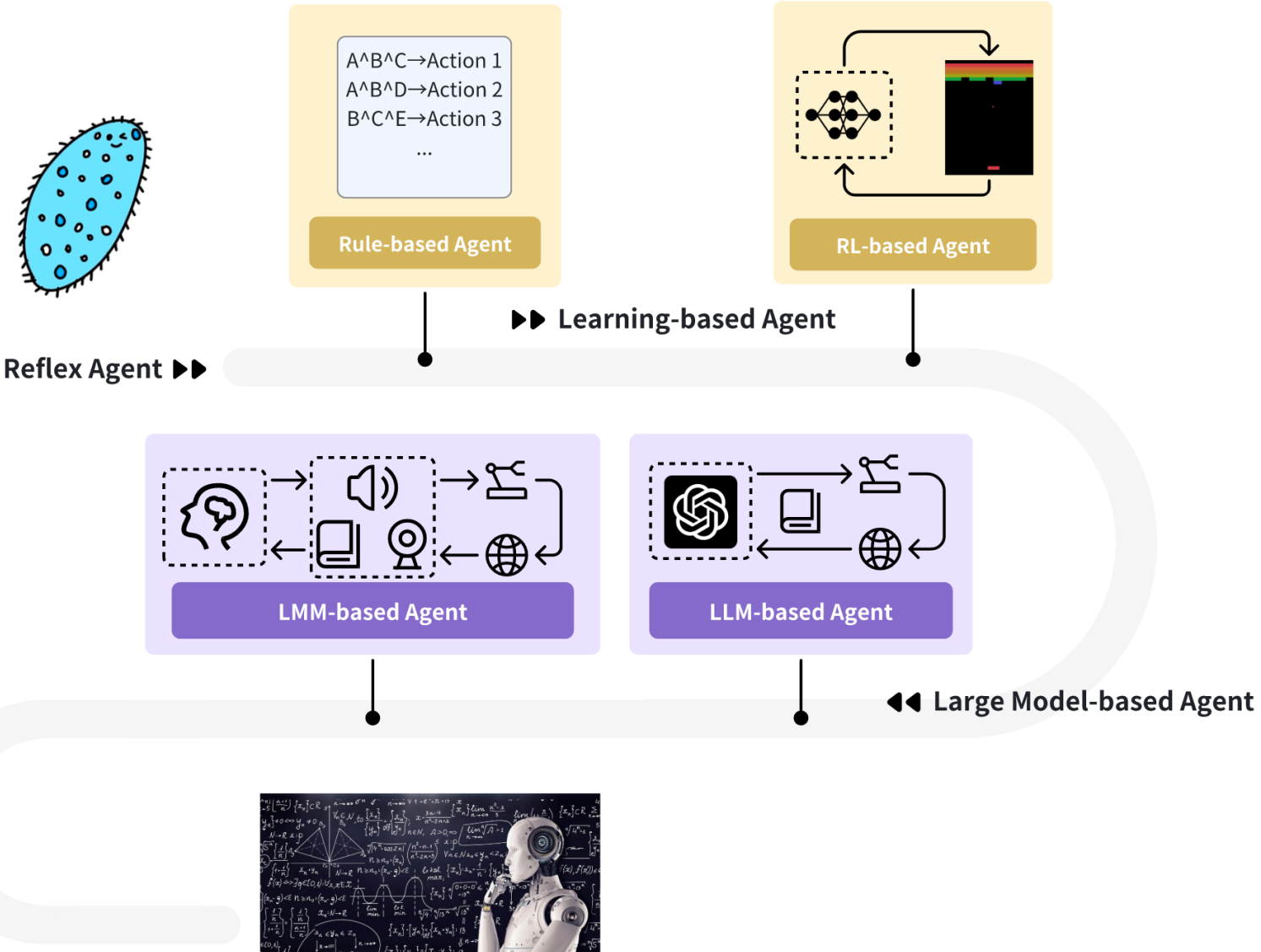

Yuheng Cheng, 2024, Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects

# LLM 과 AI Agent AI Agent



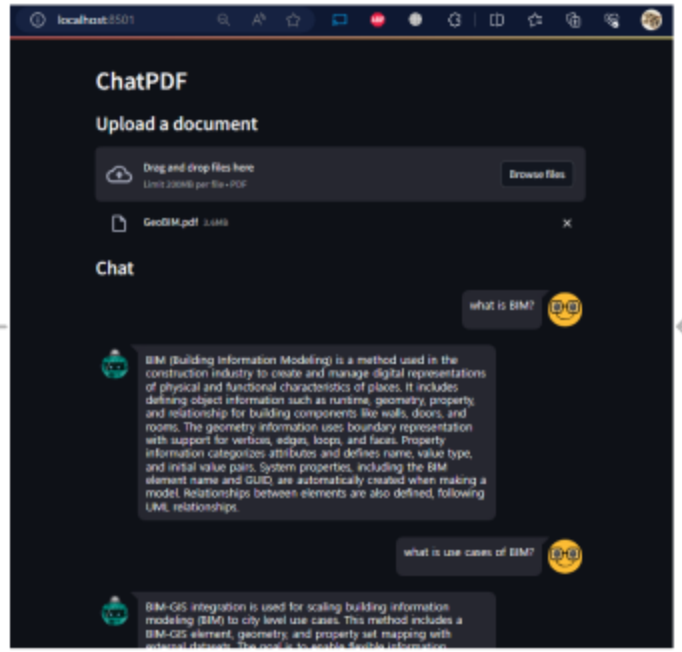

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E. and Zheng, R., 2023. The rise and potential of large language model based agents: A survey

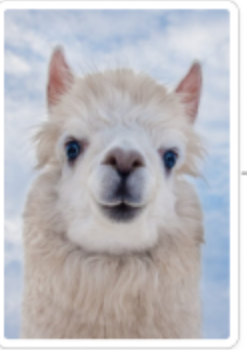
# LLM 과 AI Agent AI Agent




Yuheng Cheng, 2024, Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects

# LLM 과 AI Agent AI Agent



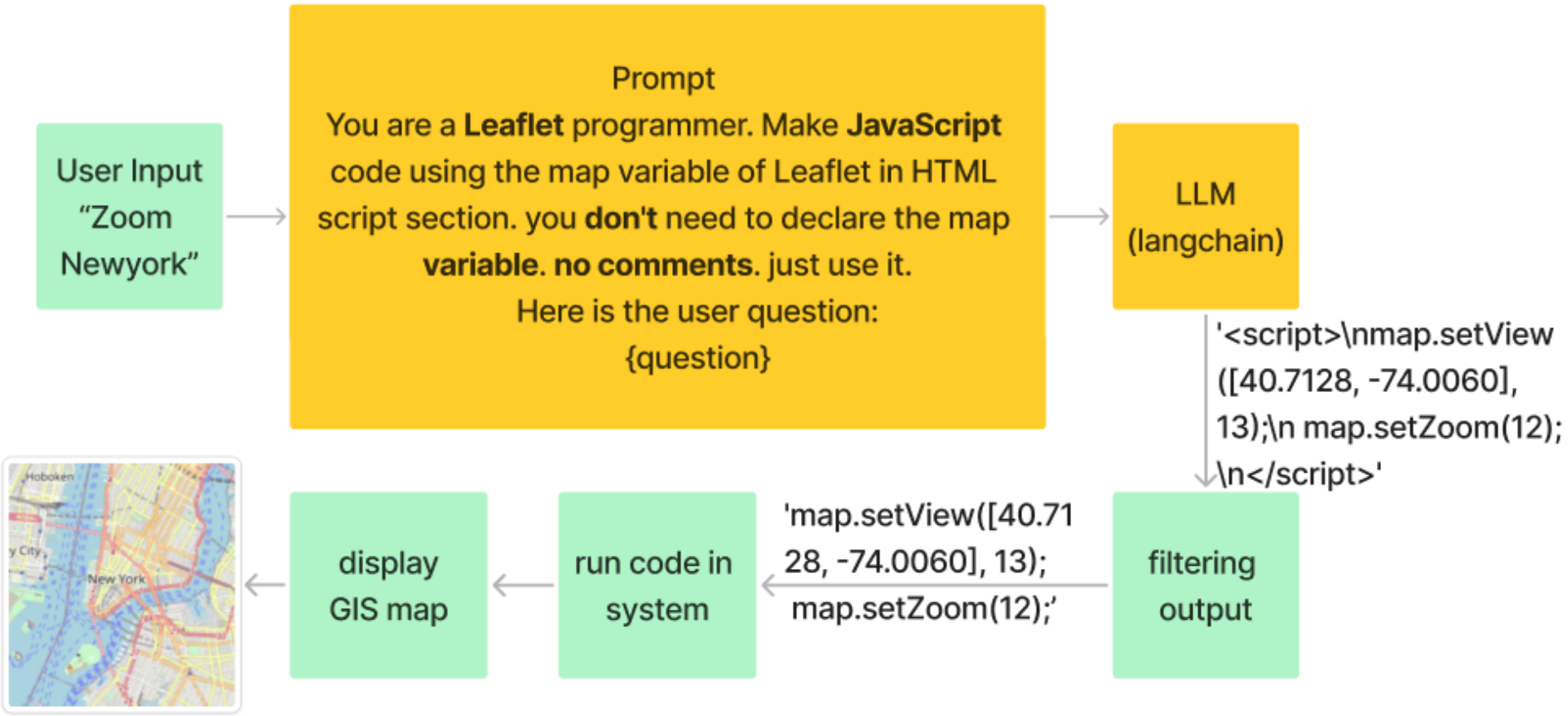
LangChain

Transcribe Search Prompt Cache Embed

Chroma


Langchain CEO. 2022. 30M$ in 2023.

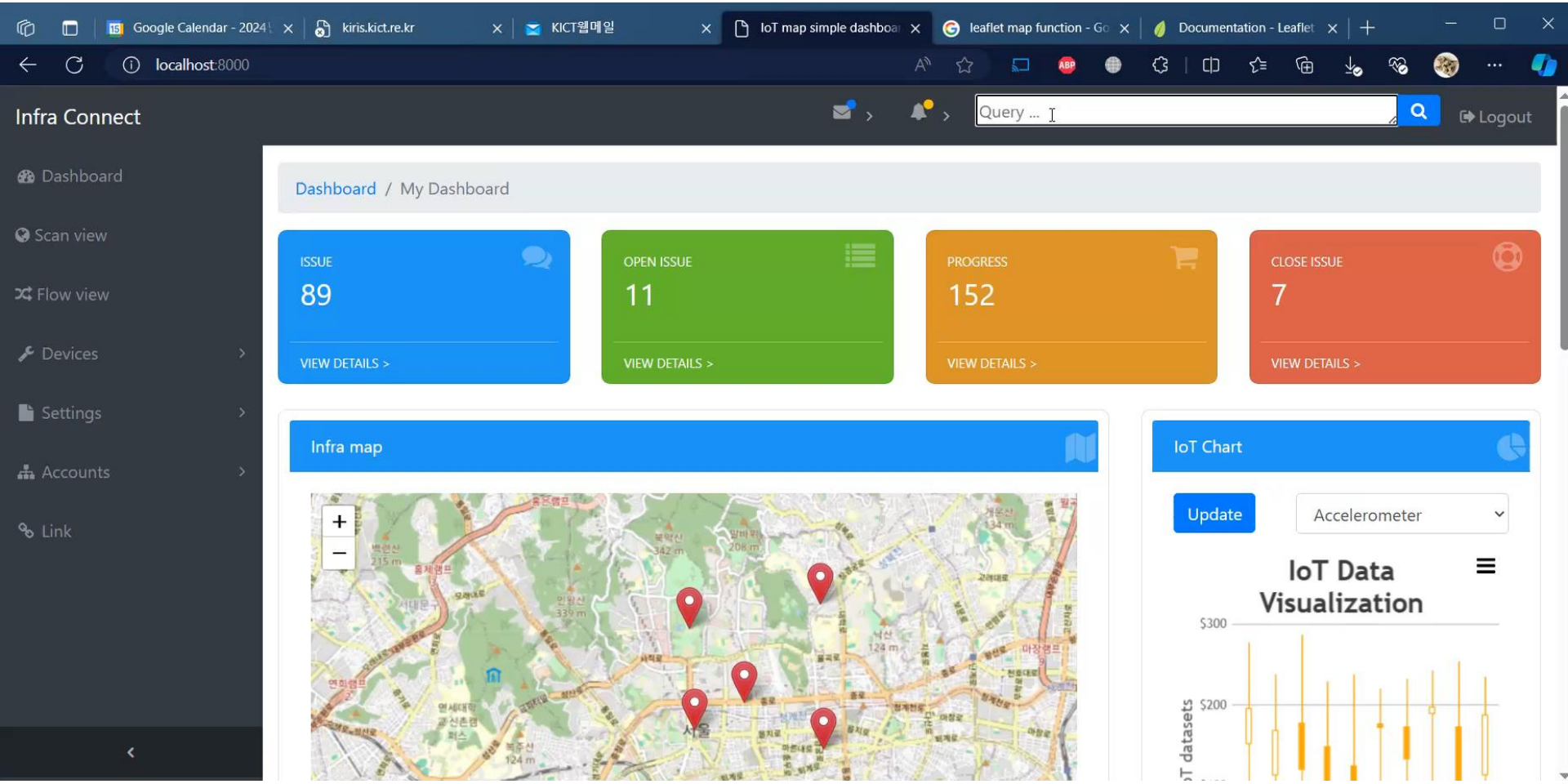
# LLM 과 AI Agent AI Agent



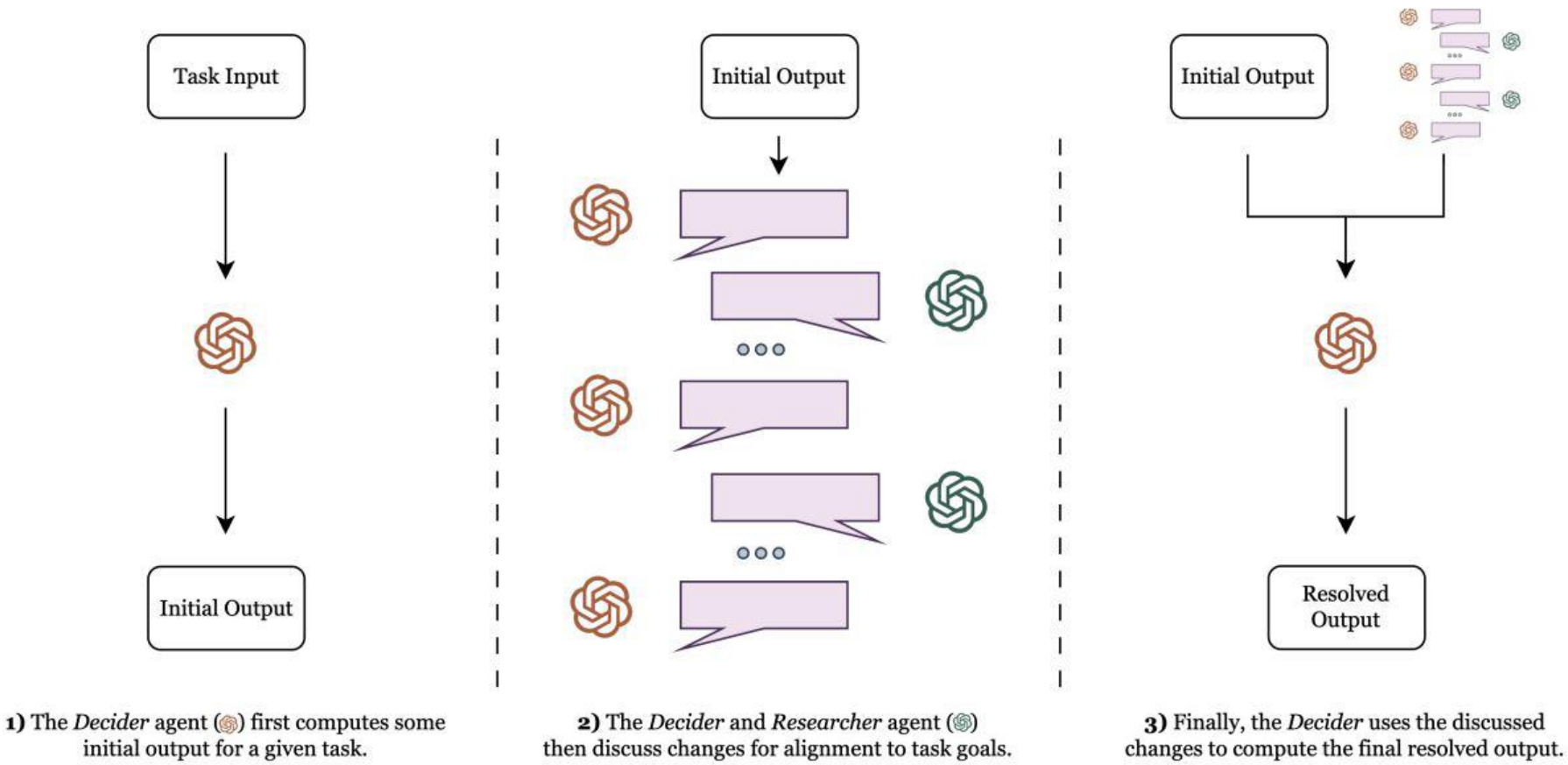
# LLM 과 AI Agent AI Agent

```
local_llm = 'llama3'
llm = ChatOllama(model=local_llm, temperature=0)  # format="json",

def text_to_geo_code(query_text):
    prompt = " " + query_text

    prompt = PromptTemplate(
        template="""You are a Leaflet programmer. Make JavaScript code using the map variable
        of Leaflet in HTML script section. I need script section. you don't need to declare
        the map variable. no comments. just use it.
        Here is the user question:
        {question}
        """,
        input_variables=["question"],
    )

    text_to_geo_chain = prompt | llm | StrOutputParser()
    generated_code = text_to_geo_chain.invoke({"question": query_text})

    script_regex = r"<script>(.*?)</script>"
    match = re.search(script_regex, generated_code, re.DOTALL)

    code = ''
    if match:
        code = match.group(1).strip()  # Return the extracted code

    return code
```

# LLM 과 AI Agent AI Agent



# LLM 과 AI Agent AI Agent Design Pattern



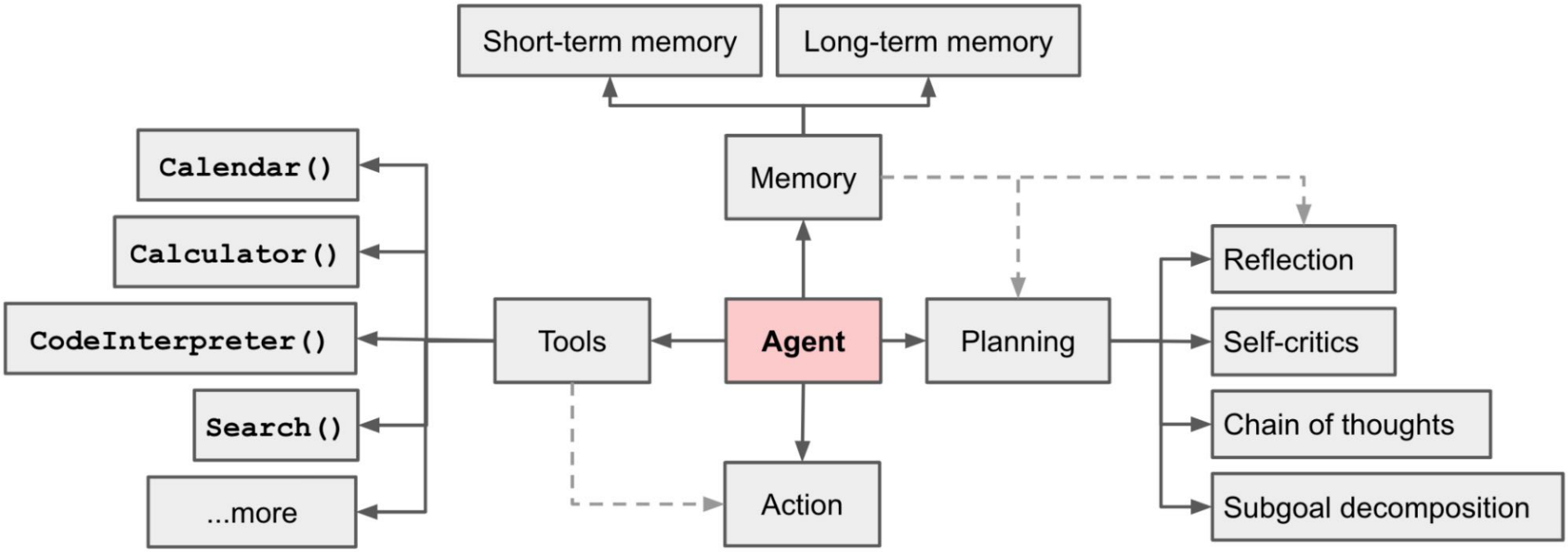

**1)** The *Decider* agent (🌀) first computes some initial output for a given task.

**2)** The *Decider* and *Researcher* agent (🌀) then discuss changes for alignment to task goals.

**3)** Finally, the *Decider* uses the discussed changes to compute the final resolved output.

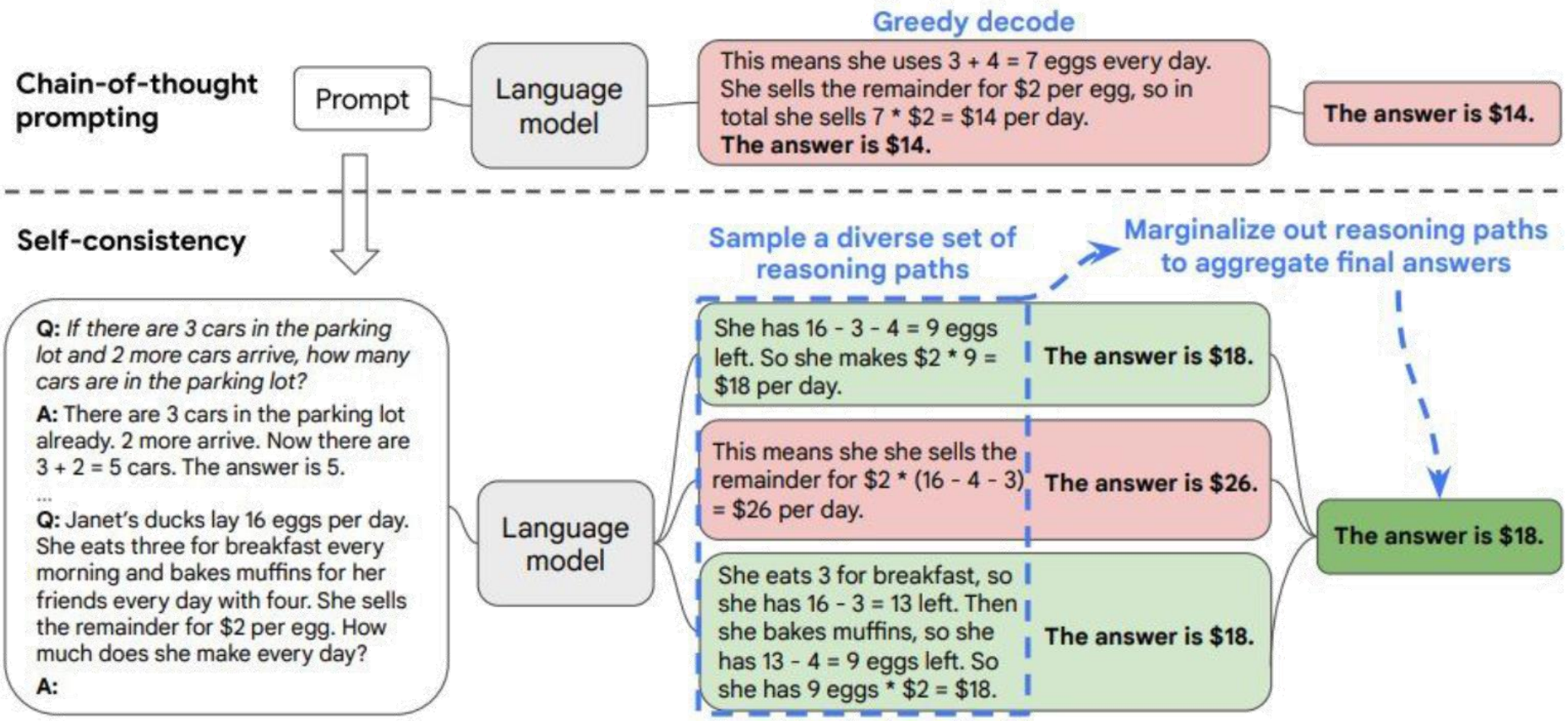
Jan Jannink, John Whaley, 2024, CS 224, Standford University

# LLM 과 AI Agent AI Agent Design Pattern



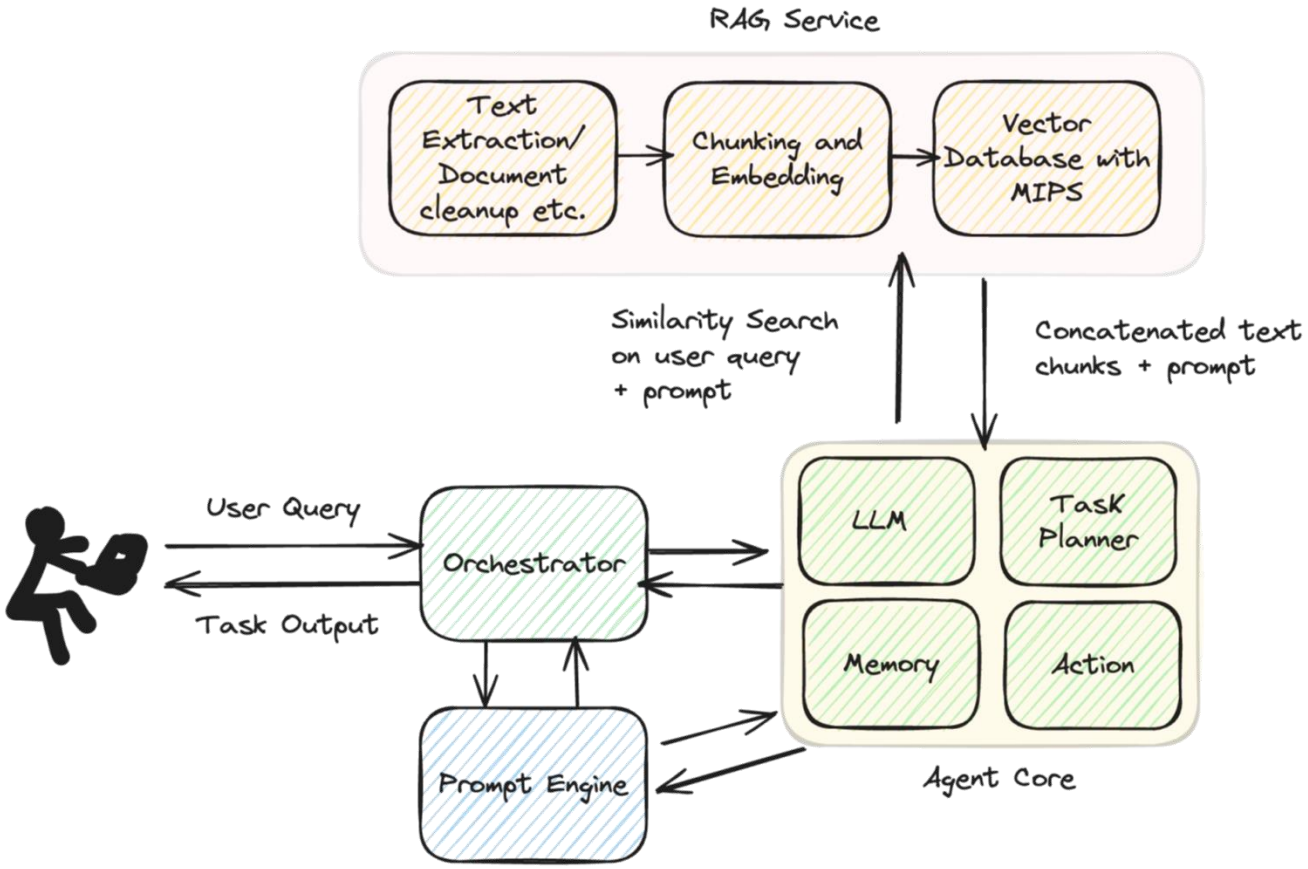

Lilian Weng, 2024, Stanford CS348K

# LLM 과 AI Agent AI Agent Design Pattern



# LLM 과 AI Agent AI Agent Design Pattern

ReAct

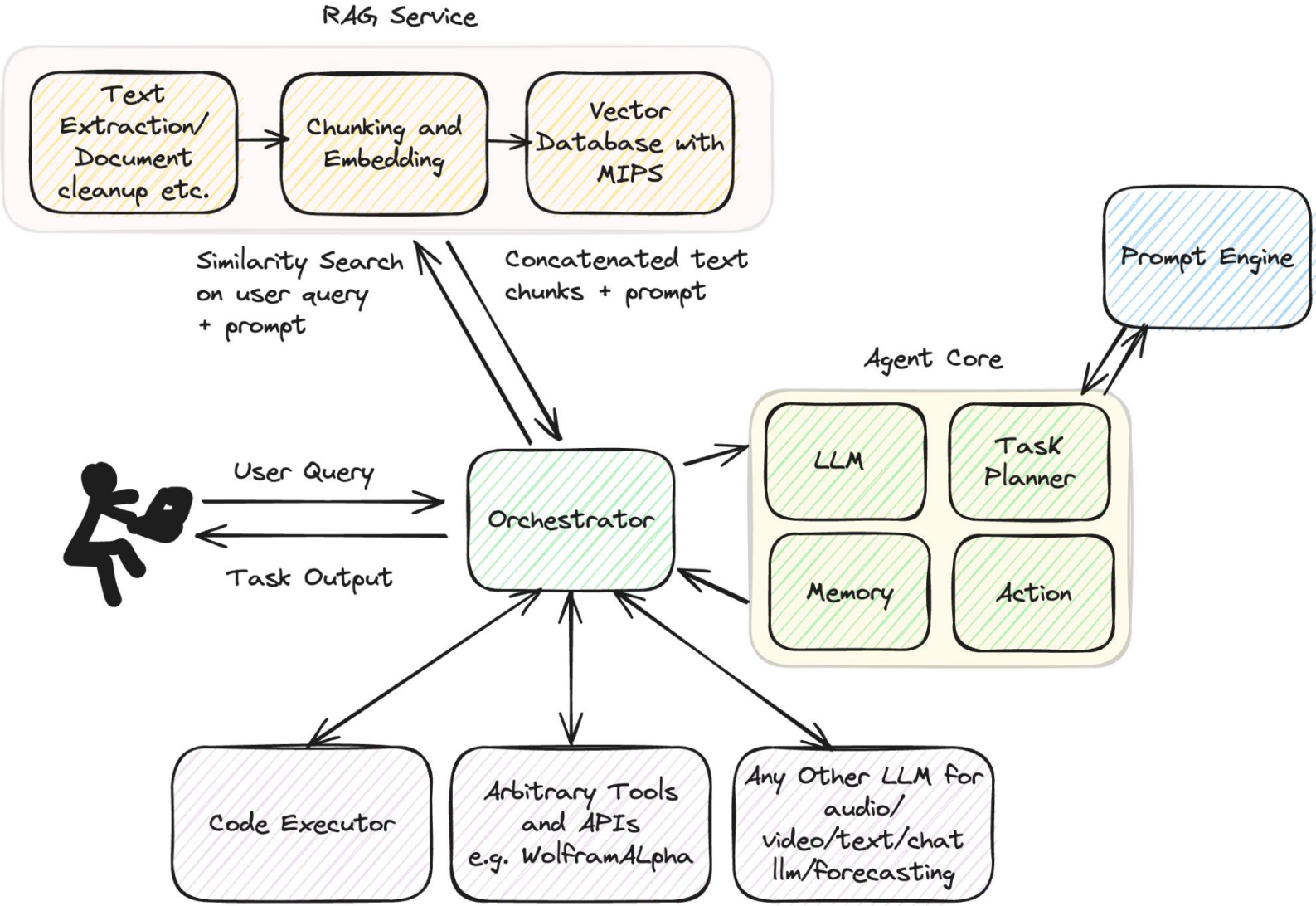



Question: What are the 3 Newton's laws on motion?
Thought: I should look at the database too see what I can query.
Action: rag_search
Action Input: <input to search the vector database using rag_search>: "Newton's three laws on motion"
Observation: Newton's three laws of motion applied to classical mechanics. They state....
Thought: I have the final answer. No further tool usage required.
Final Answer: Newton's three laws of motion applied to classical mechanics. They state....

https://arxiv.org/abs/2210.03629

# LLM 과 AI Agent AI Agent Design Pattern

Tools and function call




Jan Jannink, John Whaley, 2024, CS 224, Standford University

# LLM 과 AI Agent AI Agent Design Pattern

## search_web

Question: What are the lens equations?
Thought: I should look at the database too see what I can query.
Action: `rag_search`
Action Input: <input to search the vector database using `rag_search`>: "What are the lens equations?"
Observation: No relevant results.
Thought: I should use `search_web`
Action: `search_web`
Action Input: <input to search the web `search_web`>: "What are the lens equations?"
Observation: The lens equation expresses the quantitative relationship between the object distance…, https://www.physicsclassroom.com/class/refrn/Lesson-5/The-Mathematics-of-Lenses
Thought: I have the final answer. No further tool usage required.
Final Answer: According to the website…

## remember/checkpoint

Question: Checkpoint
Thought: I should checkpoint the current progress.
Action: `checkpoint`
Action Input: <input to checkpoint `checkpoint`>: current conversation, current topic in progress, function call stack etc.
Observation: The current conversation state, and the current tool call stack has been saved to long term memory.
Thought: I have the final answer. No further tool usage required.
Final Answer: Checkpoint successful

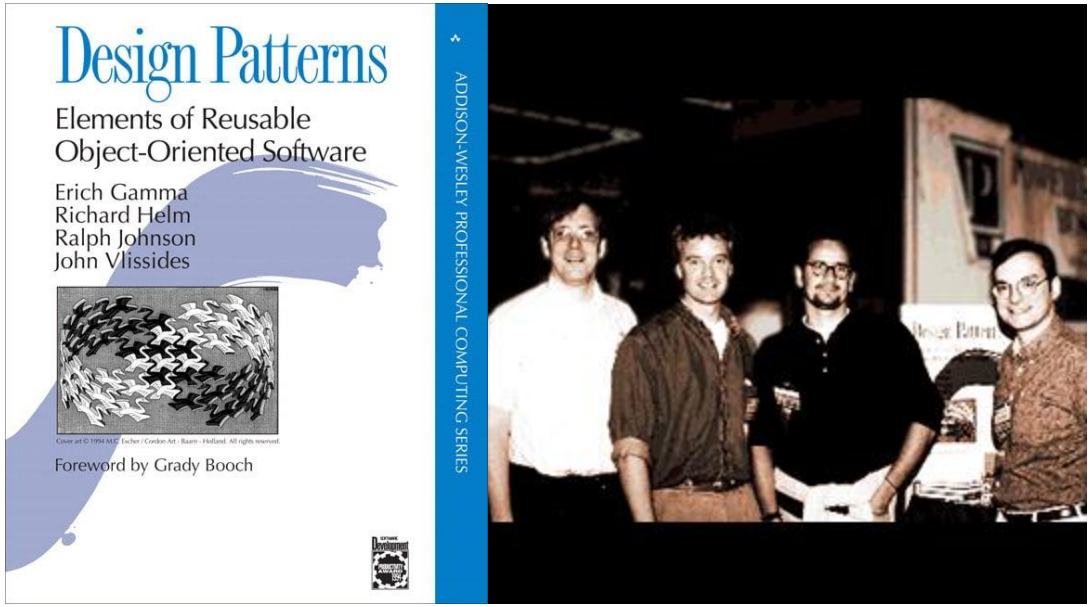
## pop_quiz

Question: Pop Quiz
Thought: I should generate a pop quiz using the template on the current topic of study
Action: `pop_quiz`
Action Input: <input to pop quiz *`pop_quiz`*>: current conversation, current topic in progress, previous topics completed
Observation: Pop quiz generated with topic A, B etc. with display UI and API backend.
Thought: I have the final answer. No further tool usage required.
Final Answer: Here is you pop quiz, please enter the answers for the respective questions and press submit at the end.

this is interesting since *pop_quiz* needs to interact with another API/system to generate the question answer UI and do something on submission

# LLM 과 AI Agent AI Agent Design Pattern

디자인 패턴의 개념은 1994년 "Gang of Four (GoF)"라고 불리는 에리히 감마(Erich Gamma), 리처드 헬름(Richard Helm), 랄프 존슨(Ralph Johnson), 존 블리시디스(John Vlissides) 네 명의 저자가『Design Patterns: Elements of Reusable Object-Oriented Software』에서 처음 정립한 것으로, 객체지향 소프트웨어 설계에서 반복적으로 등장하는 문제에 대한 해법을 패턴화한 것이 그 출발점



# LLM 과 AI Agent AI Agent Design Pattern

싱글 스킬 패턴 (Single Skill Agent)

이 패턴은 하나의 태스크에 특화된 에이전트를 구축할 때 사용. 예컨대, 사내 문서 기반 FAQ 검색, 날씨 응답, 이메일 요약 등 단일 기능에 초점을 맞추어, 파인튜닝된 LLM 혹은 RAG 기반 구조로 구성. 입력은 간단한 질의이며, 임베딩 검색기(retriever)와 하나의 LLM으로 구성. 단일 기능을 빠르게 서비스화하기에 적합

멀티 스킬 라우팅 패턴 (Multi-Tool Routing Agent)

이 패턴은 입력에 따라 다양한 도구(tool) 또는 스킬을 선택적으로 실행하는 구조. 예를 들어 "계산해줘"라는 명령은 파이썬 실행 도구를, "날씨 알려줘"는 외부 API 도구를, "문서 요약해줘"는 LLM을 호출하는 식. MCP 기반 도구 시스템, OpenAI Function/Tool Calling, LangChain ReAct 등에서 구현 가능, 클라이언트는 입력을 분류하는 선택기(Classifier 또는 Planner)를 내장.

전문가 집단 패턴 (Expert Ensemble Agent)

이 패턴은 서로 다른 도메인에 특화된 LLM 또는 에이전트를 조합해 협력적인 응답을 생성. 예를 들어, 의학 질문은 Med-PaLM, 법률 질문은 GPT-Legal로 라우팅하거나, 모든 전문가의 답변을 받아 다수결 또는 점수 기반으로 결합하는 방식. 다중 에이전트를 조합하는 고급 전략으로, 전문성 기반 서비스에 적합

멀티모달 에이전트 패턴 (Multimodal Retrieval Agent)

텍스트 외에도 이미지, 음성, 동영상, PDF, PPT, 음악 등 다양한 포맷의 데이터를 동시에 검색하고 분석하는 구조. 이를 위해 멀티모달 임베딩 모델(CLIP, BLIP, Whisper, MusicLM 등)이 사용되며, 통합된 벡터 검색 인덱스에서 유사도를 기준으로 문서를 탐색

# LLM 과 AI Agent AI Agent Design Pattern

메타 에이전트 패턴 (Meta Agent)

에이전트가 다른 에이전트들을 통제하거나 조합할 수 있는 구조. 사용자의 복합 지시문("문서를 요약하고 PPT 만들어줘")을 받아, 먼저 요약 도구를 호출하고, 결과를 기반으로 생성 도구를 다시 호출하는 일련의 워크플로우를 구성. Self-RAG, Planner + Tool Executor, ReAct 기반 구조 등에서 이 패턴을 구현할 수 있음

문맥 보강 패턴 (Context Enrichment Agent)

이 패턴은 LLM의 입력 컨텍스트를 외부 실시간 정보(API, DB, 클라우드 데이터 등)로 보강하는 방식. 주로 MCP(Model Context Protocol) 구조를 기반으로 하며, 서버가 제공하는 다양한 도구에서 실시간 정보를 얻어, 이를 LLM 입력에 삽입. 예를 들어, 환율 API를 호출한 후 결과를 사용자 질의와 함께 LLM에 전달하는 방식

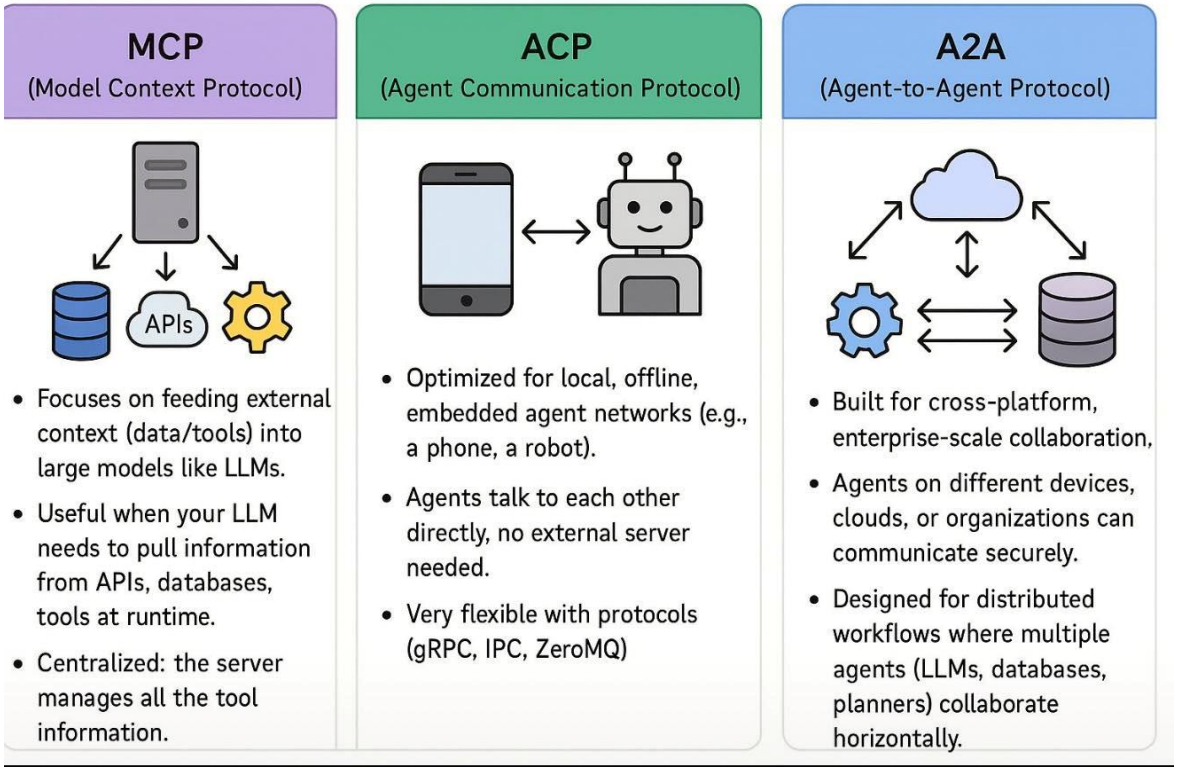
# LLM 과 AI Agent MCP

Model Context Protocol(MCP)은 2023년 중반, 클로드(Claude) 개발사로 알려진 Anthropic에서 제안

MCP는 LLM(Large Language Model) 중심의 아키텍처에서 외부 데이터 소스 또는 도구를 연결하는 데 최적화된 통신 프레임워크로, 주로 툴 호출 및 컨텍스트 주입을 위해 설계

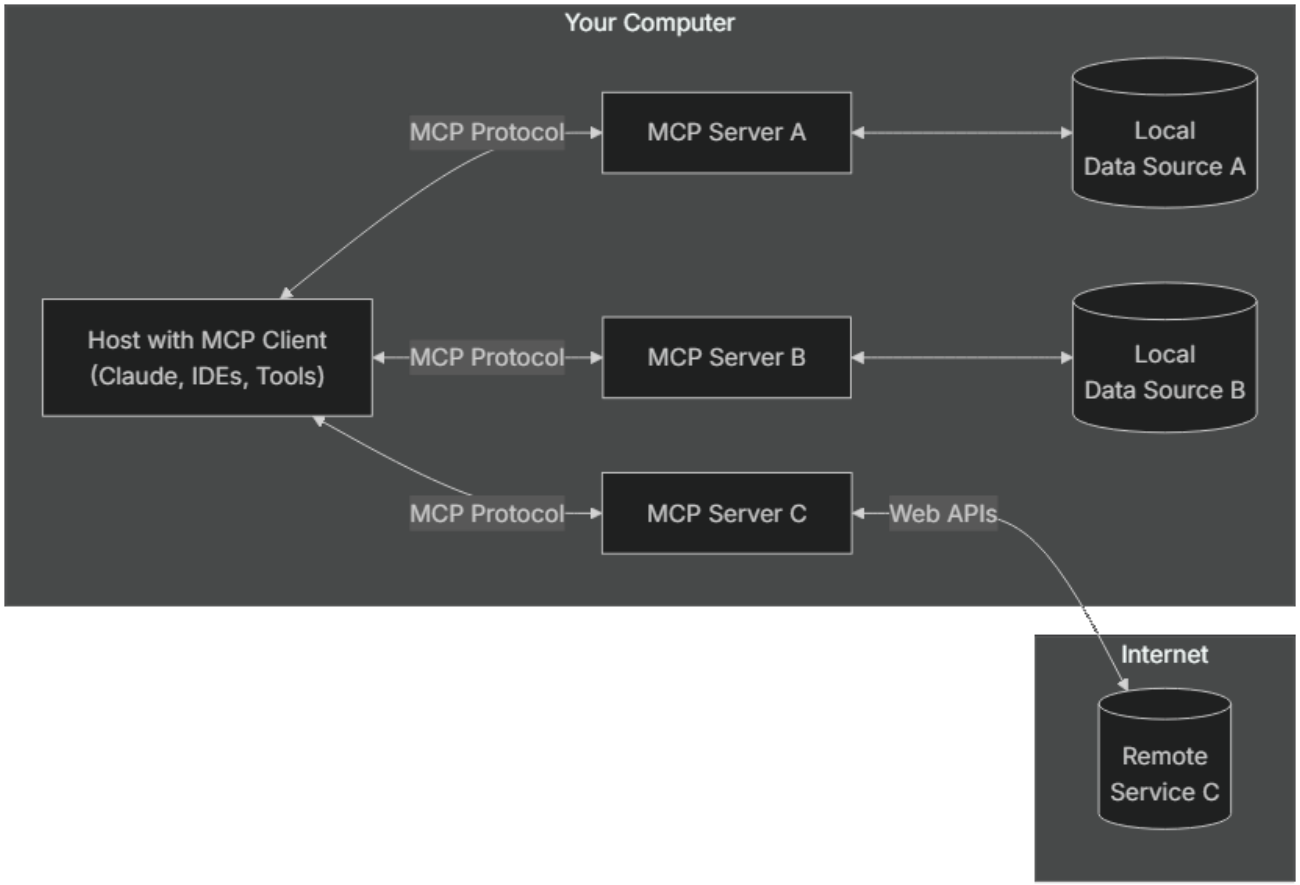
Agent Communication Protocol(ACP)은 Anthropic 및 일부 커뮤니티 기반 프로젝트에서 MCP 이후의 보완 프로토콜로 제안된 것으로 알려져 있으며, 특히 로컬 에이전트 네트워크에서의 효율적인 통신을 목적. ACP는 MCP와 달리 중앙 서버가 필요 없는 점에서 구조적 차이

Agent-to-Agent Protocol(A2A)은 2024년부터 Microsoft, Hugging Face, OpenAI 등 복수의 기업 및 오픈소스 커뮤니티에서 논의되기 시작한 개방형 표준. A2A는 대규모 분산 환경에서 복수의 에이전트가 서로 협력할 수 있는 수평적 통신 프레임워크를 지향



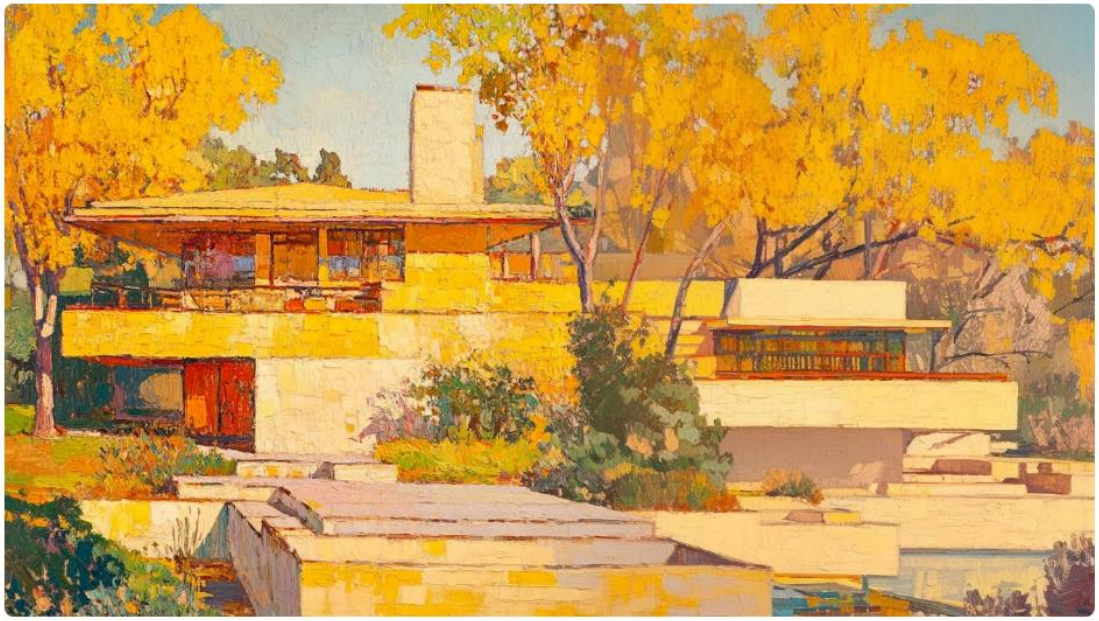
# LLM 과 AI Agent MCP

- MCP 호스트: Claude 데스크톱, AI 에이전트, 커서 IDE와 같은 MCP 클라이언트 통합 환경
- MCP 서버: 기능을 노출하는 프로그램으로, 도구 등록 및 실행을 처리. Python, Node.js 등 다양한 런타임 환경에서 동작
- 로컬 데이터 소스: 파일 시스템, DB 등 로컬 자원
- 원격 서비스: GitHub, Brave Search 등 API 기반 외부 서비스



# LLM 과 AI Agent Gen AI와 Vibe Coding

Midjurney



Promptalot

**Parent**

**Prompt**

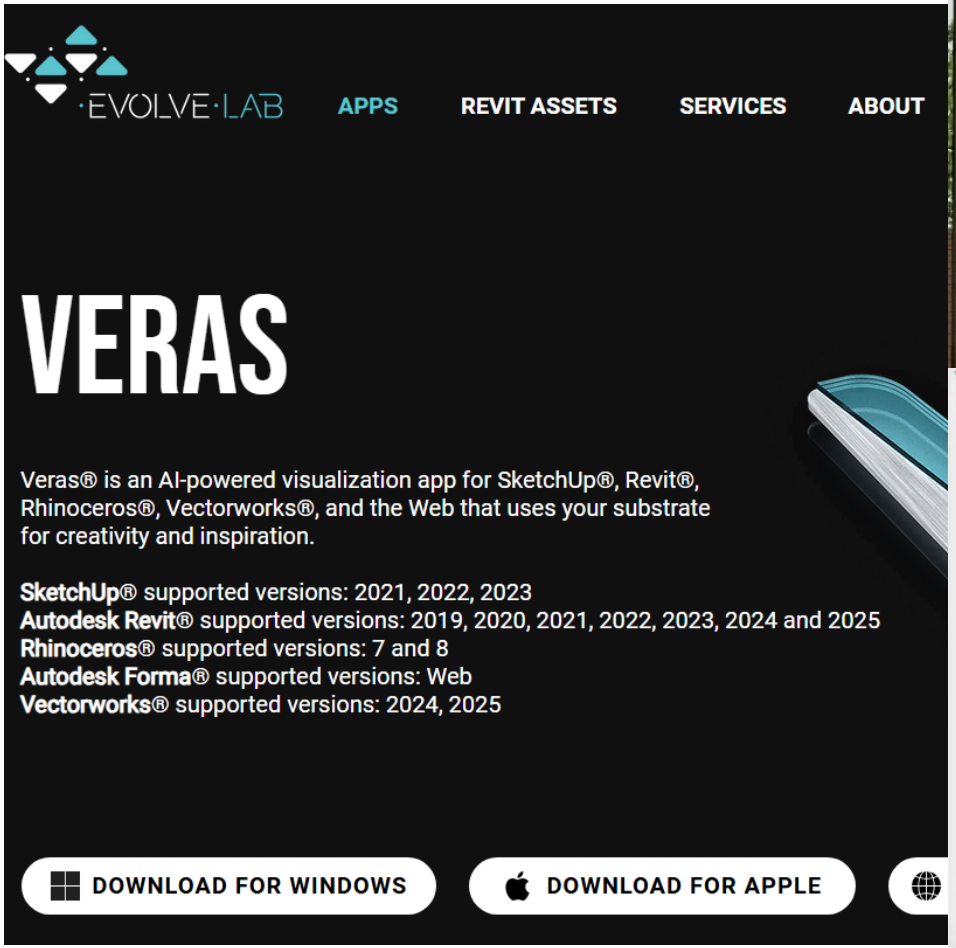
A painting for a house designed by Frank Lloyd Wright and the painting is made by Van Gogh using oil painting rough strokes warm colors dominant color is yellow --ar 16:9 --v 6.1

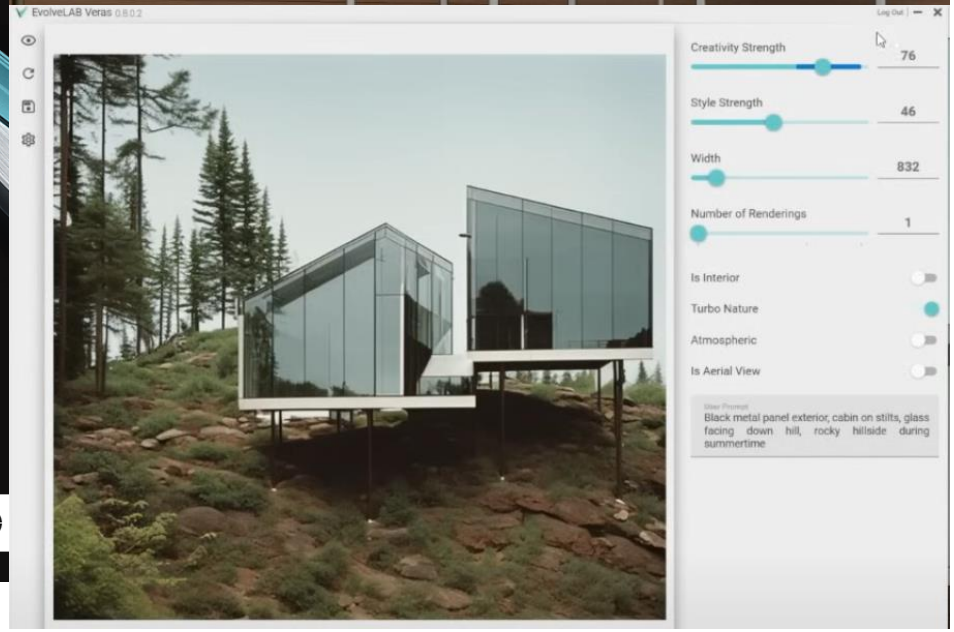
**Description**

This painting showcases a modernist house with bold yellow autumn trees in the background, emphasizing the blend of architecture and nature.

**Details**

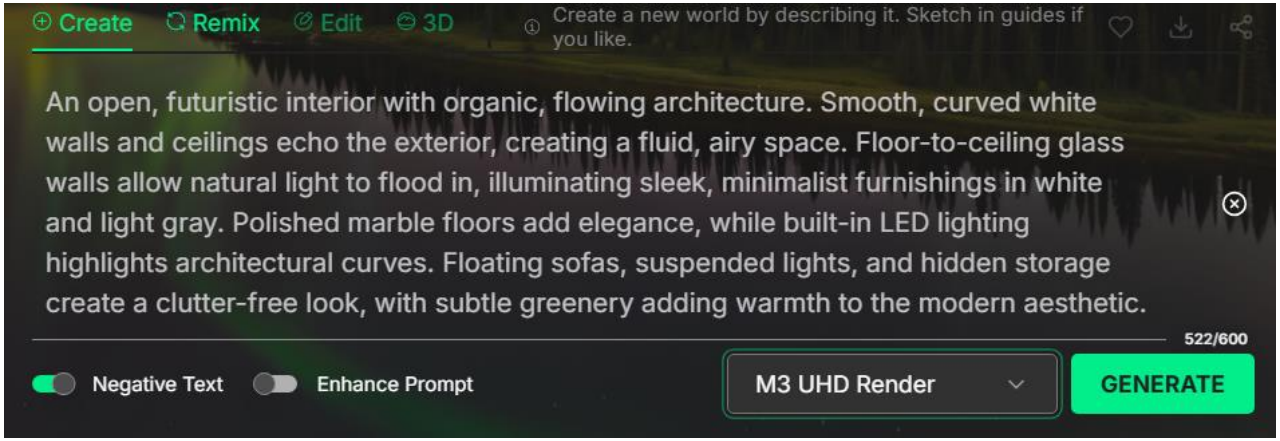
# LLM 과 AI Agent Gen AI와 Vibe Coding




VERAS | EvolveLAB

# LLM 과 AI Agent Vibe Coding




Skybox AI


Adobe Firefly

**OpenArt**

- **설명**: 텍스트로부터 이미지 생성, 스타일 변환, 커스텀 모델 학습 및 사진을 비디오로 변환하는 기능 제공.
- **링크**: https://openart.ai/

**Runway ML**

- **설명**: 텍스트를 기반으로 비디오 생성 및 편집을 지원하는 AI 콘텐츠 제작 플랫폼.
- **링크**: https://app.runwayml.com/

**Sora (OpenAI)**

- **설명**: 텍스트 설명을 기반으로 고품질 비디오를 생성하는 OpenAI의 실험적 모델.
- **링크**: https://sora.chatgpt.com/explore/videos

**Suno AI (구: Mureka AI)**

- **설명**: 텍스트 프롬프트로 음악을 생성하는 생성형 AI 서비스.
- **링크**: https://www.mureka.ai/

**D-ID**

- **설명**: 사진을 애니메이션화하고, 텍스트나 오디오를 기반으로 말하게 만드는 AI 서비스.
- **링크**: https://studio.d-id.com/

**Kling AI**

- **설명**: 캐릭터 움직임과 장면 시뮬레이션이 가능한 고성능 비디오 생성 AI.
- **링크**: https://klingai.com/global/

**Luma Labs Dream Machine**

- **설명**: 간단한 텍스트를 고품질 짧은 동영상으로 변환하는 생성형 AI 모델.
- **링크**: https://dream-machine.lumalabs.ai/

**Claude MCP (Model Context Protocol)**

- **설명**: Anthropic이 개발한 오픈 프로토콜로, Claude 같은 대형 언어 모델이 외부 도구, 시스템, 데이터 소스와 실시간으로 상호작용할 수 있도록 지원함.
- **링크**: https://modelcontextprotocol.io/

# LLM 과 AI Agent Gen AI와 Vibe Coding

바이브 코딩(Vibe Coding)은 특정 분위기나 느낌(vibe)을 조성하거나 반영하는 데 중점을 둔 코딩 방식을 의미

- ChatGPT 에 코딩 요청을 해서 생성된 파이썬 같은 코드를 복사&붙여넣기해 프로그램을 완성해 나가는 방법
- Gemini CLI, Claude code CLI, codex CLI 도구를 사용해 프로젝트 파일 및 소스코드를 생성하는 방법
- vscode 같은 개발 IDE와 연동되는 github copilot, cursor, windsurf와 같은 도구를 사용해 바이브 코딩하는 방법
- Bubble.io나 Canva와 같은 바이브 코딩 웹서비스에서 직접 요구사항을 입력하여 제공 클라우드에 앱을 생성 빌드 실행하는 방법

# LLM 과 AI Agent Gen AI와 Vibe Coding

GUI 기반 텍스트 편집기를 개발할꺼야. 파이썬과 Tkinter 라이브러리를 이용할꺼야. 메뉴는 파일 메뉴, 편집 메뉴, 테마 선택 메뉴(다크모드 하위 메뉴 포함), 찾기 메뉴로 구성됨. 이 프로그램 개발을 위한 PRD를 UI 스케치를 포함해 md 포맷으로 작성해.

사용할 라이브러리와 같은 기술 스택을 지정하는 것은 중요하다. 약간 시행착오를 하며 명확한 PRD가 작성될 수 있도록 프롬프트에 정확한 정보를 포함시킨다. md 포맷은 좀 더 명확한 문서형식작성을 위해 사용했다. md포맷은 markdown 파일형식으로 github에서 README 파일 작성할때 표준 형식이다.

# **Python/Tkinter 기반 GUI 텍스트 편집기 제품 요구사항 명세서 (PRD)**

## **1. 개요**

본 문서는 파이썬(Python)과 표준 GUI 라이브러리인 티케이인터(Tkinter)를 사용하여 개발될 데스크톱 텍스트 편집기의 요구사항을 정의합니다. 이 애플리케이션은 기본적인 텍스트 편집 기능과 함께 사용자의 편의를 위한 다크 모드, 찾기 기능 등을 제공하는 것을 목표로 합니다.

## **2. 목표 사용자**

* **초보 개발자 및 학생**: 간단한 코드나 스크립트, 메모, 과제 등을 작성하고 편집하려는 사용자.
* **일반 사용자**: Windows 메모장과 같은 가볍고 빠른 텍스트 편집기가 필요한 사용자.

## **3. 핵심 기능**

### **3.1. 파일 메뉴**

| 기능 | 단축키 | 설명 |
| --- | --- | --- |
| **새 파일** | `Ctrl+N` | 새로운 빈 편집 창을 엽니다. |
| **열기** | `Ctrl+O` | 기존 텍스트 파일 (`.txt` 등)을 불러옵니다. |
| **저장** | `Ctrl+S` | 현재 파일을 덮어쓰기 저장합니다. |
| **다른 이름으로 저장** | `Ctrl+Shift+S` | 새 이름이나 경로로 파일을 저장합니다. |
| **종료** | `Alt+F4` | 프로그램을 종료합니다. (저장되지 않은 내용이 있으면 경고) |

### **3.2. 편집 메뉴**

| 기능 | 단축키 | 설명 |
| --- | --- | --- |
| **실행 취소 (Undo)** | `Ctrl+Z` | 마지막 작업을 되돌립니다. |
| **다시 실행 (Redo)** | `Ctrl+Y` | 되돌린 작업을 다시 실행합니다. |
| **잘라내기** | `Ctrl+X` | 선택한 텍스트를 잘라내 클립보드에 저장합니다. |

## **5. UI 스케치**

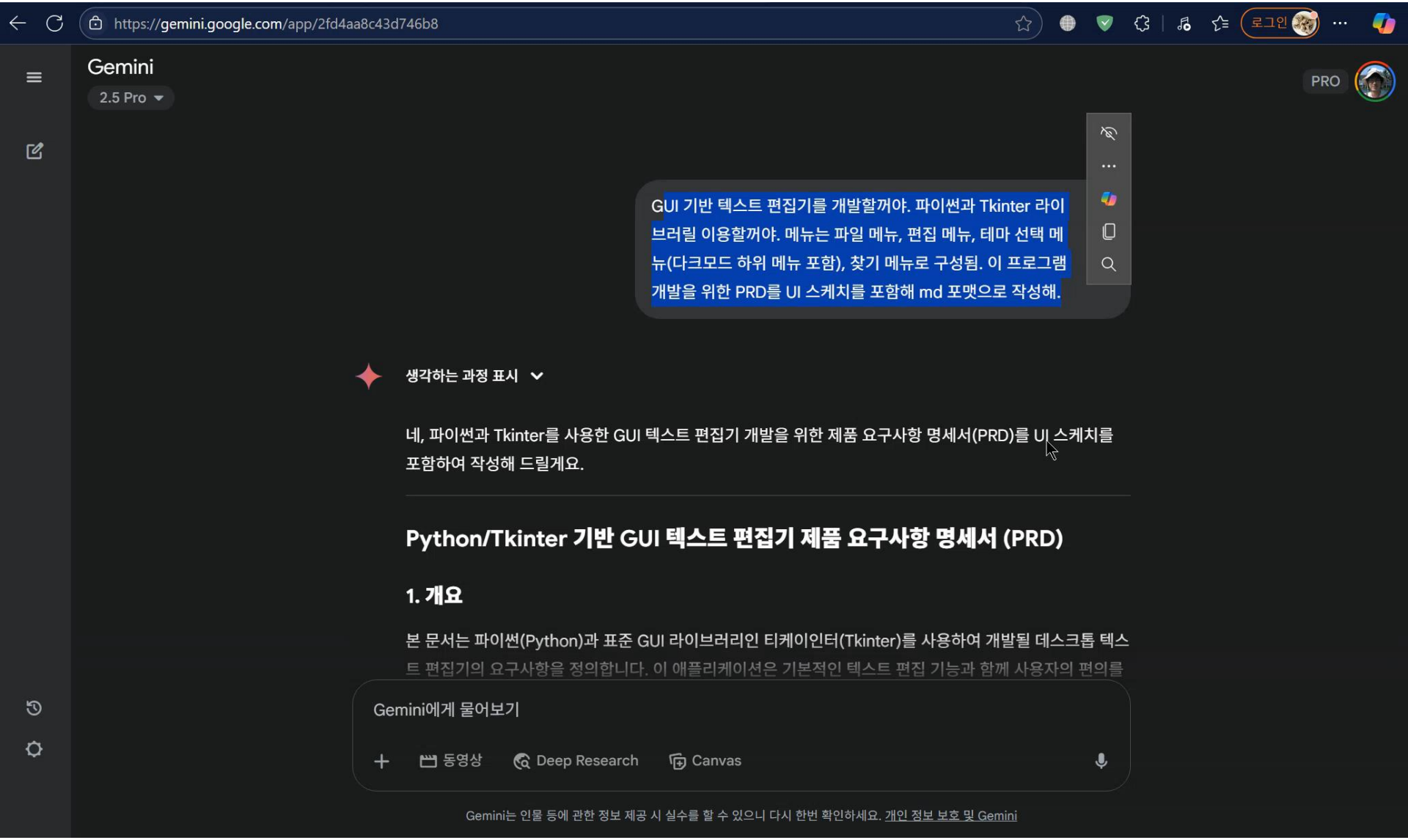
애플리케이션의 전체적인 레이아웃 스케치입니다.

```
+-------------------------------------------------------------------+
| My Text Editor                                        - □ X |
+-------------------------------------------------------------------+
| 파일(F) | 편집(E) | 테마(T) | 찾기(S)                                |
+-------------------------------------------------------------------+
| |                                                 |
| |                                                 |
| |                                                 |
| |       ##############################################        |
| |       #                                #        |
| |       #    사용자가 텍스트를 입력하는 메인 영역    #        |
| |       #                                #        |
| |       ##############################################        |
| |                                                 |
| |                                                 |
| |                                                 |
| |                                                 |
+-------------------------------------------------------------------+
| 줄: 1, 열: 1                                        UTF-8  |
+-------------------------------------------------------------------+
```

**UI 구성 요소:**

* **제목 표시줄**: 프로그램 이름과 창 조절 버튼이 표시됩니다.
* **메뉴 바**: 파일, 편집, 테마, 찾기 메뉴가 위치하며, 각 메뉴 클릭 시 하위 메뉴 항목들이 나타납니다.
* **텍스트 영역**: 스크롤 가능한 주 텍스트 편집 공간입니다.
* **상태 표시줄**: 하단에 커서의 현재 위치(줄, 열)나 인코딩 같은 간단한 상태 정보를 보여줍니다.

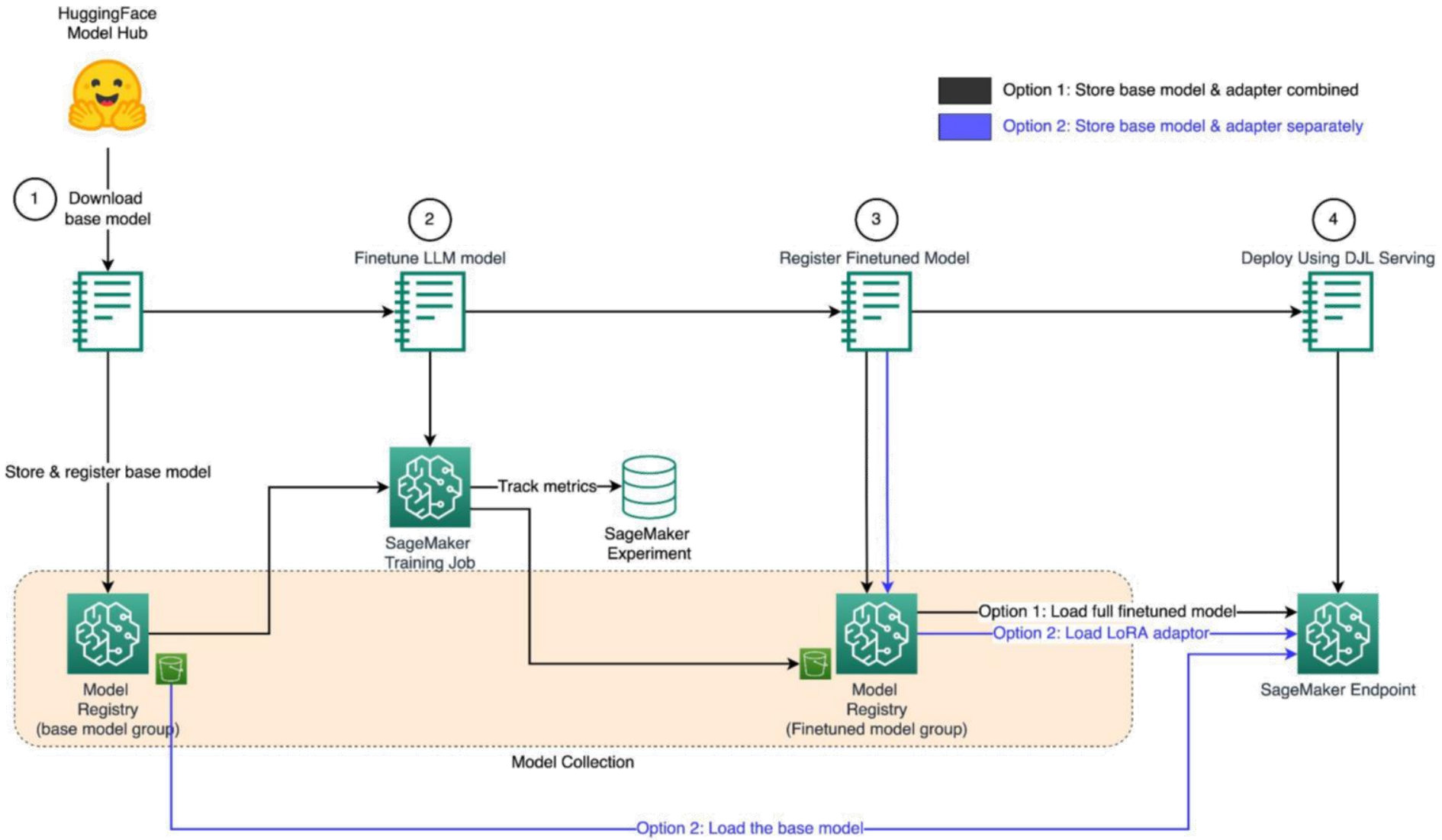
# LLM 과 AI Agent Gen AI와 Vibe Coding



# AI 모델의 서비스화 개요

- 모델 학습 vs 서빙
- API 기반 AI 서비스 아키텍처 소개
- ONNX, FastAPI, Flask 등 경량화 및 배포 전략 개요

# AI 모델의 서비스화 모델 학습 vs 서빙




https://aws.amazon.com/blogs/machine-learning/model-management-for-lora-fine-tuned-models-using-llama2-and-amazon-sagemaker/

# AI 모델의 서비스화 API 기반 AI 서비스 아키텍처 소개

**User**
hi

**Uber Order Taking Longer than expected support bot**
Hello! How can I assist you today?

**User**
my order is an hour late give me back my money

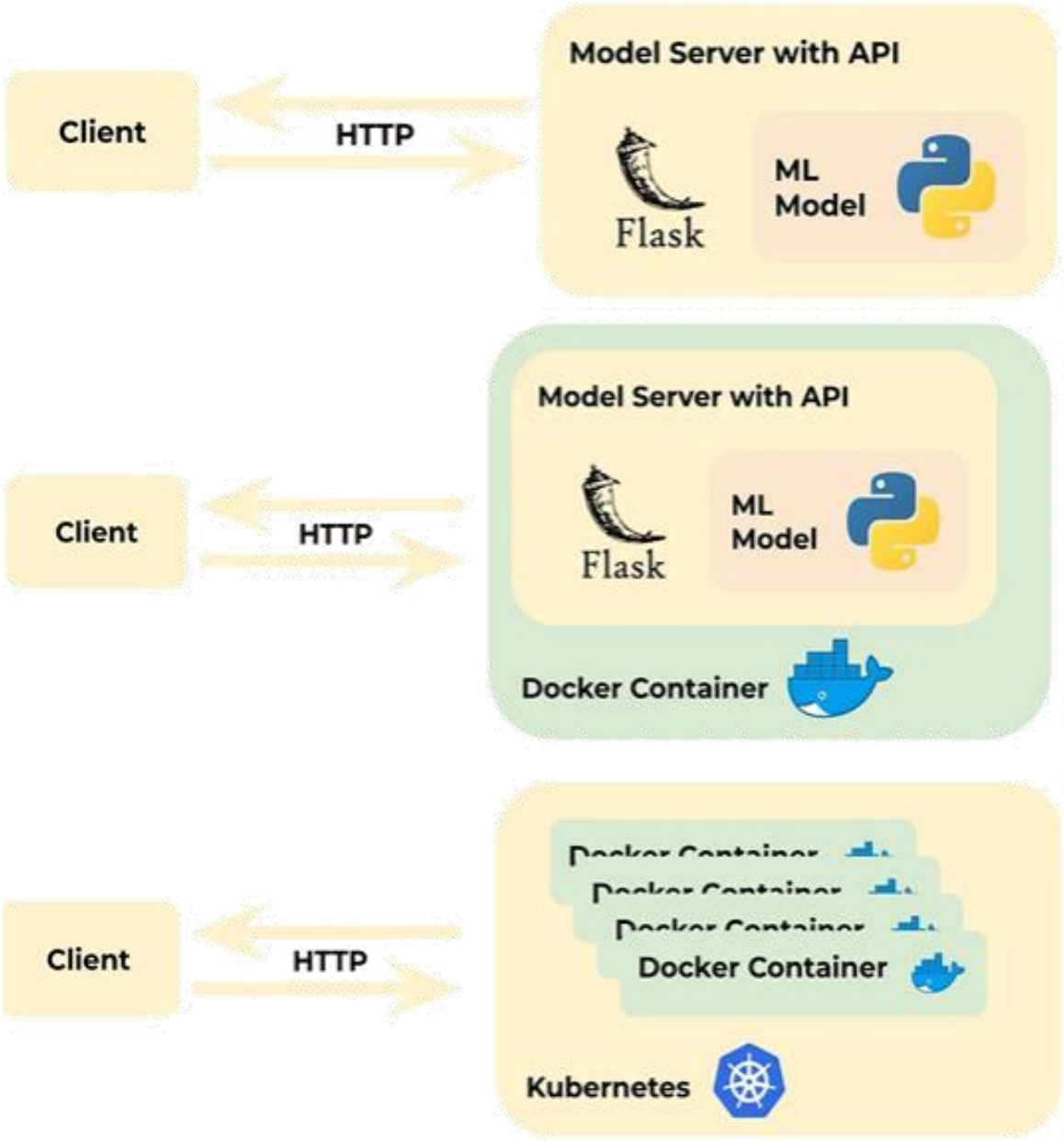
**Uber Order Taking Longer than expected support bot**
I have checked the details, and your order is still on track to be delivered on time. It's currently being prepared at {tea corner} and is expected to be delivered by 5:00pm. A courier has already been assigned to your order. Please keep an eye on the app for updates, and thank you for your patience.

**User**
you are wrong it is an hour late, give me back my money

**Uber Order Taking Longer than expected support bot**

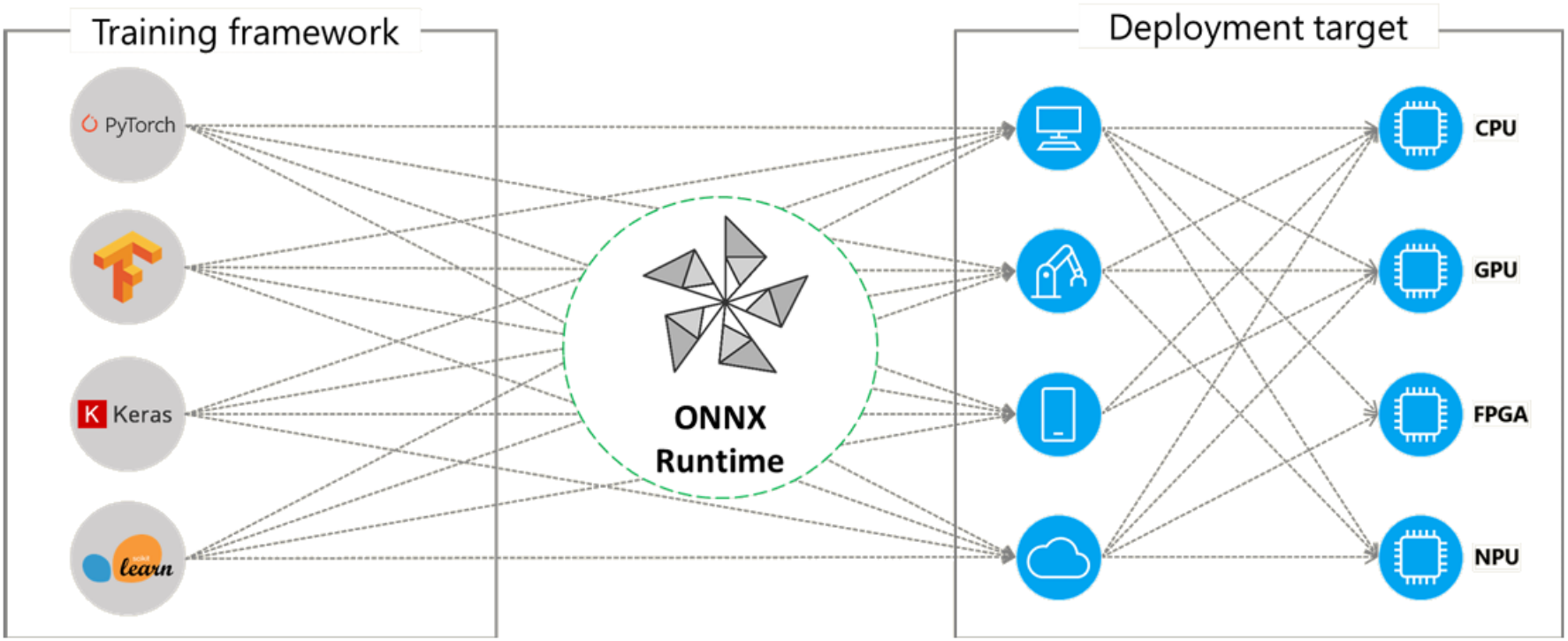
I understand this must be frustrating, but according to our records, your order is not late. It is expected to be delivered by {tea corner} between 5:00pm and 5:30pm, and we are currently before that window. A courier has been assigned to your order, indicating it is on its way. Unfortunately, if you choose to cancel now, the order will be fully charged as per our policy. Please let me know if there's anything else I can do to assist you.

# AI 모델의 서비스화 ONNX, FastAPI, Flask 등 경량화 및 배포 전략



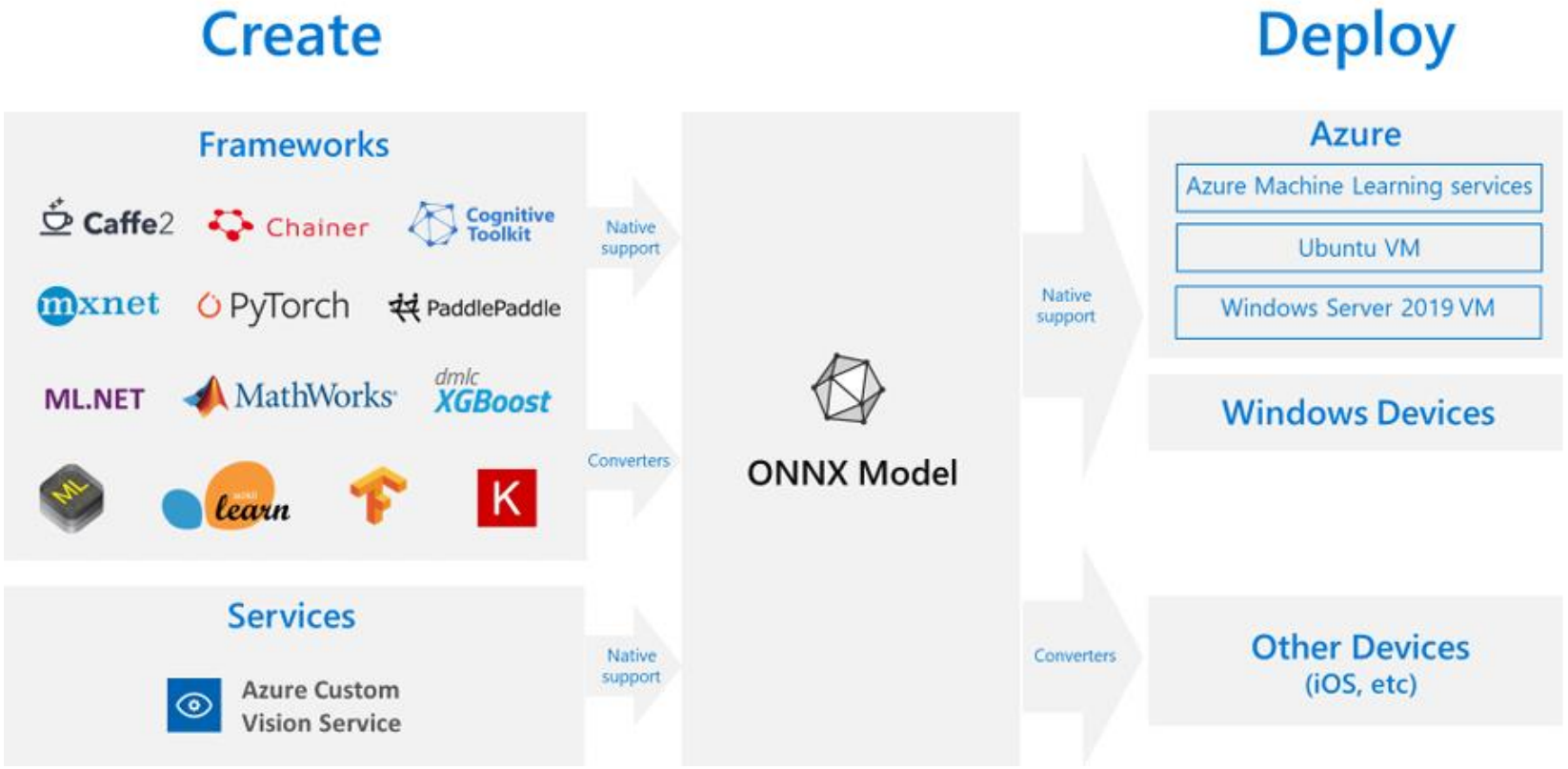

Serve your first model with Scikit-Learn + Flask + Docker | by Carl W. Handlin | Rappi Tech

# AI 모델의 서비스화 ONNX, FastAPI, Flask 등 경량화 및 배포 전략




Serve your first model with Scikit-Learn + Flask + Docker | by Carl W. Handlir
| Rappi Tech

# AI 모델의 서비스화 ONNX, FastAPI, Flask 등 경량화 및 배포 전략

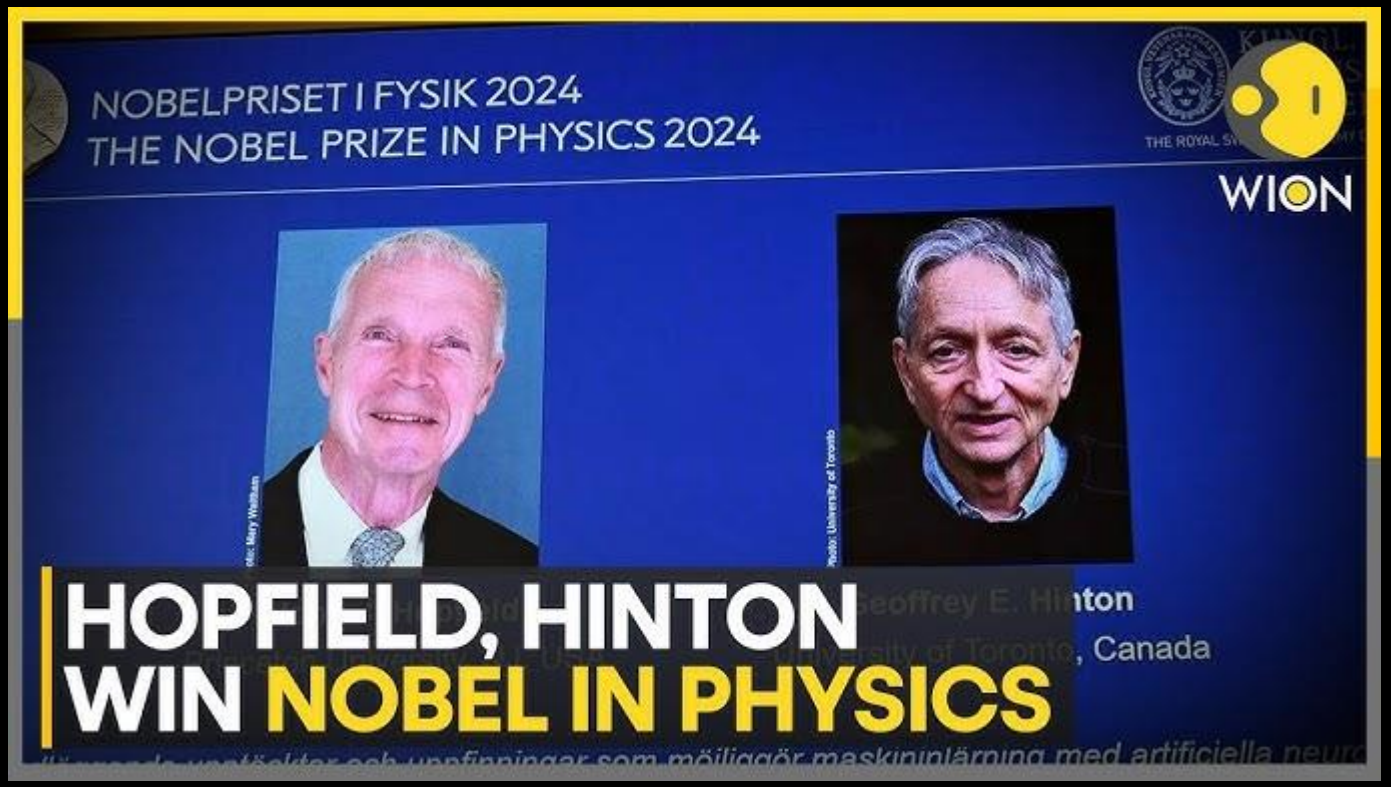



Serve your first model with Scikit-Learn + Flask + Docker | by Carl W. Handlir
| Rappi Tech

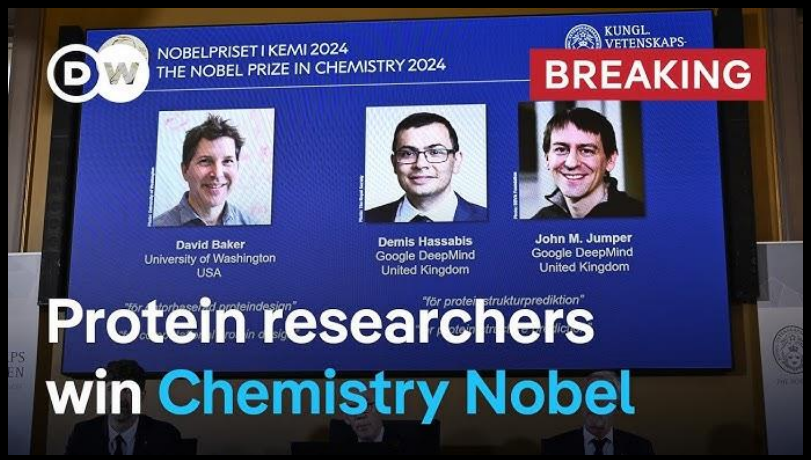
# AX전략 및 최신 트렌드 소개

- 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보 등)
- 질의응답 및 추천 학습 자료 안내

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)




AI Pioneers Geoffrey Hinton And John Hopfield Win Nobel Prize For Physics | Latest News | WION




Members of the Nobel Committee for Chemistry at the Royal Swedish Academy of Sciences explain the work of 2024 Nobel Prize in Chemistry winners David Baker, Demis Hassabis and John M. Jumper.JONATHAN NACKSTRAND/AFP via Getty Images

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

We've been testing the capabilities of
Gemini, our new multimodal AI model.

We've been capturing footage to test it on a wide
range of challenges, showing it a series of images,
and asking it to reason about what it sees.

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

All Images Videos Shopping Books **News** Maps More Tools

머니투데이

## "'추격조'에 GPU 몰아주면 韓 딥시크 10개 나와"

"글로벌 AI 추격조를 만들어 '3년간 한국 데이터를 다 가져다 써라' 해야 합니다. 정부가 연내 GPU를 1만개를 확보해 상하반기 각 5개 기업에 2000...

Feb 6, 2025

한국일보

## [현장] "GPU·데이터 몰아주면 한국에서도 딥시크 10개 나온다"

6일 과학기술정보통신부가 개최한 중국 인공지능(AI) 모델 '딥시크'의 등장 이후 국내 AI 산업의 경쟁력을 점검하는 회의에서 거대언어모델(LLM)을...

Feb 7, 2025

중앙일보

## 韓 AI 회사 "지금이 오히려 기회…딥시크 10개 만들 방법 있다" [팩플]

중국 생성 인공지능(AI) 모델 딥시크의 등장은 한국 AI 산업에 위기일까 기회일까. 국내 AI 기업들은 'AI 추격조'를 만드는 등 정부의 전폭적 지원이...

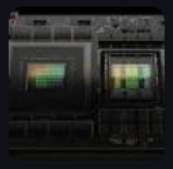
Feb 6, 2025

노컷뉴스

## [Q&A]AI 강국되려면 GPU 잡아라…"품귀현상에도 구할 수 있다"

"더 많은 인프라를 가지고 양질의 데이터를 확보해서 AI 모델을 만드는 것이 중요하다" (배경훈 LG AI연구원장) "GPU 1만 개 확보해서 10개 회사에...

1 month ago



# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

## Large Language Models as General Pattern Machines

Suvir Mirchandani Fei Xia Pete Florence Brian Ichter
Danny Driess Montserrat Gonzalez Arenas Kanishka Rao
Dorsa Sadigh Andy Zeng

general-pattern-machines.github.io

Google DeepMind Stanford TECHNISCHE UNIVERSITÄT BERLIN

| TITLE | CITED BY | YEAR |
|---|---|---|
| Large language models as general pattern machines<br>S Mirchandani, F Xia, P Florence, B Ichter, D Driess, MG Arenas, K Rao, ...<br>arXiv preprint arXiv:2307.04721 | 103 | 2023 |

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

Siemens Process Simulate (left) connects to NVIDIA Omniverse (right) to enable a full-design-fidelity, photorealistic, real-time digital twin. https://www.robotics247.com/article/siemens_xcelerator_nvidia_omniverse_accelerate_digital_twins_manufacturing

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

## 생성 AI 서비스 조사 보고서 - Deloitte

- 마케팅 컨텐츠 작성
- 코딩 개발 지원
- 고객 응대 지원
- 제품 설계 지원
- 연구 보고서 조사 작성
- 합성 및 학습 데이터 생성
- 기업 내 데이터 검색, 정리, 분석 및 요약
- 게임 컨텐츠 설계 및 개발
- 언어 번역
- 시뮬레이션 및 추론
- 고도화된 개인 맞춤 교육 및 훈련
- 현장 업무 지원


### About the Deloitte AI Institute

The Deloitte AI Institute™ helps organizations connect all the different dimensions of the robust, highly dynamic, and rapidly evolving Artificial Intelligence ecosystem. The AI Institute leads conversations on applied AI innovation across industries, with cutting-edge insights,to promote human-machine collaboration in the "Age of With™."

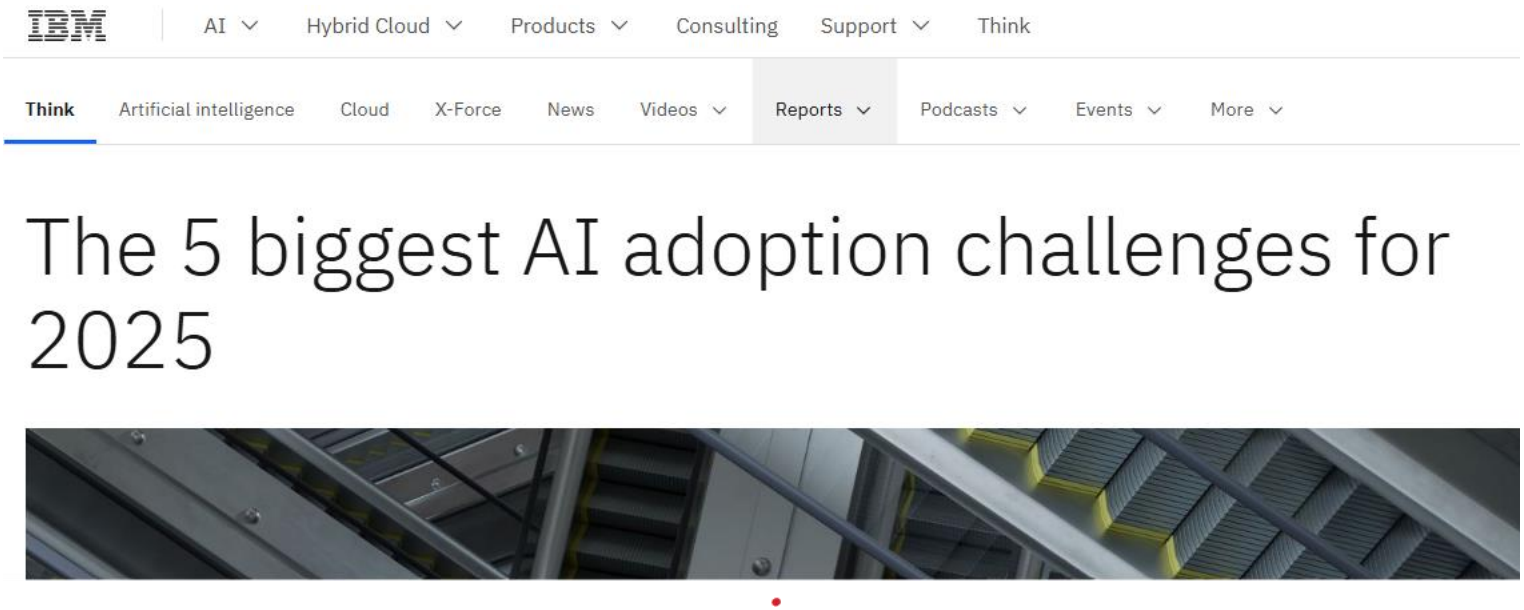
The Deloitte AI Institute aims to promote the dialogue and development of AI, stimulate innovation, and examine challenges to AI implementation and ways to address them. The AI Institute collaborates with an ecosystem composed of academic research groups, start-ups, entrepreneurs, innovators, mature AI product leaders, and AI visionaries to explore key areas of artificial intelligence including risks, policies, ethics, the future of work and talent, and applied AI use cases. Combined with Deloitte's deep knowledge and experience in artificial intelligence applications, the Institute helps make sense of this complex ecosystem, and as a result, delivers impactful perspectives to help organizations succeed by making informed AI decisions.

No matter what stage of the AI journey you are in: whether you are a board member or a C-Suite leader driving strategy for your organization—or a hands-on data scientist bringing an AI strategy to life—the Deloitte AI Institute can help you learn more about how enterprises across the world are leveraging AI for a competitive advantage. Visit us at the Deloitte AI Institute for a full body of our work, subscribe to our podcasts and newsletter, and join us at our meet-ups and live events. Let's explore the future of AI together.

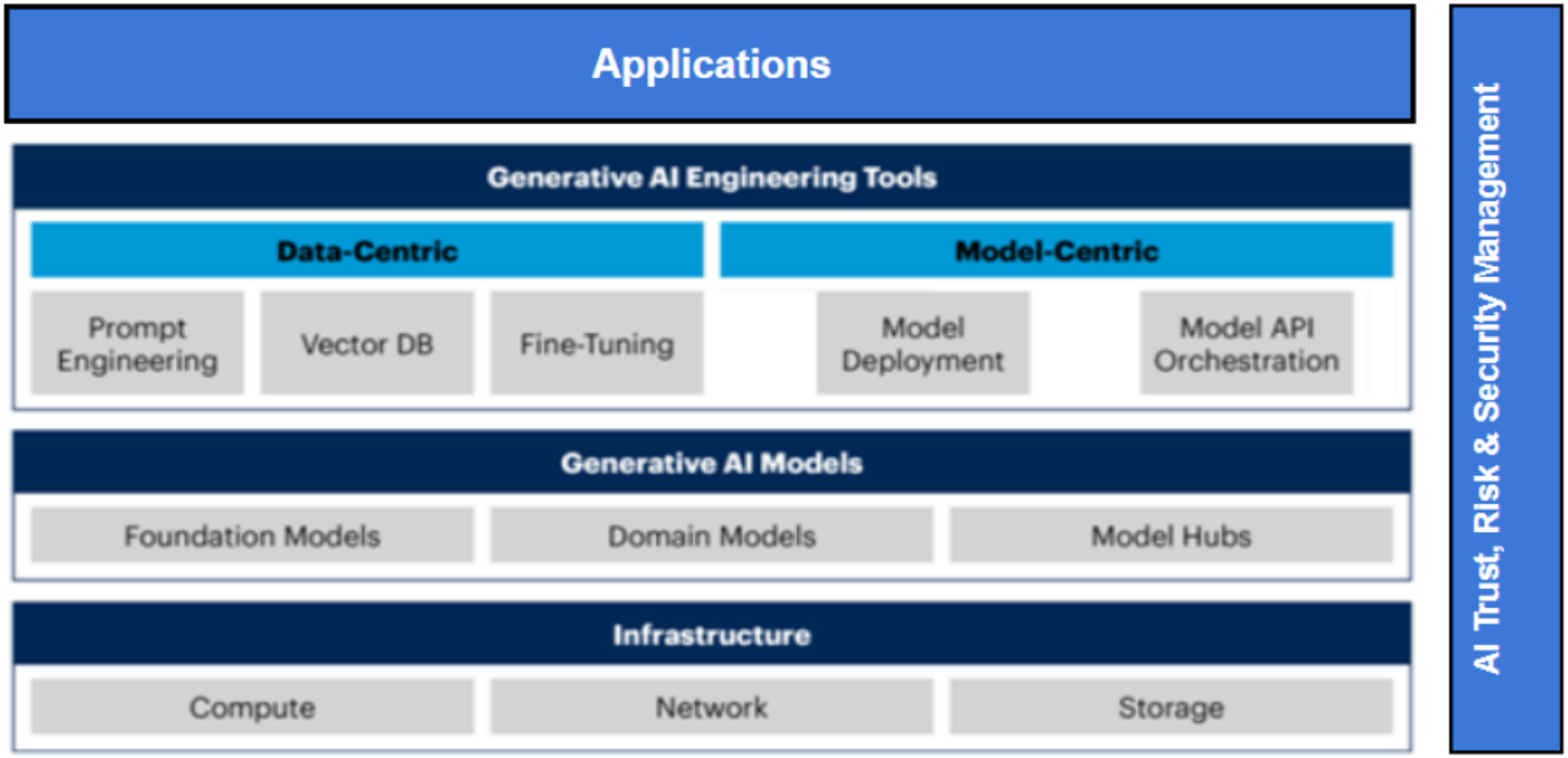
**www.deloitte.com/us/AIInstitute**

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

Concerns about data accuracy or bias (45%)
Insufficient proprietary data available to customize models (42%)
Inadequate generative AI expertise (42%)
Inadequate financial justification or business case (42%)
Concerns about privacy or confidentiality of data and information (40%)

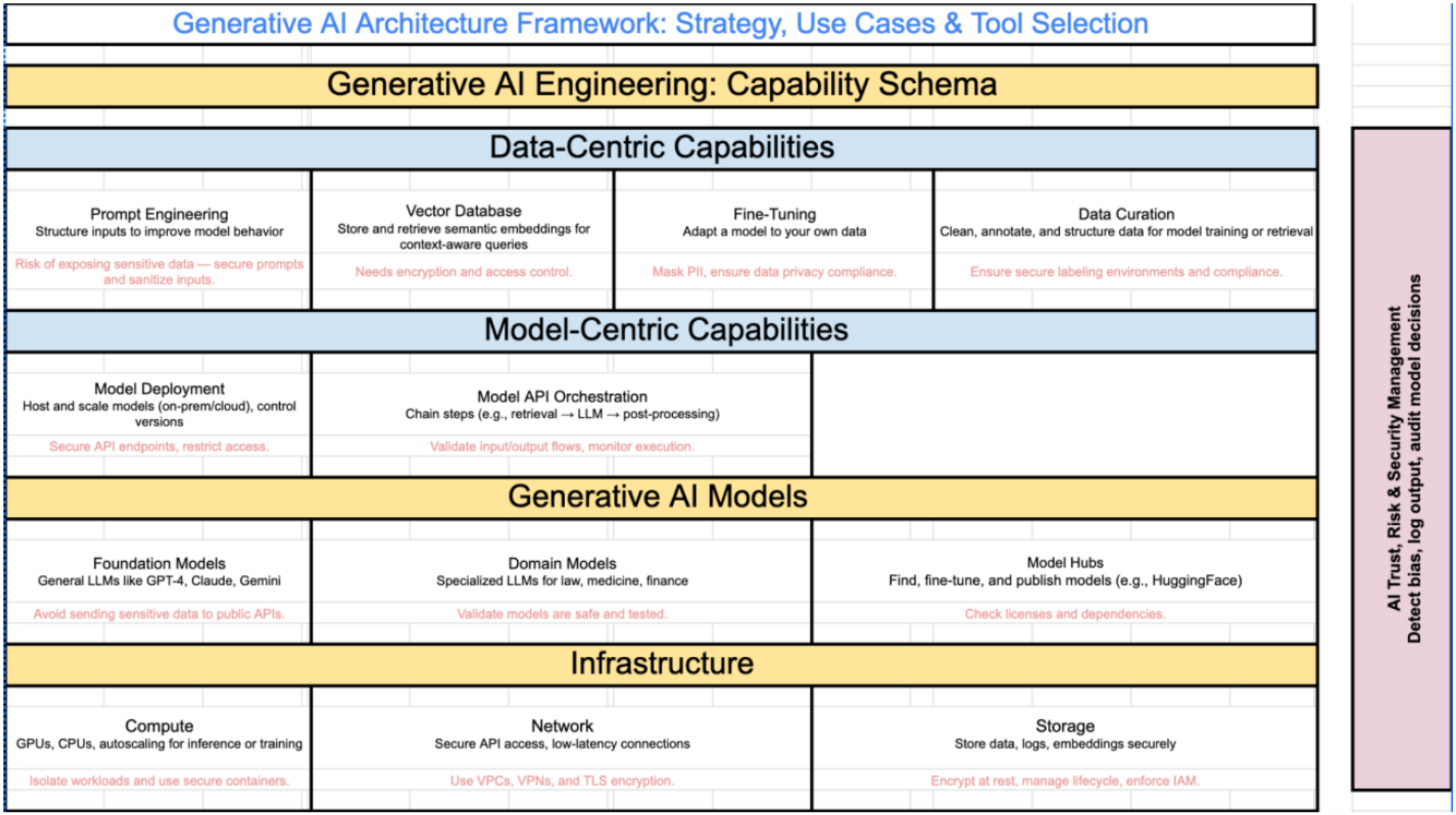


# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

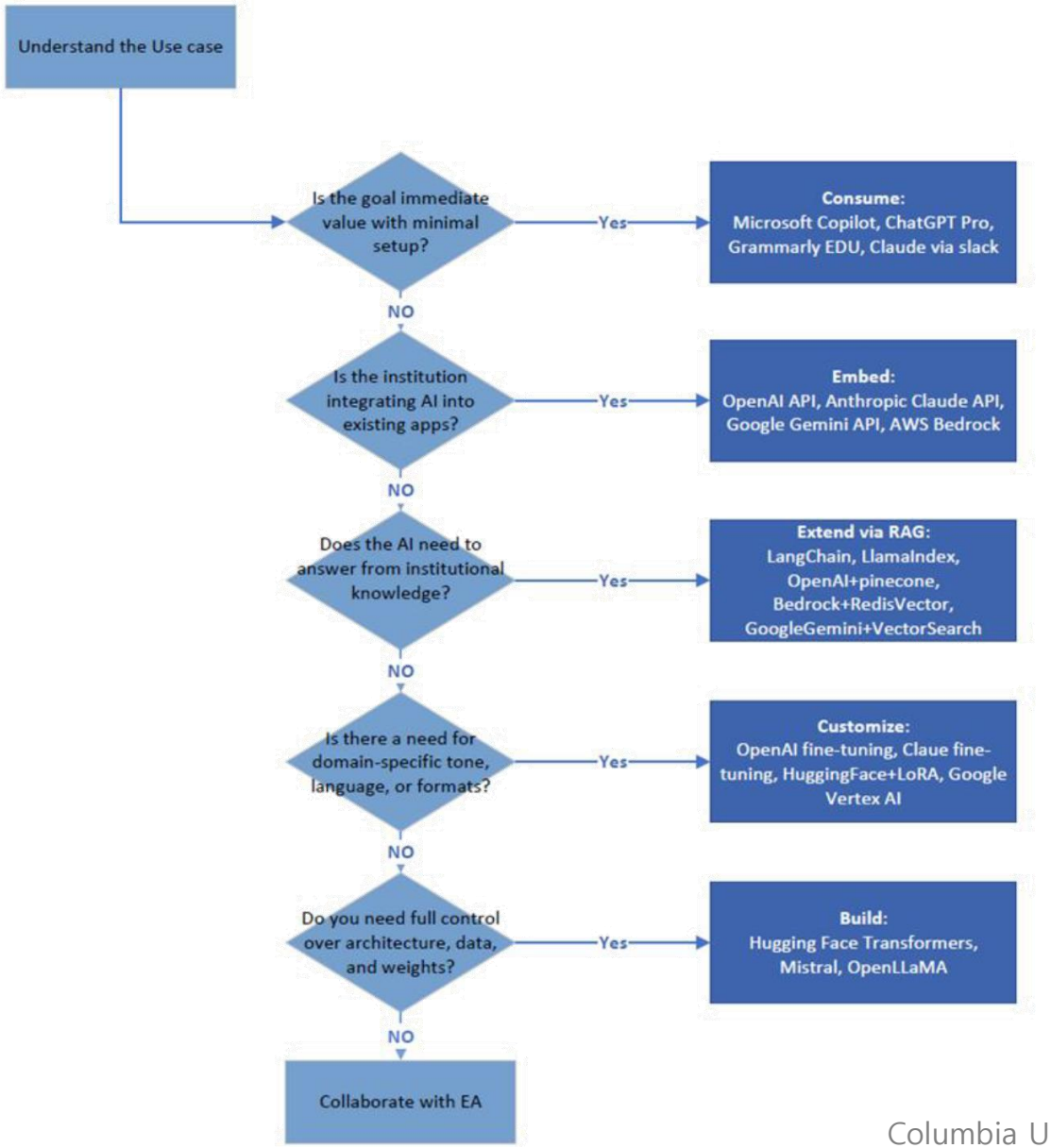



Columbia University, 2025.6, AI Framework

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)



# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)



# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

## Appendix: AI Toolset Evaluation

To ensure objective and consistent selection of AI tools, Columbia proposes a scoring framework based on six key criteria. Each tool is scored on a scale of 1 to 10 using standardized rubrics, and then weighted to reflect institutional priorities.

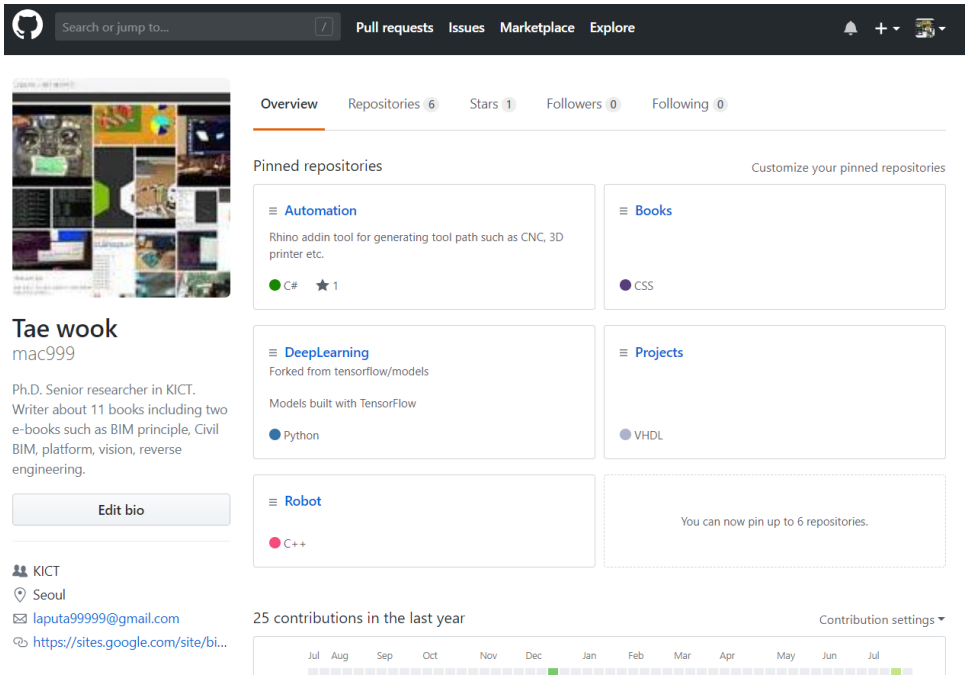
### Evaluation Criteria

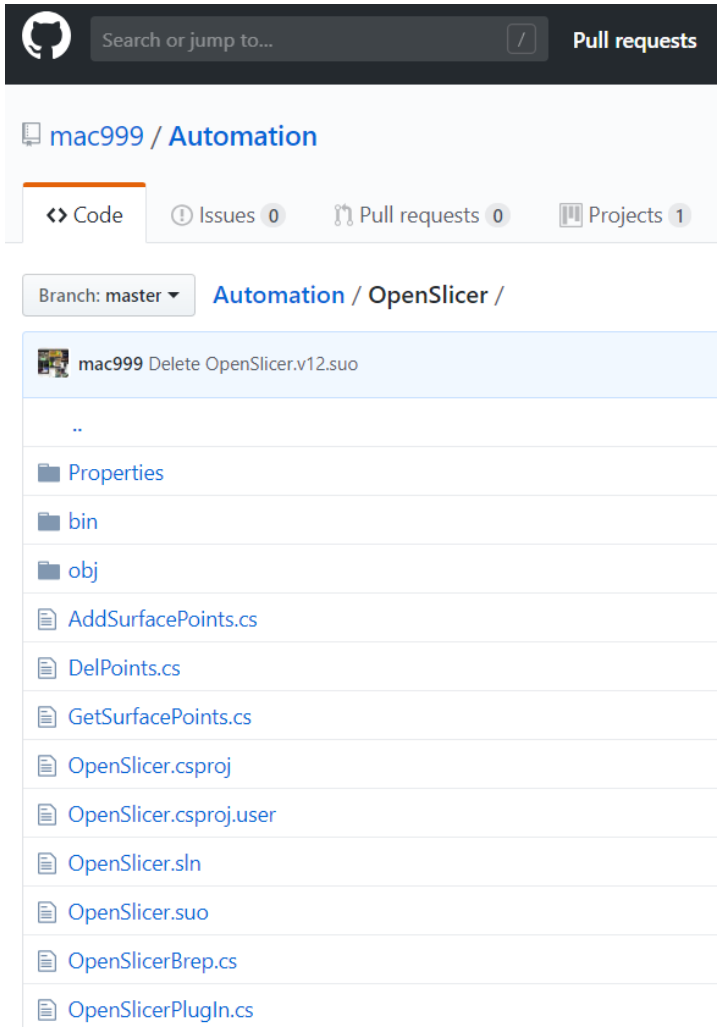
| Criterion | What It Measures |
|---|---|
| Performance | Speed, scalability, and accuracy under real-world workloads |
| Cost | Licensing, usage, infrastructure cost, and transparency |
| Customization | Ability to fine-tune or configure for domain-specific use |
| Compliance & Security | Support for regulatory/security standards (FERPA, HIPAA, etc.) |
| Integration & Tooling | Compatibility with systems, APIs, SDKs, and documentation |
| Community & Support | Docs, vendor responsiveness, and community activity |

| Criterion | Weight |
|---|---|
| Performance | 25% |
| Cost | 20% |
| Customization | 15% |
| Compliance & Security | 15% |
| Integration & Tooling | 15% |
| Community & Support | 10% |

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)



# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

GNU General Public License(GPL) 2.0
– 의무 엄격. SW 수정 및 링크 경우 소스코드 제공 의무

GNU Lesser GPL(LGPL) 2.1
– 저작권 표시. LPGL 명시. 수정한 라이브러리 소스코드 공개

Berkeley Software Distribution(BSD) License
– 소스코드 공개의무 없음. 상용 SW 무제한 사용 가능

Apache License
– BSD와 유사. 소스코드 공개의무 없음.

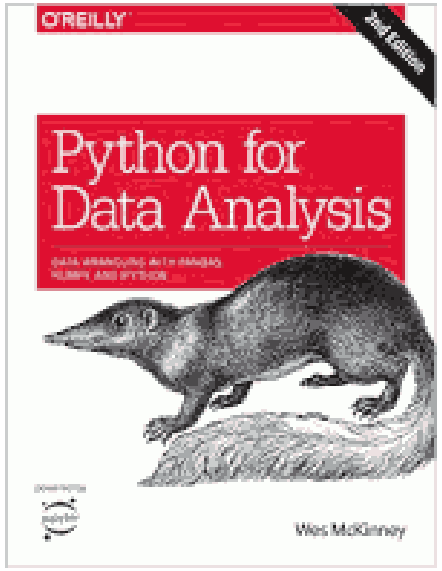
Mozilla Public License(MPL)
– 소스코드 공개의무 없음. 수정 코드는 MPL에 의해 배포. 이외 결합 프로그램 코드는 공개 필요 없음

MIT License
– 라이선스 / 저작권만 명시 조건

http://guswnsxodlf.github.io/software-license

# AX 전략과 트랜드 기업 AI 도입 시 고려사항 (비용, 성능, 데이터 확보)

# AX 전략과 트랜드 질의응답 및 추천 학습 자료 안내




NUMFOCUS
OPEN CODE = BETTER SCIENCE

TWO SIGMA

VOLTRON DATA

d-fine

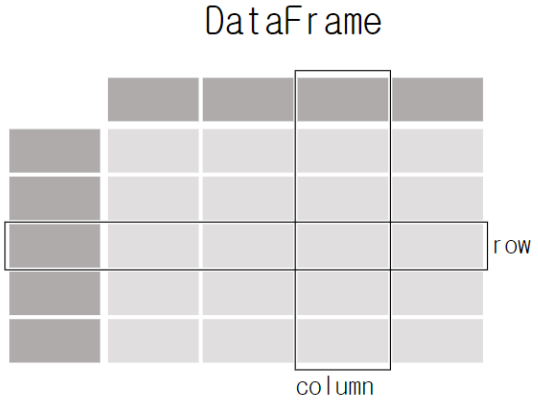
Quansight Labs

NVIDIA

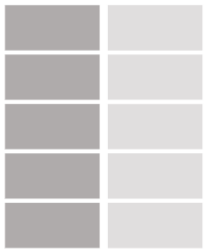
TIDELIFT

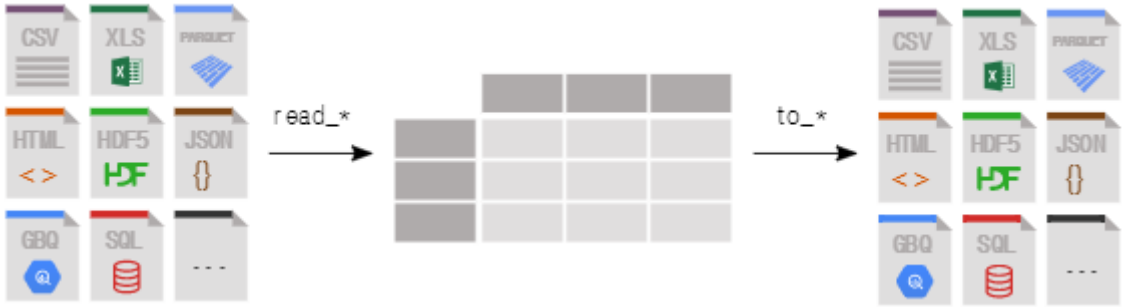
Chan Zuckerberg Initiative

bodo.ai




Each column in a `DataFrame` is a `Series`




https://pandas.pydata.org/

# AX 전략과 트랜드 질의응답 및 추천 학습 자료 안내

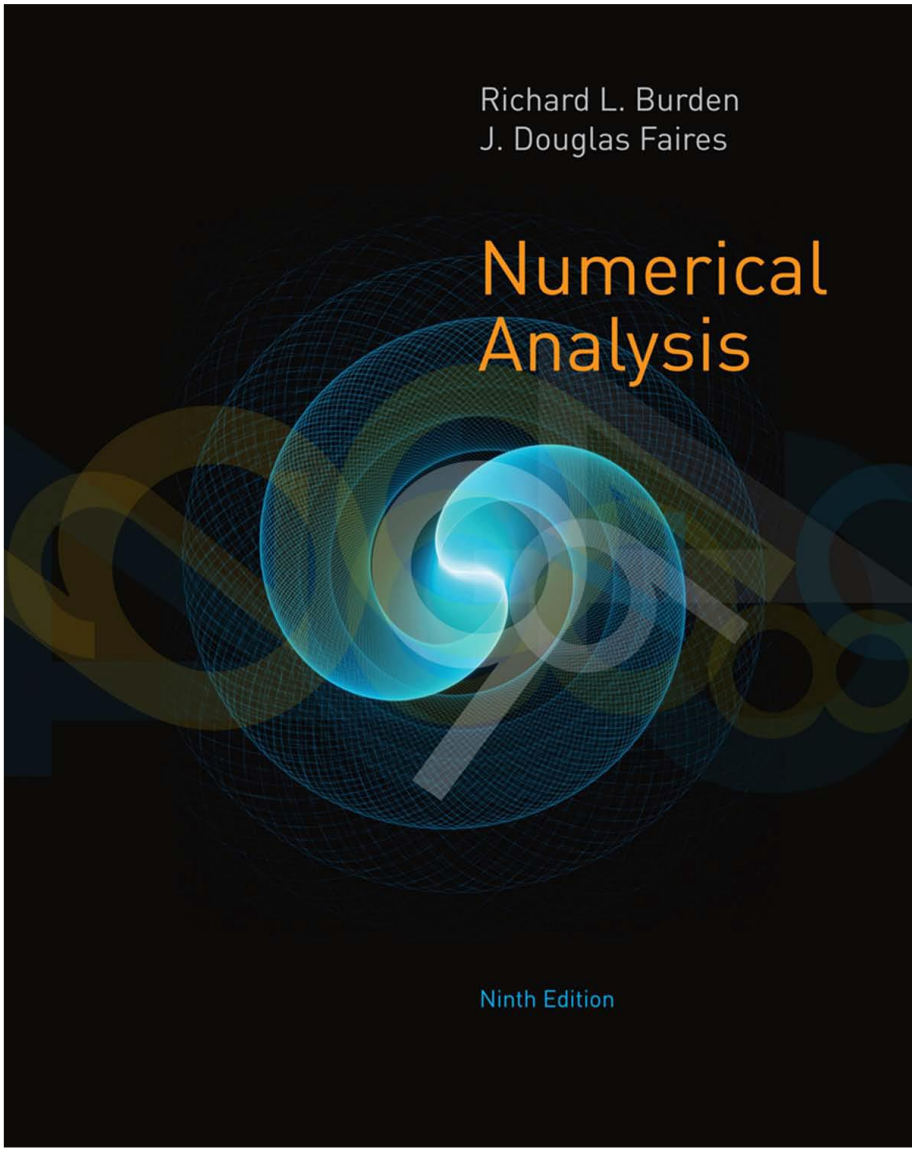
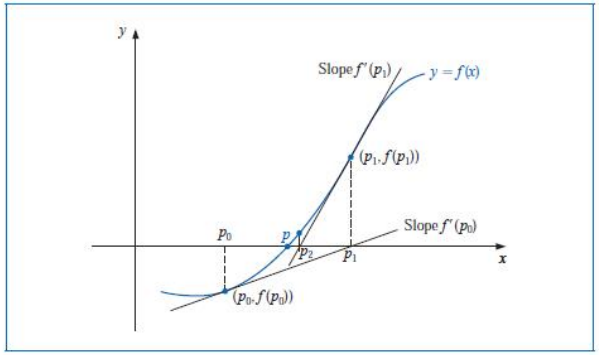





Figure 2.8




ALGORITHM
2.3

**Newton's**

To find a solution to $f(x) = 0$ given an initial approximation $p_0$:

INPUT initial approximation $p_0$; tolerance $TOL$; maximum number of iterations $N_0$.

OUTPUT approximate solution $p$ or message of failure.

*Step 1* Set $i = 1$.

*Step 2* While $i \le N_0$ do Steps 3–6.

*Step 3* Set $p = p_0 - f(p_0)/f'(p_0)$. (*Compute* $p_i$.)

*Step 4* If $|p - p_0| < TOL$ then
OUTPUT ($p$); (*The procedure was successful.*)
STOP.

*Step 5* Set $i = i + 1$.

*Step 6* Set $p_0 = p$. (*Update* $p_0$.)

*Step 7* OUTPUT ('The method failed after $N_0$ iterations, $N_0$ =', $N_0$);
(*The procedure was unsuccessful.*)
STOP. ■

The stopping-technique inequalities given with the Bisection method are applicable to Newton's method. That is, select a tolerance $\varepsilon > 0$, and construct $p_1, \ldots, p_N$ until

$$|p_N - p_{N-1}| < \varepsilon, \tag{2.8}$$

$$\frac{|p_N - p_{N-1}|}{|p_N|} < \varepsilon, \quad p_N \neq 0, \tag{2.9}$$

or

$$|f(p_N)| < \varepsilon. \tag{2.10}$$

# AX 전략과 트랜드 질의응답 및 추천 학습 자료 안내


## 목차(Contents)

http://youtu.be/Mxp1e2Zzg-A

# AX 전략과 트랜드 질의응답 및 추천 학습 자료 안내

CS231n: Deep Learning for Computer Vision

Lecture 1: Introduction

Fei-Fei Li & Ehsan Adeli CS231n: Lecture 1 - 1 April 2, 2024

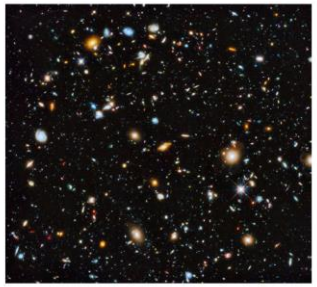
deeplearning.ai

Introduction to Deep Learning

Welcome

AndrewNg

Lecture 1: Overview



CS221 / Spring 2019 / Charikar & Sadigh

Brief Introduction to Natural Language Processing

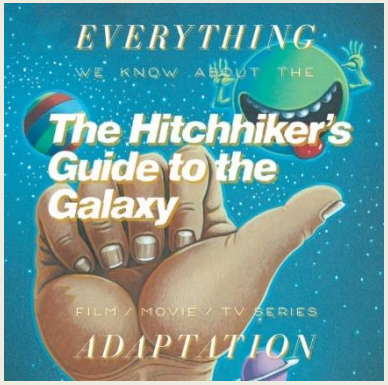
EE599 Deep Learning

Keith M. Chugg
Spring 2020

USC University of Southern California

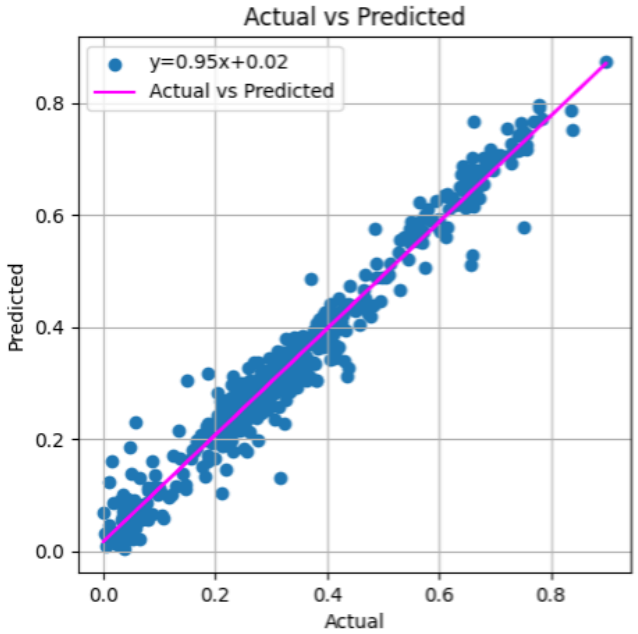
# Thanks

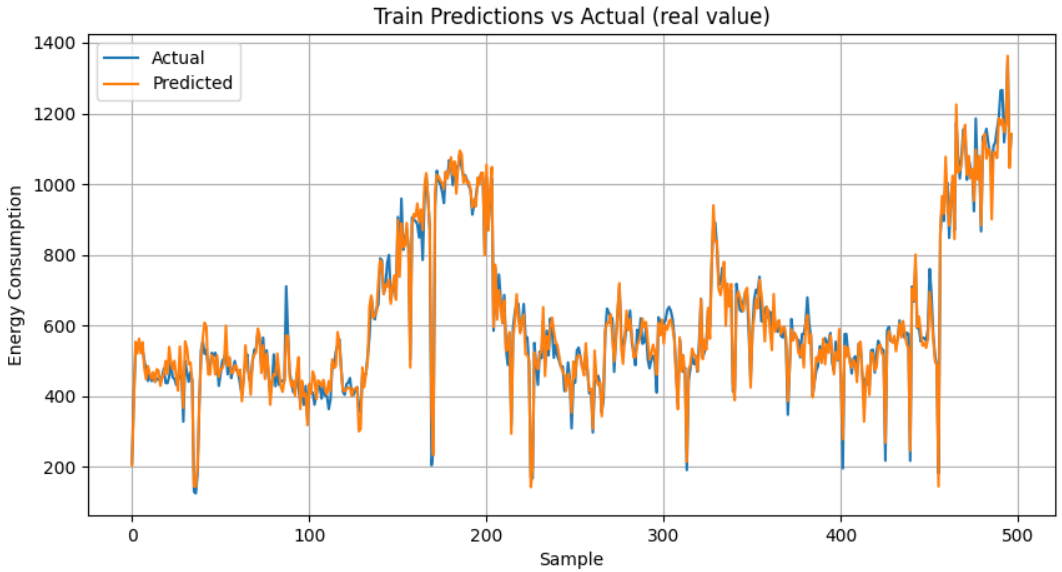
AX foundation and Trend 2025

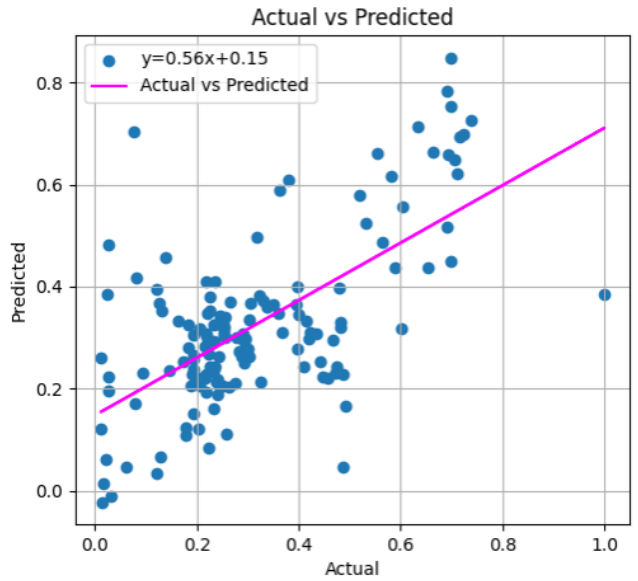


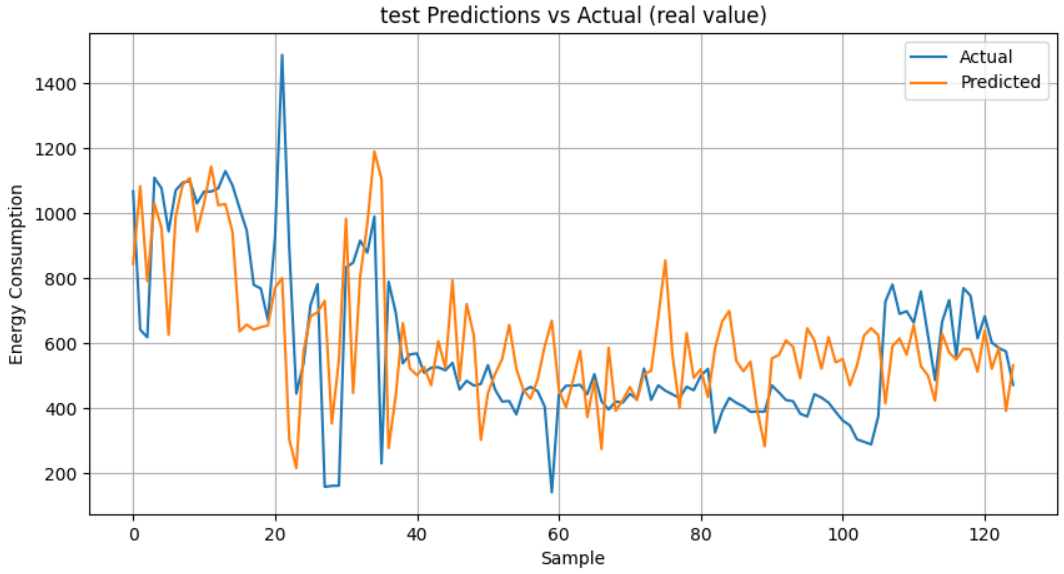

laputa99999@gmail.com

# 개발 사례 에너지 소비 예측




Loss=0.002. Train MAPE & Acc = (0.025, 95.30%). Test MAPE & Acc = (0.292, 70.76%)